參考用書：Operating System Concepts, 9th Edition
Abraham Silberschatz, Peter B. Galvin, Greg Gagne

作業系統

完整版講義

A4 直式 · 上下兩張 · Lin Cing

目錄

一、核心、介面與系統結構
第 1 章　導論 2
第 2 章　作業系統結構 18
二、行程、同步與排程
第 3 章　程序 41
第 4 章　執行緒 55
第 5 章　程序同步 68
第 6 章　CPU 排程 88
第 7 章　死結 113
三、位址轉換與虛擬記憶體
第 8 章　主記憶體 133
第 9 章　虛擬記憶體 149
四、儲存、檔案與 I/O
第 10 章　大量儲存 175
第 11 章　檔案系統介面 190
第 12 章　檔案系統實作 205
第 13 章　I/O 系統 223
五、保護與安全
第 14 章　保護 237
第 15 章　安全 248
六、虛擬化與分散式系統
第 16 章　虛擬機器 265
第 17 章　分散式系統 278
七、系統案例與設計脈絡
第 18 章　Linux 系統 292
第 19 章　Windows 7 311
第 20 章　具影響力的作業系統 334

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：作業系統如何在不受信任的程式與有限硬體之間提供方便、效率與隔離？

前置知識。 程式執行、CPU 與記憶體的基本角色、中斷概念。

學習目標

- 區分 user mode、kernel mode、中斷、例外與 trap
- 說明 programs、processes、files 與 storage hierarchy 的抽象關係
- 判斷 kernel data structure 與環境選擇對正確性及效能的影響

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

作業系統提供的不是單一功能，而是受管理的抽象

實體資源	程式使用的抽象	OS必須強制的條件
CPU／timer	process/thread執行	誰可跑、何時讓出、誰能改特權狀態
RAM／MMU	address space	映射與權限不可由一般程式任意改他人的
disk／controller	file與block服務	名稱、權限、存取次序與錯誤回報
NIC	socket／訊息	端點、buffer ownership、封包與連線狀態

方便性、利用率、response time、公平、隔離與可靠性是不同目標。例如長batch佔滿CPU可提高utilization，卻讓互動請求久等。需先指定目標與工作負載，不能以「效率高」概括所有指標。

kernel是一般系統的受信任特權核心；shell、GUI、library、daemon多在user space。呼叫library不必然進kernel；換shell也不必換kernel。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

policy決定允許什麼，mechanism強制它

例 Alice/Bob共用機器，Bob的程式不可信。policy：Bob不能讀Alice私頁與檔案、不得永久獨占CPU。mechanisms：受保護page table與每次memory access檢查；核心驗證file authority；受保護timer及scheduler入口。

單有應用的if檢查不夠：Bob可跳過該應用直接發請求。單有user/kernel mode也不夠：kernel若替Bob無條件讀敏感檔回傳，仍違反policy。enforcement point必須完整中介相關操作，且其政策／metadata不能由不可信程式改寫。

安全邊界因此包含CPU、記憶體、物件權限及可變共享狀態，不以GUI是否隱藏按鈕判定。已授權reference的撤回又是另一項契約，第11/14章分開分析。

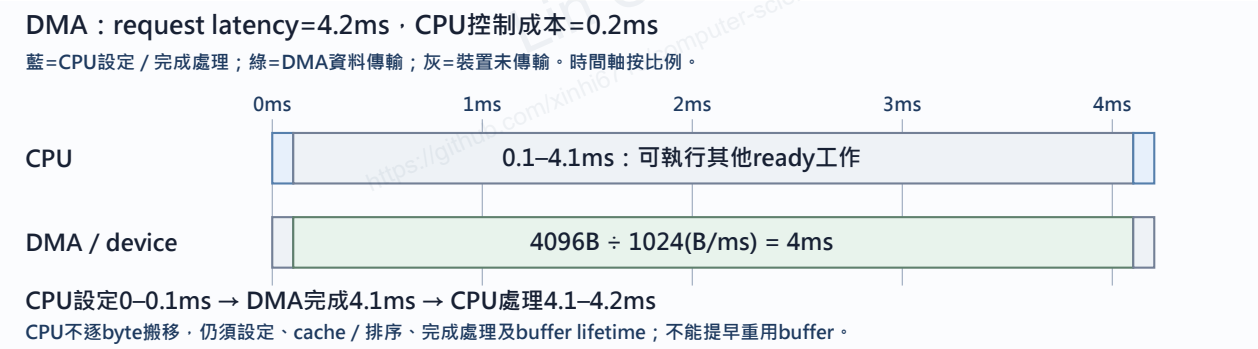
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

controller、driver與DMA各負責哪一段

controller是硬體／韌體端的裝置控制邏輯，driver是理解其register/queue/完成狀態的OS軟體。CPU提交命令、裝置搬資料、CPU處理完成是不同事件；完成可用interrupt通知或由polling讀狀態。

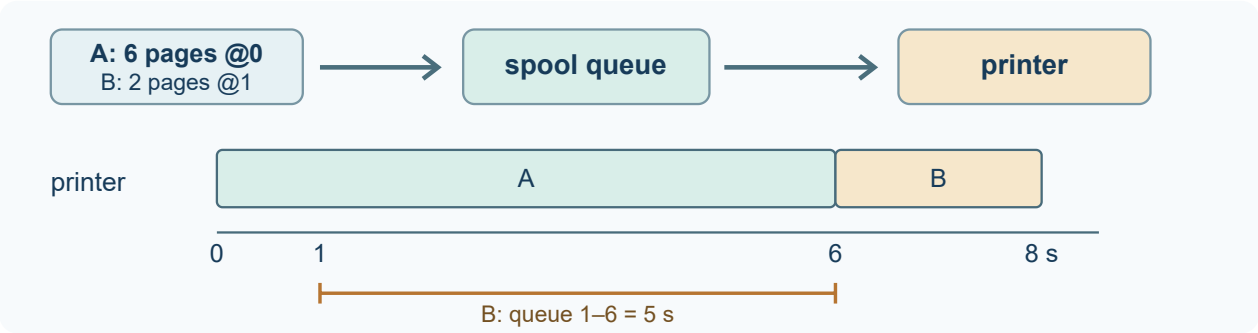
固定教學時間：driver設定0–0.1ms，DMA傳輸0.1–4.1ms，完成處理4.1–4.2ms；CPU在中間4ms可執行另一個就緒工作。DMA減少逐byte CPU搬移，不代表設定、cache同步、記憶體頻寬、完成處理都免費。

若block4096B、傳輸1024B/ms，本例transfer=4ms；CPU控制成本0.2ms，總request latency4.2ms。CPU只花0.2ms並不代表應用等待也只有0.2ms。資料必須在裝置／CPU所需時點可見，不能在DMA尚未完成就歸還buffer。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Spooling：排隊的是工作，完成的是裝置服務



假設印表機每秒印 1 頁、不可交錯工作、FCFS：A 的 6 頁在 t=0 收妥，B 的 2 頁在 t=1 收妥。印表機於 0–6 印 A，6–8 印 B；B 在 spool 佇列等待 5 秒，其完整輸出到 t=8 才完成。

應用程式「提交成功」只代表所約定的 spool 接收階段完成，不代表紙張已印出，也不自動保證斷電後仍保存。工作可存於磁碟、記憶體或其他儲存媒介；實際持久性依系統契約決定。

Spool 與 buffer 解決不同問題

機制	管理的對象	主要作用	不能直接推論
spool／job queue	完整工作及其順序、狀態	排程共享裝置，讓提交與服務解耦	一定持久、一定 FCFS 或無限容量
buffer	傳輸途中暫存的資料	吸收生產與消費速率差、協助分批傳輸	資料必在 RAM、工作已完成或已持久化

兩者可以一起使用：spool 保存列印工作，傳給驅動程式時再以 buffer 分批送資料。空間不足時依具體 API 可能阻塞、拒絕或回錯；若佇列允許優先權插隊，FCFS 時間線也不再適用。

interrupt、同步exception與system call分開分類

事件	相對執行指令	典型後續
裝置完成／timer interrupt	外部非同步	保存被打斷狀態、處理事件、可能排程
page fault／除零exception	由目前指令同步觸發	補頁後重啟、或交錯誤／終止
system call	程式主動同步請求	進受控入口、檢查參數、服務、回user

各ISA對trap/fault/interrupt的術語與返回點不同；不能把所有exception都說成下一指令繼續。裝置interrupt若發生在kernel mode，可能沒有user→kernel transition，但仍需保存／恢復當前execution state及處理重入同步。

interrupt處理完成後可以返回同一thread；system call也可能不阻塞。兩者都不必然發生process context switch。DMA也不必然每block一interrupt，可poll或合批；必須把實際completion規則寫清楚。

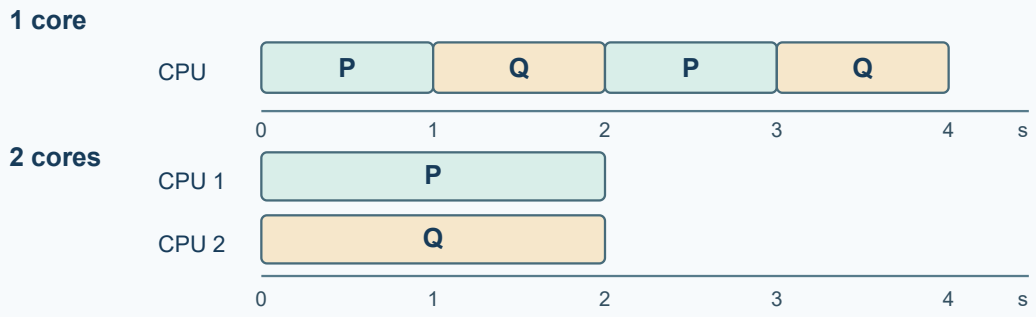
同步與失敗條件來自實際共享關係

架構	通常共享什麼	額外問題
single general-purpose CPU	一組一般指令執行資源	仍可能與DMA／interrupt並行
SMP	對等CPU、共同記憶體	共享資料同步、cache一致性
NUMA shared memory	共同addressable RAM、不同locality	位置影響latency／bandwidth
cluster	獨立節點經network合作	訊息、partial failure、重試／共識前提

core是執行核心，socket/package是封裝，hardware thread是架構暴露的執行context；它們不能不看機器就互換計數。SMP的對等控制與NUMA的成本不均是不同維度，不把所有shared-memory系統都說成UMA。

多CPU不能單獨保證容錯：共享電源、RAM、kernel或storage都可能是共同失效點。failover還需偵測、狀態恢復與防止兩個節點同時擁有排他資源。

並發與平行：看時間線，不只數 CPU



並發（concurrency）是多項工作的進度在同一段期間交錯；在單核心上，P 與 Q 可輪流執行，即使任一瞬間只有一個在該核心上跑。**平行（parallelism）**是多項工作在同一時刻使用不同執行資源，例如圖中的兩個 CPU 核心。

兩個 CPU 工作平行不代表結果正確：若同時修改共享資料，仍需同步。單核心的程式也可能與 DMA 裝置平行；因此應先說清楚比較的是 CPU 工作，還是 CPU 與裝置。

NUMA與平行加速的兩個可重算模型

假設一連串無重疊、無cache影響的memory requests，local=100ns、remote=180ns。80%local時平均0.8×100+0.2×180=116ns；移成20%local時為164ns。這是指定比例下的平均單次成本，不直接推得整個程式加速164/116。

基準工作時間100s，其中serial20s、可完美平行80s、無額外overhead：P=4時 T=20+80/4=40s，speedup=2.5。一般T(P)=100[s+(1-s)/P]，speedup≤1/s；還要加同步與資料移動成本。

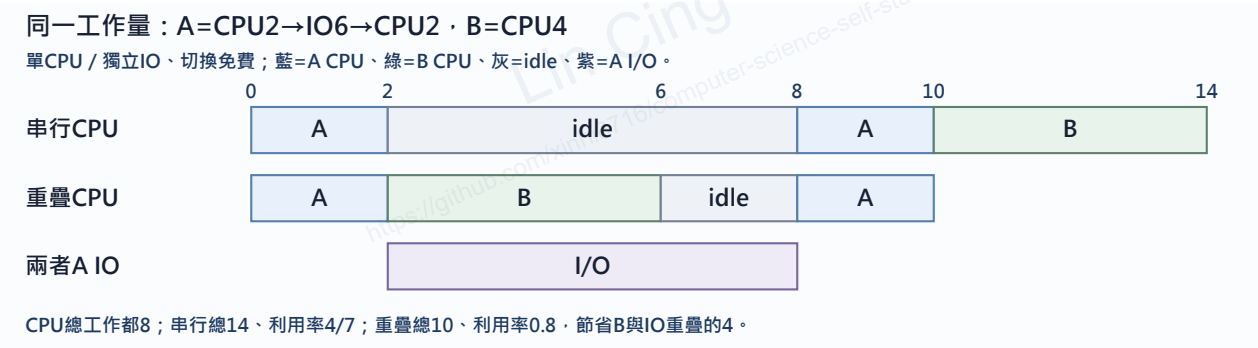
以operation count代替基準時間比例會錯；多CPU也不會自動找出可平行部分。成本比例與可同時做的工作都必須由模型或工作負載提供。

multiprogramming是把I/O等待與別人的CPU工作重疊

固定單CPU、一個I/O裝置、切換免費，A=CPU2→I/O6→CPU2，B=CPU4且無I/O；兩者t=0就緒。

方式	CPU時段	I/O時段	完成所有job	CPU利用率
A做完才跑B	A0–2，idle2–8，A8–10，B10–14	A2–8	14	8/14=4/7
A阻塞便跑B	A0–2，B2–6，idle6–8，A8–10	A2–8	10	8/10=0.8

CPU總工作仍8，不是把同一CPU同時跑A/B。節省的4正是B與A的I/O重疊；若沒有其他ready工作或I/O都等待同一瓶頸，不會憑空省時間。



time sharing再要求可回收控制權與可接受response

multiprogramming可在阻塞時換工作；若某程式CPU loop永不阻塞，單靠它自願讓出不能保證別人有進度。time sharing通常用timer及preemption，搭配ready queue給互動工作機會。

簡化模型有n個持續ready工作、每次最多跑q且每次切換成本c；固定RR輪轉，某工作剛錯過輪次至下次執行，等待上界為(n−1)q+n c（以包含切換成本的保守邊界計）。若n無界增加、kernel長時間不可搶占或更高priority可無限插入，此界不成立。

q小可降低輪轉等待但增加切換比例c/(q+c)。例q=4ms,c=0.1ms，純CPU輪轉的切換占0.1/4.1≈2.44%；這是固定一段q加一次c的模型，不是所有系統的實測開銷。

dual mode與受保護入口形成執行邊界

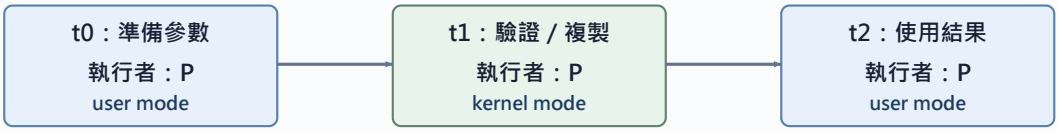
一般硬體以mode／privilege state區分允許的操作。user不能直接改timer、page-table root或中斷入口等受保護狀態。system call依ISA保存必要返回狀態並轉到受控kernel entry；kernel檢查參數／authority後才執行服務。

t	同一thread的執行	mode	必然換process？
0	user準備read參數	user	否
1	system call入口與驗證	kernel	否
2a	cache hit，複製並返回	kernel→user	否
2b	cache miss，等待I/O	kernel阻塞	可換到另一ready thread

mode switch只說特權狀態變化；context switch另保存／載入執行者。若換到同address space的另一thread，還不能因此推得address-space switch。

dual mode與受保護入口形成執行邊界 （續）

cache-hit system call：兩次mode transition，零次thread切換



藍=user，綠=kernel。受控入口與返回改變mode；P的身分與address space不變。

本例無其他事件，直接返回P；P阻塞或被搶占時可排程Q。三種切換分開計。

timer保證的是回到受控機制，公平性還需scheduler

OS在交CPU給不可信user前設定timer；倒數到期產生受硬體控制的interrupt。若user能自行關閉timer或修改counter，就能破壞回收CPU的機制，因此相關操作須受保護。

有限進度條件：timer能被服務、不可搶占kernel段落有上界、handler與scheduler會終止、ready規則不讓同一工作永久飢餓。缺其中一條，只有timer硬體不保證每個process有CPU。

中斷可能延後、合批或受優先權控制，不能把設定q說成每個指令邊界必在q精準切走。核心也會發生fault與惡意參數攻擊；mode只是enforcement基礎，不證明所有kernel行為正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

program是被動內容，process是帶狀態的執行

program bytes可存在file；process有address space、credentials、open references與至少一個thread等狀態。thread保存PC、registers、stack等執行context；同process多thread共享部分資源，卻不是共享同一PC。

事件	狀態轉移	要由誰完成
等待I/O	running→waiting	當前thread／kernel登記等待
I/O完成	waiting→ready	完成路徑使其可被排程
scheduler選取	ready→running	scheduler／dispatcher
quantum到期	running→ready	受控preemption

I/O完成不表示立即running，仍要等CPU。把waiting也算成ready-queue waiting會混淆第6章的時間指標。程序管理還包括建立／終止、同步／IPC及資源清理，不只選下一個PID。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何會有競爭：一個increment有多個步驟

共享counter=0，兩thread各做一次「load→+1→store」，不視為原子。

步	T1	T2	counter
1	load r1=0	—	0
2	—	load r2=0	0
3	store r1+1=1	—	1
4	—	store r2+1=1	1

預期2卻得到1。即使單CPU，若可在兩個步驟間切換也能發生；多CPU只是增加交錯方式。mutex或符合需要的atomic RMW能建立另一個不可分割規格，但volatile標記本身不等於互斥。完整memory ordering與同步條件在第5章展開。

address space把程式位址與實體frame分開

固定page size4096B，P的VPN1→PFN7、read/write；Q的VPN1→PFN9、read-only。相同VA=0x1234，VPN1、offset0x234=564；P對應PA0x7234，Q對應PA0x9234。

操作	P	Q
read VA0x1234	讀frame7+564	讀frame9+564
write VA0x1234	權限允許	permission fault
改自己的page table為frame7	一般user不能直接改受保護表	同樣受限制

映射存在、駐留與授權是不同檢查：not-present可能可由OS補頁；禁止寫入不一定是補頁就解決。shared memory則是OS明確讓多個映射指同一frame，並另定權限，不是VA相同自動共享。

記憶體管理要維持的集合關係

在任一時點，已配置physical frames與可回收free frames不得同時宣稱獨占同一frame；有合法共享時，references與權限須一致。回收frame前還要確定舊映射不再可被使用，包括TLB與未完成I/O等持有者。

例RAM8 frames，kernel2、P3、Q2，free1。若Q釋放1且無其他reference，free變2；若該frame仍被P共享，就不能直接加進free。OS追蹤「誰仍能到達」比只數process宣稱free更重要。

virtual memory允許非所有virtual pages同時駐留，但不是無限RAM；working set、storage I/O與commit限制會影響效能／是否能履行配置。第8/9章給頁表與replacement的一般機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

cache層次：latency模型要說明miss penalty的意義

固定每次先查L1花1cycle，hit率0.9，miss額外花50cycles，無重疊／prefetch：平均=1+0.1×50=6cycles。等價列法0.9×1+0.1×51=6。

若「50」指miss的總latency而非額外成本，則應是0.9×1+0.1×50=5.9；兩個輸入契約不同，不能只套同字母公式。平均也不代表每次都是6cycles，更不給tail latency。

cache將常用資料放快層，但引入副本一致性與失效問題。CPU cache coherence處理共享位址的快取副本，不等於filesystem durability；資料可能仍只在volatile層。write完成、flush完成與斷電後存在需分別界定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

file、block allocation與儲存故障的不同責任

應用以file名稱／offset操作；filesystem把範圍映射到blocks，block layer／driver再送裝置。allocation要防同block被兩個獨占owner拿走，free-space索引必須與物件指標一致；memory lock只解同時競爭，不解crash途中只持久一半的更新。

例裝置2^20個可配置block，以1bit/block bitmap記free狀態，需2^20bits=2^17B=128KiB。這是logical bitmap容量，不是一次query只需1bit實體I/O；未cache時仍需載入所在儲存block。

metadata毀損、裝置失效、誤刪與惡意存取需要不同機制。RAID可針對某些硬體失效，不能單獨防止誤刪；backup與protection也不能互相代替。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

protection與security先分威脅及強制位置

protection研究已知主體如何被限制使用物件；security還涉及身分驗證、入侵、惡意程式與系統威脅。兩者互相依賴，不是把一個稱「內部」、另一個永遠稱「外部」就能涵蓋所有攻擊。

問題	所需資訊／機制	單靠它仍不足
誰在請求？	authentication、可信credentials	知道身分不等於授權
可操作哪個物件？	ACL／capability、access check	UI顯示不會強制API
通訊內容被竊聽？	符合威脅模型的加密與key管理	不能修好已被攻陷端點
同址映射越權？	MMU權限與受保護頁表	kernel漏洞仍可能繞過policy

需要把資料機密性、完整性、可用性分別定義；拒絕所有請求雖可阻止某些洩漏，也會完全破壞可用性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

least privilege及授權時點

例影像轉檔程式只應讀輸入file與寫指定輸出，不應取得整個home directory的可寫authority。以已授權物件reference授予所需rights，比把全域路徑／root身分交給它更容易限制損害。

但「一般user」不是零權限，繼承的file descriptors、共享memory、network sockets都可能帶authority。反過來，撤掉某名稱的權限不必然讓所有既有handle失效；必須看建立reference時與後續操作時的契約。

一個完整判定需列主體credentials、物件身分、requested rights、政策與檢查時點。只看檔名、user顯示名稱或程式是否有特權圖示，都不足以推論實際權限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料結構必須同時符合查詢、更新與生命期

結構	適合操作與成本	必要前提／限制
linked list	已知節點／前驅後插刪O(1)	找key最壞O(n)，pointer與cache成本
stack／FIFO queue	LIFO／FIFO；正確端點可O(1)	FIFO不是priority order
balanced search tree	ordered lookup／range、O(log n)	維持特定平衡不變量
hash table	指定hash模型下平均lookup O(1)	collision可使最壞O(n)，擴容有成本
bitmap	有限整數域membership	空間隨域大小，需原子claim

BST真正條件是每個node「全部left-subtree keys < node.key < 全部right-subtree keys」（本例相異鍵）；只檢查left-child≤right-child不夠。平衡樹高度是O(log n)，不能把任意樹精確稱只有log2 n層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一組key的collision與錯誤BST反例

hash h(k)=k mod4，插5、9、13都進bucket1；chaining依序存[5,9,13]，找13要3次key比對，找17也要3次才判不存在。把n個同餘keys放一起，最壞線性；「hash O(1)」須另有分布／負載或更強設計保證。

BST例 root10、left5、right20，卻把12放在5的右子。12>5滿足局部parent比較，但12在10的left subtree，不滿足12<10；按BST搜尋12會往right20再找不到。必須驗整個subtree範圍。

並行讀者可能仍持old node reference。更新者即使已從list/tree移除節點，也不能立刻free給別人重用。鎖、reference counting或RCU式grace period提供不同生命期契約；允許讀舊值不等於允許use-after-free。

運算環境改變的是資源與失敗模型

環境	主要邊界	不能直接推論
client/server	client發請求、server提供服務	server一定不會也是別人的client
peer-to-peer	節點可同時請求／提供	完全無中央發現或驗證服務
cluster/distributed	獨立節點與network	timeout就證明remote已失效
virtual machine	guest OS經VMM使用虛擬硬體	guest權限等於host權限
container式環境	一般共享host kernel的隔離視圖	每個container都有獨立kernel
embedded／real-time	特定設備／時間約束	所有embedded都必須hard real-time

雲端的IaaS/PaaS/SaaS是提供與管理責任的層級，不是三種CPU架構；虛擬化可作基礎，但不是每個雲服務都等同一部VM。

real-time的正確性包含deadline，而非只看平均快

固定單CPU、無阻塞／overhead，週期工作A(C=1,T=D=4)、B(C=2,T=D=6)。CPU demand率1/4+2/6=7/12；只表示負載小於1，沒有指定scheduler仍不能直接保證每個deadline。

一個可行的12單位EDF排程：A0–1、B1–3、idle3–4、A4–5、idle5–6、B6–8、A8–9、idle9–12；A在4/8/12前完成，B在6/12前完成。12後同步重複，因period的lcm=12且邊界無未完成工作。

若加入非搶占critical section、release jitter或I/O等待，這個schedule證明的前提已變，需重新算response／blocking。低平均latency不能替代最壞deadline檢查；hard/soft real-time區別在違約後果及允許程度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

open source是授權與來源可用性，不是品質保證

依Open Source Definition，僅能看到source還不夠；授權還須允許符合定義的散布與衍生等權利。source available、freeware、public domain、open source不是同義詞，商業收費也不自動與open source衝突。OSI定義。

以Linux kernel搭配工具／library／服務／套件形成distribution，可區分kernel與整個可使用環境；BSD也是另一種系統家族與授權傳統。不能因某component開放就推論整個distribution每一檔同授權。

source能讓人核讀實作與重建，但需固定版本、設定與相依元件才能比較行為。原書的Linux3.2、Windows7等案例保留歷史契約；當代文件只在明確標示的補充介面使用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從介面到原始碼：如何判斷一項系統保證

要問的事	應查的證據	仍不能用什麼取代
API允許什麼結果？	該版本的介面／標準契約	只看一次成功執行
特定版本怎麼做到？	對應source、build configuration與架構	只看其他版本同名function
觀察的latency是什麼？	同工作負載的時間與資源分解	把operations比例當時間比例
更新是否持久？	儲存／flush契約及故障次序	只見write回傳成功

開源使source可查，並不自動證明沒有race、最佳效能或安全。對照算法與具體資料時仍要保留前提、錯誤結果及一般不變量；只用一個代表性範例無法推出所有輸入都成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個read請求如何連起全課程

P用已授權fd讀取file的4096B：先從user進受控kernel入口；查descriptor/OFD與buffer；cache miss時建立I/O、P等待；scheduler選Q；DMA完成後把P設ready；P再取得CPU、資料可見後複製／回傳實際bytes。

環節	要追的狀態	後續章節
呼叫與保護	mode、credentials、requested right	2、14、15
process／thread	ready/waiting/running、saved registers	3–7
映射與資料	VA/PTE/frame、cache、buffer lifetime	8、9、13
名稱與儲存	OFD offset、inode、block、完成／durable	10–12

這些state不在同一瞬間全部變成「完成」。尤其waiting→ready不等於立即running，request完成不等於其他file已flush。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最容易混淆的四組量

- mode transition／thread context switch／address-space switch分開計；同thread的cache-hit system call可只有前者。
- CPU busy時間、request latency、throughput、deadline各有分母；CPU省工不必然讓單次latency下降。
- 名稱、物件身分、已開reference分開；修改路徑不必改變既有fd。
- 平均成本、最壞成本、一般正確性分開；hash碰撞、timer延後與shared-memory競爭都有合法反例。

先固定版本與工作負載，再檢查可觀察狀態。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 1. Open Source Initiative: Open Source Definition.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：作業系統應如何分層與暴露介面，讓功能可維護，又不犧牲隔離與效能？

前置知識。 mode bit、中斷與 trap、呼叫堆疊。

學習目標

- 追蹤 system call 從使用者函式庫到核心服務的路徑
- 比較 monolithic、layered、microkernel、module 與 hybrid 結構
- 解釋除錯、產生系統映像與開機所需資料

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

服務、提供者與完成契約

作業系統服務不等於一張syscall名稱表。User interface、program execution、I/O、file manipulation、communication與error detection支援使用者工作；resource allocation、accounting、protection/security支援多個工作共存。服務可由kernel、library、daemon與應用工具合作完成，保護邊界由實際權限與物件存取決定。

複製檔案所需服務	固定實作中的提供者	能觀察的完成條件
解讀來源/目的名稱	shell與copy工具	形成兩個pathname；不是kernel解析shell語法
找檔案、檢查權限	VFS/filesystem	open成功取得descriptor或明確錯誤
傳輸資料	library API→kernel I/O	每次回傳已完成bytes；不能假定等於請求
配置/保護	kernel與filesystem	fd、cache、storage容量及存取權都合法
儲存結果	filesystem/device	檔案同步與directory entry持久性分開

User program和system program通常受相同的保護規則；「系統工具」這個名字不會自動授與kernel privilege。反過來，具管理權限的user-space daemon仍在user mode，不能把權限身分與CPU mode混為一談。

6000-byte複製：每次read後都必須寫完

固定來源是6000 bytes且不再改動，目的為新建空regular file；blocking I/O，無其他writer，buffer4096 bytes。成功路徑刻意包含short write；所有返回值皆實際參與進度計算。

事件	本次回傳bytes	來源offset	目的offset/長度	buffer尚待寫
open來源與建立目的成功	—	0	0	0
read(4096)	4096	4096	0	4096
write(4096)	1000	4096	1000	3096
write剩餘buffer[1000:]	3096	4096	4096	0
read(4096)	1904	6000	4096	1904
write(1904)	1024	6000	5120	880
write剩餘buffer[1024:]	880	6000	6000	0
read(4096)	0，EOF	6000	6000	0

不含directory額外同步，這段路徑共2次open、3次read、4次write、1次fsync與2次close，即12次API操作。表的核心不是固定次數，而是成功的short write會改變後續buffer位置；只在每次read之後呼叫一次write會漏掉3976 bytes。

複製迴圈的一般不變量與失敗界線

設已讀bytes為R、已寫bytes為W、目前buffer待寫區間為[j,n)。迴圈維持W≤R，且buffer[j:n]等於來源尚未寫入目的的連續片段。每次正回傳k≤n−j後，W增加k、j增加k，序列不變；只有j=n才能覆寫buffer並讀下一段。讀到EOF且沒有待寫bytes才可推出W=R=6000。

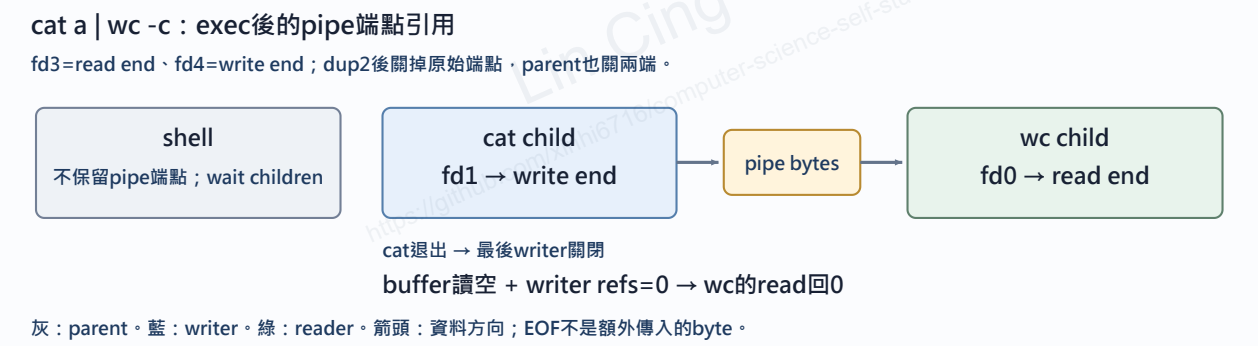
若read/write在零進度前被signal中斷，依介面處理EINTR並重試；若已返回正bytes，先前進進度，不能整段重寫。未預期的write=0必須避免無限迴圈並回報無進度。ENOSPC或EIO發生時只能聲稱已有部分輸出，不能回成功。

成功write不等於crash後持久。固定本機filesystem/device遵守flush契約時，成功fsync目的可確認檔案資料與必要metadata；新建pathname的directory entry另需同步所在directory。這仍不是「整個copy具atomic replace」：失敗時目的可能部分可見。若需求是替換舊版本，需另設同directory暫存檔、資料同步、rename與directory同步的完整協議，而不是僅加一次close。

Shell、GUI與kernel各解讀哪一層

Command interpreter將文字或script轉成程序與I/O連接；GUI將滑鼠/觸控事件轉成應用操作。兩者可以使用相同kernel服務。`cd`若只在child中執行，不會改變parent shell的cwd，所以常是shell builtin；`cat`等外部工具則可由child exec。此差異來自process資源語意，不是「內建命令必然在kernel」。

固定POSIX-style命令`cat a | wc -c`，不含alias、job control或額外子shell。Shell解析兩個command與一個pipe，先建立pipe，再fork兩個child；cat把stdout連到write end，wc把stdin連到read end。原shell仍保留自身program image並wait兩個children；兩次exec替換child的image而非另生兩個process。



端點引用決定EOF：不能只畫一條管線

假設pipe()回fd3=read end、fd4=write end，fork後三者暫時都持有這兩個descriptor。cat執行dup2(4,1)，wc執行dup2(3,0)，然後各自關掉不再需要的原始fd3/fd4；shell也關掉兩端。

process	exec後留下的pipe引用	關閉動作	為何必要
shell	無	close(3)、close(4)	避免parent自己讓writer一直存在
cat	fd1→write end	關fd3與原fd4	reader消失時不被自己的read引用掩蓋
wc	fd0→read end	關原fd3與fd4	避免wc自己持有write end而永遠等不到EOF

cat把a的6000 bytes寫入pipe並結束；wc可能用任意分段read，累加總數6000。只有pipe資料已讀空且所有write-end引用都關閉，下一次read才回0。若shell忘關fd4，cat即使已退出，wc也可能阻塞，shell又在wait wc，形成不必要的僵持。不能用「cat結束」代替引用條件。

輸出、exit status與權限不由介面外觀決定

cat失敗不保證pipeline最右端wc也失敗。例如cat開a得到ENOENT，未輸入任何bytes便退出非零；wc讀到EOF仍可能輸出0並exit0。Shell必須分別取得兩個wait status；是否把pipeline整體判為失敗取決於其明定規則。這與資料正確性是兩個不同觀測。

GUI的「複製成功」也不能只憑progress bar走完：應等待相應I/O/error回報與宣稱的同步契約。把GUI換成CLI不會改變目的檔案的權限檢查、容量或crash一致性。

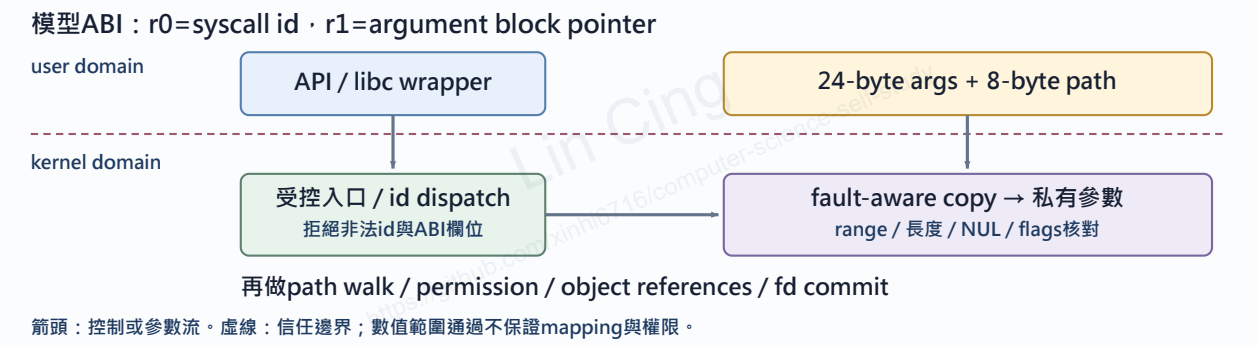
固定字串參數含空白時，parser必須區分token邊界與參數內容。例如`cat 'a b'`傳入一個pathname；`cat a b`傳兩個。Kernel只收到exec參數陣列，並不重新套用shell的引號/pipe語法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

API、ABI與system call是三個不同契約

API規定函式名稱、參數型別與可觀察行為；ABI規定machine-level calling convention、syscall number、register/stack layout與返回方式；kernel implementation處理受保護物件。Source能以同一API重新編譯，不表示原binary可在不同ISA/ABI直接執行。

`strlen()`通常可完全在user space；buffered `printf()`可能先留在library buffer，之後一次write輸出多次printf的內容；一個copy API也可組合多個open/read/write。故API呼叫次數不是trap次數。若libc將kernel錯誤碼轉成−1/errno，也應分開描述兩層返回格式。



固定教學ABI：syscall id在r0，r1指向24-byte argument block，三個64-bit欄位依序是path pointer、flags、mode。這是明定的模型ABI，**不是聲稱Linux所有open都用24-byte memory block**；真實平台可用registers傳遞，歷史實作也會變動。

參數複製、指標驗證與TOCTOU

模型pathname為`/data/x`加NUL，共8 bytes。核心先以受保護user-copy取24-byte固定區塊，再按有界string規則取至多PATH_LIMIT bytes；此例成功取8bytes，總copy32。只有複製到kernel-owned storage後，才對同一份內容解析與使用，避免之後重新讀取user陣列而得到另一條path。

步驟	核對內容	失敗結果
解碼id/ABI	id存在，欄位格式/flags合法	拒絕，不能跳到任意handler地址
複製argument block	user range與實際fault-safe copy	EFAULT或相應模型錯誤，不使用半份參數
複製pathname	有NUL且不超長；來源可存取	EFAULT或ENAMETOOLONG，無fd公布
path walk	每個directory具search權；最後物件權限符合flags	EACCES/ENOENT等
配置與公布fd	穩定引用、回滾未完成配置	成功fd；失敗不洩漏已分配引用

先查VMA「看似可讀」不是充分條件：另一thread可unmap、page可fault。真正copy操作必須可處理fault，檢查是否完整複製，再只驗證與使用同一份kernel私有副本。私有副本在複製完成後不再被user修改，**不表示複製期間讀到的是某個瞬間的原子snapshot**。

參數複製、指標驗證與TOCTOU (續)

例如user的兩欄位原為(length=4, capacity=4)。Kernel先讀length=4，user依序改length=8、capacity=8，kernel再讀capacity=8，便取得(4,8)；user曾有(4,4)、(8,4)、(8,8)，卻從未有(4,8)。因此必須把私有副本視為任意不可信輸入，重新驗證欄位間關係、長度及權限；不可宣稱一次copy保證來源多欄位一致。若API真正要求來源一致snapshot，需另外提供可強制的同步／版本協議，且不能信任惡意caller會遵守user-space鎖。Filesystem namespace在path walk中的穩定性則另需kernel物件引用與同步。

範圍與乘法溢位：先證明算式才取記憶體

固定32-bit無號machine word、user地址區間[0,U)，U=0xC0000000；要讀長度n的非空範圍。採`p < U && n <= U-p`，避免先算p+n而溢位。算式通過只表示數值範圍合法，仍須驗證實際mapping/permission並以安全copy處理fault。

p	n	exclusive end（數學整數）	範圍判斷
0xBFFFFF00	0x100	0xC0000000	合法，最後byte是0xBFFFFFFF
0xBFFFFF00	0x101	0xC0000001	非法，跨user邊界
0xFFFFFFF0	0x40	0x100000030	非法；32-bit加法會偽裝成0x30

若count個element每個4bytes，先驗證count≤SIZE_MAX/4；count=0x40000001會使數學乘積0x100000004，32-bit卻變4。只配置4bytes卻循環寫count個element就是越界來源。零長度操作是否允許任意pointer屬個別API契約，本頁判斷刻意限定非空範圍，不能任意套到所有syscalls。

System call分類應接到物件與狀態

類型	代表請求	受保護物件/狀態	成功不代表什麼
Process control	fork、exec、wait	task、image、exit status	exec不是額外建立process
File management	open/read/write/lseek/close	fd、description、inode	write成功不是持久承諾
Device management	read/write、control request	driver queue、device state	送入queue不是裝置完成
Information maintenance	get/set attributes、time	credentials與system metadata	知道物件ID不是持有權限
Communication	pipe、send/receive、shared memory	endpoint、message、mapping	byte-stream read不是一則訊息
Protection	permissions、credential操作	access policy與principal	authentication不是每次authorization

分類允許重疊：socket read同時具有I/O、通訊和保護面向；不能只因函式被放在「file」欄便假定所有fd都是regular file。相同read介面可對檔案、pipe或terminal有不同blocking/EOF/seek語意。

回傳值是狀態機的輸出，不是裝飾

固定regular-file read，count=4096且無並行修改。從offset0開始：read回3000→offset3000；下一次回1096→offset4096；再回0表示該時刻EOF。第一次short read不等於永久EOF；對pipe更可能只是目前可用bytes少於請求。

再看signal操作：模型kill(pid=100,sig=9)的兩個32-bit整數共8bytes，表示方式不等於實際copy成本。核心依pid namespace找對象並查caller權限及signal有效性；目標不存在回ESRCH，無權回EPERM。成功表示signal送達/排入相應處理機制，不表示caller已觀察到目標完全退出與所有資源回收；需要wait等生命週期同步。

若要求多步atomic交易，必須查哪些步驟共用commit point；兩次各自成功的syscall不自動變成一個atomic操作。保護、blocking、visibility、durability與termination是不同保證。

從source到running process：system programs做了哪些事

固定C程式hello.c，只需一個main與libc輸出。工具鏈路徑為preprocess→compile→assemble→link→可執行image；動態連結時，exec還需要interpreter/loader與相容libraries。Compiler與linker是user programs；它們為讀寫檔案、配置記憶體、建立子程序使用kernel服務，並不因生成機器碼而在kernel mode執行。

階段	主要輸入/輸出	可獨立發生的失敗
Preprocessor	source/headers→展開後tokens	header缺漏、macro條件錯誤
Compiler/assembler	tokens→assembly→object	syntax/type error、target ISA不合
Linker	objects/libraries→image	unresolved symbol、ABI不合
Exec/loader	image/interpreter→process image	權限/格式/loader缺漏
執行	entrypoint→API/syscalls	runtime fault、I/O錯誤

檔案管理工具、status工具、programming support、program loading、communication與background service屬不同用途；是否長期daemon化，不改變其仍受CPU模式及核心介面保護的事實。

Shell串接的錯誤傳播不能被分號吃掉

固定三個工具parser、compiler、linker，成功exit0、失敗非零。`parser && compiler && linker`只在前項成功才做下一項；`parser ; compiler ; linker`無條件繼續，最後狀態可能只反映linker。為避免舊object被誤用，build系統還需明定輸出版本與依賴，不可僅判斷檔案存在。

測試	parser	compiler	linker	合理結論
A：全部成功	0	0	0	image產生，仍未證明能run
B：parser語法失敗，&&	2	未執行	未執行	build失敗，不發布舊image
C：compiler失敗，;	0	1	0（吃到舊object）	最後0掩蓋前項失敗

Source-level API相容、binary ABI相容與runtime dependencies齊全都要分別成立。只看到link成功或某個digest一致，不能推論整個target環境可開機或應用無錯。

Mechanism與policy：用同一個timer做兩種選擇

Mechanism提供可安全執行的操作，policy選擇何時/對誰/多少。Timer interrupt、保存/還原執行狀態、ready queue操作是排程mechanisms；quantum長度與挑選順序是policy。分離不表示沒有相互限制：timer精度、nonpreemptible區段與切換成本會限制policy能達成的延遲。

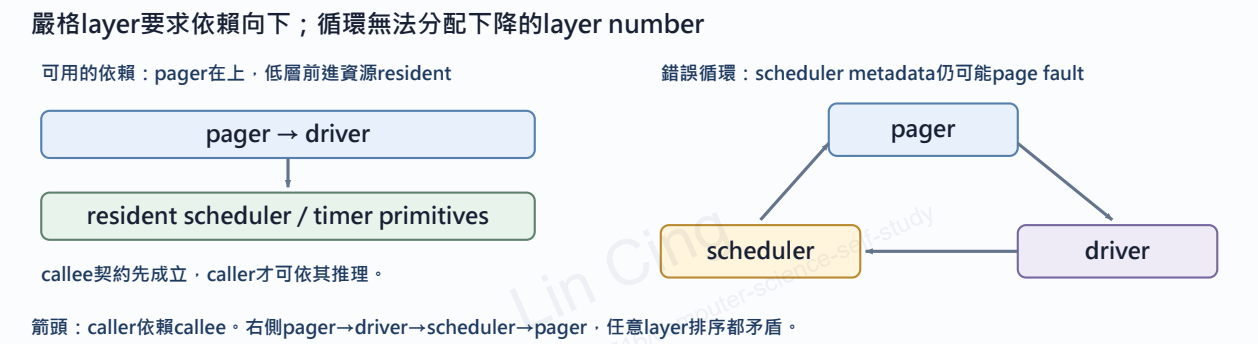
固定單CPU，A/B/C皆在t=0就緒，CPU bursts5/3/1ms，round-robin初序A,B,C，無I/O、切換成本0。相同mechanism，僅改quantum。

q	完整CPU區間	completion(A,B,C)	first response(A,B,C)
1ms	A0–1,B1–2,C2–3,A3–4,B4–5,A5–6,B6–7,A7–8,A8–9	9,7,3	0,1,2
4ms	A0–4,B4–7,C7–8,A8–9	9,7,8	0,4,7

兩者總useful CPU time都9ms；q1平均response=1ms，q4為11/3ms。q小提高反應性，不代表實機無代價：若每次真正task switch有成本s，q1有7次task switches、q4有3次；q1最後A→A不算task switch。固定成本會改變wall time，必須重算，不能把上表零成本時間直接搬用。

分層證明有前提：循環依賴破壞歸納

嚴格layered設計要求每個服務僅依賴更低層已給定的契約。若第0層滿足C0，第i層在C0…C(i−1)成立下可證Ci，則可由歸納建立上層性質。實際測試通過不是無條件「下層絕不可能出錯」；契約、前提或測試覆蓋錯誤仍會使推論失效。



例：VM pager依賴disk driver，disk driver等待時依賴scheduler；若scheduler的必要metadata又可能page fault而依賴pager，就形成pager→driver→scheduler→pager。三者無法同時滿足嚴格下降的layer number。可把scheduler必要資料與page-fault/I/O前進所需核心資源保留resident，拆出不依賴paging的底層原語；不是把循環箭頭藏起來就完成設計。

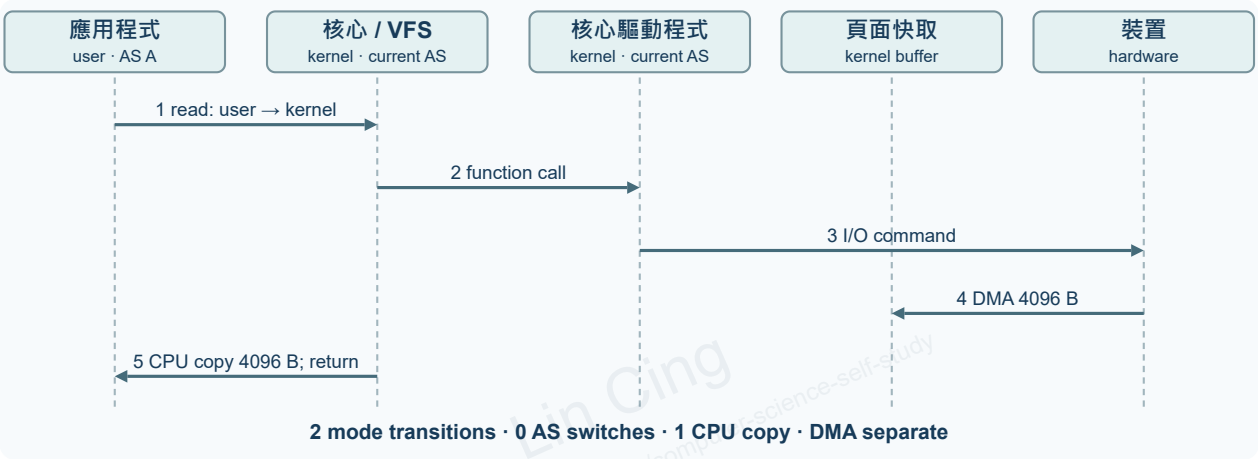
Implementation語言與可測量的設計目標

高階語言可改善可讀性、移植與compiler最佳化，但不能自動保證memory safety、atomicity或hardware ordering；C的pointer錯誤仍可破壞kernel。Assembly常留於入口、context save、特權指令等machine-specific邊界；應用同一library API不會自動消除這些ABI差異。

固定queue容量1024、相同預先產生的操作序列與machine資源。候選A無lost element；候選B雖較快卻在一個已記錄交錯把已enqueue元素遺失，B違反正確性契約而不可取。這是邏輯排除，不必等平均throughput超過門檻才決定。

效能目標須附workload與分母：單一request的CPU time、端到端latency與每秒完成數不同；p99以sorted observations的明定quantile計算。零切換成本的排程表只回答所列模型；實機比較還需量測切換、cache與等待成本，不能把不同分母的時間直接相除。

單體核心：讀取路徑泳道



固定 4096 B cold read；箭頭依事件順序由上往下。

固定檔案讀取：monolithic 與 microkernel 的完整路徑

比較設定固定為 4 KiB cold-cache read：`read(fd, user_buf, 4096)`，檔案為 `/srv/data.bin`，offset 0。ABI 明定 `fd`、`user_buf` 與 `count` 由 registers 傳遞，不把參數另行計算成 `copy_from_user`。monolithic 模型把 VFS、file system 與 block driver 放在 kernel；microkernel 模型把 file server 和 driver 放在兩個獨立 user address spaces。microkernel 的 kernel mapping 位於當前 user address space；IPC 的 sender→kernel→receiver two-copy strategy 與 user-address-space handoff 分開計數。

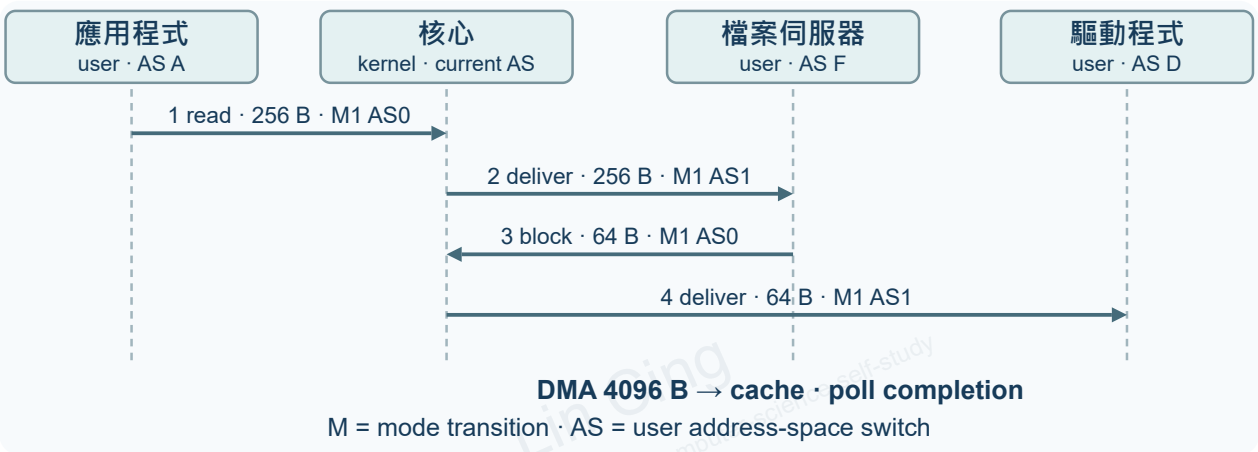
monolithic 的 function call 路徑如下。應用程式先以 libc 發出 read；CPU 從 user mode 進入 kernel mode。核心檢查 descriptor、access rights、user buffer 與 current offset，再依序呼叫 VFS、file system 與 block driver。driver 用 DMA 把 disk page 放入 page cache，核心再把 4096 bytes 複製到 user buffer，最後返回 user mode。

固定檔案讀取：monolithic 與 microkernel 的完整路徑 （續）

事件	執行位置	邊界或呼叫	成本計數
`libc read`	App user space	ABI 呼叫	0
`sys_read`	Kernel	register arguments；user→kernel mode	mode +1
`vfs_read`、file-system read、driver read	Kernel	同 address space function calls	address-space +0
disk→page cache	Driver／DMA	4096-byte DMA	CPU copy +0
page cache→user buffer	Kernel	4096-byte copy	CPU copy +1
return 4096	App user space	kernel→user mode	mode +1

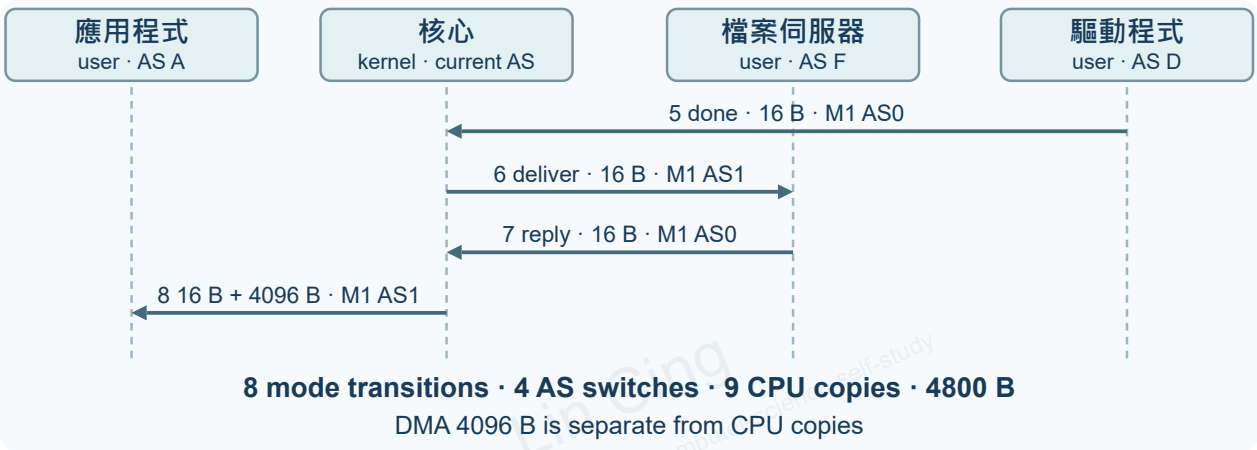
register ABI 不再增加參數 copy。因此總數是 **2 次 mode transitions、0 次 address-space switches、1 次 4096-byte CPU copy 與 1 次 4096-byte DMA**。改成 cache hit 後，driver 與 DMA 不出現，但 cache→user 的 copy 仍在。

微核心：請求與裝置工作



固定雙重複製 IPC；DMA buffer 與裝置授權預先設定。

微核心：完成與回覆



固定雙重複製 IPC；DMA buffer 與裝置授權預先設定。

microkernel：IPC、server、driver 與計數

microkernel 模型中，application、file server 與 driver 是三個 user-mode processes。Kernel mapping 隨當前 process 的 address space 生效，所以 `App→Kernel` 或 `FS→Kernel` 只做 user/kernel 的 mode transition，不另換 address space；`Kernel→FS`、`Kernel→Driver` 等交接到另一 user process 才算 AS switch。

假設核心預先配置並固定 DMA buffer，並授權驅動程式使用裝置與映射；設定／回收成本不列入本次交接計數。若有 IOMMU，也要依其映射規則限制裝置存取。

步驟	特權邊界事件	ABI／IPC	控制訊息 bytes	mode transitions	address-space switches	CPU copies
1	App→Kernel	register args；`SYS_IPC_SEND(read_req)`	256 的 sender→kernel copy	1	0	1
2	Kernel→File server	deliver read request	256 的 kernel→receiver copy	1	1	1
3	File server→Kernel	`SYS_IPC_SEND(block_req)`	64 的 sender→kernel copy	1	0	1

microkernel：IPC、server、driver 與計數 （續）

步驟	特權邊界事件	ABI／IPC	控制訊息 bytes	mode transitions	address-space switches	CPU copies
4	Kernel→Driver	deliver block request	64 的 kernel→receiver copy	1	1	1
5	Driver→Kernel	completion IPC send	16 的 sender→kernel copy	1	0	1
6	Kernel→File server	deliver completion	16 的 kernel→receiver copy	1	1	1
7	File server→Kernel	reply IPC send	16 的 sender→kernel copy	1	0	1
8	Kernel→App	deliver reply；copy page cache	16 的 kernel→receiver copy＋4096 資料	1	1	2

microkernel：IPC、server、driver 與計數 （續）

總數是 **8 次 mode transitions、4 次 user-address-space switches、9 次 CPU copies**。四則 control messages 的大小為 256、64、16、16 bytes；two-copy 後是八次 copy、704 bytes。最後再加 4096-byte 資料 copy，所以總共 copied bytes=4800。DMA 另有 1 次 4096 bytes，不混入 CPU copies。AS switches 對應 App→FS、FS→Driver、Driver→FS、FS→App，皆由 kernel handoff 到不同 user address space。

monolithic 的 function call 不會自動變成 zero-copy；microkernel 的 IPC 也不必然每次執行 eight transitions。本頁比較的是一個完整、可重算的成對模型。若 server 與 driver 合併為同一 process、kernel 使用獨立 private address space，或改用 shared-memory/one-copy IPC，就必須重新列 ABI、AS 配置與計數，不能沿用這組常數。

DMA 邊界與 IPC 成本的敏感性

前兩頁的 2/0 與 8/4 只涵蓋 request chain 內明列的服務交接。DMA 完成採 **polling**；模型中沒有其他 runnable work，也不發生 timer preemption 或 completion interrupt 喚醒。若改採 interrupt、阻塞等待或讓其他 process 在等待期間執行，就要另列 interrupt entry/exit、wakeup 與 scheduler switch；它們不包含在這組基準數字內。

變更	mode transitions	address-space switches	CPU copies	copied bytes
基準：monolithic，current-AS kernel mapping	2	0	1	4096
基準：microkernel，256/64/16/16 messages	8	4	9	4800
microkernel 的 request message 由 256 改為 128 bytes	8	4	9	4544
file server 與 driver 合併為同一 process	4	2	5	4640
private-kernel-AS 模型，monolithic	2	2	1	4096
private-kernel-AS 模型，microkernel	8	8	9	4800

DMA 邊界與 IPC 成本的敏感性 （續）

128-byte request 的 two-copy 控制資料是 `2 × 128 + 2 × 64 + 4 × 16 = 448 bytes`，再加 4096-byte 資料，得到 4544 bytes。合併 server 與 driver 後只剩 request 256 bytes 與 reply 16 bytes 兩則 IPC；two-copy 為 544 bytes，加資料後是 4640 bytes。這些列改變的是已標示的模型參數，不代表任何真實 OS 的固定 syscall 成本。

private-kernel-AS 模型也必須套用到 monolithic：每次 user/kernel mode transition 都伴隨 address-space switch，所以 monolithic 由 0 次 AS switch 變成 2 次，microkernel 由 4 次變成 8 次。頁面只比較模型，不把 polling、zero-copy 或 fast-path 條件外推為所有 read 的一般成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

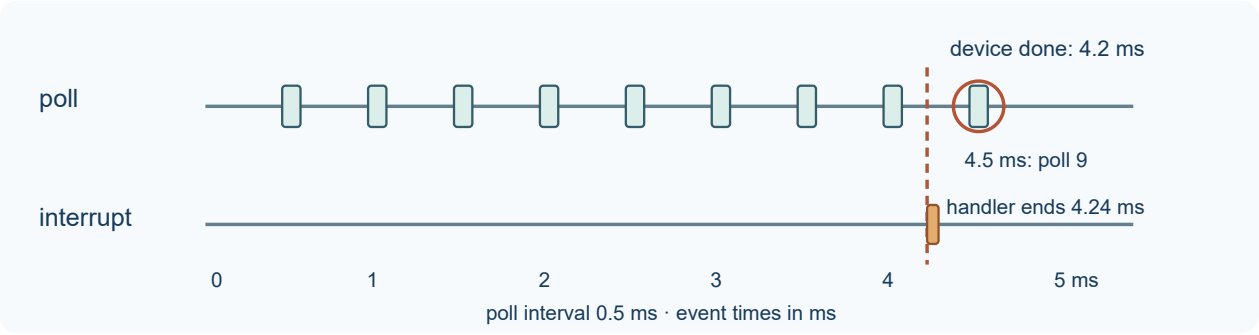
I/O 完成如何被發現？先分清三件事

Polling（輪詢）由 CPU 讀取裝置狀態；interrupt（中斷）由裝置通知 CPU 進入處理程序；DMA 則是裝置與記憶體間的資料搬移機制。DMA 的完成可以靠輪詢或中斷發現，不能把 DMA 當作第三種互斥的通知方式。

作法	等待期間 CPU 做什麼	偵測與成本
busy polling	持續查詢，該核心無法做其他工作	可能快速發現完成，但消耗 CPU
定時 polling	每隔固定時間查一次，其餘時間可排其他工作	可能延遲到下次查詢；成本含每次查詢與計時
interrupt	裝置完成後發通知，CPU 執行 handler	不必持續查詢；仍需處理中斷與同步

實際裝置可能合併通知、使用混合策略，或在高負載改為 polling。結果取決於工作負載、硬體和排程，不存在一種永遠較快的方式。

固定時間線：輪詢與中斷的比較



假設裝置在 **4.2 ms** 完成，定時輪詢在 0.5、1.0、…、4.5 ms 開始，每次消耗 CPU **0.01 ms**；檢查瞬間可靠讀出狀態，查詢安排不被延誤。第 9 次查詢在 4.5 ms 發現完成，偵測延遲 0.3 ms，九次 CPU 查詢共 0.09 ms。

另一個模型假設完成後立即交付中斷，handler 消耗 CPU 0.04 ms，於 4.24 ms 結束；不含提交、排程與返回應用的其他成本。若改成 0–4.2 ms 忙等，該核心消耗 4.2 ms，不能把忙等當作 CPU 已閒置。

喚醒不是立刻取得 CPU

裝置完成後，輪詢者或 interrupt handler 必須確認資料可見並更新完成狀態；被等待的 thread 可由 **waiting → ready**。只有 scheduler 選它並完成必要的 context 恢復，它才由 **ready → running**。

對前頁 interrupt 模型，handler 在 4.24 ms 完成並使 P ready；若另一 thread 讓出 CPU 後 P 在 4.34 ms 才執行，則 P 的 ready 等待為 0.10 ms。4.24 ms 是 handler 結束時刻，4.34 ms 才是 P 再次執行時刻；是否回傳應用與複製資料仍取決於剩餘工作。

安全回收 DMA buffer 也需要裝置完成、必要的同步及所有權交接；提早重用可能讓裝置與新使用者同時修改同一記憶體。

結構比較的四個獨立維度

結構	呼叫/依賴	權限與address space	動態擴充	故障與契約
Monolithic	多數服務kernel function calls	多數共享kernel privilege	可有loadable modules	bad pointer可污染共用核心
Strict layers	只依賴較低層介面	layer不必是硬體保護domain	可靜態或動態	契約有助逐層分析，但未自動隔離記憶體
Microkernel	最小核心原語+跨server IPC	多個受隔離user domains	server可分別替換	core較小；parser/權限/恢復仍需正確
Modular kernel	經已定義module interfaces連結	module仍可能共享kernel privilege	runtime load/unload	可維護性不等於fault isolation
Hybrid	依各子系統選部署	需逐項標domain	可混合module/server	名稱不足以推論成本或安全

Layer是dependency概念，module是組織/連結概念，protection domain是存取控制概念。把程式拆成不同檔案或module，不會自動建立user/kernel邊界；把所有軟體畫成上下方塊，也不代表依賴真的無環。

隔離可降低破壞範圍，不會自動完成恢復

固定file server crash，client已送出「append record R」，卻未收到reply。若server未commit就crash，retry才可補做；若已commit但reply遺失，無條件retry會append兩次。隔離只防止server任意寫kernel memory，沒有回答client如何辨別這兩種history。

可加入request ID與可恢復的dedup/result紀錄，使同一ID重試取回相同結果；若紀錄未持久或與append不同transaction，crash仍可能留下不一致。權限應限制server可控制的device/memory/handles；若錯誤server仍有整顆disk任意寫入權，它能破壞磁碟資料，即使無法改寫kernel。

動態module卸載也要先禁止新入口，再等既有references、DMA與callbacks結束；僅把symbol從table移除不足以保證舊function pointer不再執行。這與user-space server restart是不同生命週期問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

教材歷史系統：混合元件不等於相同邊界

以本書出版時的Mac OS X案例：Aqua/Cocoa在上層，kernel environment包含Mach、BSD與I/O Kit/extension。圖中列出Mach並不代表BSD服務一律在獨立user process；判定mode/AS成本仍要查實際部署。iOS以Cocoa Touch、Media Services、Core Services、Core OS表達分層服務。

同一歷史Android案例為applications→application framework→libraries與Dalvik runtime→Linux kernel。Dalvik是該教材的runtime，不把它當現今所有Android的固定實作；Linux kernel提供process/memory/device等服務，不執行Java API的所有高層語意。

三者要比較的是「哪一層提供哪個contract、在哪個domain、跨哪條邊界」。上層framework名稱與同一張stack圖不能替代前述逐事件read trace；也不能把歷史架構直接外推至新版本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Failure analysis：證據反映哪個時刻

User core dump保存失敗process的部分執行/記憶體狀態，kernel crash dump則保存核心故障現場；log是程式選擇輸出的事件，trace可觀察呼叫、排程、I/O等時間順序。它們互補，並非任何一種都能還原完整歷史。

證據	可以回答	不能只憑它斷言
fault PC、registers、stack	哪個指令/呼叫鏈失敗	第一個破壞記憶體的位置必在此處
heap/object snapshot	crash當時物件值	值何時被哪個thread改壞
timestamped trace	已記錄事件先後與時間分解	未開probe的事件沒有發生
CPU sampling	哪些stack累計CPU活動	I/O等待最多的request一定被採到

若filesystem本身壞了，要求故障kernel先透過同一filesystem寫完整dump可能再次失敗。需預留可靠dump路徑/空間與重啟後擷取方法；蒐集失敗不能當作「沒有故障」的證據。

10ms延遲：CPU、blocked與ready分開加總

固定單一request，clock是可比較的monotonic時間，事件無遺失。t0進handler，t1送I/O並阻塞，t8裝置完成而task變ready，t9得到CPU，t10回傳。CPU執行[0,1)+[9,10)=2ms，blocked[1,8)=7ms，ready queue等[8,9)=1ms，end-to-end=10ms。

把7ms都叫「磁碟CPU時間」會誤判；把t8完成當成t8返回會漏掉ready delay。若probe開銷改變排程，量測系統已不同，需對照未插樁時的輸出與可觀察時間，而不是假設debug build零干擾。

八個request latency為1,1,1,1,1,1,1,100ms，mean=107/8=13.375ms；採nearest-rank p99，rank=ceil(0.99×8)=8，p99=100ms。小樣本p99只是此樣本的order statistic，不構成穩定尾端機率估計。

觀測會改交錯：race必須以同步條件排除

初始x=0，兩個threads都做read x、加1、write x，沒有鎖。合法交錯T1:r0→T2:r0→T2:w1→T1:w1，最後x=1；serial順序則為2。加入printf可能把原本重疊改成serial，輸出2不等於data race已修好。

正確性要求的不是「debug跑了十次沒錯」，而是所有容許交錯都保證兩次increment不可分割，例如兩次用同一mutex保護完整read-modify-write，或使用滿足需求的atomic RMW。Lock若只保護write而不保護read，兩者仍可先讀0再各寫1，反例未消失。

動態probe應保留事件ID、時間域、drop計數及必要object identity；每CPU timestamp的排序假設也要明示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SYSGEN：把硬體配置接到可滿足的依賴

System generation可以在compile time裁切source，在link time選預編譯元件，或runtime依table/偵測結果選功能。三者取捨是映像大小、可移植性、硬體變更成本與早期可用性，不能僅以「有沒有重新編譯kernel」二分。

配置項	固定機器需求	缺漏的可觀察後果
CPU/ABI	kernel與使用者image為相容ISA/ABI	未支援指令/exec格式不合
RAM與保留區	使用可用實體範圍，避開device/firmware reserved	覆寫保留資料或非法映射
Boot storage	controller driver能讀root所在device	看不見root device
Filesystem	root filesystem driver早期可用	device存在仍無法mount
Root/init	正確root識別、可執行init及其loader	mount成功卻無法啟動userspace

Runtime hardware discovery需要合適driver，不能把「bus上找得到device」當成已有可用I/O。模組檔案存在也不代表kernel版本/ABI可接受，更不代表在需要它之前能讀到它。

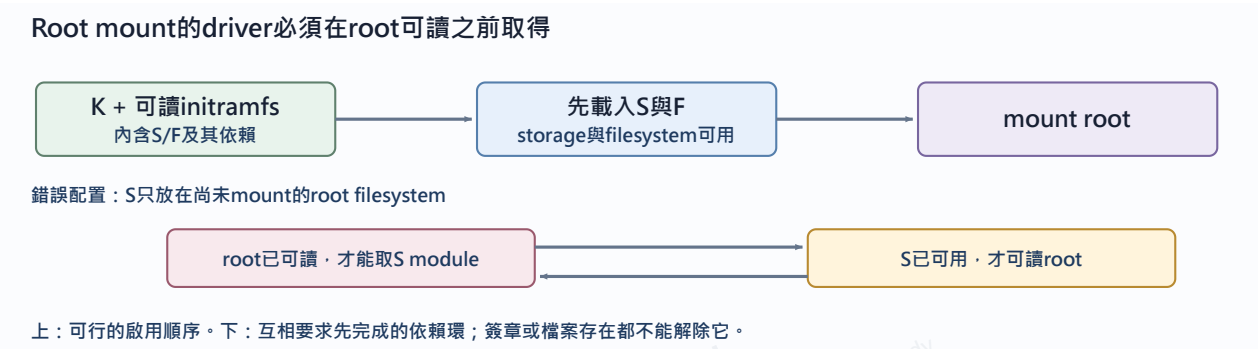
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root device的driver不能只放在尚未mount的root

固定root在storage S，使用filesystem F；K已內建其餘必要依賴。以B表示built-in、I表示initramfs可載入的相容module、R表示僅放在root。早期需求是S與F在root mount之前皆可用。

S位置	F位置	initramfs內容	能否到root mount
B	B	不需S/F modules	能
I	I	S、F及各自依賴皆有	能，先載入再mount
R	B	缺S	不能；須先讀root才取得讀root的driver
B	R	缺F	不能；有device但不認root格式
I	I	只有F，S缺漏	不能；必要依賴未閉合

Root device的driver不能只放在尚未mount的root （續）



這是dependency reachability問題：由built-ins與initramfs可讀元件起始，反覆加入「所有prerequisites都已可用」的元件；若mount-root永遠未加入，就有缺依賴或循環。依賴閉包成立只排除這一類缺件，仍不能證明hardware、driver邏輯或root內容都正確。

從reset到init：每次交接都有最低前提

固定教學機器的firmware可找到bootloader；loader能載入kernel與initramfs並提供memory map/command line。這是角色模型，不把BIOS單扇區流程與UEFI流程硬合成同一個實作。

階段	主要輸入	產生的下一階段前提
Reset/firmware	固定reset入口、硬體配置	基本可用CPU/RAM、可執行loader
Loader	boot設定、kernel/initramfs images	image載入到正確區域，參數交接
Early kernel	hardware/boot資訊	page tables、interrupt、scheduler、memory基本設施
Root setup	storage/filesystem drivers	root可mount，userspace檔案可讀
Userspace init	init image、interpreter、libraries	建立服務/登入環境；權限依各服務policy

Linuxv3.2 `start_kernel()`完成早期初始化後，`rest_init()`建立執行kernel_init的task；不是`start_kernel()`直接跳去執行任意shell。`kernel_init()`完成initcalls/root準備與後續交接，最後嘗試執行相應init。initramfs有可執行`/init`時可先走它；沒有合適早期init時需要準備root namespace。錯誤位置不同，留下的診斷也不同。

Boot失敗的狀態比「不能開機」更有資訊

固定可驗證階段F→L→K→R→I：firmware找到loader、loader交接kernel、kernel可讀root、root可讀init、init成功exec。令gate只在其輸入成立時前進。

故障注入	最後成功階段	首個無法完成的動作	不應歸咎
loader找不到kernel	L	載入K	不能先怪init服務
缺S driver	K	讀root device	不只是root pathname拼錯
F driver缺漏	K	mount已存在的root device	不等於disk不存在
init loader缺漏	R	exec init	不等於kernel沒起來

這張table是可追蹤的role model，不宣稱所有Linux錯誤都以同一exit code回傳到firmware。Kernel panic、loader錯誤畫面、userspace非零退出是不同失敗通路，無法用「任一stage回1就停止」描述完整真實開機。

Verified boot、measured boot與相容性分開

若採verified-boot設計，可信前階段驗證下一階段的簽章/授權policy，並在失敗時拒絕執行；只計算或記錄digest屬量測，沒有自動阻擋。下一階段包含哪些image/config/initramfs，須在policy中明列，不能只驗kernel檔名。

即使所有簽章合法，舊且已撤銷的版本仍可能不應接受；需要版本/撤銷policy。被授權但本身有漏洞的image也不因簽章正確而無漏洞。反過來，ABI匹配與driver依賴閉合只能支持相容性，不證來源可信。

前一頁Linuxv3.2的啟動控制流與這個抽象信任鏈分開：不宣稱v3.2 mainline自帶所有此處政策，也不假設init立即移除所有capabilities。服務啟動後的權限分配需依具體工作與安全設計決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個read跨越哪些契約

以2.7的4096-byte cold read收束：應用API提供fd/buffer/count；ABI決定registers；受控入口建立kernel執行上下文；核心驗證fd及buffer，VFS/filesystem依描述物件找到資料；driver/DMA完成cache填入，再copy到user buffer，回4096。Function call、mode transition、AS handoff、CPU copy、DMA是不同事件，不能重複或互換計數。

把實作改成microkernel，API可以維持同樣read契約，內部卻增加server/driver IPC。把shell換成GUI，前端改變但read保護規則不因此消失。把kernel拆成modules，載入方式改變但未自動縮小每個module的kernel privilege。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功、故障與恢復要接到真正的完成點

讀回4096表示此次bytes可供應用使用，不表示後續write已完成或storage已持久；新pathname的持久化也不同於檔案資料同步。對pipe，讀到EOF取決於buffer空且所有writers消失；對server request，收不到reply不能判定操作未執行。

設計時用契約與不變量連起服務；除錯時用物件ID與時間線找第一個被破壞的條件；SYSGEN/boot用依賴閉包確認所需機制在交接前可用。這些推論依賴的前提都能落在本章的buffer進度、fd引用、地址範圍、事件表或module依賴上，而不是由架構名稱決定。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapters 1–2. write(2); fsync(2); pipe(7); Linux 3.2 init/main.c.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：作業系統如何表示、切換與協調彼此獨立又可能合作的執行單元？

前置知識。 CPU 暫存器、記憶體位址空間、中斷與 system call。

學習目標

- 追蹤 process state 與 PCB 內容
- 區分排程器、dispatcher 與 context switch 成本
- 設計具明確 blocking、buffering 與終止語意的 IPC

Lin Cing https://github.com/xinhi6716/computer-science-self-study

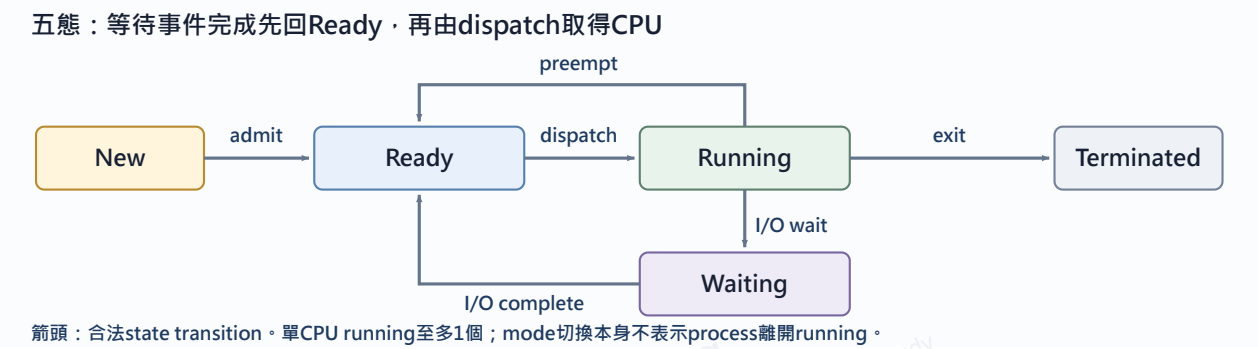
程式、行程與 PCB 的差別

程式（program）是可執行檔中的指令與資料；行程（process）是程式執行時的實例，具有身分、資源與執行狀態。同一程式啟動兩次，就有兩個不同的行程。

項目	P100	P101	可以共享嗎？
PID、PC、暫存器 r1	100、0x40、7	101、0x120、3	執行狀態各自獨立
位址 0x5000 的私有計數器	0 → 1	0 → 1	若未建立共享映射，兩次遞增不會相加
檔案描述符 fd 3	引用開啟檔案 A 的紀錄	引用開啟檔案 B 的紀錄	描述符數字相同，對象不一定相同
程式碼	同一程式的唯讀指令	同左	可以共用實體頁面，不表示共用執行狀態

核心用 PCB（process control block，行程控制區塊）追蹤行程身分、狀態、排程資訊，以及記憶體和 I/O 資源的引用。PCB 可指向頁表和開啟檔案的紀錄，但不必把整個 heap 存在 PCB 裡。

五態模型：每次轉移的原因

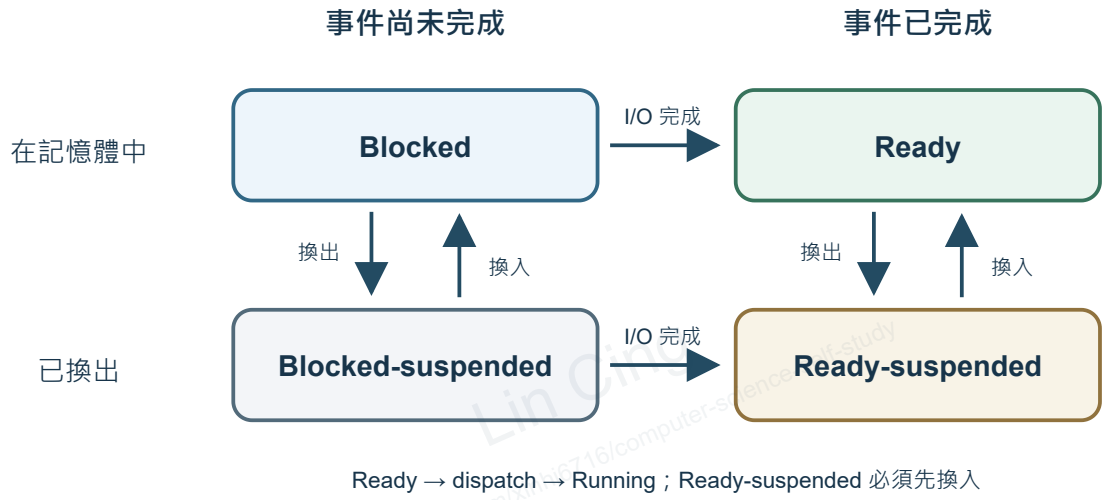


先假設一顆 CPU，而且每個行程只有一條執行流：同一時刻最多一個行程處於 running。**Ready** 表示已具備執行條件、正在等待 CPU；**waiting** 表示還在等待某個事件，不能直接參與排程。

行程獲准進入系統後由 new → ready；派遣時由 ready → running；被搶占時由 running → ready；等待 I/O 時由 running → waiting；事件完成後由 waiting → ready；結束時由 running → terminated。

I/O 完成只使等待者回到 ready，何時真正執行仍由排程策略決定。在單 CPU 模型中，中斷處理常式可以喚醒等待者；等待者不需要先自己執行。此模型省略實際系統的細分狀態，terminated 也不代表所有紀錄立刻消失。

在記憶體外等待：七態模型



這是教學用 whole-process swapping 模型：**blocked** 表示事件尚未完成；**suspended** 表示行程暫不具備在記憶體中執行的資格。兩者互相獨立。現代系統的虛擬記憶體可逐頁回收，不一定把整個 process 換出。

事件完成不等於立刻執行

時刻	事件	A 的狀態	能否分配 CPU？
0	A 發出 I/O	blocked	否，事件未完成
1	排程器換出 A	blocked-suspended	否，事件未完成且已換出
2	I/O 完成	ready-suspended	否，仍須載入
3	換入 A	ready	尚未執行，等待選取
4	dispatcher 選取 A	running	是

另一個 process 可在時間 1–3 執行。若先換入再等到 I/O 完成，則路徑是 blocked-suspended → blocked → ready；兩條路徑最後條件相同，順序不同。

行程切換要保存什麼？

假設 P100 正在執行，PC＝0x40、r1＝7、SP＝0x9000；P101 先前保存的值是 PC＝0x120、r1＝3、SP＝0xA000。中斷或核心入口先依處理器架構保存必要狀態；核心再保存 P100 的執行上下文，載入 P101 的上下文及記憶體管理狀態。

時刻	CPU 正在執行	狀態
切換前	P100	PC＝0x40、r1＝7、SP＝0x9000
核心保存 P100	核心	將 P100 的可恢復狀態與它的紀錄關聯
核心載入 P101	核心 → P101	載入 P101 的 0x120、3、0xA000
稍後再選中 P100	P100	恢復 P100 自己的狀態

實際保存的 PC 及是否重試指令，取決於架構與事件種類。

佇列、排程器與派遣器的分工

Ready queue 收集可執行但尚未分到 CPU 的工作；事件或裝置等待佇列則保存正在等待條件成立的工作。短期排程器從 ready 工作中選擇執行流，派遣器（dispatcher）負責交接並恢復上下文。長期准入與中期暫停分別管理工作進入系統及暫時移出記憶體；實際系統不一定各有獨立元件。

模式切換不一定切換行程；同一行程的系統呼叫就可能只有 user/kernel mode 的轉換。同一行程中的執行緒切換可更換暫存器與堆疊，卻共用使用者位址空間。

例如 P1 執行時發出磁碟請求並阻塞，P2 從 ready 變成 running。請求完成使 P1 回到 ready；若採非搶占式策略，P2 可以繼續執行。重複完成通知不得讓同一工作多次進入 ready queue，也不能讓同一執行流同時在兩顆 CPU 上執行。

若每輪有 10 ms 的有用工作，另花 1 ms 排程與派遣，總時間為 11 ms，額外負擔占 **1/11**。比值 1/10 則是相對於有用工作量的額外成本，兩者分母不同。此固定成本模型沒有計入快取與 TLB 干擾。

喚醒遺失：檢查之後仍有競賽

等待者先檢查 `done=false`、登記並釋鎖，即使之後看到自己仍是 WAITING，也不能直接保證安全：完成事件可能恰好發生在這次檢查之後、實際睡眠之前。

時序	等待者	完成方
1	釋鎖後讀到 WAITING	—
2	尚未睡眠	設 `done=true` 並發送喚醒
3	依舊執行先前決定，進入睡眠	不再有完成事件

如果排程器沒有與喚醒協調，第三步仍可能永久等待。要排除此交錯，必須由同步原語保證「釋鎖並進入等待」對喚醒者不可分割，或以等價的排程狀態協定保存提早到達的喚醒。

以原子等待連接條件與喚醒

在同一把鎖 `L` 下保護條件與等待佇列。等待者取得 `L`，只要 `done=false`，就登記後呼叫 `park_and_release(L)`；返回時重新取得 `L`，再以 **while** 檢查條件。完成方取得 `L`，設定 `done=true`，喚醒等待者，再釋放 `L`。

`park_and_release` 是抽象原語：相對於持 `L` 的完成方，釋鎖與進入等待的交接必須不可分割；若喚醒發生於交接期間，不得遺失。它不是一般的 `unlock(); sleep()`。實作可使用具有此契約的 condition variable 或核心等待佇列／排程器協定。

此處的完成方可在允許取得 `L` 的執行環境中執行；若是真正的硬體中斷處理常式，不能使用可能睡眠的 mutex 或 condition variable，需採用適合中斷情境的鎖及喚醒機制，或將工作延後。

完成先取得鎖時，等待者看到 `done=true` 不睡；等待者先取得鎖時，完成方只能在登記與原子交接後設定條件並喚醒。被喚醒只代表具備執行資格；排程器仍要選取它。迴圈亦處理虛假喚醒及條件被其他執行緒改變。

CPU 密集與 I/O 密集的工作

假設工作 A 交替執行 CPU 1 ms 與等待 I/O 9 ms；工作 B 需要 CPU 10 ms。在單 CPU 且 I/O 可獨立進行的模型中，A 等待 I/O 時，CPU 可以執行 B。

CPU-bound 表示工作經常受限於運算資源；**I/O-bound** 表示它經常等待 I/O。這是工作負載的特性，不是行程永遠不變的身分。增加可執行工作可能改善 CPU 利用率，也可能使裝置佇列或記憶體過載。

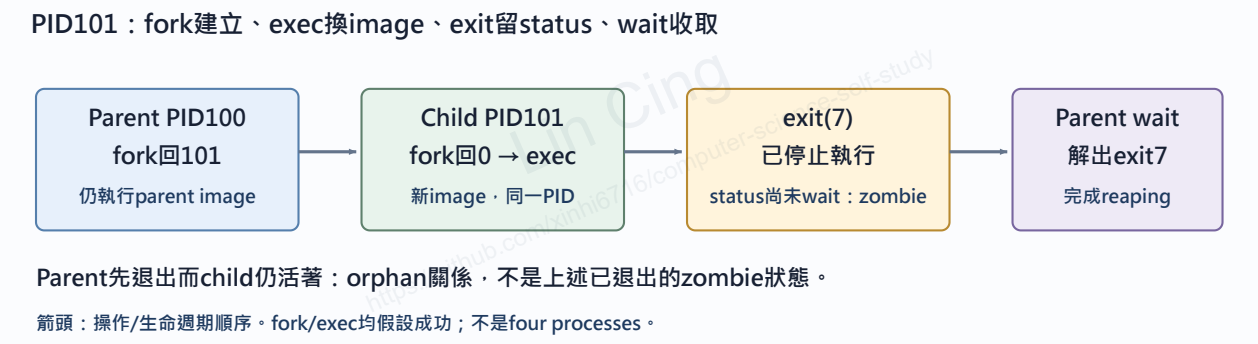
若各行程的工作集超出可用頁框，頻繁缺頁可能導致 thrashing，增加行程數反而降低吞吐量。暫停與重新載入是中期資源管理；每次從 ready queue 選擇下一個工作則是短期排程。第 6 章會用共同工作負載比較具體排程策略。

fork、exec、exit、wait 各做什麼？

fork 建立子行程；子行程有自己的行程與位址空間管理資訊，也取得必要的資源引用。私有記憶體頁可透過寫入時複製（copy-on-write，COW）延後實體複製。

exec 以新程式映像取代呼叫者的映像，成功時不會回到原呼叫點，也不會新增行程。exit 結束執行並釋放大部分資源；父行程用 wait 取得子行程的結束狀態，完成回收（reaping）。

例如 PID 100 執行 fork：父行程得到子行程 PID 101，子行程得到 0。子行程成功執行 exec 後仍是 PID 101；若它以 7 結束，父行程應先確認正常結束，再從 wait 的狀態解出 7。



Zombie 是已結束但狀態尚未被回收的子行程；orphan 則是父行程先結束、仍存活的子行程。重新指定父行程與回收的細節取決於作業系統及其命名空間與 subreaper 政策。

兩次 fork 為何產生四個行程？

假設兩次 fork 都成功、沒有提前結束，且每個行程都會執行下一行。第一次由 P0 建立 P1；第二次由 P0 建立 P2，也由 P1 建立 P3。

已完成的呼叫	執行該呼叫的行程	新增子行程	存活總數
初始	P0	無	1
第一次 fork	P0	P1	2
第二次 fork	P0、P1	P2、P3	4

一般而言，若每個現有行程均執行下一次成功的 fork，則 N(0)=1、N(k+1)=2N(k)，由歸納法得 N(k)=2^k。若只有原父行程繼續、子行程提早 exec 或結束，或者 fork 失敗，就不能套用此式。例如只有 P0 再建立一個子行程時，總數為 3。這棵樹也不決定實際 PID 或輸出先後。

繼承描述符與獨立開啟檔案

假設父行程的 fd 3 引用開啟檔案紀錄 O1，其 offset 為 0。`fork` 後父子各有描述符表項，但兩個表項仍可引用同一個 O1，因此共享 offset。

在檔案足夠長、沒有其他操作且每次確實讀到指定數量的條件下，父行程讀 100 bytes，offset 由 0 變 100；子行程接著讀 50 bytes，從 offset 100 開始，最後變成 150。若另外 `open` 同一檔案，新的開啟紀錄 O2 可有獨立的 offset。這與使用者記憶體的 COW 是不同的共享關係。

`fork` 失敗應走錯誤處理，不能誤認為子行程。`exec` 失敗通常回傳錯誤，但特定實作跨過不可回復點後，不能無條件假設舊映像完整。父行程若從不等待已結束的子行程，可累積未回收的結束狀態。

多執行緒行程呼叫 `fork` 時，子行程只保留呼叫者這條執行緒；複製過去的鎖狀態可能沒有存活的持有者。子行程在 `exec` 前可安全執行的操作須遵守系統介面的限制。

共享記憶體與訊息傳遞

共享記憶體先建立共同的儲存物件與映射，再由各行程讀寫資料；仍需權限檢查與同步。訊息傳遞則透過 `send`／`receive` 交換資料，其等待、容量與複製行為由介面契約決定。

必須約定的事項	例子	不釐清的風險
對象與命名	行程、端點或 mailbox	權限、識別碼重用與清理
等待	send／receive 各自可否阻塞	send 完成可能只代表訊息已入列
容量	零容量、有界或理想無界	緩衝區滿時的 backpressure；實體資源仍有限
資料邊界	完整訊息或 byte stream	一次 read 不一定對應一筆紀錄
生命週期	close、unlink、remove、對方失敗	EOF、錯誤與佇列消失的語意不同

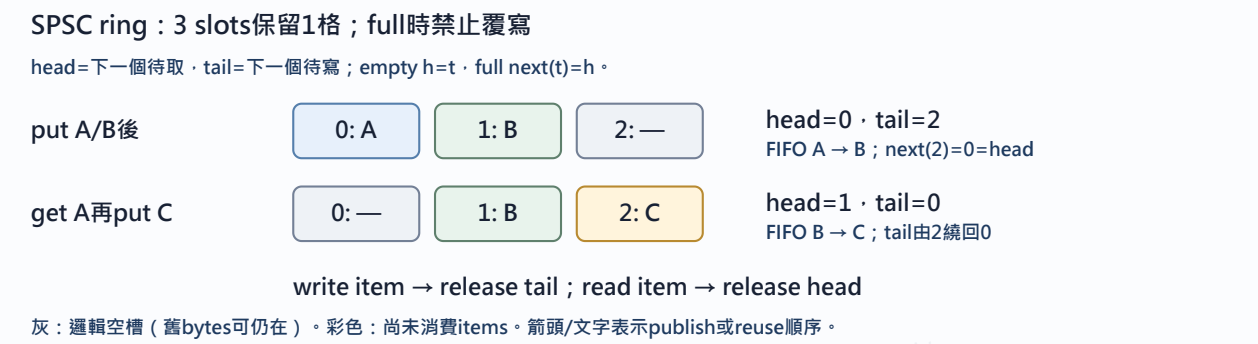
例如容量為 2 的抽象通道先接收 J1、J2，接著 J3 必須等到接收者取走 J1 才能入列。即使兩端都使用阻塞呼叫，只要 send 完成代表「成功入列」，它就不一定是零容量 rendezvous。

單生產者單消費者環形佇列

一個 producer 和一個 consumer 共用三個槽位 `slots[3]`，以及初值均為 0 的原子索引 `head`、`tail`。`head` 指向下一筆待讀資料，`tail` 指向下一個待寫位置。保留一格後，空佇列滿足 `head=tail`；滿佇列滿足 `next(tail)=head`，其中 `next(x)=(x+1) mod 3`。因此容量是 2，而非 3。

事件	head	tail	FIFO 中尚未取出的資料	結果
初始	0	0	[]	空
放入 A	0	1	[A]	發布 tail＝1
放入 B	0	2	[A,B]	滿，next(2)＝0＝head
嘗試放入 C	0	2	[A,B]	佇列滿，不能發布 C
取出 A	1	2	[B]	釋放槽位 0
放入 C	1	0	[B,C]	tail 繞回，仍為滿
依序取出 B、C	0	0	[]	空

單生產者單消費者環形佇列 （續）



發布資料與重用空間的先後關係

Producer 以 acquire 讀取 `head`；確認有空間後，先寫入 `slots[tail]`，再以 release 發布新的 `tail`。Consumer 以 acquire 讀取 `tail`；確認有資料後，先讀取 `slots[head]`，再以 release 發布新的 `head`。只有 producer 寫 `tail`，只有 consumer 寫 `head`。

第一條先後關係：slot 寫入先於發布 `tail`；consumer 的對應 acquire 觀察到該發布後，才能安全讀取已寫完的資料。第二條：slot 讀取先於發布 `head`；producer 的對應 acquire 觀察到釋放後，才能重用該槽位。空／滿檢查與唯一寫入者的前提共同維持容量 0 至 2 及 FIFO 次序。

普通共享整數不能替代原子操作與記憶體順序：C/C++ 中衝突的非同步存取可能構成 data race。若先發布 `tail` 再寫資料，讀者可能看見舊內容；若增加第二個 producer，也必須重新設計索引競爭。跨行程使用原子物件仍須平台明確支援。有限追蹤能檢查索引，不能代替同步論證。

從位元組串流組出完整紀錄

假設每筆紀錄以 2-byte big-endian 長度開頭，後接 payload；payload 最多 8 bytes。兩筆資料 `ABC`、`DE` 編成：

```
00 03 41 42 43 | 00 02 44 45
```

串流中的直線僅供說明，不會由 `read` 保證。三次讀取可能分別得到 `[00]`、`[03 41 42]`、`[43 00 02 44 45]`。解析器先累積兩個 header bytes，解出長度 3，再累積 `ABC` 並輸出；接著解出長度 2，累積並輸出 `DE`。

如果 EOF 出現在半個 header，或 payload 尚未收齊時，就是截斷（truncation），不能當成完整訊息。收到長度後先驗證不超過 8，並檢查配置大小的算術，再接收 payload。TCP 與 pipe 的 byte stream 不替應用程式劃分紀錄；datagram 與 message API 則須依各自的邊界及截斷規則處理。

共享記憶體的名稱、描述符與映射

建立者以名稱 `/course-ring` 建立物件、設定長度為 4096 bytes，再以 `MAP_SHARED` 映射；另一個行程以相同名稱開啟及映射。假設 P1 的映射起點是 0x100000，P2 是 0x700000，則物件 offset 16 分別對應虛擬位址 0x100010 與 0x700010。兩者仍可能存取同一個 backing 的位置。

因此共享結構可儲存 offset 或 index；不應將 P1 的絕對指標直接交給 P2。關閉 descriptor 不會自動解除既有映射；`shm_unlink` 移除名稱，阻止後續透過舊名稱開啟，但現有引用與映射可以繼續使用，直到各自生命週期結束。

`MAP_SHARED` 本身不保證初始化先於讀取。建立者應完成物件大小、版本、長度與欄位初始化，再以支援跨行程的同步機制發布 ready 條件。消費者不能假設對尚未設定大小、超出有效範圍的區域存取只是讀到零；固定 sleep 一秒也無法保證發布順序。

移除 System V 與 POSIX 佇列

物件	移除操作	已有使用者受到的影響
POSIX message queue	`mq_unlink` 移除名稱	已開啟的引用仍可使用，直到最後引用關閉後才銷毀
System V message queue	`IPC_RMID` 移除佇列	阻塞中的 send／receive 被喚醒並收到 `EIDRM`
自訂應用通道	停止傳送、排空，最後銷毀	只能依應用程式明定的狀態機推理

不能把「移除前必定等待消費者排空」當作這兩種原生 API 的共同保證；移除名稱也不等於自動向每個接收者發送 EOF。若需要排空與正常停止，應用程式必須另行約定協定。

用結束訊息通知消費者

假設一個 producer 和一個 consumer 透過容量為 4 的 FIFO 交換工作，正整數是合法工作，0 保留為 STOP。Producer 先送 1、2、3，停止接受新工作後再送 0：佇列依序為 `[1]` → `[1,2]` → `[1,2,3]` → `[1,2,3,0]`。

Consumer 依次完成 1、2、3，再收到 STOP 並回覆確認；管理者收到確認後才清理資源。這是應用層協定，並非核心自動提供的排空保證。

若 0 也是合法工作，應改用帶有明確類型標記的訊息。若有兩個 consumer 而只送一個 STOP，另一個可能永遠等待；可以各送一個 STOP，或設計關閉狀態與廣播。若 consumer 在確認前故障，還須另外規定恢復或重派，不能由正常路徑推論工作恰好完成一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach：連接埠權限與訊息順序

以教材的歷史 Mach port 模型為例，server 持有 receive right，client 需要對應的 send right。訊息 header 指明目的地與回覆 port；帶型別的內容可包含資料或受控的權限。知道 port 的本地名稱不代表擁有接收權，也不能把它當成全系統通用的指標。

假設 client A 依序送出 A1、A2，client B 送出 B1。在本模型中，同一 sender 的 A1 應先於 A2，但不同 sender 之間沒有自動建立全域順序；例如 A1、B1、A2 或 B1、A1、A2 都可能出現。Server 回覆 A1 時，A2 與 B1 仍可能等待；send 成功也不等於 reply 已成功收到。

佇列滿時是等待、逾時還是立即失敗，須依所選操作的契約判定。Out-of-line memory 可透過虛擬記憶體映射或 COW 減少大資料複製，但不能因此宣稱所有訊息與跨主機傳輸都不複製。權限和共享記憶體的生命週期仍需由協定處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows ALPC 的連線與私有通道

依原書的歷史 Windows 案例，server 公開 connection port；取得授權的 client 送出連線請求，server 接受後建立雙方可用的通訊端點。Connection port 用於建立通道，並非每傳送一筆資料就重建一次連線。

教材以小訊息複製與大資料使用 shared section 區分路徑，並以當時案例的 256 bytes 為分界；這不是所有現代 Windows 版本固定的 API 上限。在該模型下，128-byte 請求可採小訊息路徑，4096-byte payload 可放入預先建立的 section，再以 offset 和 length 指示範圍。

即使用 section 減少重複複製，仍須驗證映射範圍、權限及可寫入者，不能信任 client 任意提供的指標。應用程式通常透過公開 RPC 等介面間接使用此類本機機制；Mach、ALPC 和 POSIX queue 的命名、關閉與權限語意不能直接互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Socket 端點與 RPC stub 的職責

Socket 端點由位址家族、位址與 port 等資訊識別；常見的串流伺服器先 `bind`、`listen`，再用 `accept` 取得連線端點。原本的 listening socket 仍可接受後續連線。Port 不等於 PID；連線成功也不表示應用程式的操作已完成。

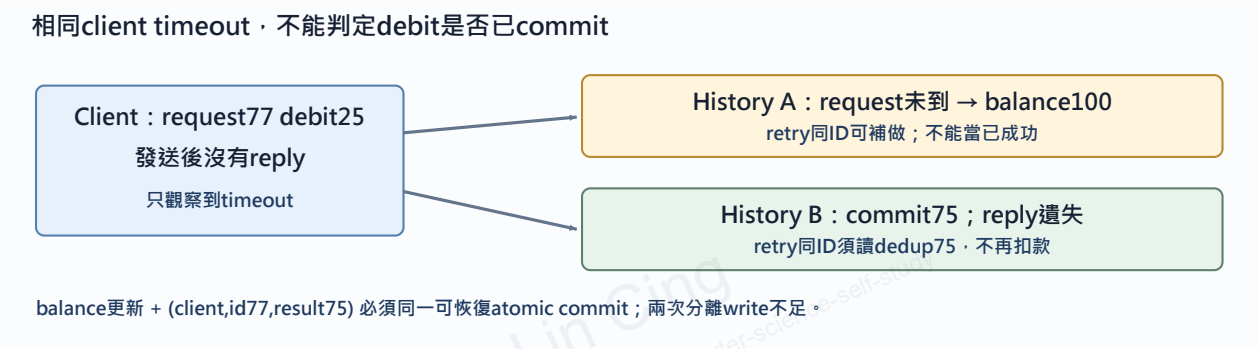
RPC client stub 將操作與參數編碼為訊息；server stub 驗證並解碼訊息、呼叫服務，再編碼結果。不同機器的整數寬度、endianness、對齊方式及指標值不能直接當成共通的 wire format。

例如規定 request ID 為 32 bits、opcode 為 16 bits、amount 為 32 bits，全部使用 big-endian。ID＝77、opcode＝1、amount＝25 時，10 bytes 為 `00 00 00 4D | 00 01 | 00 00 00 19`。接收者先驗證長度與 opcode，再解碼數值；傳送 client 的指標位址不會讓 server 自動取得該記憶體的存取權。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逾時：兩種不同的執行歷程

帳戶原有 100，request 77 要扣款 25。情況 A：請求沒有到達 server，餘額仍為 100；情況 B：server 已提交扣款、餘額為 75，但回覆遺失。Client 在兩種情況下都只觀察到 timeout。



因此，改用新 ID 盲目重試可能讓情況 B 再扣一次、變成 50；完全不重試則可能讓情況 A 的操作遺漏。若要重試且避免重複扣款，server 可用 `(client identity, request ID)` 查詢去重紀錄，並把餘額更新與結果紀錄放在同一個可恢復的原子交易中。

第一次 request 77 將 100 改為 75 並記錄結果 75；重送同一 ID 時回傳 75，不再扣款。並行的重複請求也須由交易或唯一性約束協調，不能都先查到「不存在」再各自扣款。

至多執行一次與可用性

在去重紀錄有效的期間，同一 request ID 可以限制為至多一個效果。若先扣款、再另外寫入紀錄，兩者之間故障就會留下重複執行的機會；若紀錄只存在 RAM，重啟後也會消失。紀錄何時可以刪除，必須與重試期限、ID 作用範圍及 epoch 等規則配合。

At-most-once 允許請求永久遺失、一次也未執行。At-least-once 在持續重試與最終可達等條件下提高完成機會，但可能重複非冪等操作；扣款 25 就不是冪等操作。若要談 exactly-once，必須明確界定交易、持久紀錄、故障與通訊假設，不能只靠名稱保證。

Pipe 和 named FIFO 支援本機串流；socket 可支援更一般的端點及跨主機通訊。應用仍須處理 framing、backpressure 與對方失敗；IPC 成本也包括喚醒、同步、引用與重試。

行程與資料的生命週期

假設 producer P 與 consumer C 使用容量 2 的 FIFO。P 送入 J1、J2 後，嘗試送 J3 時因佇列已滿而等待；C 取走 J1，釋出一個槽位並喚醒 P；P 回到 ready，在被選中執行後才送入 J3。過程中 FIFO 次序不變，容量也不超過 2。

取出訊息不表示工作的效果已經持久完成；行程結束也不代表父行程已經以 `wait` 收取狀態。PID 與 PCB 描述執行，描述符表項引用開啟檔案紀錄，共享記憶體 offset 指向資料，而 request ID 用來辨認重試。這些身分與生命週期必須分開追蹤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

檢查安全性的關鍵條件

行程切換時，保存與恢復必須對應同一條執行流；等待與喚醒必須圍繞同一個受同步保護的條件。環形佇列須先寫入資料再發布、讀取完才允許重用空間；串流解析器只交出完整且長度合法的紀錄；RPC 的實際效果與去重結果必須一同提交。

逐步範例用來核對中間狀態；不變量則解釋更多允許的事件順序為何仍然安全。有限測試不能證明所有執行交錯或故障都正確。若更換 API、增加生產者或消費者、改變記憶體順序或持久化策略，就要重新檢查各項前提。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 3. read(2); pipe(7); msgctl(2); mq_unlink(3).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：多個執行流如何共享 process 資源，同時維持排程與資料正確性？

前置知識。 process 位址空間、CPU 排程基本概念、stack 與暫存器。

學習目標

- 區分 process 與 thread 的資源邊界
- 比較 threads 映射模型與並行來源
- 辨識 cancellation、signal、TLS 與 fork 的執行緒問題

Lin Cing https://github.com/xinhi6716/computer-science-self-study

執行緒私有狀態與行程共享資源

執行緒（thread）各自擁有程式計數器（PC）、暫存器、堆疊、執行狀態與身分。同一行程（process）中的執行緒通常共享程式碼、全域變數、heap、位址空間與開啟檔案等資源。共享方便合作，但錯誤指標也可能破壞其他執行緒使用的資料。

各執行緒通常使用不同的堆疊區域，但這是使用方式與物件生命週期的區別，並不是同一位址空間內的硬體隔離：若將指向尚未結束生命週期的堆疊物件的指標交給另一執行緒，後者可能存取該物件。

兩個 worker 的資料	W1	W2	協調方式
暫存器／區域索引	自己的索引 0	自己的索引 0	執行上下文獨立
暫存緩衝區	B1 暫存 110	B2 暫存 120	各自寫入不同區域
共用工作佇列	取得工作 11	取得工作 12	所有取出操作遵守同一同步協定
全域總和	合併 110	合併 120	使用原子操作或 mutex；也可在 join 後由一方合併

執行緒可能提高回應性、降低部分資源建立成本，並利用多核心；實際效果仍取決於工作分解及硬體。增加執行緒不會自動消除鎖競爭、記憶體頻寬限制或序列相依。

全域變數、執行緒區域儲存與生命週期

假設兩條執行緒各執行一次 `counter++`。若它們未同步地修改同一個非原子 C/C++ 物件，衝突存取會構成 data race，行為未定義。以抽象的讀取、加一、寫回模型說明：兩者都讀到 0，再各自寫回 1，便可能遺失一次更新；這只是機制示例，不能宣稱是未定義行為唯一的輸出。

若改用每條執行緒獨有的 TLS（thread-local storage）計數器，兩者各自從 0 增至 1，消除了對同一計數器的共享寫入。要得到總和 2，worker 必須在結束前將各自的值複製到生命週期足夠長、互不衝突的共享結果槽；主執行緒成功 join 兩者後，再讀取結果槽並相加。執行緒結束後不可直接讀取已銷毀的 TLS 物件。

TLS 可跨函式呼叫保存，直到該執行緒結束，與單次函式呼叫的自動區域變數不同。若 thread pool 重用 worker，TLS 不會在每個工作開始時自動歸零；應在工作邊界明確初始化及清理。同步也無法使已失效的堆疊物件重新有效。

並行、平行與工作分解

Concurrency（並行處理）表示多個工作流可以交錯前進；即使只有一顆 CPU，也能透過切換工作達成。Parallelism（平行執行）表示工作在同一時間執行，需要足夠的硬體執行資源。SMT 的多個邏輯執行環境會共享部分核心資源，因此每個邏輯 CPU 不等於一顆完整獨立核心的算力。

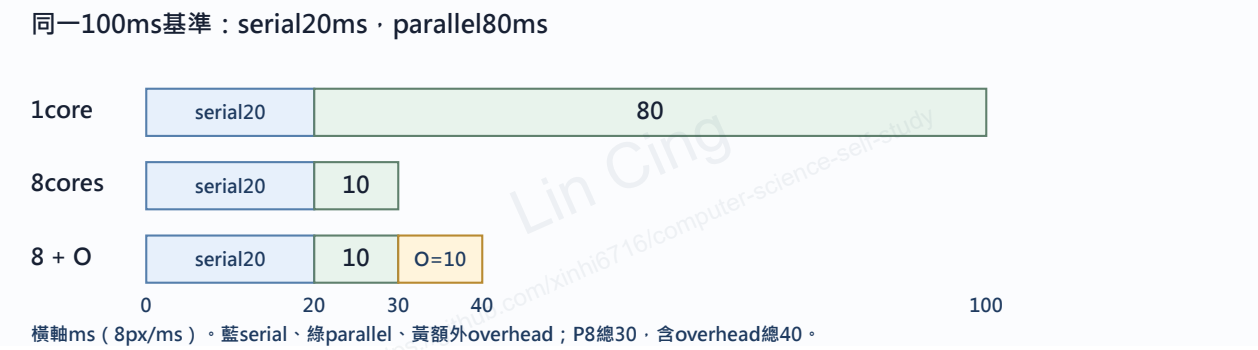
資料平行將相同操作分配到不同資料區段；任務平行則讓不同功能或階段並行。設有 64 MiB 的 32-bit 整數陣列，共 16,777,216 個元素；分成 8 份後，每份 8 MiB、2,097,152 個元素。每個 worker 使用足以容納該輸入總和的 64-bit 累加器先計算區段和，再將 8 個部分和以 7 次加法合併。此做法仍須讀取整個陣列。

若每個元素都以 atomic add 更新同一全域計數器，會有 16,777,216 次共享 read-modify-write；分區版本則在完成時計算並發布 8 個部分和。這是操作數量比較，不是實測加速倍數：記憶體頻寬、快取與 NUMA 配置仍可能成為限制。

Amdahl 定律：以基準時間推算上限

固定相同問題規模：單核心基準時間 $T_1 = 100$ ms，其中 20 ms 必須序列執行，80 ms 可理想地平均分給 P 個核心，且先不計額外開銷。令序列部分占基準時間的比例 $s = 0.2$，則 $T_p = 100[s + (1-s)/P]$ ms，加速比 $S_p = 1/[s + (1-s)/P]$。

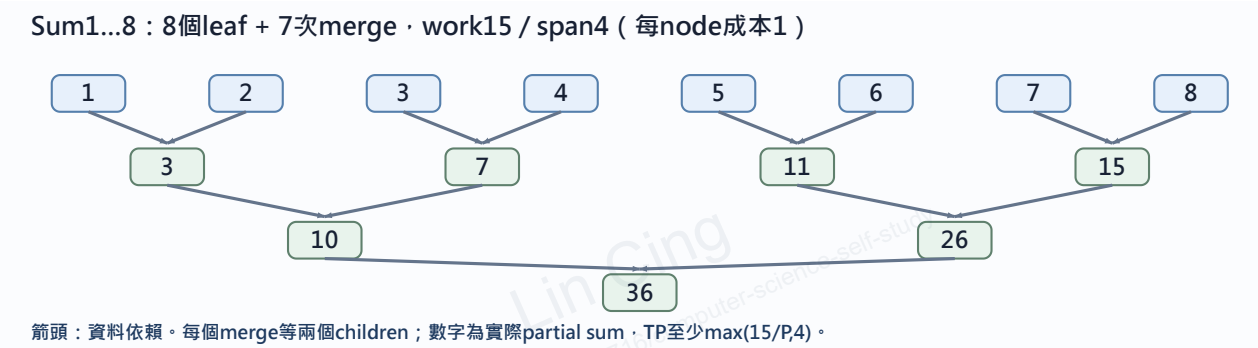
P＝8 時，$T_8 = 20 + 80/8 = 30$ ms，加速比為 100/30＝10/3，效率為 (10/3)/8＝5/12；當 P 趨近無限大，加速比的上限為 1/0.2＝5。



若 8 核心總共多出 10 ms 的啟動與同步成本，$T_8 = 40$ ms，加速比降為 2.5。操作數比例也不能直接當時間比例：80 個操作各需 1 cycle，另 20 個各需 10 cycles，基準時間是 280 cycles；只將前者加速 8 倍後為 10＋200＝210 cycles，加速比僅 4/3。

工作量、關鍵路徑與歸約

將數值 1 至 8 分別放在 8 個 leaf，以二元加法合併；假設每次讀取 leaf 與每次 merge 的抽象成本均為 1 單位。總工作量 work＝8＋7＝15；最長依賴路徑依序經過 leaf、pair、quartet、root，故 span＝4。



第一層得到 3、7、11、15；第二層得到 10、26；最後得到 36，共進行 7 次加法。每個節點都必須等待前驅完成，例如 root 不可能在兩個 quartet 完成前執行。

對任意 P 處理器排程，$T_p \geq \max(15/P, 4)$，因此加速比不超過 work/span＝15/4。這個比值是平均平行度；它不代表超過 3.75 個處理器後就一定沒有額外收益。若某一工作區段特別大，總時間也至少要等最慢的區段完成，不能只看平均值。

False sharing 與 data race 的差別

假設 cache line 為 64 bytes、整數為 4 bytes，且陣列起點按 64 bytes 對齊。W1 反覆寫入 `a[0]`、W2 寫入 `a[1]`：兩者是不同整數物件，在沒有其他衝突存取時可以不存在語言層級的 data race；但它們位於同一 cache line，可能使快取一致性協定反覆轉移該 line 的寫入權，降低效能。

將兩個寫入位置分別放在符合對齊要求、間隔至少 64 bytes 的區域，可降低此模型中的干擾；是否真的改善、改善多少，仍須在目標系統實測。若原本兩者修改的是同一個計數器，padding 不會修復 data race，仍需同步或重新設計資料所有權。

為更新加 mutex 可能提供互斥，但也可能增加序列化；改用每執行緒的計數器則改變資料表示，仍需定義結果如何合併。正確性、cache line 共享成本與結果合併應分別分析。

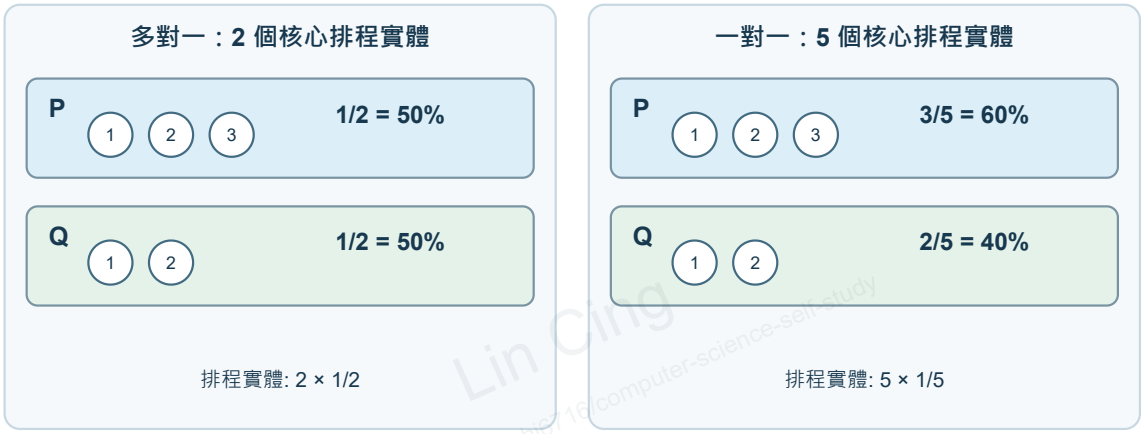
使用者執行緒與核心排程實體的映射

模型	使用者執行緒與核心排程實體的關係	一個真正阻塞的 system call	平行執行限制
Many-to-one	多條使用者執行緒共用一個 carrier	若沒有非同步替代，carrier 阻塞後，同一行程的其他使用者執行緒無法另由核心排程	同時最多一個 carrier 執行
One-to-one	每條使用者執行緒對應一個核心實體	一條阻塞，其他就緒實體仍可排程	受就緒實體數和核心數限制
Many-to-many	runtime 將多條使用者執行緒排到多個 carrier	影響取決於 carrier 數量與阻塞補償策略	受可執行 carrier 數和核心數限制
Two-level	M:N，且允許部分執行緒綁定 carrier	綁定與未綁定執行緒依各自規則運作	綁定會限制 runtime 的排程選擇

使用者程式呼叫哪個 thread library API，不能單獨決定其唯一映射；大量使用者執行緒也不必然代表同樣數量的核心物件。若 runtime 以 nonblocking I/O 和 event loop 等待完成，許多任務可以在邏輯上等待，而 carrier 本身不必阻塞；這與真正阻塞的 system call 不同。

排程單位會影響行程的 CPU 份額

圓圈：使用者執行緒；核心實體長期等份額



假設單 CPU、所有執行緒持續可執行、等優先權，且排程器長期平均分配 CPU 給每個可執行的核心排程實體。P 有 3 條使用者執行緒，Q 有 2 條。

在此例的 many-to-one 模型中，每個行程各有一個 carrier，共兩個核心實體：P、Q 分別取得 1/2 的 CPU。改為 one-to-one，五條執行緒各有一個核心實體、各取得 1/5：P 合計 3/5，Q 合計 2/5。

換一種公平規則，答案也會改變

仍假設單 CPU、P 的三條和 Q 的兩條執行緒持續可執行。若排程器先讓 P、Q 兩個行程群組各得 1/2，再於各群組內平均分配：

分配方式	P 的總份額	P 每條	Q 的總份額	Q 每條
五條核心實體各得 1/5	3/5	1/5	2/5	1/5
先對兩個行程群組平分	1/2	1/6	1/2	1/4

例如觀察時間為 60 ms，群組公平模型給 P、Q 各 30 ms；再平均分配時，P 每條得到 10 ms，Q 每條得到 15 ms。這是模型計算，沒有計入切換成本。

不同權重、執行緒阻塞或多核心也可能改變結果。Many-to-one 模型下，使用者層 runtime 如何分配 P 的 1/2，還需另行約定；不能僅由核心份額推出。

100 條執行緒與 10 個 carrier：阻塞與補償

假設有固定的 10 個 carrier K1 至 K10，不提供替代 carrier，也不使用非同步 I/O；100 個使用者執行緒各想發出一次會阻塞的 I/O。U1 使 K1 阻塞後，還有 9 個 carrier 可執行；當 K1 至 K10 都已阻塞，便沒有 carrier 可執行。此時其餘 90 條使用者執行緒仍在 runtime 的 ready queue，尚未有機會發出它們的 system call。

「最多 10 個核心實體同時阻塞」只來自上述固定 10 個 carrier 的前提，不能推成所有 M:N 系統的通則。若另一策略允許最多 20 個 carrier，原本 10 個阻塞後，K11 至 K20 仍可執行並發出更多阻塞呼叫。若採非同步 I/O，100 個任務也可能都在邏輯上等待完成，而 carrier 不一定阻塞。

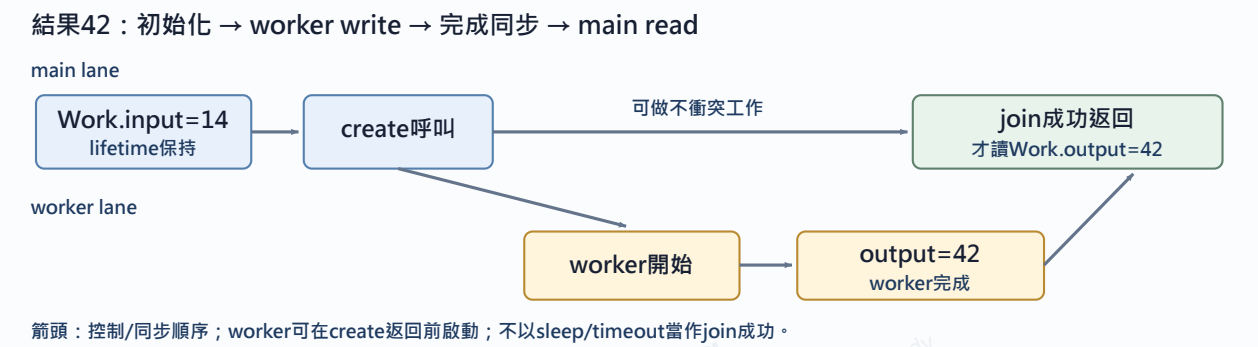
Scheduler activations／upcalls 是協調核心資源通知與使用者層排程的歷史機制：核心通知阻塞或可用資源，runtime 再選擇執行緒。它必須處理 runtime 鎖、通知上下文及 carrier 配置，並不會創造額外 CPU 算力；two-level 綁定也不表示同一條執行緒可同時在兩個 carrier 執行。

建立、完成與 join：結果的生命週期

假設主執行緒持有 `Work`，輸入為 14；worker 計算 `output=3×14=42`。主執行緒在建立 worker 前初始化輸入，並讓 `Work` 至少存活到成功 join 後。建立後不能假設 worker 尚未開始，也不能在缺乏同步時讀取 worker 正在寫入的輸出。

階段	主執行緒	worker	必須維持的條件
建立前	初始化 input＝14	尚未建立	參數與物件有效
建立成功	可做其他不衝突的工作	可開始並寫 output＝42	執行先後不固定
完成	可等待 join	不再使用 Work	完成事件依 API 契約同步
join 成功後	讀 output＝42，之後才結束 Work 的生命週期	已完成	不與 worker 寫入重疊

建立、完成與 join：結果的生命週期 （續）



睡眠 10 ms 不能證明 worker 已完成。Detached thread 不再透過 join 收取，但仍需要另一個完成通知與所有權協定；不能 detach 後立刻釋放它仍可能使用的參數。

Pthreads、Windows 與 Java 的建立和等待

Pthreads 的 `pthread_create` 接收入口函式及參數；成功 `pthread_join` 會等待可 join 的目標結束並取得其結果。應處理函式回傳的錯誤碼，不能一律只查 `errno`。同一目標的 join 所有權必須明定，也不能回傳指向已失效 stack 區域變數的指標。

Windows 可用 `CreateThread` 建立執行緒；只有 `WaitForSingleObject` 對 thread handle 成功取得 signaled 結果，才按該完成契約讀取結果。Timeout 與 `WAIT_FAILED` 不是完成證明。使用完仍須 `CloseHandle`；關閉 handle 本身不會終止執行緒。

Java 8 的 `Thread.start` 啟動新執行緒，直接呼叫 `run` 則是目前執行緒的普通方法呼叫。啟動前的動作 happens-before 新執行緒的動作；worker 的動作 happens-before 另一執行緒成功從 `join` 返回。`InterruptedException` 不代表目標已完成。

執行緒池與 fork-join 的等待風險

假設 8 個工作各需 1 個 CPU 時間單位，4 個 worker、FIFO 排程且沒有額外成本：J1–J4 在 [0,1) 執行，J5–J8 在 [1,2) 執行，完成時間為 2。Pool 可重用 worker；worker 數和等待佇列容量是不同限制，仍需處理 backpressure、取消與錯誤。

Fork-join 將問題拆成子工作再合併。若任務太細，建立、佇列與同步成本可能超過有用工作；任務的合併也必須等待其前驅完成。

例如只有兩個 worker 的 pool 同時執行 A、B；兩者各自提交 A1、B1 到同一個 pool，再阻塞等待子工作。A1、B1 已 ready，卻沒有空閒 worker，因而產生 starvation deadlock，即使資料依賴圖本身沒有環。具備明確 help-join 行為的 runtime 可讓等待者協助執行子工作，但不能假定所有 pool 都有此功能；改用 continuation 也是一種解法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OpenMP reduction 與 GCD 佇列

OpenMP 的 `parallel for` 可分配迭代；若每次迭代寫入不同的 `c[i]`，可分別計算 `c[i]=a[i]+b[i]`。相反地，多個執行緒同時執行 `sum+=a[i]` 需要 reduction 或其他同步。

假設實際建立四個 worker，分別處理 [1,2]、[3,4]、[5,6]、[7,8]，得到部分和 3、7、11、15；一種合法合併順序為 10＋26＝36。OpenMP 不保證這組分配或合併順序；浮點加法並非精確結合，改變次序可能改變最後位元。

GCD 將工作提交到 dispatch queue。Serial queue 在同一時間只執行該 queue 的一個工作；concurrent queue 可重疊，先入列不代表先完成。若正在 serial Q 執行的工作同步提交另一工作到同一 Q 並等待，兩者互相等待，可能卡死。不同 queue 之間的共享物件不會自動互斥。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多執行緒 fork 與 signal 傳遞

假設同一行程中 T2 持有 mutex M，而 T1 呼叫 POSIX `fork`。子行程只保留呼叫者 T1；複製的記憶體中 M 卻可能仍呈現由 T2 持有，而子行程沒有 T2 能釋放它。子行程在成功 `exec` 前須遵守 async-signal-safe 的限制；`pthread_atfork` 只有配合整體鎖與函式庫設計才能處理已知問題。

Signal disposition 通常由行程中的執行緒共享，signal mask 則是每條執行緒各自維護。同步 fault 對應造成事件的執行緒；process-directed signal 依遮罩與系統規則交給符合條件的執行緒，不是每條執行緒都執行一次 handler。使用專門的 `sigwait` 執行緒時，其他執行緒也須協調遮罩。

成功 `exec` 會替換程式映像，原本其他執行緒不繼續執行舊映像；建立一般執行緒則會共享現有行程資源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延後取消與安全的清理時機

`pthread_cancel` 成功只表示取消請求已提出，不等於目標已終止。Deferred cancellation 在指定的 cancellation point 處理；asynchronous cancellation 可能在更新資料或持鎖期間發生，因此更難維持不變量。

假設 worker 擁有 mutex M 和 buffer B，並在 `pthread_cond_wait` 等待穩定的條件。先停用取消，建立資源及清理程序，依序登記 unlock-M、free-B；再啟用 deferred cancellation。若等待期間處理取消，`pthread_cond_wait` 先重新取得 M，清理程序再依後進先出執行：先釋放 B，後解鎖 M。主執行緒成功 join 後才能回收需要等待 worker 完成的共享參數。

清理程序只處理資源，不會自動修復資料結構的半完成更新；應在不變量穩定處設取消點，或設計可驗證的 rollback。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取消前後的資源表與不變量

階段	mutex M	buffer B	釋放 B 次數	解鎖 M 次數
資源與清理程序已建立	worker 持有	有效	0	0
condition wait 期間	原子釋放 M 並等待	有效	0	0
開始處理取消	依等待契約重新取得 M	有效	0	0
第一個 cleanup	worker 仍持有 M	已釋放	1	0
第二個 cleanup	已解鎖	已釋放	1	1

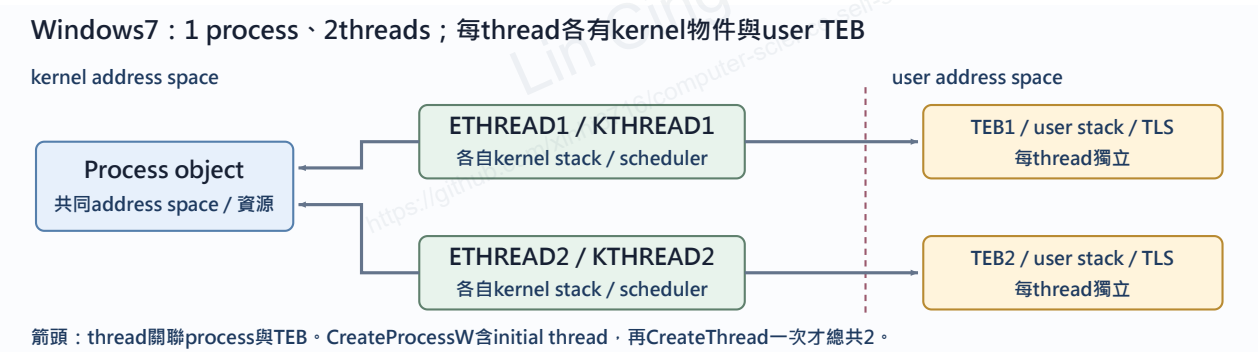
正常、錯誤和取消路徑都要配對清理並明定所有權，避免重複釋放或解鎖。不能一邊留下半更新的共享資料，一邊只靠 unlock 宣稱恢復正確；必要時需延後取消或回復狀態。

Windows 7 與 Linux 3.2 的執行緒物件

以原書的 Windows 7 及 Linux 3.2 歷史案例為例，比較行程擁有的資源與核心實際排程的單位；本頁不是對現代版本實作細節的通用保證。

Windows 的 process 保存位址空間與行程層級資源；thread 分別有核心中的 ETHREAD、KTHREAD，以及使用者空間的 TEB。`CreateProcessW` 已建立一個初始 thread；若接著成功呼叫一次 `CreateThread`，這個案例共有兩個 thread，而非一個。

Linux 3.2 用 `task_struct` 表示每個可排程 task。一般 `fork` 建立新的 task 與 kernel stack，且未共享 `CLONE_VM` 時建立不同的 mm 和頁表；`clone` 則可透過 flags 選擇共享 mm、檔案表與 signal handler 等結構。



Windows 7 與 Linux 3.2 的執行緒物件 （續）

比較項目	Windows 7	Linux 3.2
可排程單位	kernel thread	task
個別狀態	暫存器、堆疊、TEB	暫存器、kernel stack、排程狀態
共享範圍	同一 process 的位址空間及資源	由 clone flags 決定 mm、files、fs 和 signal handlers

在指定的雙執行流例子中，Windows 有 1 個 process、2 個 thread 與 2 個 TEB。Linux 若用一般 fork，則有 2 個 task、2 個 mm；若以題設的 thread-group clone 共享 mm，則有 2 個 task、1 個 mm，但各自有 kernel stack。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建立呼叫後的狀態與版本反例

Windows 案例中，每個 thread 有獨立的暫存器、user／kernel stack 與 TEB，但引用同一個 process 和位址空間。第一條 thread 已由 `CreateProcessW` 建立，新增一條後總數才是 2。

Linux 的一般 `fork` 建立 child 自己的 task、kernel stack、mm 與頁表；某些私有頁面可先透過 COW 共享實體 frame，這不表示兩者共享 task 物件。對仍共享的 COW 頁面寫入時，核心處理寫入保護 fault，必要時才建立私有實體副本；若符合重用條件，不一定發生複製。

指定的 thread-group clone 包含 `CLONE_THREAD`：兩個 task 的 TID 不同、TGID 相同。Linux 3.2 中 `CLONE_THREAD` 要求 `CLONE_SIGHAND`，而後者要求 `CLONE_VM`。即使只使用 VM／FS／FILES／SIGHAND 共享資源，缺少 `CLONE_THREAD` 也不能推論兩者屬於同一 thread group。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建立呼叫後的狀態與版本反例 （續）

共用位址空間不代表衝突的 heap 寫入自動安全，仍須同步。成功 exec 改變程式映像；特定版本在不可回復點前的錯誤與之後的失敗，處理方式可能不同，不能保證所有失敗都能繼續執行原映像。

從結果、完成契約與依賴評估執行緒

計算 1 至 8 的和可以由單一執行緒完成，也可以分成四份或使用 fork-join；答案都應是 36。平行版本還須確認部分結果存活、依完成契約同步，並在前驅完成後才合併。建立執行緒本身不能保證結果正確。

固定工作量的 Amdahl 模型使用基準時間比例；work／span 下界來自工作總量與依賴路徑；carrier 數則限制可供核心排程的實體。這三者描述不同限制，不能由其中之一推論「執行緒數等於核心數必然最佳」或「超過平均平行度就一定沒有收益」。

實作改變哪一項，便重新檢查哪個前提

變更	需要重新檢查的條件
從 1:1 改成 M:N	阻塞時 carrier 是否補償，以及 runtime 的排程規則
從各自暫存區改成共享資料	衝突存取、同步與資料所有權
從 join 改成 detach	完成通知與參數、結果的生命週期
多執行緒行程呼叫 fork	子行程保留哪些執行緒，以及遺留鎖的處理
加入 cancellation	正常、錯誤與取消路徑的資源清理和資料不變量

比較系統效能時，應固定輸入及正確結果、可用核心與記憶體配置，明定計時範圍，再量測吞吐量與延遲分布。平均吞吐量可能掩蓋鎖競爭或工作池耗盡造成的長尾，也應記錄失敗與等待情況。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：在任意交錯下，如何保證共享狀態正確，同時保留進度與等待上限？

前置知識。 process 與 thread 狀態、共享記憶體、交錯與原子操作。

學習目標

- 證明 mutual exclusion、progress 與 bounded waiting
- 用 semaphore 建立明確順序與背壓
- 辨識 deadlock、starvation、priority inversion 與 lost wake-up

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

競態條件與共享資料

多個執行流同時存取共享資料時，若至少一方會修改資料，最終結果可能取決於操作的交錯順序。這種時序造成的不確定性稱為 race condition（競態條件）。判斷程式是否正確，要先說清楚共享狀態應維持的不變量，以及哪些操作必須協調。

例如 producer 對 `counter` 加一，consumer 對它減一；若兩次操作都完整執行，且期間沒有其他修改，期望值應與初值相同。但一個「加一」或「減一」可能分成讀取、計算、寫回：兩者都讀到舊值時，較晚的寫回可能覆蓋較早的結果。

以下範例使用抽象機器：每一步讀取或寫回都可單獨觀察，同一執行流的步驟保持先後順序。這不代表 C/C++ 中未同步的非原子共享操作有固定結果；若構成 data race，其行為未定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交錯執行：為何 counter 變成 4？

初始 `counter=5`。Producer 要加一，consumer 要減一；兩者各自先讀取、在暫存器計算，最後寫回。

時間	Producer	Consumer	counter	說明
T0	`R1=5`	—	5	producer 讀到 5
T1	`R1=6`	—	5	加一的結果尚未寫回
T2	—	`R2=5`	5	consumer 也讀到舊值
T3	—	`R2=4`	5	減一的結果尚未寫回
T4	`counter=6`	—	6	producer 寫回
T5	—	`counter=4`	4	consumer 覆蓋了 producer 的更新

若交換最後兩次寫回，結果是 6；若其中一個完整操作在另一個開始前完成，則結果是 5。此例的問題不是某一步算錯，而是兩個複合操作沒有受到共同同步協定保護。

何時需要同步

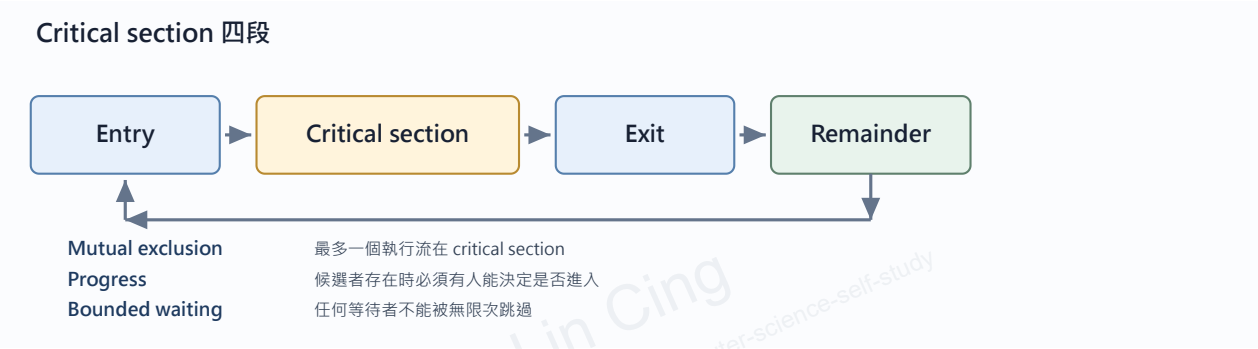
分析共享狀態時，可依序問四個問題：

1. 哪些可變位置會被多個執行流存取？是否有衝突的讀寫？
2. 抽象狀態要維持什麼不變量？例如 `0 ≤ size ≤ capacity`，或「有 writer 時不能有 reader」。
3. 哪些操作需要多個步驟，期間可能暫時破壞不變量？
4. 哪些觀察者不能看見中間狀態？使用什麼鎖或其他協定保證互斥、順序與資料可見性？

單一原子操作只保證它自己的不可分割性。例如更新鏈結串列時，即使修改 `next` 與 `size` 的兩個 store 各自原子，另一執行流仍可能在兩次 store 之間觀察到不一致。保護多欄位關係需要涵蓋所有相關存取的協定。

三個要求的精確界線

每個執行流反覆經過 entry section、critical section、exit section 和 remainder section。進入協定的三項要求各自回答不同問題：



要求	意義	違反時可能發生
Mutual exclusion（互斥）	同一時刻至多一個執行流在受保護的 critical section	兩個操作同時破壞共享不變量
Progress（進展）	臨界區空閒且有人要進入時，只由相關候選者參與決定，不能無限期延後選擇	臨界區空著，候選者卻一直無法決定誰進入
Bounded waiting（有界等待）	某執行流提出進入請求後，在其獲准之前，其他執行流能進入的次數有有限上界	系統持續工作，但特定等待者一直被插隊

三個要求的精確界線 （續）

互斥是安全性；進展與有界等待涉及活性及公平。證明活性時也要交代執行流會繼續被排程、已進入的臨界區會結束等假設；「其他人最多進入有限次」本身不是等待時間的毫秒上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三個彼此獨立的反例

互斥成立，但沒有 progress。 兩人使用嚴格輪替的 `turn`，每人只能在輪到自己時進入。若輪到 P1，但 P1 在 remainder section 不想進入，P0 即使想進入且臨界區空著，也只能等待 P1 改變 `turn`。

互斥與整體 progress 成立，但缺乏 bounded waiting。 不公平的鎖每次空出後總能讓某個競爭者取得，但同一位等待者可能被無限次越過。系統一直有進展，該等待者仍可能飢餓（starvation）。

沒有互斥。 entry section 不做任何檢查，所有候選者可立即進入；等待很短，卻可能同時修改共享資料，安全性立即失效。

這些例子說明，看到「有人能進入」不能當作每個人都有等待上界的證明；看到「沒有人等待」也不能代替互斥證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

進入與離開協定的檢查

對進入協定 `enter_i` 和離開協定 `leave_i`，依序檢查：

1. 對所有允許的相對速度與交錯，是否都不會有兩個執行流同時位於同一受保護的臨界區？
2. 臨界區空閒且存在候選者時，選擇是否能完成，而不是由不想進入的執行流阻擋？
3. 某個請求提出後，其他執行流能在它之前進入幾次？是否存在有限上界？
4. 協定依賴哪些原子操作、記憶體順序、排程與持鎖者完成假設？錯誤或取消路徑是否仍符合契約？

有限測試可以找到反例，不能枚舉任意時間與所有交錯。安全性通常要對所有可達狀態證明；活性與公平需另行列出必要的環境假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Peterson 演算法的完整協議

Peterson 演算法只處理兩個執行流。令 `j=1-i`，共享 `flag[0]=flag[1]=false` 與變數 `turn`。此處先假設共享讀寫原子且具有 sequential consistency（順序一致性）。

```
P_i:
  flag[i] = true
  turn = j
  while (flag[j] && turn == j) { /* 等待 */ }
  critical_section_i
  flag[i] = false
  remainder_i
```

`flag[i]` 表示 Pi 想進入或仍位於臨界區；`turn` 在兩者都想進入時決定哪一方先等待。它不是鎖的持有者欄位：若另一方沒有競爭，某執行流即使最後寫了 `turn`，也可能直接進入。

互斥性證明

假設所有 `flag` 與 `turn` 的存取都原子且順序一致，反設 P0、P1 這兩次臨界區重疊。設 W0 是 P0 本輪寫入 `turn=1`，W1 是 P1 本輪寫入 `turn=0`；順序一致性保證兩者有先後。

若 W0 在 W1 之前，P0 的 `flag[0]=true` 也已在 W1 之前發生。當 P1 通過 while 時，P0 尚未離開本輪臨界區，因此 `flag[0]` 仍為 true；最後的本輪 `turn` 寫入為 W1，值為 0。P1 的等待條件 `flag[0] && turn==0` 成立，P1 不可能通過，矛盾。

若 W1 在 W0 之前，對稱地可得 P0 的等待條件成立，同樣矛盾。因此兩個臨界區不可能重疊。證明所比較的是兩次重疊嘗試中的寫入；進入臨界區後，該執行流直到離場才清除自己的 flag。

進展、有界等待與記憶體順序

Progress。 若只有 Pi 想進入，對方 flag 為 false，Pi 不必等待。若兩人都想進入，在順序一致的模型中 `turn` 為 0 或 1，等待條件不會同時擋住兩人；已在臨界區者離開時清除自己的 flag。

Bounded waiting。 從 Pi 完成 `flag[i]=true; turn=j` 起算，Pj 至多再進入一次。Pj 離開後清除 flag；若它再次競爭，必須先把 `turn` 設為 i，讓仍在等待的 Pi 不再因 `turn` 而受阻。若只從設旗標起算，Pi 可能尚未寫 turn，而 Pj 可能在旗標寫入之前已通過檢查；這時不能直接套用一次的界線。要推論 Pi 最終執行，仍需它和已進入者得到執行機會、臨界區能結束。

記憶體模型。 上述論證依賴順序一致的共享原子存取；在 C/C++ 中使用未同步的普通共享變數可能構成 data race，不能直接套用證明。可使用各相應位置的原子操作與 `memory_order_seq_cst`，或證明其他實作提供等價的順序與可見性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bakery 演算法：同號也能決定先後

有 n 個固定參與者，每人最多同時提出一個請求。`choosing[i]` 初始為 false，`number[i]` 初始為 0；0 表示尚未排隊。此處為便於證明，所有共享變數讀寫均假設為 sequentially consistent 的原子操作，票號不溢位。這是較強記憶體假設的教學版本；Lamport 原始論文討論的讀寫模型更弱。

```
choosing[i] = true
number[i] = 1 + max(number[0..n-1])
choosing[i] = false
for each j != i:
  while choosing[j]: wait
  while number[j] != 0 and (number[j],j) < (number[i],i): wait
critical section
number[i] = 0
```

配對使用字典順序：先比票號，票號相同再比身分。掃描 max 不是一個不可分割的快照：兩人可以同時看到最大值 0，兩人都選 1。若 P0、P1 都完成取號且分別為 (1,0)、(1,1)，P0 排在前面，P1 必須等待 P0 將自己的號碼清為 0。

Bakery：為何需要 choosing？

`choosing[j]` 表示 Pj 正在選號；觀察者先等它結束，才能依它已發布的號碼做比較。若省略這一步，初始全為 0 時可能出現：

修正：P1 先等 choosing[0] 結束，再比較票號

正確協議中 P1 會因 `choosing[0]=true` 停住，等 P0 發布結果後發現 (1,0) 優先。互斥證明依每人檢查對方的選號完成與票號順序，不能只用「取號必然不同」解釋。

當 Pi 完成取號後，已有或正在選號的其他請求至多 n−1 個；各參與者重新請求時，若 Pi 的號碼仍有效，新號碼須排在 Pi 之後。要推得 Pi 最終進入，還要假設選號／臨界區最終完成、相關執行緒持續獲得排程；這不是固定時間上限。

Bakery：一般 n 人的互斥證明

反設 Pi 和 Pj 的兩次請求同時在臨界區。Pi 檢查 Pj 時，Pi 的號碼已發布，直到 Pi 離開臨界區前都不變。

若 Pj 在 Pi 檢查它時已發布本輪非零票號，Pi 等到 `choosing[j]=false` 才比較。要讓 Pi 通過，必須 `(number[i],i) < (number[j],j)`。若 Pj 尚未完成對 Pi 的檢查，嚴格全序使 Pj 必須等待。若 Pj 早已檢查並忽略 Pi 的本輪請求，則它的選號已發布且會維持到離開；Pi 後來選號時會讀到 Pj 的號碼，選出更大的值，反而不能通過。若兩人選號交疊，`choosing` 會讓檢查等待選號完成，再以配對次序判定。

另一種時序是 Pi 讀到 `choosing[j]=false` 後，Pj 才開始選號，而 Pi 接著讀到 `number[j]=0`。兩次讀取並非原子快照，這種交錯可以發生。不過 Pj 是在 Pi 完成取號後才開始；若兩人臨界區重疊，Pj 掃描時 Pi 的非零票號仍有效，故 Pj 的新號碼嚴格較大，必須等待 Pi 清號。若 Pi 讀取 `choosing[j]` 時 Pj 已在選號，Pi 會等該次選號結束；旗標不保證 Pj 在 Pi 檢查之後不會開始新請求。

兩個情況都與同時進入矛盾，因此在所列 SC、無溢位和單一未完成請求假設下，任意 n 個參與者滿足互斥。

test-and-set 與 compare-and-swap

`test_and_set(L)` 原子地讀取 L 的舊值，並將 L 設成 true；`compare_and_swap(L,0,1)` 則只有在舊值為 0 時才寫入 1，並回傳舊值。下列是以 test-and-set 表示的抽象 spinlock：

```
acquire(L):
  repeat old = test_and_set(L) until old == false

release(L):
  L = false
```

假設 L 初值為 false、只有成功將 false 轉為 true 的執行流取得鎖，而且每次只有持有者在完成臨界區後釋放：第一次操作可讀到 false；在它釋放前，其他原子 test-and-set 只能讀到 true。因此同時最多一個持有者。

這個互斥推理不自動提供資料可見性：實際程式中的成功 acquire 與 release 寫入還必須具有適當的記憶體順序，才能讓下一個持有者看見受保護資料。

spinlock 的邊界

在 spinlock 中，等待者反覆嘗試，不主動睡眠。若臨界區極短、持鎖者能在另一顆核心繼續執行，這可能省下睡眠與喚醒的成本；長時間等待、單核心或持鎖者無法執行時，則可能浪費 CPU，甚至阻礙持鎖者前進。

單核心上的典型危險：程式持有鎖時被中斷；中斷處理常式又自旋等待同一把鎖。被中斷的持鎖者要等中斷返回後才可執行，處理常式卻等它釋鎖，因而無法前進。中斷環境需依系統規則處理，例如使用適當的中斷遮罩與非睡眠同步原語。

簡單 test-and-set 鎖只說明誰能成功取得，不保證等待者的 bounded waiting；特定等待者可能一直競爭失敗。要保證公平性，需另外設計及證明排隊／交接規則。

原子操作與 acquire／release 的區別

原子讀改寫保證該操作不被觀察為一半；鎖還需處理它保護的其他資料何時可見，以及編譯器或處理器可否重新排序。

同一把鎖上，前一持有者先更新共享資料、再執行 release；下一持有者成功執行與之配對的 acquire 後，才能依語言與 API 的同步契約讀取那些更新。所有共享存取都必須遵守相同協定；普通共享旗標不能取代原子與記憶體順序。

CAS 單獨只能提供指定位置的原子比較與更新。若用於 lock-free 結構，還要另行處理 ABA、節點生命週期和記憶體回收，以及系統與個別操作的 progress 保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mutex 的使用契約

Mutex 提供互斥及其 API 定義的同步契約。凡是由同一不變量約束的共享存取，都必須遵守對應的鎖規則：

```
acquire(m)
  更新或讀取受 m 保護的共享狀態
release(m)
```

具所有權語意的 mutex 通常要求由持有者解鎖；semaphore 的 signal 通常不要求呼叫者是先前 wait 的執行流。不同 API 的錯誤檢查與限制須依其契約確認，不能只憑名稱假設。

等待 mutex 的實作可以阻塞、短暫自旋或混合兩者；被喚醒只表示有機會繼續競爭，是否能立即取得鎖取決於實作契約。例外、錯誤和取消路徑也必須維持正確的所有權及釋放次數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

依固定順序取得鎖

為所有可能同時持有的鎖指定嚴格全域順序，例如 `ord(A)=1`、`ord(B)=2`；每個執行流只在已持有的所有鎖都小於新鎖時，才可要求新鎖。

假設仍出現等待環。對環上的每個執行流，它持有某把鎖、正等待另一把，因此等待的鎖順序都必須比它所持有的鎖大。沿整個環走一圈便得到 $\mathrm{ord}(L_1) < \mathrm{ord}(L_2) < \dots < \mathrm{ord}(L_1)$，與嚴格順序不可能小於自身矛盾；因此遵循此規則的鎖等待邊不能形成環。

此結論依賴所有相關路徑都遵循同一順序；callback、鎖升級、例外與取消路徑也不能例外。動態物件可用穩定且唯一的 key 排序後取得鎖；只按轉帳的 from／to 參數順序取得兩把鎖，仍可能出現相反順序。本論證排除這類鎖循環，不能因此保證排程公平或排除其他資源形成的死結。

Semaphore 的兩種實作

資源型 semaphore 以 permit 表示可使用的資源。抽象的 binary semaphore 最多保有一個可用 permit；另一種帶號誌計數器的 blocking 實作可用負數表示等待者，內部整數不能直接當成抽象資源數。

Busy-wait 版本的關鍵是「確認有 permit 並扣減」必須是**同一個原子動作**。以下以原子 CAS 描述概念，假設 S 的所有操作均有正確的原子性與可見性；資源型 signal 也不得超出約定容量：

```
wait(S):
  loop:
    old = atomic_load(S)
    if old > 0 and CAS(S, old, old-1) succeeds: return
signal(S): atomic_fetch_add(S, 1)
```

若只寫 `while(S<=0) {}; S--` 而沒有原子檢查及扣減，S＝1 時兩個執行緒都可能先看到 1，然後都通過；只有一個 permit，卻產生兩個持有者。真實實作還須選擇相應的記憶體順序與處理上限。

阻塞式 semaphore：等待者與 permit 交接

下列是特定的 signed-counter 教學模型：更新計數、登記等待者及提交睡眠須與 signal 原子協調；被選中的等待者取得保留的 permit，即使尚未排到 CPU，新來者也不能取走該 permit。

```
wait(S):
  原子地 S--；若 S < 0，登記並阻塞
signal(S):
  原子地 S++；若 S <= 0，選出等待者並交付 permit
```

容量 1 的例子：T0 `wait` 使 S：1→0，取得 permit；T1 `wait` 使 0→−1 並等待；T0 `signal` 使 −1→0，選中 T1 並保留 permit 給它。T1 恢復時從原 wait 返回，不再扣一次；它之後 `signal` 才使 0→1，留下可用 permit。

穩定邊界下可用 permit 為 max(S,0)，佇列中等待者為 max(−S,0)；已交接但尚未執行者的 permit 已分配。這是特定實作，不是所有 POSIX semaphore 的內部表示。資源型 semaphore 可用「空閒＋已分配＝容量」檢查；事件型 semaphore 不一定有固定容量。

有界緩衝區的操作順序

令 `empty=3`、`full=0`、`mutex=1`，buffer 為空。下列是合法事件的前後狀態。

已完成的操作	empty	full	buffer	可觀察事實
初始	3	0	`[]`	三個空槽
put A	2	1	`[A]`	一項已發布
put B	1	2	`[A,B]`	`empty+full=3`
put C	0	3	`[A,B,C]`	下次 put 必須等待
get A	1	2	`[B,C]`	取出後釋放容量
put D	0	3	`[B,C,D]`	容量仍為 3

producer 應 `wait(empty); wait(mutex); ...; signal(mutex); signal(full)`。consumer 對稱地先 `wait(full)`。先取得 mutex 再等資源可能持鎖睡眠，令能產生該資源的對方也卡在 mutex。

死結、飢餓與 priority inversion

設 S、Q 初值皆為 1。P0 先取得 S、P1 先取得 Q；若 P0 接著等 Q、P1 等 S，兩者各自持有對方需要的資源，形成 deadlock。相反地，系統仍不斷完成工作，但特定等待者一直被跳過，是 starvation。

低優先權 L 持有高優先權 H 所需的鎖時，若中優先權 M 不斷搶占 L，H 會間接等待 M，這是 priority inversion。Priority inheritance 可暫時提高 L 的有效優先權，讓它有機會完成並釋鎖。

單鎖且沒有其他 donation 的簡化例子中，釋鎖後 L 可恢復原優先權；若它仍持有其他鎖、存在其他高優先權等待者或鏈式繼承，必須依剩餘等待關係重新計算，不能一律恢復原值。繼承改善特定優先權反轉，仍不等於一般的公平或無死結保證。

有界緩衝區的正確性條件

容量為 n 的緩衝區使用三個不同的同步物件：`empty` 限制可用空槽，`full` 限制可取出的項目，`mutex` 則保護槽位、`head`、`tail` 與 `size` 的複合更新。安全條件包括 `0 ≤ items ≤ n`；producer 先等待 `empty` 才取得 mutex，consumer 對稱地先等待 `full`。若 producer 持有 mutex 卻等待空槽，consumer 可能因無法取得 mutex 而不能取出資料，形成等待環。

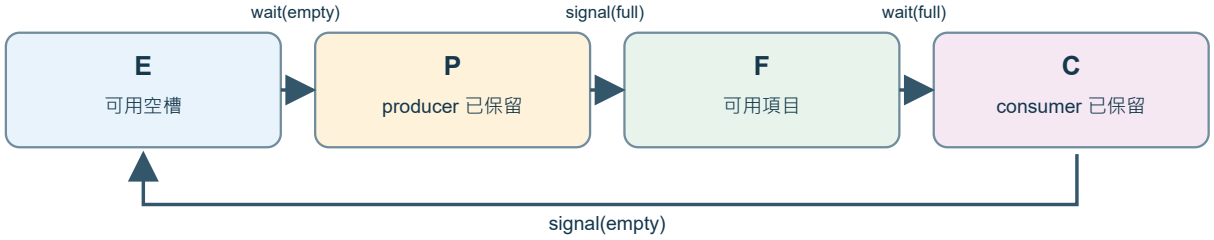
一般記帳必須計入已授權但尚未完成的操作。令 E、F 是尚未保留的 empty、full 可用 permit；P 是已取得 empty permit、尚未完成 `signal(full)` 的 producer reservation 數；C 是已取得 full permit、尚未完成 `signal(empty)` 的 consumer reservation 數。直接交給已喚醒但尚未執行者的 permit，自授權時就算入 P 或 C。

在正常執行且每次保留都會依協定完成的模型中，各原子協定轉移邊界維持 `E + F + P + C = n`。Producer 成功等待將 E 轉成 P；發布 full 時，P 轉成 F，或直接交給等待中的 consumer 而轉成 C。Consumer 對稱地將 F 轉成 C，釋出 empty 時再轉成 E 或直接轉成 P。寫入或取出資料本身不改變這項 permit 分類。

例如 n＝3，初始 `(E,F,P,C)=(3,0,0,0)`；一個 producer 剛取得 empty permit、尚未寫入時為 `(2,0,1,0)`，此時 `E+F=2`，不是 3。只有沒有在途 reservation 的靜止邊界才有 `E+F=n`。E、F 是非負可用 permit，不能直接拿可能含等待者的 signed semaphore 原始計數代入；F 在任意時刻也不一定等於 items。若加入取消或失敗，還須設計 reservation 的回復規則。

在途保留如何維持容量守恆

容量 3：permit 在四種狀態間流轉



直接交棒：P 可直接轉 C；C 可直接轉 P

事件	E	F	P	C	說明
空緩衝區	3	0	0	0	三個空槽尚未保留
producer 成功 wait(empty)	2	0	1	0	空槽已保留，尚未發布項目
producer 完成寫入並 signal(full)	2	1	0	0	一個項目可供取得
consumer 成功 wait(full)	2	0	0	1	該項目已保留，尚未釋放空槽
consumer 取出並 signal(empty)	3	0	0	0	回到空緩衝區

每一列 E＋F＋P＋C 均為 3。圖中箭頭描述 permit 的分類，而非每個資料複製步驟；直接交棒時可由 P 直接轉 C，或由 C 直接轉 P。取消／故障路徑須另外回復或完成已保留的授權。

讀者與寫入者的公平策略

策略	新 reader 遇到等待中的 writer	可能長期等待者
Reader-preference	writer 尚未取得資源時，新 reader 仍可加入	writer
Writer-preference	writer 開始等待後，新 reader 不再開始	reader

Reader-preference 的典型協議用 `read_count` 記錄目前讀者數，並以同一把 mutex 保護計數更新。第一個 reader 以 wait 取得保護共享資料的 `rw_sem`；最後一個 reader 以 signal 釋放它，允許 writer 進入。

第一位與最後一位 reader 可能不是同一個執行緒，所以這裡的 `rw_sem` 必須是允許跨執行緒 signal 的 binary semaphore 等無 owner permit；不能直接把它換成只允許持有者解鎖的 POSIX mutex。保護 `read_count` 的 mutex 則由取得它的 reader 自己釋放。

哲學家進餐：互斥與死結

五位哲學家共用五支筷子，每人需要左、右兩支才能進食。若一律先拿左筷，再拿右筷：

```
wait(chopstick[i]);
wait(chopstick[(i+1)%5]);
eat;
signal(chopstick[(i+1)%5]);
signal(chopstick[i]);
```

可能的交錯是：五人各拿到自己的左筷，再各自等待已被鄰居持有的右筷。每個人都等待另一人先釋放，形成五節循環等待；每支筷子雖然仍保持互斥，系統卻無法前進。

常見設計包括：限制最多四人同時進入取筷協定；用受保護的協定只在兩支筷子均可取得時才授權進食；或者讓奇偶哲學家使用相反取得順序，破壞這個環形等待。以第一種方法為例，若四人各持左筷，仍有一支筷子未被拿作左筷；相鄰的一位可取得第二支並完成，前提是可執行者最終被排程、進食後會釋放。

排除這個循環等待不等於保證每人都有有限等待時間。是否會飢餓仍取決於等待者選擇、排程與反覆競爭的公平性。

等待的是條件，不是一張通知票

採 Mesa 語意；共享佇列 `q` 初始為空，P 只放入一筆 A，C1、C2 各呼叫一次 `get()`。佇列的所有修改和 `q.empty()` 檢查均持有同一把 mutex `m`，`not_empty` 是 condition variable。

```
get():                          put_A():
  lock(m)                         lock(m)
  while q.empty():                q.push_back(A)
    wait(not_empty, m)            signal(not_empty)
  x = q.pop_front()               unlock(m)
  unlock(m)
  return x
```

`wait` 相對於其他執行緒取得 m 並通知的行為，原子地完成釋鎖與進入等待的交接；成功返回前必須重新取得 m。`signal` 不會立即把鎖交給等待者，也不為它保留 A。沒有等待者時，通知本身不會累積成可供將來取用的 permit。

被通知，不代表下一個拿到鎖

時間向下；**P** 只放入一個 **A**；本例沒有額外喚醒

	Consumer C1	Producer P	Consumer C2	mutex / queue
T0	wait：釋鎖 + 入 CV			m=—, q=[]
T1		lock；放入 A		m=P, q=[A]
T2	被通知；尚未持鎖	signal C1；仍持鎖		m=P, q=[A]
T3		unlock →	lock；取 A；unlock	m=—, q=[]
T4	重取鎖；發現 q 空			m=C1, q=[]
T5	while → 再 wait			m=—, q=[]

依圖追蹤：C1 先在空佇列上等待；P 放入 A 並通知 C1，但仍持有 m。T3 依序是 P 釋鎖、C2 先取得 m 並取走 A，然後 C2 釋鎖；此例不假設 mutex 依 FIFO 選取等待者。

C1 在 T4 從 wait 返回時確實重新取得 m，但佇列已再次變空。這不是 spurious wakeup：若只用 `if` 檢查一次，C1 會對空佇列執行 `pop`；使用 `while` 則會重新檢查並在 T5 繼續等待。

把三種狀態分開記錄

下表中的 CV 是 condition variable 的等待佇列；E 是已收到通知、但尚未重新取得 `m` 的執行緒集合，並不是核心 ready queue。表格依序記錄每個事件完成後的狀態。

事件	m 的持有者	佇列 q	CV	E
T0：C1 進入 wait	無	`[]`	`{C1}`	`{}`
T1：P 放入 A	P	`[A]`	`{C1}`	`{}`
T2：P 通知 C1	P	`[A]`	`{}`	`{C1}`
T3：C2 取走 A 並釋鎖	無	`[]`	`{}`	`{C1}`
T4：C1 重新取得 m	C1	`[]`	`{}`	`{}`
T5：C1 再次 wait	無	`[]`	`{C1}`	`{}`

T2 的通知改變 C1 的等待資格，但不保留佇列中的 A；是否取得 mutex 及何時執行仍是另外的狀態。

遺失喚醒：等待之前的空窗

錯誤：自行拆開 unlock 與等待

1. C1：檢查空，unlock；尚未入 CV
2. P：放 A、signal；沒有等待者
3. C1：現在才入 CV，睡著；q=[A]

有資料卻睡著；通知不會累積。

正確：wait 的原子契約

1. C1：檢查空，wait 釋鎖並登記
2. P：取得鎖，放 A、signal，再 unlock
3. C1：重取鎖，重檢非空，取走 A

P 先取得鎖時，C1 看到 A 就不 wait。

左圖是假設某個錯誤的等待實作先解鎖，再另外登記並阻塞；它不是合法的 `pthread_cond_wait` 實作契約。P 可能在這個空窗更新條件並通知，但當時 CV 還沒有等待者；C1 隨後才睡著，若再無事件就可能永久等待。

正確的 condition wait 以原子交接排除此空窗。若 P 先完成，且 A 尚未被其他 consumer 取走，C1 在持有 m 時看到佇列非空，就不等待；若 A 已被取走，C1 看到空佇列並繼續等待是正確行為。通知沒有累積成獨立的「票」，所以等待者始終應依共享條件判斷。

分開證明安全性與活性

安全性：每次 `pop` 前都在持有 m 的情況下確認 `!q.empty()`；從檢查到取值之間，其他執行緒不能取得 m 改變 q。Wait 成功返回只保證重新取得 mutex，並不保證條件仍成立。

活性：`while` 防止錯誤取值，卻不保證每位 consumer 都能取得資料。本例只有一筆 A，若 C2 已取走，C1 繼續等待是合理結果。要證明某個等待者最終完成，還需條件再次成立、相應通知、排程與鎖取得等假設；不能只由 `while` 推出。

若 broadcast 喚醒兩位 consumer，但只有一筆 A，先取得 m 的一人取走 A；另一人仍須在重新取得 m 後檢查條件並等待。通知更多人不會增加資料，也不自動建立有限等待上界。

Hoare 與 Mesa 如何交棒

前述反例使用 Mesa：通知者繼續持有鎖並執行，等待者之後才重新取得鎖。不同 monitor 的交接契約不同，不能把某一模型的條件保證直接套用到另一模型。

交接方式	signal 後的執行權與鎖	等待者如何判讀
Hoare signal-and-wait	立即交給被通知者，通知者等待	只有在 signal 前已建立目標條件，且模型排除虛假喚醒、逾時等情況時，才能利用交接點的條件保證
Mesa signal-and-continue	通知者繼續執行並持鎖，等待者之後重取	其他執行緒可能先改變共享狀態，因此必須重檢 predicate
Concurrent Pascal signal-and-exit	通知者離開 monitor 後再交棒	依該語言定義的交接與條件語意判讀

固定版本的同步機制

以下以原書第 9 版討論的 Windows 7、Linux 2.6 和 Solaris 案例為歷史參考。各系統的版本、核心設定與呼叫情境會影響機制；不能把一個案例的內部實作當成所有版本的 API 契約。

案例	同步工具	使用時需檢查的條件
Windows 核心	spinlock 保護短時間共用資料	`KeAcquireSpinLock` 要求呼叫時 IRQL ≤ DISPATCH_LEVEL，將目前處理器的 IRQL 提高到 DISPATCH_LEVEL；配對釋放時恢復舊值
Windows 使用者模式	mutex、semaphore、event 等等待物件；另有 process 內 critical section	等待物件與 critical section 的所有權、可見範圍和喚醒條件不同
Linux 教材案例	atomic 操作、spinlock、mutex 與 semaphore	是否可睡眠，以及是否須額外處理本機中斷或核心搶占，取決於鎖、設定與執行環境
Solaris 教材案例	adaptive mutex、reader-writer lock、turnstile	持鎖者正在其他 CPU 執行時可考慮 spin；否則可睡眠；turnstile 可支援 priority inheritance
POSIX	mutex、condition variable、reader-writer lock 及 semaphore	檢查對應 API 的等待、錯誤、所有權與程序間共享契約

固定版本的同步機制 （續）

`KeAcquireSpinLock` 持鎖期間，一般執行緒排程不能搶占目前處理器上的持鎖路徑，但更高 IRQL 的硬體中斷仍可能發生。ISR 使用的 interrupt spinlock 和 DIRQL 有另一套呼叫規則，不能把 executive spinlock API 直接帶入所有中斷處理常式。持 spinlock 的區段也不能執行可能睡眠的等待。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為何同名 primitive 不能直接互換

Windows 的 user-mode critical section 是同一 process 內的遞迴互斥工具；它不等於同名的核心程式碼區段。可設定 spin count：在多處理器競爭時先嘗試，再依 API 進入等待；單處理器的 spin count 會被忽略。等待執行緒取得所有權的順序不保證 FIFO，實作的等待物件也不應被概括成「必定配置 kernel mutex」。

Linux 教材中的一般 spinlock 用於不能睡眠的短區段；在某些單核心及核心設定下，不需真正跨核心自旋，但仍須處理核心搶占和可能的中斷競爭。`spin_lock`、`spin_lock_irqsave` 的中斷處理效果不同；不能把「停用 preemption」誤當成「所有中斷都被遮罩」。較新 Linux 的 PREEMPT_RT 對 `spinlock_t` 有不同實作與規則，須另按目標版本確認。

比較替代方案時，至少釐清：呼叫發生在 user／kernel／interrupt context；等待期間能否睡眠；鎖是否有 owner、能否跨 process；是否允許重入、取消或跨執行緒釋放；以及 API 對可見性、喚醒次序與錯誤的保證。名稱相似不能替代這些契約。

等待時間與公平。 支援 timeout 只讓呼叫者在條件未滿足時可走逾時路徑，不代表它在期限內必然取得鎖，也不建立 FIFO 或 bounded waiting。實際返回時間還受排程與時鐘等 API 條件影響。

交易記憶體

交易記憶體以 transaction 表示一組共享讀寫：在實作承諾的隔離範圍內，成功 commit 時整組效果生效；abort 時交易記憶體寫入不生效，之後可重試。以下是概念偽碼，`atomic` 不是可直接貼入 C++ 的通用語法。

```
atomic transaction:
  x = x - 25
  y = y + 25
```

例如初始 `x=100, y=40`，成功 commit 後為 `(75,65)`，總和仍為 140；若交易 abort，則這次交易未提交的記憶體更新不生效。在所選 transaction 系統提供的隔離契約下，另一個遵守同一協定的觀察者不會把只減 x、未加 y 的中間狀態當作已提交結果。

並行衝突可能觸發 abort；容量限制、排程或實作限制也可能導致失敗。重試次數與進度須另作設計，交易不保證公平或每次最終成功。I/O、網路傳送等不可逆外部效果通常不能靠回滾記憶體撤銷，必須移出可重試區段或採用明確的補償／提交協定。

STM 通常由 compiler／runtime 記錄並驗證讀寫；HTM 依硬體機制追蹤衝突與資源限制。若加入鎖作 fallback，交易路徑和鎖路徑必須共同遵守同一互斥協定；不能假設不參與交易的普通讀寫自動受到保護。交易式編程可避免某些手寫鎖順序問題，但仍可能反覆失敗或飢餓。

OpenMP、immutability 與實際邊界

OpenMP `#pragma omp critical(name)` 保護其結構化區塊：在同一 contention group 內，相同名稱的 critical 區段一次最多一個 thread 執行，不因 thread 所屬 team 不同而改變。未命名的 critical 區段視為具有同一個未指定名稱；不同名稱不提供彼此之間的互斥。

例如兩個區段都更新 `total`，若分別使用 `critical(A)` 與 `critical(B)`，不能靠這兩個不同名稱保證彼此互斥。所有會衝突的存取必須遵守相同同步協定；單獨保護其中一個寫入點並不足夠。

Immutable value 建立後不再原地修改，因此多個執行流可在有效生命週期內共享同一個舊值；但建立新版本、發布給讀者、物件回收以及檔案、網路或 mutable database 仍可能需要同步。使用 immutable 並不自動保證整個程式沒有競爭。

Message passing 可透過複製或依協定轉移所有權，減少直接共享可變資料；仍須明定通道容量、順序、backpressure、取消和失敗重送。比較方案時也要計入複製、分配與等待成本，而不是只比較語法。

如何選擇同步工具

問題	必要資訊	應檢查
只有一個 critical section	臨界區長度、是否可 sleep、是否有 owner	lock acquire/release 是否成對
有固定數量 resource	capacity、初始 permits、誰可 signal	semaphore 是否可能超額 signal
等待 predicate	predicate 變數、通知條件、喚醒後重檢	CV 是否綁定同一 mutex
多個 locks	全域順序、動態 key、取消路徑	每條路徑是否仍依序取得
多 readers 與 writers	公平版本、writer 是否可等待	是否出現 writer 或 reader starvation

完成選型後，逐事件列出 mode、owned resources、waiting set 與 predicate。只在平均測試中看到正確答案，不能排除少見交錯；一般性證明與可重跑反例要分開提出。

本章重點

- critical section 三性質分別對應安全、活性與公平
- semaphore 與 monitor 的錯誤多在等待條件而非 API 語法
- 經典假設不必然適用弱記憶體或現代硬體

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra’s Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

一條時間線，四個不同的問題

CPU 排程先決定「下一個誰能執行」，再討論公平、效率或期限。**同一張時間表，可以讓平均等待變短，也可能讓某個工作更晚完成。** 本章會逐次列出選擇依據，再計算結果。

學習順序	要能自己回答的問題
狀態與事件	工作還在等 CPU，還是在等 I/O？到達是否一定搶占？
一般排程	FCFS、SJF、SRTF、RR 與回饋佇列，當下各選誰？
多核心與即時	遷移成本、工作相依與期限，如何改變判斷？
系統案例	版本、計時器與權重，如何產生具體時間線？

前置知識是行程／執行緒狀態、上下文切換，以及基本代數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先分清 ready、running 與 blocked

Ready（就緒）表示資源條件已滿足，只差 CPU；running（執行中）表示正在占用 CPU；blocked（阻塞）表示還在等 I/O 或某個事件，此時多給 CPU 也無法繼續。

一次 CPU burst 是工作從可執行狀態算到下次阻塞或完成所需的一段 CPU 需求，搶占只是暫停這段需求，並不把它完成或重算。短期排程器從可執行者中選下一個；dispatcher（分派器）再保存／恢復上下文並移交控制權。

Ready queue 是「可選集合」的概念名稱，實作不一定是 FIFO：也可能是優先佇列或樹。單 CPU 任一時刻至多有一個 running 工作；同一工作不能同時出現在 ready 與 running。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些事件必須換人？哪些只是重新比較？

狀態改變與選取決策是兩個步驟

新到達 → Ready；完成 → 離開系統（圖中省略出口）

Blocked 不在 ready queue；搶占者保留剩餘 CPU 需求。

Running 工作阻塞或完成後，CPU 才能交給另一個 ready 工作；若沒有候選，就閒置。新工作到達或 I/O 完成則只增加候選，**是否搶占，要再看政策與比較鍵**。

非搶占政策讓目前工作執行到阻塞、完成或主動讓出；可搶占政策允許提早收回 CPU。「blocked→ready 發生了」本身不等於已把目前工作趕下 CPU。

切換成本：忙碌，不一定在推進應用程式

單 CPU，P1 先執行 5 ms 後阻塞；假設此時 P2 已就緒、需要 2 ms，且由 P1 切換至 P2 需 1 ms，沒有其他成本：

區間（ms）	CPU 在做什麼？	應用工作是否前進？
[0,5)	P1	是，5 ms
[5,6)	保存／恢復上下文	否，1 ms 成本
[6,8)	P2	是，2 ms

整段沒有閒置，忙碌率為 8/8=100%；但有用工作占比只有 7/8=87.5%。兩個比例的分子不同，不能都稱為「效率」。本例不把 P1 的後續 I/O 完成當成已知，也不宣稱它已完成整個工作。

長期 admission（接納）決定哪些工作進系統；短期排程決定下一個執行者；傳統中期排程可暫停／換出工作以降低記憶體壓力。三者影響的時間尺度不同。

先固定指標，再比較政策

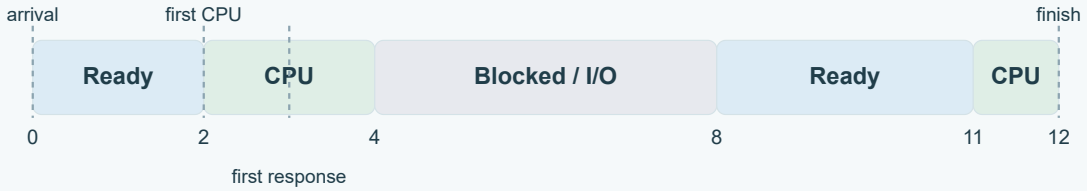
令 a 為到達、s 為首次取得 CPU、o 為首次產生回應、f 為完成時刻。

指標	計算方式	不應包含或混用的量
等待時間 W	所有 ready 區間的總長	不含 CPU 執行及 blocked 時間
周轉時間 T	f−a	包含途中等待與 I/O
首次 CPU 延遲 R_CPU	s−a	不能推定應用程式已輸出
首次回應延遲 R_out	o−a	不等於全部輸出或工作完成

本章數值排程題若只寫 response，固定採 R_CPU。 原書 §6.2 從互動請求的首次回應說明 response；它與第一次分派 CPU 不必同時。題目若量輸出，會明寫 R_out，不把教材的計算慣例當作兩者恆等。

等 I/O 的四毫秒，為什麼不算 ready waiting？

同一工作 A：等 CPU、執行、等 I/O 分開累加



Ready waiting	CPU service	Blocked I/O	Turnaround
2+3 = 5 ms	2+1 = 3 ms	4 ms	12 ms

首次 CPU 延遲 = 2；首次回應延遲 = 3；5+3+4 = 12。

工作 A 在 0 到達，ready [0,2)、CPU [2,4)、blocked [4,8)、ready [8,11)、CPU [11,12)，首次回應在 3。故 W=2+3=5、CPU service=3、blocked=4、T=12，且 **5+3+4=12**。

R_CPU=2，R_out=3。首次取得 CPU 並未立即回應；首次回應時也尚未完成。若把 I/O 的 4 ms 算進等待，會誤得 W=9；只有沒有 blocked 等其他區間時，才可直接用 T−CPU service 求 W。

排程不是背名稱，而是固定選取規則

以下一般排程例子固定單 CPU、每工作一段 CPU burst、已知需求、忽略切換成本。除特別指定外，同鍵按到達順序，再按編號；區間一律寫成 [開始,結束)。

政策	比較什麼？	何時重新選取？
FCFS	ready 入列順序	目前工作讓出 CPU 時
非搶占 SJF	下一段 CPU burst 長度	目前工作讓出 CPU 時
SRTF	尚需的 CPU 時間	到達、完成等事件；更短者可搶占
RR	FIFO 次序與剩餘時間片	完成、阻塞或時間片用完
Priority	明定方向的優先權	依政策決定是否搶占

把 burst 視為已知是教學假設。實機通常只能估計，錯估時不能沿用精確長度下的最適保證。

FCFS：長工作先進來，短工作也得等

P1/P2/P3 都在 0 抵達，CPU 需求為 8/4/2 ms，FCFS 入列順序為 P1、P2、P3。

工作	執行區間	完成 f	等待 W	周轉 T
P1	[0,8)	8	0	8
P2	[8,12)	12	8	12
P3	[12,14)	14	12	14

平均等待 20/3≈6.67 ms，平均周轉 34/3≈11.33 ms。後方短工作一起等長工作，是 convoy effect 的基本現象；若再交錯 I/O，還可能降低裝置與 CPU 的配合效率。

不能因此稱 FCFS 在所有工作負載下「最差」。在每筆前方只有有限工作、每段需求有限的 FIFO 模型中，可證有限等待；沒有到達或服務上界，就沒有共同的數值等待上限。

SJF：同一批輸入，先做短者

沿用 8/4/2 ms，但順序改為 P3→P2→P1。下表仍以 **P1、P2、P3** 排列，避免把執行順序誤當工作編號。

工作	執行區間	完成 f	等待 W	周轉 T
P1	[6,14)	14	6	14
P2	[2,6)	6	2	6
P3	[0,2)	2	0	2

平均等待 8/3≈2.67 ms，比 FCFS 少 4 ms；但 P1 的完成時刻從 8 變成 14。平均值改善，不代表每一個人都改善。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SJF 的交換論證，在哪些條件下成立？

所有工作同時可用、單一 CPU burst、已知長度、非搶占且沒有切換成本。考慮相鄰兩工作 x>y，它們開始前已有共同等待 h。

順序	第一筆等待	第二筆等待	兩筆等待總和
x 再 y	h	h+x	2h+x
y 再 x	h	h+y	2h+y

交換後少 x−y>0；兩筆總執行長度不變，所以後方工作的開始時間不變。反覆消除逆序，得到短者在前的排列，因此等待總和最小。工作數固定，平均等待也最小；總 CPU 需求固定，平均周轉隨之最小。

不同到達、允許刻意閒置、權重、期限或估計誤差，都不能未經推導直接套用此結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不知道下一段 burst，怎麼估計？

以指數平均保存歷史：$\hat{b}_{n+1} = \alpha b_n + (1-\alpha)\hat{b}_n$，其中 b_n 是剛量到的 burst、$\hat{b}_n$ 是先前估計，$0 \le \alpha \le 1$。

α=1/2，初值估計 10 ms	本次量到 b_n	下一次估計
第一次	6	(6+10)/2=8
第二次	4	(4+8)/2=6
第三次	8	(8+6)/2=7

展開遞迴後，舊觀測的係數為 $\alpha(1-\alpha)^j$；$0<\alpha<1$ 時愈舊權重愈小。$\alpha=0$ 完全忽略新觀測，$\alpha=1$ 只用最近一次。

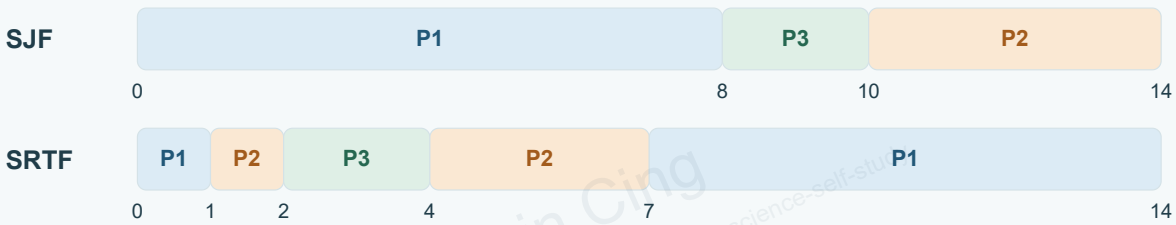
預測不是保證：工作突然進入較長計算階段時，舊歷史可能低估需求。排程器須另外處理公平與飢餓，不能把「估計最短」當成「實際最短」。

SRTF：新來者要和「剩餘量」比較

固定 P1=(到達0,需求8)、P2=(1,4)、P3=(2,2)，單位 ms。同剩餘量時保留 current；否則選最短者。

相同到達與需求，只有是否搶占不同

P1: a=0, b=8 P2: a=1, b=4 P3: a=2, b=2 (ms)



t=1: 4 < 7 t=2: 2 < 3 t=4: P2 剩 3，接著做完

P1/P2/P3 等待：SJF = 0/9/6；SRTF = 6/2/0 ms

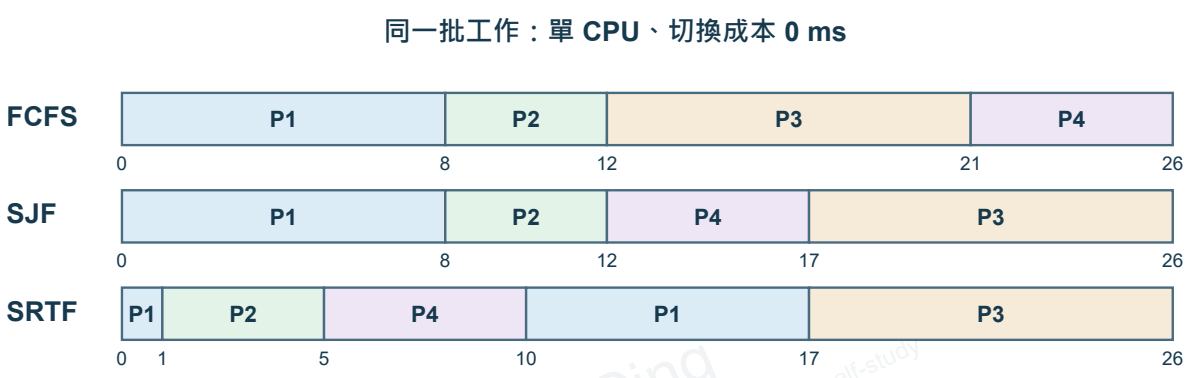
t=1，P1 剩 7，P2 需 4，所以搶占；t=2，P2 剩 3，P3 需 2，再搶占。t=4 P3 完成，剩餘量是 P2=3、P1=7，先恢復 P2；**不是把 P2 從原始 4 重新開始**。

SRTF：逐事件重建時間表與指標

事件時刻	可選者的剩餘量	下一段	理由
0	P1=8	P1 [0,1)	唯一候選
1	P1=7、P2=4	P2 [1,2)	4<7
2	P1=7、P2=3、P3=2	P3 [2,4)	2 最小
4	P1=7、P2=3	P2 [4,7)	3<7
7	P1=7	P1 [7,14)	唯一候選

依 P1/P2/P3，完成為 14/7/4、周轉 14/6/2、等待 6/2/0，平均等待 8/3 ms；首次 CPU 延遲皆為 0。非搶占 SJF 則完成 8/14/10，等待 0/9/6，平均 5 ms。兩者的到達與需求完全相同，差別才可歸因於政策。

同一批工作，三種排程結果



單 CPU、沒有切換成本，每個工作只有一段 CPU burst。輸入為 P1＝(到達 0，需求 8)、P2＝(1，4)、P3＝(2，9)、P4＝(3，5)，單位皆為 ms。FCFS 依到達排序；非搶占 SJF 在每次 CPU 空出時選已到達者中最短者；SRTF 選剩餘時間最短者。

SRTF 在 t＝1 以 P2 的 4 對比 P1 的剩餘 7，因此搶占；P2 於 t＝5 完成時，P4 的 5 小於 P1 的 7 與 P3 的 9，所以接著執行 P4。SJF 不在到達時搶占，因此 P1 先執行至 t＝8。

用完成時間核對等待時間

本例沒有 I/O，故每個工作的等待時間 W＝完成時間 − 到達時間 − CPU 需求。下表依工作編號排列，避免將時間線的執行順序誤當成工作編號。

工作	FCFS 完成／等待	SJF 完成／等待	SRTF 完成／等待
P1	8／0	8／0	17／9
P2	12／7	12／7	5／0
P3	21／10	26／15	26／15
P4	26／18	17／9	10／2
平均等待	35/4＝8.75	31/4＝7.75	26/4＝6.5

例如 SRTF 的 P1 在 [0,1) 使用 1 ms，剩餘 7 ms；直到 t＝10 才恢復，完成於 t＝17，等待 17−0−8＝9 ms。這是指定工作負載的比較；納入搶占成本或不同目標時，不能直接沿用結果。

RR：未完成者才回隊尾

P1/P2/P3 皆在0到達，需求5/2/3 ms，初序P1→P2→P3、q=2、成本0。完成者離開，其他工作用滿片額才回隊尾。

依 P1/P2/P3，RR 完成10/4/9、等待5/2/6、首次 CPU 延遲0/2/4。t=4 的P2已完成，不再入列。SJF平均等待7/3，RR為13/3 ms；兩者工作量皆10 ms。

RR：回應上界與切換成本要分開算

固定 n 筆始終就緒的工作，時間片 q、每次交接成本 c。某工作剛回到隊尾，在再次開始前至多有 n−1 個其他片段；若把到它的交接也計入，等待上界為 **(n−1)q+nc**。這是固定候選集、無更高類別插入且無阻塞的模型上界。

持續有其他工作，c=0.1 ms	q	一個滿片額的有用時間占比 q/(q+c)
較大片額	4 ms	4/4.1≈97.56%
較小片額	0.1 ms	0.1/0.2=50%

q 大到每筆 burst 都能一口氣完成時，RR 對這個單 burst 工作集退化成 FCFS。q 小能縮短輪候，卻增加交接占比；只剩一個 runnable 工作時，時間片到期不一定有跨工作切換，不能照表硬加成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Priority、aging 與「等很久」的差別

優先權數字方向必須先約定；有的系統數字大者優先，有的相反。固定優先權若持續有高優先權工作 ready，低者可能永遠輪不到。

有限追蹤只能證明「到目前仍未執行」，不能證明無限飢餓。反例模型：低者 L 在 0 就緒，高者 H_k 在每個整數 k 到達、各需 1 ms；若高者可完全占滿 CPU，L 永不前進。

Aging（等待愈久逐步提高優先權）必須配合有上限的等級、同級公平、有限前方工作與服務需求等條件，才可推出有限等待；只寫「會提高」不構成上界。即時系統若改動固定優先權，還須重做期限分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多級佇列與回饋佇列：差在工作能不能換級

Multilevel queue 把類別固定分在不同佇列；各級內部及級間都需要選取規則。Multilevel feedback queue（MLFQ，多級回饋佇列）則依執行／等待行為換級。

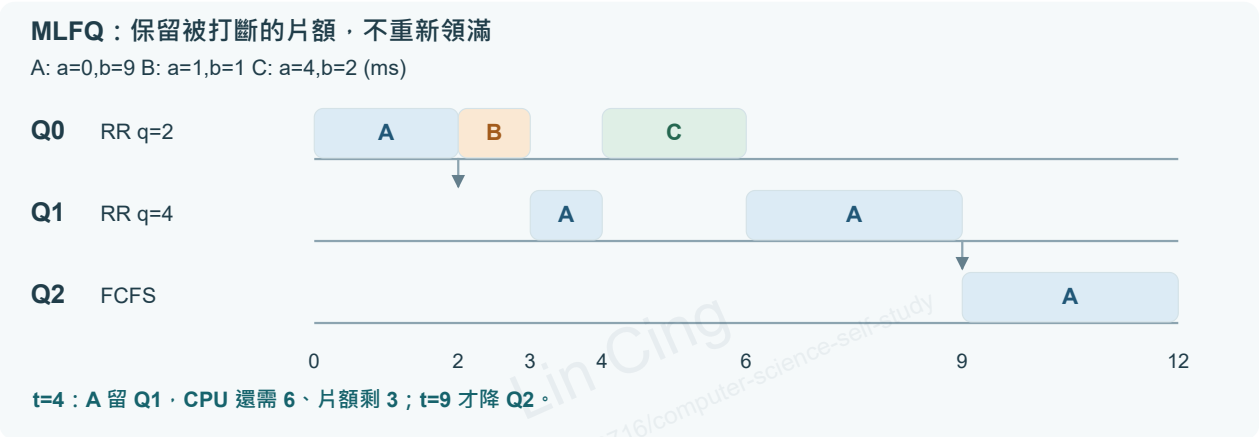
以下採一個明確的教學變體，**不是宣稱所有 MLFQ 都相同**：

規則	本例定義
由高到低	Q0：RR q=2；Q1：RR q=4；Q2：FCFS
新到達	進 Q0；高級非空就搶占低級
同級片額用完	未完成者降下一級；Q2 留原級
被高級搶占	留在原級隊首，保留剩餘片額與需求
升級／阻塞	本例沒有阻塞；有限追蹤內不升級

不同的重新入列、升級與片額重設規則，都可能改變答案。

MLFQ：被搶占不等於片額重新領滿

令 A=(到達0,需求9)、B=(1,1)、C=(4,2)，單位 ms，沿用前頁規則。



A 在 Q0 做 [0,2)，剩 7 並降 Q1；B 在同級不插隊，於 [2,3) 完成。A 在 Q1 做 [3,4) 後，被新進 Q0 的 C 搶占，**剩餘片額是 3，需求是 6，兩者不同**。

C 在 [4,6) 完成；A 回 Q1 做 [6,9)，用完餘下 3 ms，仍需 3，才降 Q2 並在 [9,12) 完成。

MLFQ：每次移動都能核對佇列與預算

表中列事件處理後、分派前的佇列；括號為「剩餘 CPU／該級剩餘片額」，— 表示 Q2 不設片額。

t	Q0	Q1	Q2	下一段
0	A(9/2)	空	空	A 0–2
2	B(1/2)	A(7/4)	空	B 2–3
3	空	A(7/4)	空	A 3–4
4	C(2/2)	A(6/3)	空	C 4–6
6	空	A(6/3)	空	A 6–9
9	空	空	A(3/—)	A 9–12

若每次被高級打斷都重領 Q1 的 4 ms，A 會延後降級，已是另一種政策。沒有 aging／boost 的嚴格級間優先權，也不能保證低級在無限到達下不飢餓。

誰在排程 user thread？誰在排程 CPU？

User-level thread 由函式庫管理；核心選取的是可被核心辨識的執行實體。Process-contention scope（PCS）在行程內分配可用執行實體；system-contention scope（SCS）在系統層競爭 CPU。

映射	T1 阻塞時，T2 能否繼續？	多 CPU 能否同時執行兩者？
Many-to-one	若阻塞唯一核心執行實體，就不能	只有一個核心實體，不能
One-to-one	T2 自己的核心執行實體仍 ready 才能	還要有至少兩個可用 CPU

例如 T2 需 4 ms，T1 等網路；one-to-one 也不能無條件說 T2 於 4 完成，還須假設它從 0 起獨占一個 CPU。映射提供執行的可能性，政策與負載才決定實際時間。

多核心：多給 CPU，還要付資料搬動成本

Processor affinity（處理器親和性）讓工作留在熟悉的快取／TLB 上；load balancing（負載平衡）則把工作搬到較空的 CPU。兩者有取捨，不能只看佇列長度。

同一工作不能被兩個 CPU 同時取走。遷移必須在同步保護下移除舊入列、移交唯一擁有者，再加入目的佇列；已在執行的工作與尚未開始者也不能混作同一種「偷取」。

以下保留一個可完整重算的八核心例子：所有工作在 0 就緒，不可切分，單工作不得同時跑在兩核；每次移交與快取暖機合計 1 ms。這些是假定成本，並非量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八核心：移交待執行工作，能省下多少時間？

固定8cores、所有jobs在t=0就緒、不可切分、單job不得同時跑在兩core。Jobs A/B各8ms、C/D各4ms、E/F/G/H/I/J各2ms，共36ms有用工作。初始CPU0 queue=[A,I,J]；CPU1至7分別只有B、C、D、E、F、G、H。

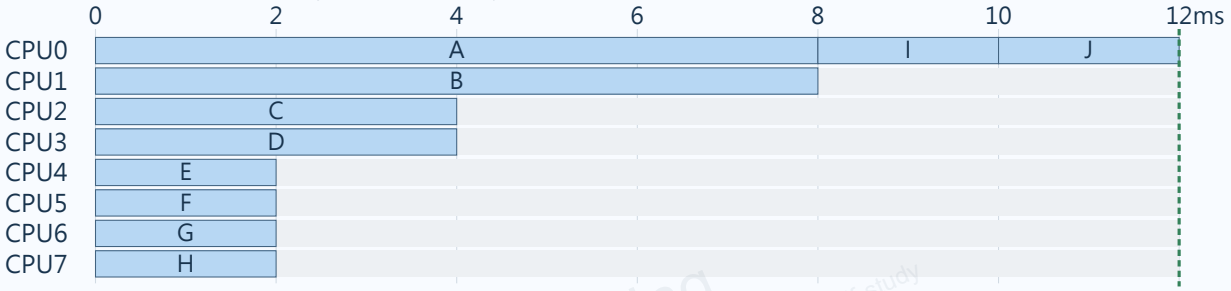
不遷移時CPU0依序A[0,8)、I[8,10)、J[10,12)，makespan=12ms。t=2時CPU4/5已做完E/F；在queue lock下各移除尚未執行的I/J，移交其唯一owner。每次遷移明定0.5ms queue/transfer加0.5ms cache warmup，故兩core各在[2,3)付成本、[3,5)執行I/J。CPU0只執行A至8，makespan降為8ms，speedup=12/8=1.5；有用工作仍36ms，新增成本共2ms。這是給定成本的教學模型，不是實機測量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搬移前：誰在等待？

不遷移：CPU0 的 A→I→J 決定 makespan=12ms

藍=有用工作；灰=idle。所有 jobs 在 t=0 ready；同一時間軸 0–12ms。

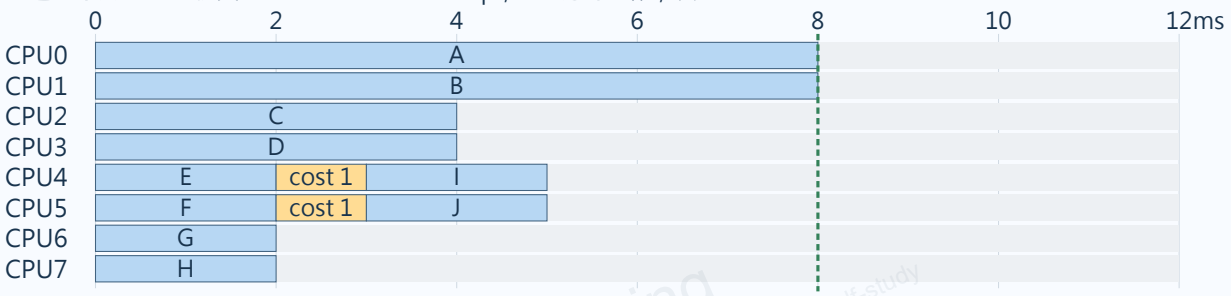


t=2 時 CPU4/5 已完成 E/F，但 I/J 仍在 CPU0 queue 等 A 完成。

搬移後：連同成本重新計時

t=2 遷移 I/J：含成本的 makespan=8ms

橙色=每次 0.5ms 移交 + 0.5ms cache warmup；藍=有用工作；灰=idle。



CPU4/5：成本[2,3)，I/J執行[3,5)；A/B於8完成。有用36ms + 成本2ms。

單job A長8ms給下界T≥8，所以本例達到此下界。若每次遷移成本改為c，makespan=max(8,2+c+2)=max(8,4+c)；c<8才比12ms嚴格改善。資料相依、硬性affinity或不同cache成本會改變模型，不能只因core閒置就宣稱遷移一定有益。

即時排程：先分清 task、job 與期限

週期工作（task）τ_i 反覆產生一筆筆待執行的 job。以下統一使用執行時間上界 C_i、週期 T_i、相對期限 D_i。除非另有說明，本節數例的每筆 job 實際需求均為 C_i。從時刻 0 同步釋放時，第 k 筆 job（k 從 0 起）的釋放時刻與絕對期限為

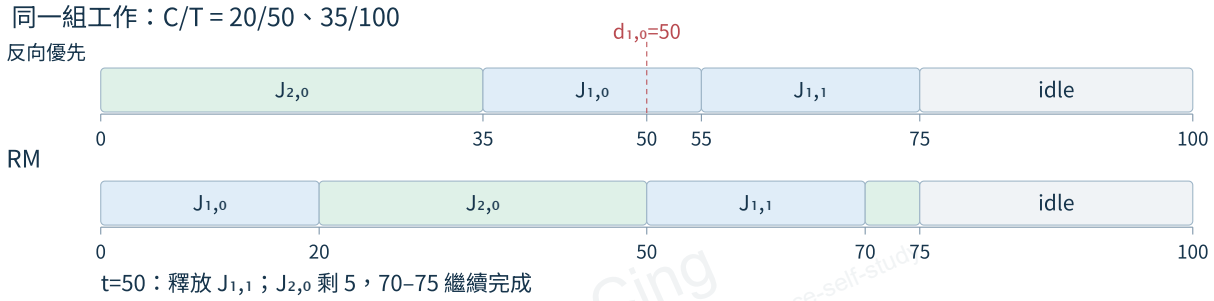
$$r_{i,k} = kT_i, \qquad d_{i,k} = r_{i,k} + D_i.$$

本節基本模型：單 CPU、可立即搶占、工作彼此獨立，沒有鎖阻塞、I/O、自我暫停、切換或中斷成本；除末頁外 $D_i=T_i$。 到達時只把新 job 放進 ready set，未必換人執行。完成恰在期限上算成功；時刻 t 先計入 [前一事件,t) 的執行，再處理 t 的到達。

量	問的問題	不能混用的量
C_i	每筆最多需要多少 CPU 時間？	不是 response time
T_i	同一 task 隔多久產生下一筆？	不是當前 job 剩餘期限
D_i	從釋放起多久必須完成？	絕對期限為 $r_{i,k}+D_i$
$U=\Sigma C_i/T_i$	長期 CPU 需求占比是多少？	不單獨描述短時間擁塞

RM：週期短的 task 優先，不是剩餘工作少者優先

Rate-monotonic（RM）把**較短週期設為較高固定優先權**；新來的高優先權 job 可搶占。令 τ_1=(C_1=20,T_1=D_1=50)、τ_2=(35,100,100)，時間單位均為 ms。



若把 τ_2 排在前面，$J_{1,0}$ 只能在 [35,55) 執行，超過期限 50。RM 則先做 $J_{1,0}$；$J_{2,0}$ 在 [20,50) 已做 30，t=50 被 $J_{1,1}$ 搶占，**剩下的 5 不會重做**，於 [70,75) 完成。

RM 的完成時刻為 $J_{1,0}$=20、$J_{1,1}$=70、$J_{2,0}$=75，對應期限 50、100、100。t=100 是共同週期邊界，舊 job 全已完成；同一 ready 狀態隨後重現，故整個排程可每 100 ms 重複。

RM 利用率界：通過可保證，沒通過要再分析

在本節獨立、可搶占、$D_i=T_i$ 的理想單 CPU 模型，n 個週期 task 的經典**充分條件**為

$$U = \sum_i C_i/T_i \le n(2^{1/n} - 1).$$

n=2 時界為 $2(\sqrt{2}-1)\approx 0.8284$；前例 U=20/50+35/100=0.75，通過。$n\to\infty$ 時界趨近 $\ln 2\approx 0.6931$。這些數字是對所有符合模型工作集的保證界，**不是每個 RM 工作集最多只能用這麼多 CPU**。

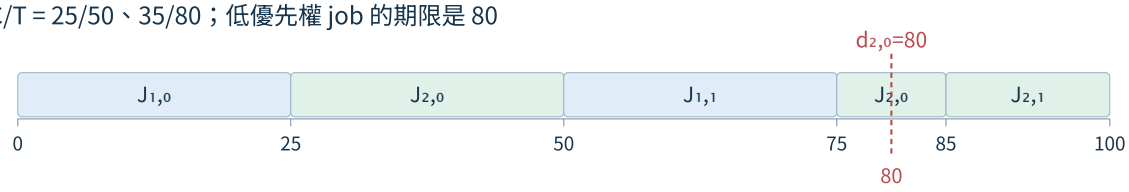
反例：τ_1=(C=1,T=D=2)、τ_2=(2,4,4)，U=1。RM 在 [0,1) 做 $J_{1,0}$，[1,2) 做 $J_{2,0}$，[2,3) 做 $J_{1,1}$，[3,4) 做 $J_{2,0}$ 的剩餘 1。三筆完成於 1、3、4，期限分別為 2、4、4。t=4 無積欠且同時再釋放，故可重複，RM 仍可排程。

判斷順序：先核對假設，再用充分測試；不通過時，以更精確的干擾分析或可證明重複的時間表判定。單憑 U>0.8284 不能宣告失敗；U≤1 也不能保證任意固定優先權成功。

同樣不到滿載，RM 仍會讓某筆 job 超時

改成 τ_1=(25,50,50)、τ_2=(35,80,80)。U=25/50+35/80=15/16=0.9375；RM 仍讓 τ_1 優先。

RM：C/T = 25/50、35/80；低優先權 job 的期限是 80



t=80：舊 $J_{2,0}$ 還剩 5；新 $J_{2,1}$ 另行加入

$J_{2,0}$ 在 [25,50) 做了 25，剩 10；[50,75) 被 $J_{1,1}$ 搶占；[75,80) 只能再做 5，**到了期限 80 仍剩 5**，所以此時已失敗。若超時 job 不丟棄，續做到 85 才完成。t=80 釋放的 $J_{2,1}$ 與舊 $J_{2,0}$ 是兩筆不同 job，圖中沒有把它們合併。

U<1 只表示長期總量未超過 CPU 供給；固定優先權仍可能把某個短區間擠滿，讓低優先權 job 錯過期限。證明失敗的是 t=80 的剩餘需求，不是「超過 RM 充分界」。

用干擾次數核對 RM 的完成時刻

只分析兩個同步釋放的 task，且 τ_1 優先、$C_1 \le T_1$、沒有阻塞。$J_{2,0}$ 在完成前長度 R 的區間 [0,R) 內，τ_1 於 $0, T_1, 2T_1, \cdots$ 釋放；共有 $\lceil R/T_1 \rceil$ 筆會干擾。**在 R 才釋放的新 job 不算入先前區間**。

$$w^{(0)} = C_2, \qquad w^{(k+1)} = C_2 + \left\lceil w^{(k)}/T_1 \right\rceil C_1.$$

從 C_2 往上代入，求出這筆同步 job 的最早固定點。此處以各 job 實際需求均為上界 C_i 的同步序列計算；若遞增值已超過 D_2，這個序列的 job 期限內不可能完成；固定點 $\le D_2$ 才通過此筆檢查。

迭代不減：右側對 w 單調，且首步不小於 C_2。若存在期限內固定點 R^*，歸納得每一步 $w \le R^*$；因此迭代超出期限即可排除期限內固定點。這裡只推導同步兩 task 的首筆；任意相位、多 task 或有 blocking 的完整最壞回應時間分析，還需要相應的 critical-instant 與干擾模型，不能由這張表直接推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三組工作集，逐次代入核對

工作集	$w^{(0)}$	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	核對結果
(20,50)、(35,100)	35	55	75	75	75≤100，對上圖
(25,50)、(35,80)	35	60	85	85	85>80，已超時
(1,2)、(2,4)	2	3	4	4	4=4，恰在期限完成

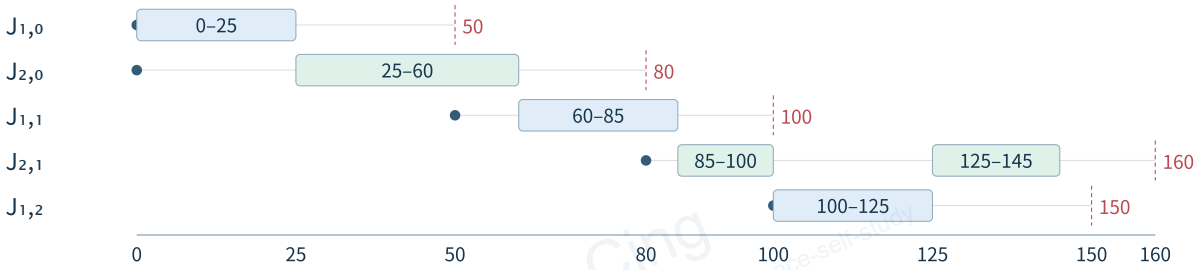
例如第二列：35+⌈35/50⌉25=60，下一步35+⌈60/50⌉25=85；85>80，已超過期限。第一列則在75固定，且75≤100。數值迭代的意義來自前頁的干擾上界，不能只背表格結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EDF：比較的是每筆 job 的絕對期限

Earliest-deadline-first（EDF）每次從 ready jobs 選**絕對期限最早**者。沿用 τ_1=(25,50,50)、τ_2=(35,80,80)；平手時先維持 current，否則依 task 編號、job 編號。

EDF：● 釋放；虛線為各 job 的期限；色塊才是實際執行



t=50 新 $J_{1,1}$ 的期限 100 比 current $J_{2,0}$ 的 80 晚，故 **$J_{2,0}$ 續作至 60 完成**。t=100 新 $J_{1,2}$ 的期限 150 比 current $J_{2,1}$ 的 160 早，才搶占。

圖只展開前 150 ms；有限前綴通過不代表未來全通過，後面的 EDF 定理才給一般保證。被搶占者保留剩餘量，不重新從 C_i 開始。

EDF 每一次選擇，都能回到 ready set 核對

表中列事件處理後、下一段執行前的 ready jobs；括號依序為「剩餘 CPU，絕對期限」。只列尚未完成的 job。

t (ms)	ready set	下一段執行	選擇理由
0	$J_{1,0}$(25,50)、$J_{2,0}$(35,80)	$J_{1,0}$：0–25	50<80
25	$J_{2,0}$(35,80)	$J_{2,0}$：25–50	唯一 ready
50	$J_{1,1}$(25,100)、$J_{2,0}$(10,80)	$J_{2,0}$：50–60	80<100，續作
60	$J_{1,1}$(25,100)	$J_{1,1}$：60–80	唯一 ready
80	$J_{1,1}$(5,100)、$J_{2,1}$(35,160)	$J_{1,1}$：80–85	100<160，續作

前五個事件的重點：$J_{2,0}$ 在 50 面對新到達仍續作；$J_{1,1}$ 在 80 也續作。到達只改 ready set，只有 current 不再是最早到期者時才搶占。

EDF 在 100 搶占：同一規則為何得到不同選擇？

仍以（剩餘 CPU，絕對期限）列 ready jobs。

t (ms)	ready set	下一段執行	選擇理由
85	$J_{2,1}$(35,160)	$J_{2,1}$：85–100	唯一 ready
100	$J_{1,2}$(25,150)、$J_{2,1}$(20,160)	$J_{1,2}$：100–125	150<160，搶占
125	$J_{2,1}$(20,160)	$J_{2,1}$：125–145	唯一 ready
145	空	idle：145–150	沒有可執行 job

J_1 的前三筆完成於 25、85、125，期限 50、100、150；J_2 的前兩筆完成於 60、145，期限 80、160。**job 到達、優先權重排、實際切換是三件事**；50 與 80 都有到達，但 current 仍是最佳選擇。

為什麼理想 EDF 在 D=T 時只需 U≤1？

考慮前述單 CPU 模型，自 t=0 無積欠開始，允許不同 task 有不同非負釋放相位。若 U>1，取每筆 job 實際需求均為上界 C_i 的合法序列。到期需求為 Ut−O(1)，最終超過 CPU 供給，故不能保證所有合法需求序列都準時。若實際需求較低，某次執行仍可能準時。

反過來，假設 U≤1，EDF 卻首次在時刻 b 錯過期限。只看絕對期限 ≤b 的 jobs；向前找它們**持續有未完成工作**的最後一段 [a,b)，長度 L=b−a。在 a 前沒有這類積欠，故區間內要完成的這類 jobs 都在 a 或之後釋放。EDF 在整段只執行這類工作。

因 $D_i=T_i$，同一 task 在 [a,b] 內「釋放且到期」的 jobs 最多 $\lfloor L/T_i \rfloor$ 筆：首筆最快在 $a+T_i$ 到期，以後每次至少再隔 T_i。因此必須在這段處理的總需求滿足

$$\sum_i N_i C_i \le \sum_i \lfloor L/T_i \rfloor C_i \le L \sum_i C_i/T_i = LU \le L.$$

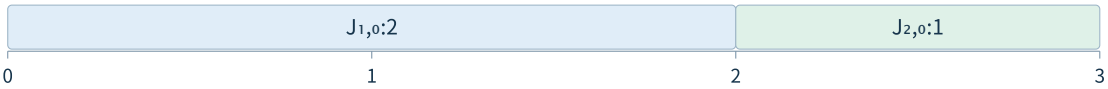
但首次超時表示 EDF 已連做 L 時間仍沒做完，需求必須 **>L**，矛盾。所以模型內 U≤1 充分；加上前段必要性，即為保證所有合法需求序列準時的充要條件。這個證明靠的是每個區間的需求上界，並非「平均沒滿就一定沒事」。

前例 U=15/16，故 EDF 可保證所有 jobs；實機切換成本、鎖阻塞、多核心或 D<T 都會改變上述前提，不能照搬結論。

相對期限短於週期時，低利用率也可能無解

令 τ_1、τ_2 都是 C=2、T=10，但 D=3，同步在 0 釋放。U=0.4，然而兩筆 job 都須在 t=3 前完成，合計需要 4 ms；單 CPU 在 [0,3) 只供應 3 ms。**不論 RM、EDF 或其他政策，至少有一筆超時**。

C=2、T=10、D=3：兩筆都在 0 到達，3 之前需要 4



供給 3 < 需求 4；t=3 時 $J_{2,0}$ 還剩 1

對一般 $D_i \le T_i$，長度 L 內可能「釋放且到期」的最大需求為

$$\mathrm{dbf}_i(L) = \max\left(0, \left\lfloor \frac{L - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

首筆需留 D_i 時間；若 $L \ge D_i$，此後每隔 T_i 可再放一筆，故得到上式。此例 L=3，每個 task 的 dbf=2，合計 4>3，已足以證明不可排程。只檢查某一個 L 通過不能證明全部區間都通過。

即時分析的第一步是把**需要多少 CPU、何時到達、最晚何時交付**分開寫。priority inheritance 可處理特定鎖造成的優先權反轉，但不會把這個 4>3 的容量矛盾消掉。

系統名稱，不能代替選取規則

本節固定以下歷史版本與事件模型：Solaris TS/IA、Windows 7，以及 Linux 3.2 CFS。比較的是特定模型的行為，並非現代版本的效能排名。

系統案例	選取結構與主要規則	不可省略的條件
Solaris TS/IA	類別優先權轉全域優先權；用盡片額與睡醒可改等級	等級表、片額、睡醒規則
Windows 7	32 級 ready queue；同級輪流、動態提升	基礎等級、boost、tick 單位
Linux 3.2 CFS	依 vruntime 的樹與公平選取提示	權重、計時器與喚醒門檻

原書不把上述三者化成一個共同 RR 片額。以下 CFS 案例先把時序條件固定，再逐事件核對。

Linux 3.2 CFS：先固定輸入，再推切換時刻

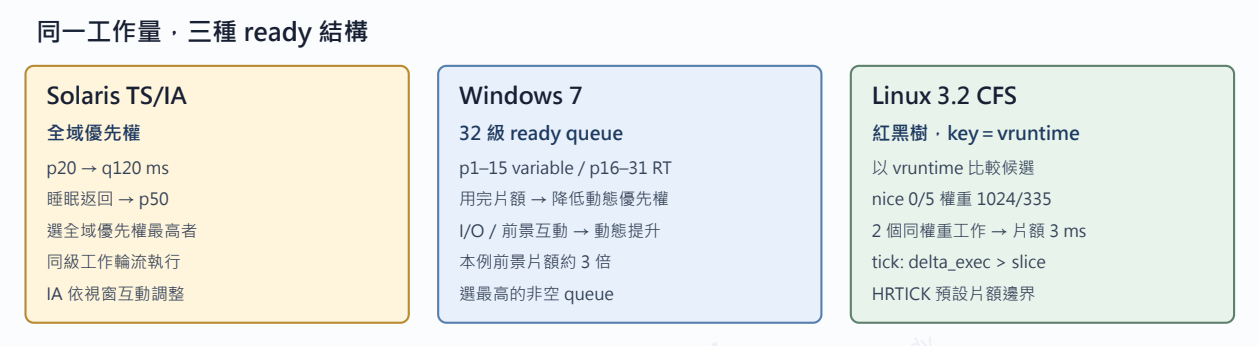
單 CPU；P1、P2 在 0 就緒，分別需要 8、4 ms；P3 需要 2 ms，在 9 ms 喚醒。權重均為 1024，初始 vruntime 正規化為 0；同鍵先選 P1。忽略切換、interrupt 與 timer jitter 成本。

本例固定 target latency=6 ms、minimum granularity=0.75 ms、wakeup granularity=1 ms，且 GENTLE_FAIR_SLEEPERS=true。編譯設定 CONFIG_SCHED_HRTICK=y，執行時 HRTICK=true，CPU 的高解析度計時器可用；DOUBLE_TICK=false。這些是**本例條件**，不代表每個 Linux 3.2 系統都啟用 HRTICK。

可執行數 n≤8 時 period=6 ms；n>8 時 period=0.75n ms。等權重時，每人的理想片額為 period/n：兩人 3 ms、三人 2 ms。

在 t=9，先處理 P3 的喚醒入列，再處理原 HRTICK 到期；兩件事同時發生時的順序是本例指定的。一般 tick 仍可能發生；只假設它不額外造成切換。

三種 ready 結構，不能共用一套片額規則



本例 current 不放在 ready tree；仍可執行的舊current，才會依更新後vruntime回樹。睡眠者要等喚醒，完成者直接離開。選定current後，ready欄不再包含它。

CFS 逐事件表：每次切換都要有原因

列出事件處理與選取後的狀態；ready 不含 current，片額單位是 ms。

時刻／事件	current／片額	其他 ready	min_vruntime	選取理由
0 初始	P1／3	P2@0	0	同鍵，P1 先入樹
3 HRTICK	P2／3	P1@3	0	P1 的片額到期
6 HRTICK	P1／3	P2@3	3	同鍵，P1 是既有最左者
9 P3 入列	P3／2	P2@3、P1@6	3	新片額縮為 2，P1 已跑 3；入列更新要求重排
11 P3 完成	P2／3	P1@6	3	比較 ready 鍵 3<6
12 P2 完成	P1／6	空	6	只剩 P1，需求尚有 2
14 P1 完成	無／—	空	8	無可執行工作

t=9 的喚醒：鍵值小於基準，基準卻不倒退

P1 自 t=6 執行至 9，vruntime 從 3 增至 6；P2 等待，鍵為 3。P3 依 GENTLE_FAIR_SLEEPERS 規則放回 max(0, 3−6/2)=0；min_vruntime 仍為 3，不因新鍵較小而倒退。

步驟	具體變化
P3 入列	runnable 從 2 增至 3，每人的理想片額從 3 縮為 2 ms
hrtick_update	P1 本次已跑 3 ms，2−3<0，對 current 設定重新排程
重新選取	P3 的鍵 0 小於 P2 的 3 與 P1 的 6，選 P3

這個時序中，重排已由入列後的 HRTICK 更新要求；不能把 6−0>1 的喚醒比較當成實際觸發原因。P3 在 t=8 仍睡眠，不能提早執行。

把不同計時器的時間線並排比較

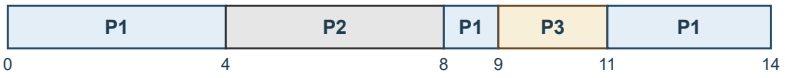
同一批工作，不同事件模型

Linux 3.2 CFS · 單一 CPU · 時間單位 ms

HRTICK
P3 在 9 喚醒

3、6：片額到期；9：喚醒入列 · 重算片額後要求重排。

一般 tick
P3 在 9 喚醒



4：週期檢查判定用量超過片額；9：喚醒比較觸發搶占。

HRTICK
P3 在 8.5 喚醒

8.5：喚醒入列後 · P1 已超過重算出的 2 ms 片額。

P1 / P2 / P3 的需求：8 / 4 / 2 ms；圖中的數字是各次事件時刻。

tick、HRTICK 與 sleep placement 的敏感度

設定	時間線（ms）	完成時刻 P1／P2／P3
HRTICK；P3 在 9 喚醒	P1 0–3、P2 3–6、P1 6–9、P3 9–11、P2 11–12、P1 12–14	14／12／11
無 HRTICK；每 1 ms tick；P3 在 9 喚醒	P1 0–4、P2 4–8、P1 8–9、P3 9–11、P1 11–14	14／8／11
HRTICK；P3 在 8.5 喚醒	P1 0–3、P2 3–6、P1 6–8.5、P3 8.5–10.5、P2 10.5–11.5、P1 11.5–14	14／11.5／10.5

一般 tick 的片額檢查使用嚴格不等式：3>3 為假，所以無 HRTICK 時 P1 到 t=4 才切出。最後一列在入列後重算理想片額為 2 ms，而 P1 已跑 2.5 ms：hrtick_update 要求重排。第二列則在 t=9 由 5−2>1 的喚醒比較要求重排；兩者觸發路徑不同。

週期檢查、喚醒比較與 buddy 提示

一般 tick 在已達 minimum granularity 後，也可能比較 current 與樹中最左候選的 vruntime 差距；這是另一項公平性檢查。到達或喚醒不必然使目前工作離開 CPU。

Linux 3.2 的 wakeup preemption 會檢查是否已經要求重新排程。若 P3 入列後 hrtick_update 已設定重排，便不能再宣稱是 wakeup comparison 導致這次切換；若尚未要求重排，才需依規則進行喚醒比較。

last／next／skip buddy 是兼顧局部性與排程開銷的候選提示，仍受公平性限制；不是額外的絕對優先權，也不能只因 P3 被選中就推定它必然被設為 next buddy。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

nice、weight 與跨案例比較

Linux 3.2 的 nice 不直接加減 priority queue；它改變 weight。v3.2 的 nice=0 weight 為 1024，nice=5 weight 為 335。若只有這兩個 task runnable 且 period=6 ms：

task	nice	weight	ideal_runtime	實體每 1 ms 增加的 vruntime
A	0	1024	6×1024/1359≈4.521 ms	1 ms
B	5	335	6×335/1359≈1.479 ms	1024/335≈3.057 ms

所以 nice 低者取得約 75.35% 的 CPU，nice 高者取得約 24.65%；權重並非線性 CPU percentage。n>8 時 period 隨 runnable 數延長，讓 minimum granularity 避免 slice 無限變短。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一工作量，為何沒有唯一的系統時間表？

系統	可比較的選取機制	要重建時間線，還缺什麼？
Solaris TS/IA	類別與全域優先權、用盡片額及喚醒後的調整	初始優先權、dispatch table、片額與事件順序
Windows 7	多級 ready queue、同級輪替與動態 boost	初始優先權、量子、tick 單位、boost 及佇列次序
Linux 3.2 CFS	vruntime、權重、timer 與睡眠放置	權重、timer 設定、喚醒時刻與同鍵規則

同樣的 CPU 需求與喚醒時刻不足以確定 Solaris 或 Windows 的 Gantt 圖；缺少上述參數，就不能推算各工作的完成時間。前面的 Linux 範例則在列明條件後逐事件推得時間線，不能拿它當成三個產品的效能排名。

比較政策：用同一批輸入，也要看未完成者

方法	能回答什麼？	不能直接保證什麼？
確定性時間表	指定輸入下每次選擇與指標	其他到達分布也相同
佇列模型	在到達／服務假設下推導平均	未滿足假設時的尾端延遲
模擬	指定成本與分布下的政策行為	未建模的硬體或工作負載
實作量測	特定版本與機器上的觀測	所有機器與未來負載

固定三筆工作皆在0抵達、需求5/2/3 ms且無切換成本，SJF平均等待7/3、RR q=2為13/3；完成三筆皆需10 ms。平均等待較低，不能推成吞吐量也較高。若只統計已完成的短工作而漏掉長工作，平均值還會受到選樣偏差影響。

CPU 一直忙，等待仍可能變長

沿用 RR q=2、到達全為0、初序P1/P2/P3，只把三筆需求同乘k。本批 p99 採 nearest-rank，三筆觀察的結果是最大值，不能當作母體 p99 估計。每次取隊首，執行 min(2,剩餘需求)，未完成才放回隊尾。忽略切換成本，時間單位為 ms。

k；bursts	completion P1/P2/P3	waiting P1/P2/P3	throughput jobs/ms	本批p99首 CPUresponse	最大 waiting
1；5/2/3	10/4/9	5/2/6	3/10	4	6
2；10/4/6	20/10/16	10/6/10	3/20	4	10
3；15/6/9	30/16/25	15/10/16	1/10	4	16

三批的首次取得 CPU 都是0/2/4，利用率都100%；最大等待卻由6增至10再增至16。首次回應好，並不代表長工作很快完成。這是有限批次的比較，不能據此證明持續到達下也沒有 starvation。

把 k=2 的每個時間片列出來

完成時間 20／10／16，減去需求 10／4／6，得到等待 10／6／10。沒有 I/O。表中每格按時間順序對應。

時間	工作	執行後剩餘	結果
0–2；2–4	P1；P2	8；2	均回隊尾
4–6；6–8	P3；P1	4；6	均回隊尾
8–10；10–12	P2；P3	0；2	P2 完成；P3 回隊尾
12–14；14–16	P1；P3	4；0	P1 回隊尾；P3 完成
16–18；18–20	P1；P1	2；0	同一工作續執行至完成

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：系統不確定資源何時釋放時，什麼條件足以造成永久等待，如何用可負擔的策略處理？

前置知識。 process state、mutex 與 semaphore、向量的逐分量比較。

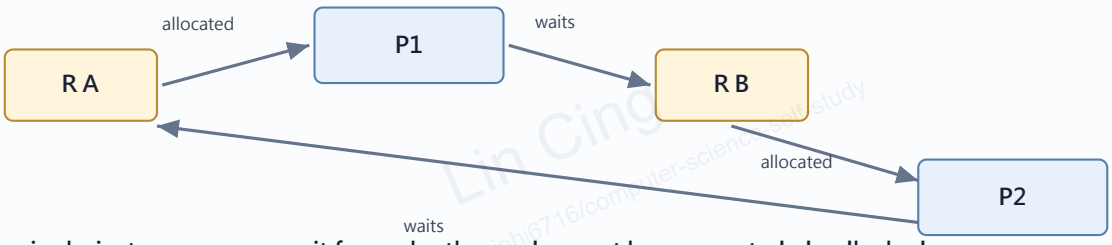
- 證明必要條件不等於充分條件
- 設計破壞 circular wait 的 lock ordering
- 以 Banker's algorithm 檢查 safe state

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

系統模型

- 資源型別有固定 instance 數；process 對資源執行 request、use、release。request 可能立即成功，也可能在資源不足時阻塞。
- 資源配置圖中 process node 指向 resource 表示等待，resource instance 指向 process 表示持有。圖模型只描述等待與持有關係。

Single-instance cycle: A → P1 → B → P2 → A



Two single-instance owners wait for each other and cannot be preempted: deadlocked.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資源、狀態與 deadlocked set

resource type 是一組功能等價的 instance。若兩個 printer 可互換，它們屬於同一 type；若位置或功能不可互換，就必須分成不同 type。process 對每個 resource 依序執行 request、use、release。

request 可以立即成功，也可以在 instance 不足時等待。system table 至少保存：

- 每個 type 的 instance 總數。
- 每個 instance 的 free 或 allocated 狀態。
- allocated instance 的 owner。
- 每個 waiting request 的 requester 與 resource type。

一個 process set 處於 deadlocked state，表示集合內每個 process 都在等待一個事件，而該事件只能由集合內的等待者產生，且在既定模型中沒有其他可行途徑解除等待。等待者可以是自己：例如執行緒持有 nonrecursive mutex，又以阻塞方式重複取得同一把鎖，而唯一持有者只能在取得後釋放，就形成自我死結。

兩個已死結的具體狀態

同一 resource type。 三個 CD-RW 共 3 個 instance，P1、P2、P3 各持有 1 個，接著各再請求 1 個。Available=0，且沒有任何 owner 能完成並 release，三者形成 deadlock。

不同 resource types。 系統有 1 個 printer 與 1 個 DVD。P1 持有 DVD 並等 printer，P2 持有 printer 並等 DVD。此狀態滿足：

Process	Allocation	Request
P1	DVD	printer
P2	printer	DVD

資源種類不同不影響 deadlock；判斷依據是 request/hold 關係與是否仍有可完成的 transition。

四個必要條件

條件	在此題中的含義	若持續成立
Mutual exclusion	至少一個 resource 不可同時共享	其他 requester 必須等
Hold and wait	process 已持有至少一個 resource，又等待另一個	等待可能連結兩份 ownership
No preemption	resource 不能被系統強制收回，只能持有者自願 release	外部不能直接拆環
Circular wait	`P0→P1→...→Pn→P0` 的等待鏈	每個參與者的下一步依賴下一人

circular wait 已蘊含 hold and wait，因此四個條件不是彼此邏輯獨立。四者同時成立仍只是 deadlock 的必要條件，不能用來斷言目前已經無限期卡住。

resource-allocation graph 的判準

process node 為圓形，resource type 為矩形，instance 為矩形內圓點。

- request edge `Pi→Rj` 表示 Pi 正在等待 Rj 的 instance。
- assignment edge `Rj→Pi` 表示 Rj 的某個 instance 已分配給 Pi。
- 若 request 立即成功，request edge 立即轉成 assignment edge。
- release 時刪除對應 assignment edge。

以下採一致快照及阻塞請求模型：請求不能取消或逾時，持有資源不可被搶占，等待者取得所需資源前不會提前釋放持有資源；每次請求一個同型實例。

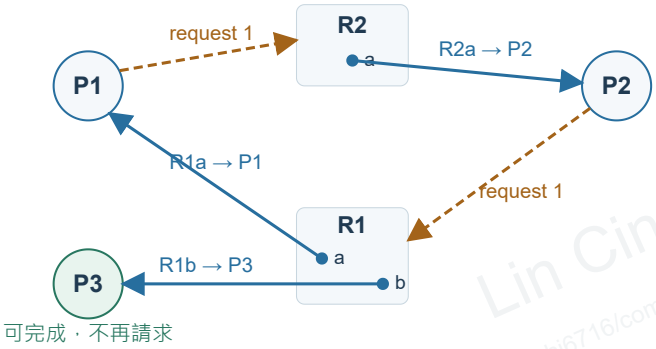
定理。 無 cycle 必無資源死結。若全圖每個 resource type 都只有一個 instance，則存在 cycle **若且唯若**存在死結。即使其他地方有多 instance type，只要某一具體 cycle 上的每個 type 都是單 instance，該 cycle 仍足以證明其參與者死結。

若 cycle 涉及多 instance type，光看這個 cycle 不能判斷是否死結：需檢查其他 instance 的持有者是否能完成並歸還。不能反過來說，多 instance 系統中每個死結都必有一條全由單 instance type 組成的 cycle。

有環，為什麼仍能完成？

R1 有兩個可互換的實例 R1a、R1b，R2 只有 R2a。三個 process 都遵守「取得目前請求後，即可完成並歸還全部持有資源」；P3 不再請求資源，可直接完成。初始沒有空閒實例。

初始狀態：Total=(2,1)，Available=(0,0)



箭頭從誰指向誰？

實線：實例 → 持有者

虛線：等待者 → 資源型別

R1 的兩個點是兩份獨立實例。

環：P1 → R2 → P2 → R1 → P1

P3 不在環上，卻持有另一份 R1。

先讓 P3 完成，就能解開等待。

環上的 P1、P2 目前都在等待，但 R1b 的持有者 P3 並未等待。**有環不等於所有可解環的釋放途徑都被堵住。**

先完成 P3，再依序解除等待

同一個完成順序 P3 → P2 → P1；向量順序為 (R1,R2)

事件	R1a owner	R1b owner	R2a owner	Available
初始：P1、P2 等待	P1	P3	P2	(0, 0)
P3 完成，歸還 R1b	P1	空閒	P2	(1, 0)
授予 P2：R1b	P1	P2	P2	(0, 0)
P2 完成，歸還 R1b、R2a	P1	空閒	空閒	(1, 1)
授予 P1：R2a	P1	空閒	P1	(1, 0)
P1 完成，歸還 R1a、R2a	空閒	空閒	空閒	(2, 1)

授予資源會減少 Available，真正完成並歸還才會增加。只寫完成順序 P3→P2→P1，會省略 P2、P1 取得請求時的兩次扣除；上圖把這些中間狀態一併列出。

Detection 的 Work 是模擬帳，不是即時庫存

向量順序固定為 (R1,R2)。目前尚未滿足的 Request 與既有 Allocation 為：

Process	Allocation	Request	檢查前 Work	模擬完成後 Work
P3	(1,0)	(0,0)	(0,0)	(1,0)
P2	(0,1)	(1,0)	(1,0)	(1,1)
P1	(1,0)	(0,1)	(1,1)	(2,1)

每列先檢查 Request≤Work；若可滿足，再令 Work←Work+Allocation。它把「先授予 Request，之後歸還 Allocation+Request」合成一步，所以淨增加只有 Allocation。Work 中的實例可由已模擬完成者歸還，**不代表那些 process 已真的執行；初始 Available 仍是 (0,0)。**

此處比較的是**目前未滿足的 Request**，不是 Banker's algorithm 的最大剩餘需求 Need。完整 detection 程序見 7.6。

只改 P3 的下一步，就會真的死結

保留圖中的全部持有關係與 P1、P2 的請求，只把 P3 改成「也必須再取得一份 R2 才能完成」，即新增 P3→R2。

初始狀態	P3 的 Request	Work=(0,0) 時可先完成者	結果
原例	(0,0)	P3	完成順序 P3→P2→P1
改例	(0,1)	無	P1、P2、P3 皆死結

改例的 P1 與 P3 都等待 P2 持有的 R2a；P2 等待的 R1，兩份都在這兩個等待者手上。所有實例都困在等待集合內，沒有能先歸還的持有者。此結論假設資源不可搶占、沒有取消或額外實例，也沒有完成前的提前釋放。單純把 P3 暫時移出 CPU 不會改變原例的資源可行性；排程飢餓與資源死結須分開判讀。

lock deadlock 的可重現交錯

thread A 依序取得 `first`、`second`；thread B 依序取得 `second`、`first`。

時刻	動作	持有
t0	A 取得 first	A:first
t1	B 取得 second	A:first; B:second
t2	A 等 second	A 持 first 等 B
t3	B 等 first	B 持 second 等 A

四條件皆成立且沒有非 waiting holder，兩 thread 永久等待。若 A 在 t1 前完成全部工作並 release，這個特定交錯不會發生。deadlock 需要特定相對時序，單次壓力測試不能證明不存在。

四種處理策略的資訊需求

策略	採用的資訊	保證	主要成本
Prevention	只靠限制 request 的協議	至少破壞一個必要條件，理論上不進入 deadlock	低利用率、拒絕 request、rollback
Avoidance	process 事先宣告最大需求	每次配置後仍存在 safe sequence	需要可信 claim，runtime safety test
Detection/recovery	runtime allocation、request 與 snapshot	允許形成 deadlock，之後找出並恢復	資訊維護、偵測延遲、victim 損失
Ignore	不維持 deadlock protocol	不提供一般 deadlock 保證	卡住時由應用、timeout 或重啟處理

第一與第二種都避免「系統進入無法完成全體工作的狀態」，但 prevention 限制 request 形狀，avoidance 則根據未來需求判斷一個目前有空閒的 request 是否仍該等待。

Lock ordering 不變量

- 若每條執行路徑都只依遞增全域順序取得鎖，等待邊永遠由低階指向高階，無法形成嚴格循環。
- 例外路徑、callback、condition wait 與升級鎖也必須遵守順序；只在主路徑排序不足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐一破壞必要條件

Mutual exclusion。 將 resource 改為可共享，例如多人同時讀取 read-only file。mutex 對受保護的可寫資料本質上不可共享，所以這條通常只能局部使用。

Hold and wait。 要求啟動前一次取得全部 resource，或每次 request 前先 release 所有 resource。前者可能讓印表機閒置到程式最後；後者可能破壞尚未持久化的中間資料。

No preemption。 新 request 無法滿足時，收回該 process 現有 resource，待舊與新資源都可重新配置後 restart；或只搶收回應中 waiting process 的資源。只有 state 可保存與重建的資源適合此做法。

Circular wait。 為所有 resource type 指定嚴格順序，只能向較高順序 request。這是實作上最常見的破壞方式，代價是違反順序的 request 必須改成等待或重新設計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

resource ordering 的反證法

令 `F:R→N` 為一對一順序。協議要求 process 持有 `Ri` 時，只能再取得 `Rj`，且 `F(Ri)<F(Rj)`。

假設仍有 circular wait，沿等待邊列 resource 會得到

```
F(R0)<F(R1)<...<F(Rn)<F(R0)
```

由遞移性得到 `F(R0)<F(R0)`，矛盾。因此不可能存在嚴格順序下的 cycle。

動態鎖的實際版本。 兩帳戶 from/to 的管理鎖不能按參數順序取得。先對帳戶 id 排序，再依小 id 到大 id 取得兩鎖；release 可反向或按 API 規則，但新的 acquire 必須維持順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單 instance avoidance 的 claim edge

單 instance graph 額外畫 claim edge `Pi→Rj`，表示 Pi 未來可能 request Rj。request 到來時，claim edge 先轉 request edge；若 grant，再轉 assignment edge。

grant 前要檢查：把提出的 request edge 轉為 assignment edge 後，graph 是否新增 cycle。若無 cycle，保留分配；若有 cycle，Pi 等待，狀態不變。

claim edge 必須在 process 開始前已知，或只能在該 process 尚無 assignment edge 時加入。若未來需求宣告不完整，cycle 檢查無法覆蓋真實 request。cycle detection 可用 adjacency matrix 的 `O(n^2)` 做法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Banker 的資料與 safety 規則

令 n 為 process 數，m 為 resource type 數，`Allocation_i` 是 Pi 現有資源，`Need_i` 是完成前最多還會請求的差額。

```
Need = Max - Allocation
```

safe sequence 的每一步都要求 `Need_i <= Work`。完成 Pi 後：

```
Work = Work + Allocation_i
Finish[i] = true
```

此步驟表示 Pi 能取得它宣告的剩餘需求，完成任務後把全部已配置與新取得資源返回。若所有 Finish 都能變成 true，當下 state 是 safe；否則 unsafe。

Safety algorithm的一般正確性

假設每個process的Max宣告合法、所有resource分量非負、process取得Need後能在有限時間完成並歸還全部資源。Work起始Available；若某Pi的Need_i<=Work，先給它Need，再收回Allocation_i+Need_i，淨變化是Work−Need_i+(Allocation_i+Need_i)=Work+Allocation_i。故完成一步不會減少其他process未來可用的資源，也不把Need平白加進總量。

若存在某安全完成序列，而演算法先挑了序列中不是第一個的可完成Pi，可以先完成Pi再維持其餘原順序、略過原來的Pi位置。其餘每一步面對的可用資源只會更大，不會較小；因此任選一個Need<=Work不會破壞仍存在的安全序列。這說明greedy選擇不需對每個候選分支回溯。

若演算法停止但仍有unfinished，反設原state有安全序列，取該序列中第一個尚未被演算法完成的Pj。它在該序列之前的process都已在演算法的完成集合內；當前Work包含這些Allocation，且可能更多，所以Need_j必<=當前Work，與停止時沒有候選矛盾。故找不到候選而仍有unfinished便unsafe；若全部完成，記錄的順序本身就是安全證書。

resource-request 演算法

Pi提出m維Request_i時，先要求每個分量都是非負整數且維度正確；本節x<=y一律表示每個j都有x[j]<=y[j]。其否定是「存在至少一個分量較大」，不是「每個分量都較大」。

1. 若存在j使Request_i[j]>Need_i[j]，拒絕並視為超過最大宣告。
2. 否則若存在j使Request_i[j]>Available[j]，Pi等待現有資源。
3. 暫時令 `Available-=Request`、`Allocation_i+=Request`、`Need_i-=Request`。
4. 對 trial state 執行 safety algorithm。
5. safe 才真正授予；unsafe 要撤銷 trial 修改，讓 Pi 等待。

反例：Total=(2,1)、唯一Pi的Allocation=(1,0)、Need=(1,1)、Available=(1,1)是合法state。Request=(2,0)在第一維同時超過Need與Available，即應在第1步error；若把向量>誤解為兩維都大，會錯放而得到Available=(−1,1)，已違反非負資源。

完整流程的必要性是：第 3 步是 transaction，不能在 unsafe 後留下部分扣款或部分 allocation。每輪 safety 至多檢查 n 個 process，可做 n 次，因此典型複雜度為 `O(m n^2)`。

Banker T0 完整矩陣

Total=`(10,5,7)`，Available=`(3,3,2)`。

Process	Allocation	Max	Need=Max-A
P0	(0,1,0)	(7,5,3)	(7,4,3)
P1	(2,0,0)	(3,2,2)	(1,2,2)
P2	(3,0,2)	(9,0,2)	(6,0,0)
P3	(2,1,1)	(2,2,2)	(0,1,1)
P4	(0,0,2)	(4,3,3)	(4,3,1)

以下的 safety trace 證明 `<P1,P3,P4,P0,P2>` 是 safe sequence：

完成順序	Work 轉移
初始	(3,3,2)
P1	(5,3,2)
P3	(7,4,3)
P4	(7,4,5)
P0	(7,5,5)
P2	(10,5,7)

五種資源：求最小安全庫存

考慮 4 個 process 與 5 種不可搶占、可重複使用的資源。每個 process 取得其剩餘最大需求後，能在有限時間完成並歸還全部資源；向量依資源 A、B、C、D、E 排列。

Process	Allocation	Max	Need = Max − Allocation
P1	(1,0,2,1,1)	(1,1,5,1,2)	(0,1,3,0,1)
P2	(2,0,1,1,0)	(5,2,2,1,0)	(3,2,1,0,0)
P3	(1,1,0,1,0)	(3,1,3,1,0)	(2,0,3,0,0)
P4	(1,1,1,1,0)	(1,1,2,2,1)	(0,0,1,1,1)

已知 Available = (0,0,x,1,1)，x 是非負整數。每個 x 對應資源總量 Total = ΣAllocation + Available = (5,2,4+x,5,2)。求能使狀態安全的最小 x；不同 x 是不同的候選總量，不是憑空改變同一系統的資源帳。

從 Work 的變化找出最小 x

x = 2：每一步先核對 Need ≤ Work

完成者	舊 Work	其 Need		新 Work
P4	(0,0,2,1,1)	(0,0,1,1,1)	→	(1,1,3,2,1)
P1	(1,1,3,2,1)	(0,1,3,0,1)	→	(2,1,5,3,2)
P3	(2,1,5,3,2)	(2,0,3,0,0)	→	(3,2,5,4,2)
P2	(3,2,5,4,2)	(3,2,1,0,0)	→	(5,2,6,5,2)

每步新 Work = 舊 Work + 該 process 的 Allocation

x=0 時，四列 Need 都無法逐分量小於等於初始 Work，沒有第一個候選。x=1 時只有 P4 可先完成；模擬歸還後 Work=(1,1,2,2,1)，P1 缺 C，P2 缺 A、B，P3 缺 A、C，因此無下一個候選。

x=2 時依圖依次核對 Need ≤ Work，得到安全序列 ⟨P4,P1,P3,P2⟩。所以最小的整數 **x=2**。Work 是檢查中的模擬帳；檢查通過並不表示 process 已真正完成或 Available 已變成最後的 Work。

單一資源類型：何時能保證避免死結？

一種資源共有 m 個相同且可重複使用的實例；有限的 n 個 process 各自最多需要 Max_i 個，且 $1 \le Max_i \le m$。已分配資源在完成前不被搶占；資源請求只在可用數不足時阻塞，取得所需資源後 process 能有限時間完成並歸還所有持有量。

對所有尚未完成者，令 $Need_i = Max_i - Allocation_i$。若

$$\sum_{i=1}^{n} Max_i < m + n,$$

則每個合法配置都至少有一個尚未完成者滿足 $Need_i \le Available$。依次讓這些 process 完成，可得到一個安全完成序列。

例：m=10，四個 process 的 Max 均為 3，則 ΣMax=12 < 14；依此模型，無論目前合法配置如何，都有安全序列。此條件**充分但非必要**：若有五人各 Max=3，雖然 ΣMax=15 不小於 m+n=15，但全部尚未持有資源時 Available=10，仍有安全序列。

為什麼每一輪都找得到可完成者？

令目前可用實例數為 v。若所有 n 個尚未完成者都不能滿足最大剩餘需求，則每個 $Need_i > v$；數量為整數，故 $Need_i \ge v+1$。因 $\Sigma Allocation_i = m - v$，得到

$$\sum_i Max_i = \sum_i Allocation_i + \sum_i Need_i \ge (m - v) + n(v + 1) = m + n + (n - 1)v \ge m + n,$$

這與假設矛盾；因此有一人能先完成。這個推理也包含 v>0，不只適用於資源全被占滿。

完成者歸還資源後，從總最大需求中移除其 $Max_i \ge 1$，而人數減一，所以嚴格不等式對剩餘者仍成立；重複此論證即可完成全部 process。這是存在安全序列的資源模型保證；實際系統仍需合理授予請求並讓可執行者持續進展，不能僅靠不等式排除不公平排程造成的飢餓。

P1 request 與 unsafe 拒絕

P1 提出 `Request1=(1,0,2)`：

1. `(1,0,2)<=Need1=(1,2,2)`，未超過 claim。
2. `(1,0,2)<=Available=(3,3,2)`，可做 trial。
3. trial 後 Available=`(2,3,0)`、Allocation1=`(3,0,2)`、Need1=`(0,2,0)`。

Trial safety 可依 `P1→P3→P4→P0→P2` 完成，因此授予此 request。

反例：在上述 trial 後，P0 請求 `(0,2,0)`。資源量足夠，但 trial 後 Work=`(2,1,0)`，沒有任何 P0 到 P4 的 Need 小於等於 Work，safety 失敗，因此 P0 必須等待。

unsafe 不代表此刻已 deadlock。若 process 自願提前釋放資源，仍可能完成；avoidance 的保守保證是「不能保證存在完成序列」，不是「必然永久卡住」。

同樣有空閒，為什麼仍可能拒絕？

以下另用兩類資源 A、B，比較兩次**獨立**試配。總量 (4,3)，初始狀態稱 S，Available=(1,1)。向量一律先 A 後 B。

Process	Allocation	Max	Need=Max−Allocation
P0	(1,0)	(3,2)	(2,2)
P1	(1,1)	(2,2)	(1,1)
P2	(1,1)	(3,2)	(2,1)

逐分量核對：已配置合計 (3,2)，加 Available 得 (4,3)。每列 Allocation+Need=Max，且 Max 不超過 Total。此例假設資源不可搶占、完成前不歸還；取得實際所需資源後可在有限時間完成。

S 是安全狀態：P1 的 Need=(1,1)≤Work=(1,1)，模擬歸還後 Work=(2,2)；再完成 P0 得 (3,2)，最後 P2 得 (4,3)。這個順序只是一份安全證書，不要求 CPU 真的照此排程。

同一份請求，試配給誰有差別

兩次請求各從 S 開始，都是 (1,0)，也都沒有超過該 process 的 Need 或 Available。第二次不接續第一次的試配。

同一個起點，同一份請求，不同的未來

S：Available=(1,1)；兩條分支各自從 S 開始

P0 請求 (1,0)

試配：Available=(0,1)

Allocation0=(2, 0)

Need0=(1, 2)

沒有 Need ≤ (0,1)

unsafe：撤銷全部三項更新

P1 請求 (1,0)

試配：Available=(0,1)

Allocation1=(2, 1)

Need1=(0, 1)

P1 → P0 → P2 可模擬完成

safe：提交試配的狀態

兩邊都只剩 (0,1)，但持有者與剩餘需求不同。

Safety 的完成順序是模擬證書，不是已發生的排程。

安全性不只看剩餘庫存。P1 多取得一份 A 後，只缺一份 B 就能完成；同一份 A 若給 P0，卻沒有人能在最大剩餘需求下先完成。

拒絕 P0：每一列究竟卡在哪裡？

P0 請求 (1,0) 的試配狀態稱 U。Available=(0,1)，只有 P0 改為 Allocation0=(2,0)、Need0=(1,2)。令 Work=(0,1)，所有 Finish 初始均為 false。

候選	試配後 Need	無法開始的具體分量
P0	(1,2)	A：1>0；B：2>1
P1	(1,1)	A：1>0
P2	(2,1)	A：2>0

沒有可先完成者，安全性檢查立即失敗；不是任意選錯第一個 process。實際系統必須回到 S：Available 加回 (1,0)，Allocation0 減去 (1,0)，Need0 加回 (1,0)。其他列與 Max 都不變。

只加回 Available 會把同一份 A 同時計為空閒與已配置。這時 Available+ΣAllocation=(5,3)，已違反 Total=(4,3)。試配是不可被外部觀察到一半的交易；可用鎖保護整段檢查與提交，或在提交時檢查快照版本並重試。

授予 P1：安全性檢查不會真的執行 process

重新從 S 試配 P1 的 (1,0)：Available=(0,1)、Allocation1=(2,1)、Need1=(0,1)。本表只更新 safety 的 Work 與 Finish。

模擬完成者	Need≤舊 Work	加回既有 Allocation	新 Work	Finish，依 P0/P1/P2
初始	—	—	(0,1)	F/F/F
P1	(0,1)≤(0,1)	(2,1)	(2,2)	F/T/F
P0	(2,2)≤(2,2)	(1,0)	(3,2)	T/T/F
P2	(2,1)≤(3,2)	(1,1)	(4,3)	T/T/T

檢查通過後，**提交的是試配狀態，Available 仍為 (0,1)**。不能把最後 Work=(4,3) 寫回 Available，也不能把模擬的 Finish 當作真實終止紀錄。要等 process 真正完成，才會歸還實例。

為何 Work 只加 Allocation？先給 Need、完成時再收回 Allocation+Need，淨變化就是 −Need+(Allocation+Need)=Allocation；Need 不會憑空增加資源總量。

Unsafe 不等於已死結：給出一條真實完成路徑

Banker 會拒絕 U。現在只是反事實比較：假設繞過安全性檢查，真的把 P0 的 (1,0) 授予，停在 U。Max 是上界，不是每個 process 一定會用滿的配額。

假設繞過拒絕：unsafe 狀態仍可能走到完成

A 共 4 份、B 共 3 份；每格代表一份實例，灰色為空閒。

由左至右讀；括號為真實 Available；三次請求依序是 (0,1)、(1,0)、(1,0)。
各 process 使用少於 Max 的資源便完成；持有的資源在完成時一次歸還。

這條路徑中，P0 只再要 (0,1)，P1、P2 各再要 (1,0) 就能完成；三者皆未超過 U 的 Need。它證明 U **存在**完成的實際未來，沒有提供應付所有合法未來需求的保證。

同一個 U，也能走向死結

若在 U 中，P0、P1、P2 分別提出整份 Need：(1,2)、(1,1)、(2,1)，每份都合法，但 Available=(0,1) 無法滿足任何一份。三者全都阻塞且仍持有 A；沒有取消、搶占或提前釋放時，便形成死結。

兩條未來使用同一個 Allocation、Max、Available，差在實際請求。安全性檢查使用最大剩餘 Need，要求能保守地保證一個完成順序；detection 則必須看當下真正阻塞的 Request。不能把未來可能請求的 Need 自動當成目前 Request。

Unsafe 狀態沒有安全序列；它仍可能有成功的實際執行。 Safe sequence 是依最大宣告定義的專門概念，不能把任何事後完成順序都叫成它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單 instance：wait-for graph

移除資源節點，保留 process 節點。若存在 request edge `Pi→Rq` 與 assignment edge `Rq→Pj`，合併成 `Pi→Pj`。此邊表示 Pi 等 Pj release Rq。

若每個 resource type 都只有一個 instance，deadlock 存在若且唯若 wait-for graph 有 cycle。cycle 上的每個 process 都不能靠 cycle 外的 instance 自行解除等待。若任何 cycle 經過一個多 instance resource，就必須回到 allocation/request 狀態做可完成性分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多 instance detection 演算法

輸入 `Available`、`Allocation`、`Request`。

```
Work = Available
Finish[i] = (Allocation_i == 0)

while exists i:
    Finish[i] == false and Request_i <= Work:
        Work = Work + Allocation_i
        Finish[i] = true

if any Finish[i] == false:
    Pi is not shown able to complete in this snapshot
```

每次比較採一致快照；標準 reduction 樂觀地假設某 process 的目前 Request 一旦滿足，就能在有限時間完成、歸還全部 Allocation，且完成前不再提出新請求。將沒有持有資源者先標成完成，是因它們不持有其他人所需的資源；不表示這些 process 真的已終止或不在等待。

所有 Finish 為 true 只說明在上述模型與目前 Request 下可構造一個解除等待的模擬順序，不保證真實未來不會再請求或死結。若有 Finish 為 false，它們是此 reduction 無法排除的持有資源者；宣稱它們永久死結還需請求不取消、資源不搶占或提前歸還等假設。複雜度為 `O(m n^2)`。

完整 reduction 例

T0 的 Available=`(0,0,0)`。

Process	Allocation	Request
P0	(0,1,0)	(0,0,0)
P1	(2,0,0)	(2,0,2)
P2	(3,0,3)	(0,0,0)
P3	(2,1,1)	(1,0,0)
P4	(0,0,2)	(0,0,2)

完成	Work 加回 Allocation 後
P0	(0,1,0)
P2	(3,1,3)
P3	(5,2,4)
P1	(7,2,4)
P4	(7,2,6)

所有 Finish 為 true，表示在「滿足目前 Request 後有限時間完成且不再請求」的模型中，此快照有可行的解除等待順序；這不是對真實未來永不死結的保證。

P2 多請求後的 deadlocked set

修改 P2 Request=`(0,0,1)`。Work=`(0,0,0)`：

1. P0 Request=`(0,0,0)` 可完成，Work 變成 `(0,1,0)`。
2. 其他 Request 都不能由 Work 滿足。
3. Finish P1、P2、P3、P4 保持 false。

在請求不取消、資源不搶占、不提前釋放的模型下，這個 snapshot 的死結集合為 `{P1,P2,P3,P4}`。

終止 P1 後，重新檢查存活者

前面的P0完成只是偵測演算法的**模擬**，不會真的把資源放回Available。若P0尚未實際完成，終止P1並安全回收其Allocation=(2,0,0)後，真實Available由(0,0,0)變成(2,0,0)，不是(2,1,0)。重新以Work=Available開始，對存活者做reduction：

模擬完成者	當前Request	加回Allocation後的Work
初始，P1已終止	—	(2,0,0)
P0	(0,0,0)	(2,1,0)
P3	(1,0,0)	(4,2,1)
P2	(0,0,1)	(7,2,4)
P4	(0,0,2)	(7,2,6)

每列先驗Request≤Work，完成後只加回原Allocation。在目前 Request 獲得滿足後各存活者都能有限時間完成、無新請求的假設下，這個序列證明存活者有可行完成順序；只有它們真的執行並歸還資源，真實Available才依序增加。若P0早已實際完成，終止P1後才可直接由Available=(2,1,0)開始。

偵測頻率越高，越早找出最接近最後等待的 requester，但執行成本也越高；固定 interval 可能一次看到多個 cycle。

終止 process 的恢復

全部終止。 一次 abort 所有 deadlocked processes，保證拆環，但損失全部未完成工作。

逐一終止。 每次 abort 一位 victim，回收其全部 resource，再重跑 detection。未終止者若因資源返回而可進展，就停止犧牲。

victim 的選擇是 policy：

- priority 與互動程度。
- 已執行時間、剩餘估計時間。
- 持有與需要的 resource 數量。
- 資源是否容易 reclaim。
- 需要終止多少個 process 才能拆環。

若 process 正在修改 file 或驅動裝置，直接 abort 可能留下外部半完成狀態。恢復設計要把 rollback、journal 或 device reset 納入成本，不能只把 lock 視為可回收物件。

resource preemption 與 starvation

resource preemption 要回答三個獨立問題：

1. **Victim。** 要收回哪位 process 的哪些 resource，成本如何計算。
2. **Rollback。** 被收回者要回到哪個一致狀態。最容易是全部 abort 後 restart；較省成本的是回到 checkpoint，但需要保存足夠 undo state。
3. **Starvation。** 同一 process 不能永遠因 cost 最低而被選為 victim。

`cost_i = execution_i + rollback_count_i × k`（k>0）會提高反覆回滾者的相對成本，是減少重複選取的啟發式。只有在競爭集合與其他候選成本受適當限制時，才可推導它何時不再最低；任意新到達或成本變動下，不能據此保證公平。明定每個 process 最多連續被選 r 次（r 有限）可限制連續受害次數，但仍須交代沒有其他可選 victim 時的恢復策略；A、B 仍可能交替無限次被選，因此此局部限制不能保證全體最終完成。有限等待或完成保證須由完整 workload 與選取規則推導。

deadlock 判斷的完整決策表

你觀察到什麼	可以先得到什麼	還需要什麼
四必要條件成立	deadlock 可能	是否存在無可進展 transition
single-instance wait-for graph 無 cycle	未 deadlocked	不需再做 cycle 充分性判斷
single-instance graph 有 cycle	deadlocked	確認所有 cycle resource 都是單 instance
multi-instance graph 有 cycle	deadlock 可能	availability/reduction 或 safety 分析
unsafe state	avoidance 不能提供完成保證	實際 request 可能尚未造成 deadlock
某 process 長期未執行但系統持續前進	starvation 可能	公平 policy 與等待上界

deadlock 是等待集合沒有可完成的內部 transition。starvation 是系統仍有 transition 與進展，但某個體沒有 finite wait bound。兩者可以同時存在，也可以各自獨立發生。

本章收束

- 必要條件不能單獨證明充分
- avoidance 依賴最大需求宣告
- 檢測、恢復與資料一致性必須一起設計

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 7.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：作業系統如何把連續的 logical address 安全映射到有限且分散的 physical memory？

前置知識。 二進位位址、暫存器與 memory bus、process 隔離。

- 推導 base/limit、segment、page 的轉換
- 比較 internal/external fragmentation
- 分析 TLB 命中率與 page table 層級

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

位址保護：先檢查，再重定位

在單一連續區段模型中，base 是該行程實體區段的起點，limit 是區段長度。CPU 產生 offset d 後，硬體先檢查 `0 ≤ d < limit`；通過才計算 physical address `base + d`，否則觸發保護例外。base 與 limit 的設定由特權程式控制。

例如 base=`0x8000`、limit=`0x2000`，d=`0x1FFF` 的實體位址是 `0x9FFF`；d=`0x2000` 正好等於長度，必須拒絕。若一次存取跨越多個位元組，整個範圍都要在界內，不能只檢查起始位址。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swapping

- swapping 把整個 process 移到 backing store，以釋放主記憶體；需要可容納 process 的連續或可映射空間與恢復狀態。
- 若每個 page 都可選擇性載入，demand paging 比 whole-process swapping 細緻；成本與政策不同。

例如 80 KiB 的行程換出至可用空間 100 KiB 的 backing store，須保存必要狀態；若 DMA 仍使用某個 buffer，須等待完成或採 pin 等方式防止其 frame 被回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續配置：總空間足夠也可能放不下

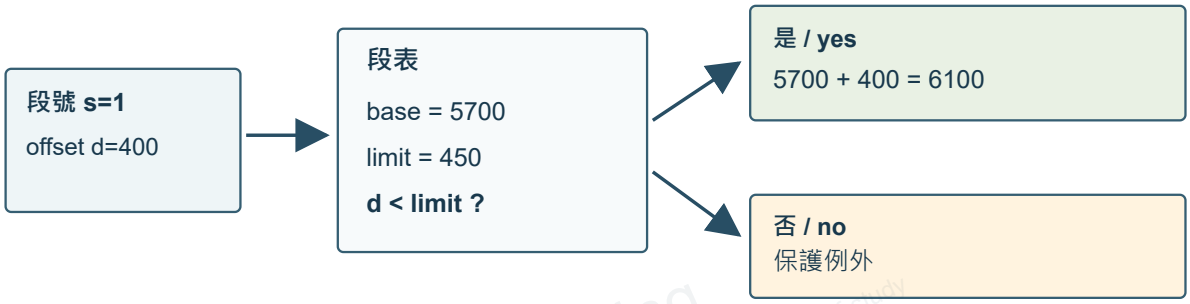
空洞依位址順序為 100、500、200 KiB；新的連續需求為 180 KiB。first-fit 選第一個足夠的 500，留下 320；best-fit 選 200，留下 20；worst-fit 選最大的 500，留下 320。三種策略都是針對同一初始狀態，各自獨立試配。

另一狀態有四個分離的 5 KiB 空洞。總共雖有 20 KiB，仍沒有單一區段容納 10 KiB：這是 **external fragmentation**。若使用 4 KiB 配置單位滿足 6 KiB 需求，配置 8 KiB、其中 2 KiB 未使用，則屬 **internal fragmentation**。壓縮需要能重新定位，且必須協調正在進行的 I/O 和位址引用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Segmentation：每個段都有自己的界線

段位址的保護檢查



offset=449 合法；offset=450 超界

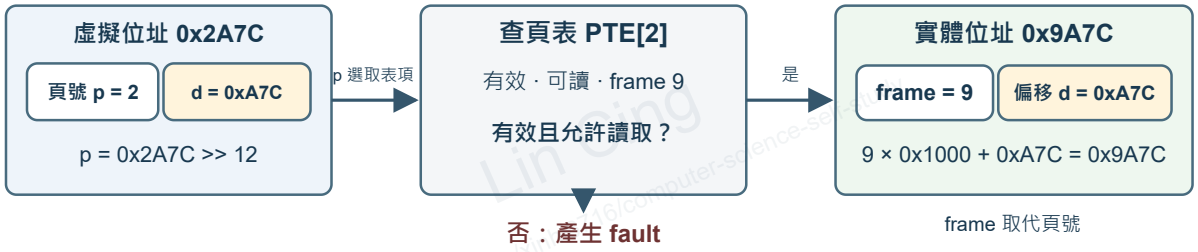
指定 segment 0 的 base=1000、limit=339，segment 1 的 base=5700、limit=450，segment 2 的 base=6300、limit=1280；數值均為 byte。`<0,330>` 因 `330<339` 合法，轉為 1330；`<1,400>` 轉為 6100；`<2,1380>` 因 `1380≥1280` 觸發例外。

limit 是長度而非最後合法 offset：segment 1 的 offset 449 合法，450 不合法。不同 segment 可以有不同長度、權限和實體起點；同一段內則依此模型使用連續實體區間。

Paging

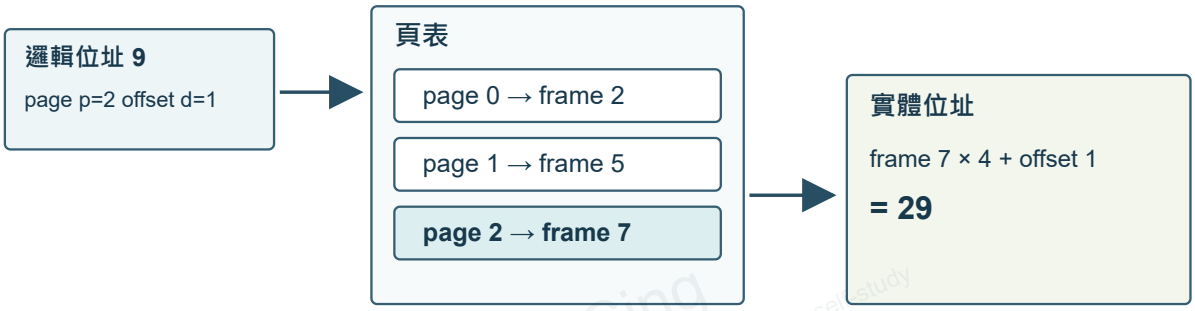
- logical address 分成 page number p 與 offset d。page table[p] 給 frame number f，實體位址為 f×page_size+d。
- page 大小是 trade-off。較小頁減少 internal fragmentation，但增大 table 與轉換 overhead；較大頁降低項目數，卻可能浪費每個物件的尾端空間。

4 KiB 分頁：從虛擬位址到實體位址



一次頁面轉換：從位址 9 到 29

4-byte 頁面：先分解，再查表



offset 不變；頁號經頁表轉為 frame 號

page size = 4 bytes。對 logical address 9，先算 `p = floor(9/4) = 2`、`d = 9 mod 4 = 1`；查表得到 frame `f = 7`，因此 physical address 為 `7×4+1 = 29`。offset 原樣保留；page 與 frame 的編號不必相等。

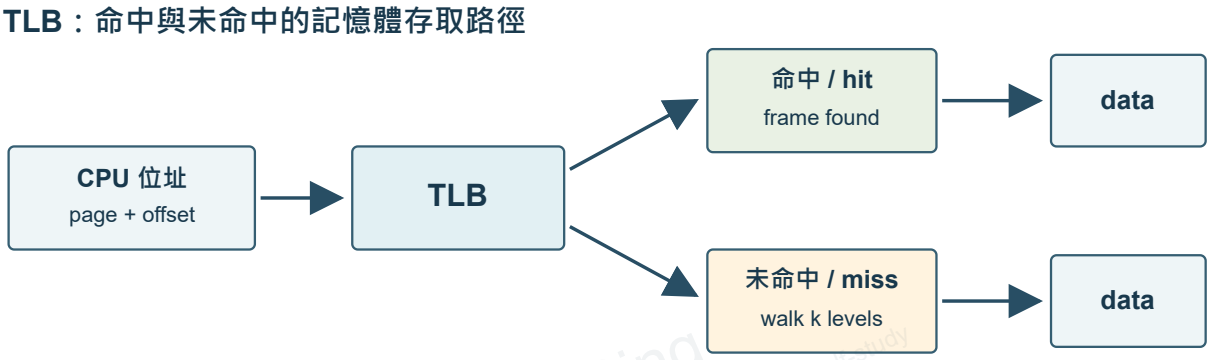
本例只列有效的 page 0、1、2；若查不到有效的 page-table entry，不能依頁號猜 frame，而應依系統規則產生 fault 或處理缺頁。

位址位元與記憶體碎片

每頁 1 KiB = 2^{10} bytes、每個 address space 有 8 = 2^3 pages，則 logical address 需要 3 個 page bits 加 10 個 offset bits，共 **13 bits**。若實體記憶體有 32 = 2^5 frames，physical address 需要 5 個 frame bits 加同樣的 10 個 offset bits，共 **15 bits**。假設位址以 byte 為單位，且兩種位址空間均使用完整編碼範圍。

用 10 KB 的頁框配置一段 32 KB 的連續邏輯區域，需要 4 頁、分配 40 KB；若前三頁填滿，最後一頁用 2 KB，內部碎片為 8 KB。這裡 KB 代表 1000 bytes，僅用於容量算例；二進位位址拆分的 1 KiB 則是 1024 bytes。

TLB 命中與未命中如何計時？



串列模型：hit = TLB + data；miss = TLB + k 次表項讀取 + data

TLB hit 時取得 frame 後訪問資料；miss 時需要查 page table，再訪問資料。以下採串列查找、各次記憶體訪問成本相同、固定頁表層數，且不計 cache、page-walk cache、替換與 page fault。

一層頁表時，TLB lookup = 10 ns，RAM access = 200 ns，hit rate = 0.8。hit 成本 `10+200=210 ns`；miss 成本 `10+200+200=410 ns`。因此 EAT = `0.8×210 + 0.2×410 = 250 ns`。這是此模型下的平均值，不是所有 CPU 的實測延遲。

頁表多一層，miss 成本怎麼變？

改用兩層頁表、TLB lookup = 20 ns、RAM access = 200 ns、hit rate = 0.8；其他假設不變。hit 是 `20+200=220 ns`；miss 先後讀兩層 entry 和資料，為 `20+2×200+200=620 ns`。

所以 EAT = `0.8×220 + 0.2×620 = 300 ns`。比較兩個數例時，同時改了 TLB lookup 時間和頁表層數；不能把 50 ns 的差距全部歸因於多一層頁表。實機也可能平行查找或快取中間表項。

一般而言，hit rate 為 h、TLB 時間 t、每次 memory 時間 m、miss 查 k 層時，EAT = h(t+m)+(1−h)(t+km+m)。

保護與共享

- 每個 page table entry 可含 valid、read/write、execute、user/kernel 等位元；另一 process 只有在其 page table 映射同一 frame 時才可共享。
- 共享唯讀 code 可節省記憶體，但寫入或可執行權限若設錯，就會破壞隔離。

例如 P1 與 P2 的頁表各映射同一實體 frame，且兩邊均為 user-readable、read-only；兩者可讀取同一內容，user write 則被拒。移除其中一個映射後，另一個仍可存在；回收前須確認所有引用及 TLB 失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩層頁表：空間取決於實際映射

32-bit byte address、4 KiB pages、每筆 entry 4 bytes。一張 4 KiB 表可放 $1024 = 2^{10}$ 筆 entry；位址分成 `10-bit root index | 10-bit leaf index | 12-bit offset`。每張 leaf table 覆蓋 1024 頁，即 4 MiB 虛擬位址。

映射一頁時，若根表必須完整配置且恰好只需一張 leaf，表格本身為 `4 KiB + 4 KiB = 8 KiB`；資料頁不含在內。全部 2^{20} 個虛擬頁都映射時，需要 1024 張 leaf，合計 `4 KiB + 1024×4 KiB = 4 MiB + 4 KiB`。稀疏配置節省未使用的 leaf，不能把 8 KiB 當成所有兩層頁表的最大值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倒排頁表：按 frame 而非虛擬頁計數

若實體記憶體為 1 GiB、page/frame 為 8 KiB，則 frame 數為 $2^{30} / 2^{13} = 2^{17}$。假設每個 frame 的記錄恰好 4 bytes，這些記錄共 $2^{17} \times 4 = 2^{19}$ bytes = 512 KiB。

這只是依 frame 編號排列的記錄空間；用來加速以 (address-space ID, virtual page) 查詢的 hash/index、碰撞鏈結及共享 frame 的多重映射紀錄都可能額外占空間。記錄格式亦須足以保存所需識別與權限，4 bytes 是本例假設，不是通用實作大小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IA-32 protected mode：從 linear address 到 page entry

本模型固定 IA-32 protected mode、CR0.PG=1、CR4.PAE=0 與 4 KiB page；輸入是已通過 segment descriptor 檢查的 32-bit linear address。segment selector 先用 LDT/GDT descriptor 的 base 與 limit 形成 linear address，paging unit 再把 linear address 轉成 physical address。因此 0x12345678 是 linear address，不把它誤稱為尚未經過 segmentation 的 logical address。

每張 IA-32 page directory 與 page table 有 $4096/4 = 1024$ entries，索引各 10 bits。令 linear address 為 L：

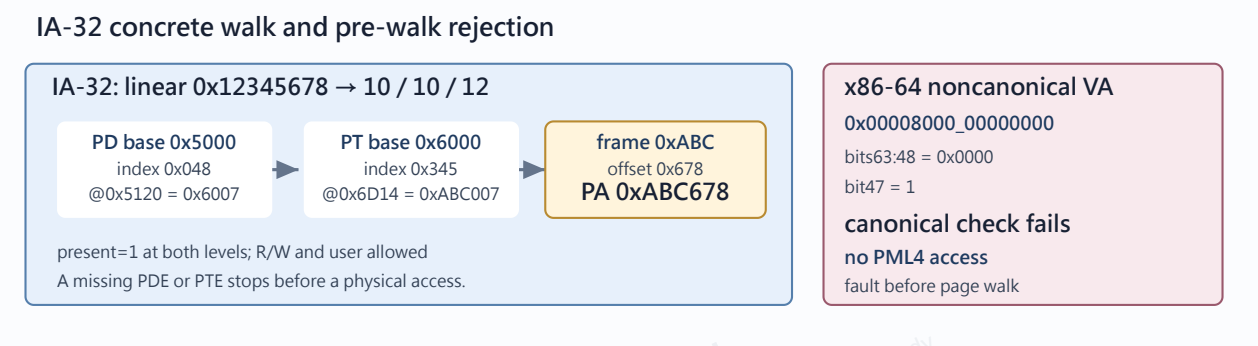
$$\mathrm{PDI} = (L \gg 22)\&0x3FF, \quad \mathrm{PTI} = (L \gg 12)\&0x3FF, \quad \mathrm{offset} = L\&0xFFF.$$

固定 CR3 page-directory base=0x5000、下一層 page-table base=0x6000。entry address 由 base+index×4 得到：

層級	index	entry 位址	表項內容	解讀
page directory	0x048	0x5000+4×0x048=0x5120	0x00006007	present；next PT base=0x6000；R/W、user 允許
page table	0x345	0x6000+4×0x345=0x6D14	0x00ABC007	present；4 KiB frame=0xABC；R/W、user 允許

因此 offset=0x678，physical address=0xABC000+0x678=0xABC678。這一路徑使用 page directory、page table 與目標 page 三個 4 KiB 記憶體頁；table 本身的 page-table 配置成本另計，但由上表的 entries 可完全重建。

IA-32 protected mode：從 linear address 到 page entry （續）



若 PDE 的 present=0，paging unit 在讀下層 PT 前就 fault；若 PTE 的 present=0，不能把 frame 欄位解成有效記憶體。若 CPU 支援 4 MiB page 且 `CR4.PSE=1`，PDE 的 `PS=1` 會讓 PDI 之後的 22 bits 成為 offset，直接跳過 page table；這是不同的 encoding。若 `CR4.PSE=0`，32-bit paging 忽略 PDE 的 PS 位，改把該 PDE 當作指向 4 KiB page table，不能當成 4 MiB 映射。

x86-64 canonical address 與四層頁表

OSC 9e 的 x86-64 案例使用 48-bit virtual address 與四層 4 KiB paging。有效位元為 `[47:39]`、`[38:30]`、`[29:21]`、`[20:12]`，每層 9-bit index；offset 為 `[11:0]`。每張 table 有 $4096/8 = 512$ entries。

本頁固定48-bit canonical模式，輸入先限定0≤v<2^64。Canonical condition要求bit47複製到所有`[63:48]`，不能只比較bit63與bit47：

$$\text{canonical}(v) = \left((v \gg 48) = \begin{cases} 65535, & ((v \gg 47)\ \&\ 1) = 1, \\ 0, & ((v \gg 47)\ \&\ 1) = 0. \end{cases} \right)$$

反例`v=0x0001_0000_0000_0000=2^48`的bit63=bit47=0，但bits63:48=1而非0，故noncanonical，不能walk。合法lower-half位址若單獨把任一bit48…63翻成1，都破壞sign extension；合法upper-half若把任一對應bit翻成0亦然。四個邊界為0與0x00007FFFFFFFFFFF合法、0x0000800000000000至0xFFFF7FFFFFFFFFFF非法、0xFFFF800000000000至0xFFFFFFFFFFFFFFFF合法。

固定 CR3=`0x1000`，沿路 table bases 為 PDPT=`0x2000`、PD=`0x3000`、PT=`0x4000`。目標 VA=`0x00007FFF_FFFFF123` 的 bit47=0、bits63:48=0，所以 canonical。各層都提 9 bits：

x86-64 canonical address 與四層頁表 （續）

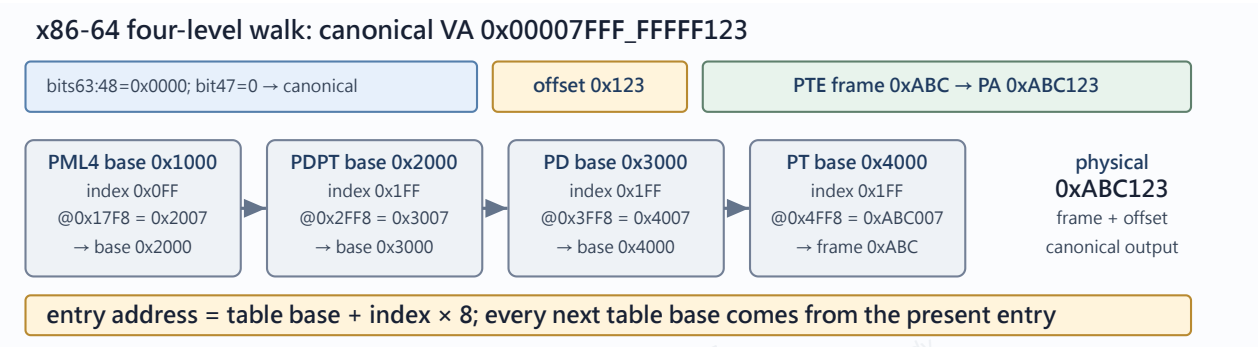
層級	index	entry offset	entry 位址	entry 值	下一個 frame/table
PML4	`0x0FF`	`0x7F8`	`0x17F8`	`0x2007`	PDPT base `0x2000`
PDPT	`0x1FF`	`0xFF8`	`0x2FF8`	`0x3007`	PD base `0x3000`
PD	`0x1FF`	`0xFF8`	`0x3FF8`	`0x4007`	PT base `0x4000`
PT	`0x1FF`	`0xFF8`	`0x4FF8`	`0xABC007`	data frame `0xABC`

entry$_\ell$ 位址的一般式是

$$address_\ell = base_\ell + index_\ell \times 8.$$

最後 offset=`0x123`，故 physical address=`0xABC000+0x123=0xABC123`。若只算 indices，仍不足以重建表格；本例另給每層 base、entry 位址與表項值，任何一步都可從上表重新展開。

x86-64 canonical address 與四層頁表 （續）



2 MiB 與 1 GiB large-page encodings 會在較上層終止 walk，並把較多低位址 bits 當 offset。它們需要不同 entry format 與 alignment；本頁的 `9/9/9/9/12` 只描述指定的 4 KiB 路徑。

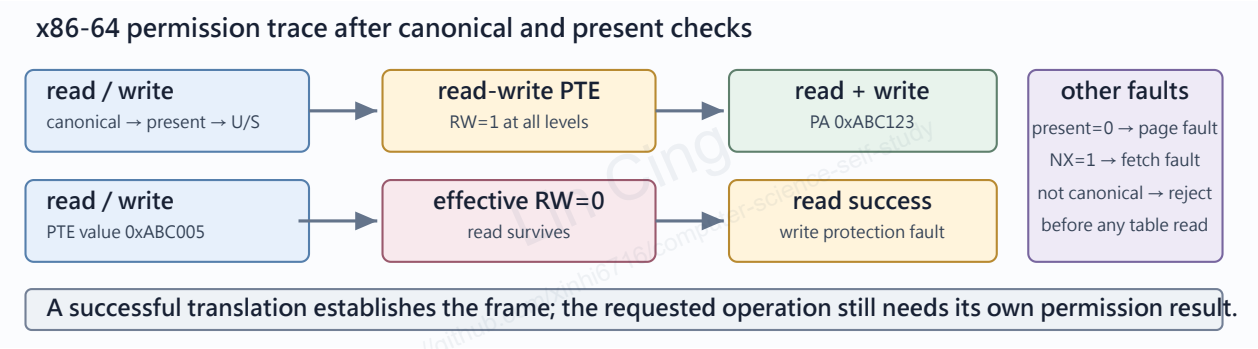
合法、非法與權限 fault 的完整分流

本頁固定四層 48-bit x86-64 paging、user-mode `CPL=3`，CPU 支援 NX 且 `IA32_EFER.NXE=1`；SMEP、SMAP、PKE 等額外保護均不介入。canonical、present、U/S、R/W 與 NX 回答不同問題。canonical 失敗時不產生 page-table memory access；canonical 通過後，page walk 先找 mapping，再依操作合併權限。

案例	canonical	mapping	effective permission	read	write	execute
本頁正常路徑	true	四層 present	全層 U/S=1、R/W=1、NX=0	`0xABC123`	`0xABC123`	允許
PTE 改為 `0xABC005`	true	四層 present	全層 U/S=1；R/W=0；NX=0	`0xABC123`	protection fault	允許
本次PML4[0x0FF] absent	true	present=0	不成為有效映射	page fault	page fault	instruction page fault
PTE 加上 NX	true	四層 present	R/W 允許；NX=1	資料讀取成功	資料寫入成功	instruction fetch fault
`0x00008000_00000000`	false	不應 walk	無法定義	reject	reject	reject

合法、非法與權限 fault 的完整分流 （續）

canonical check 不可取代 U/S；user 存取要求沿途 entry 的 user 權限允許。本例 user write 需要 effective R/W=1；supervisor write 另受 `CR0.WP` 等條件影響，不可直接套用。NX 區分資料讀取與 instruction fetch；若 `EFER.NXE=0`，表項的 XD/NX bit 為保留位，將其設為 1 會造成 reserved-bit page fault，不能只視為禁止執行。root 高位、table entry 與 page frame 都存在，也不等於任意操作有權限。



這個模型固定 48-bit 4-level 設定。PAE、large pages、5-level paging 或不同 physical-width 會改變欄位與 entry 數，必須換成對應模式；不能把本頁的 masks 當成所有 Intel CPU 的通則。TLB 有效期則由 entry 更新與必要的 invalidation 決定。

ARMv7-A AArch32：short-descriptor 的兩層 walk

本頁固定 ARMv7-A、AArch32、short-descriptor translation format、`TTBCR.N=0`、`TTBCR.PD0=0`、`SCTLR.AFE=0`、PL1 建表及 4 KiB small pages。P1、P2 使用不同且尚未重用的 ASID，例如 1、2。這段不是 AArch64 long descriptor。

short-descriptor 格式的 L1 table 有 4096 個 4-byte entries，總大小為 $4096 \times 4 = 16,384$ bytes，即 16 KiB。`TTBCR.N=0` 時所有輸入 VA 都由 TTBR0 選 root，並以 VA `[31:20]` 索引完整 L1 table；TTBR0 的 table base 因此必須 16 KiB 對齊。

$$L1_address = TTBR0 + ((VA \gg 20)\&0xFFF) \times 4.$$

coarse L1 page-table descriptor 指向 L2 table。short-descriptor 的 L2 table 有 256 個 4-byte entries，總大小 1 KiB，base 只需 1 KiB 對齊；硬體使用 descriptor 的 `[31:10]` 作為 table base，因此不能任意插入非對齊位址。

$$L2_address = L2_base + ((VA \gg 12)\&0xFF) \times 4, \qquad PA = frame + (VA\&0xFFF).$$

ARMv7-A AArch32：short-descriptor 的兩層 walk （續）

本例的四個 table 區間完整且互不重疊：

storage	base	range	size	alignment check
P1 root	`0x8000`	`0x8000–0xBFFF`	16 KiB	`0x8000 % 0x4000=0`
P2 root	`0xC000`	`0xC000–0xFFFF`	16 KiB	`0xC000 % 0x4000=0`
P1 L2	`0x10000`	`0x10000–0x103FF`	1 KiB	`0x10000 % 0x400=0`
P2 L2	`0x10400`	`0x10400–0x107FF`	1 KiB	`0x10400 % 0x400=0`

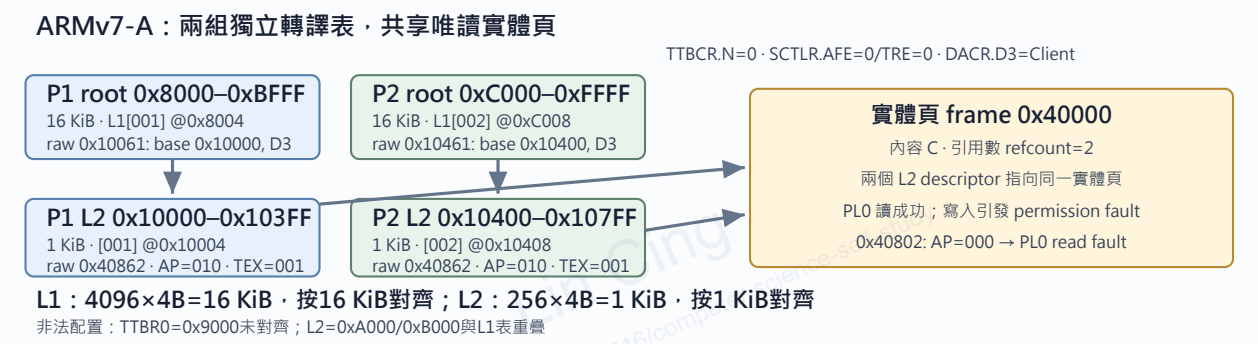
P1 VA `0x00101000` 與 P2 VA `0x00202000` 的實際位址為：

process	VA	L1 index → entry address	L2 base	L2 index → entry address	frame	PA
P1	`0x00101000`	`0x001→0x8004`	`0x10000`	`0x001→0x10004`	`0x40000`	`0x40000`
P2	`0x00202000`	`0x002→0xC008`	`0x10400`	`0x002→0x10408`	`0x40000`	`0x40000`
P1，非零 offset	`0x00101A5C`	`0x001→0x8004`	`0x10000`	`0x001→0x10004`	`0x40000`	`0x40A5C`

第三列的 page offset 是 VA `[11:0]=0xA5C`；它不參與任一層索引，會完整保留至輸出位址。只保留最低 8 位會錯把 PA 算成 `0x4005C`。

ARMv7-A AArch32：short-descriptor 的兩層 walk （續）

每個 address space 都保留完整 root 儲存。即使 VA 只使用少數 entries，也不能只保存那幾筆 descriptor；其餘 entries 必須有架構允許的 invalid 或其他有效內容。兩張 16 KiB roots 與兩張 1 KiB L2 tables 佔用不同的 physical ranges。



raw descriptors、domain、AFE 與 AP

本例在每個 L1 page-table descriptor 使用 domain 3，並設定 `DACR.D3=0b01`，即 Client。domain 只存在 L1 page-table descriptor；L2 entry 繼承該 domain。

當 `SCTLR.AFE=0` 時，使用 full AP[2:0] 模型。small page 的 AP[2] 在 bit9、AP[1:0] 在 bits[5:4]；不能套用 section descriptor 的 bits[11:10]。本例 L2 解碼後 `AP[2:0]=010`：PL1 read/write、PL0 read-only，所以 PL0 write 產生 Permission fault。

可從 raw word 直接解碼：

descriptor	raw word	解碼
P1 L1	`0x10061`	bits[1:0]=01 coarse；table base=`0x10000`；domain=3
P2 L1	`0x10461`	bits[1:0]=01 coarse；table base=`0x10400`；domain=3
P1/P2 L2	`0x40862`	frame=`0x40000`；type bits[1:0]=10；XN=0；nG=1；AP[2:0]=010；TEX=001、C=B=0

small-page descriptor 的 `field = (raw_word >> shift) & mask` 必須逐項套用：`AP2=(raw>>9)&1`、`AP1=(raw>>5)&1`、`AP0=(raw>>4)&1`、`TEX=(raw>>6)&7`、`C=(raw>>3)&1`、`B=(raw>>2)&1`、`XN=raw&1`。本例得到 AP010、TEX001、C=B=0、XN=0、nG=1。

raw descriptors、domain、AFE 與 AP （續）

`TEX=001,C=B=0` 在 `SCTLR.TRE=0` 下解為 Normal non-cacheable。instruction fetch 不能只由 XN=0 推出；本模型另固定 `SCTLR.WXN=UWXN=0`、mapping 可讀、domain Client，且沒有其他實作限制。若只改記憶體屬性為 strongly ordered，不能憑 XN=0 宣稱一定可直接執行。

同一個 frame 的 descriptor 可以給出不同權限。比較下列兩個設定：

raw word	正確解碼	結果
`0x40802`	AP[2:0]=000；TEX=0,C=B=0	AP 禁止 PL0 read/write；frame 位址有效也無法通過 read 權限檢查
`0x40022`	AP[2:0]=010；TEX=0,C=B=0	只示範 data AP；Strongly-ordered 不可藉 XN=0 推 instruction fetch

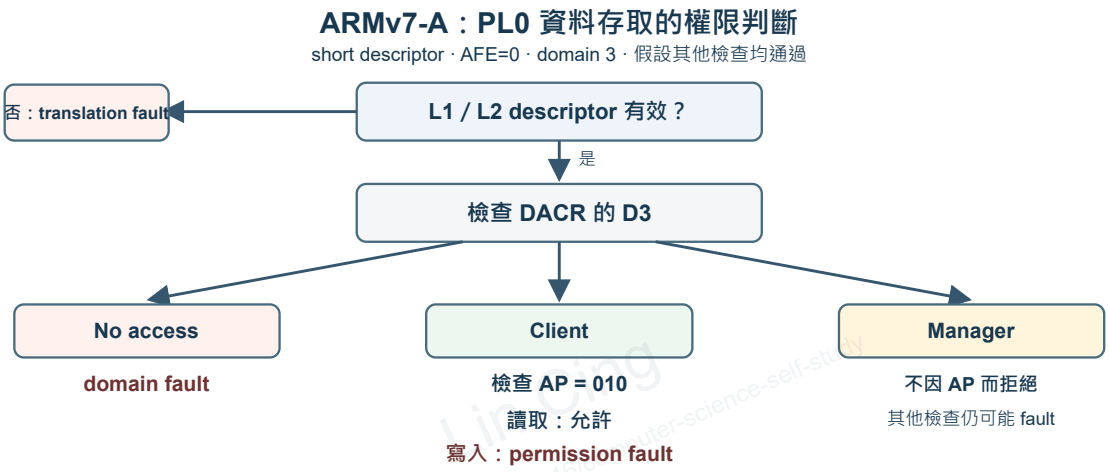
解碼時必須分別讀取 type、frame、AP 與記憶體屬性；只認出 frame 位址，尚不足以判定存取結果。

permission 不是只看 L2 word。DACR D3=Client 時才檢查 AP；若 D3=Manager（`0b11`），short-descriptor 的 AP 不被檢查，因此不能再用相同 L2 word 推出 write 必 fault。若 `SCTLR.AFE=1`，AP[0] 被重新定義成 Access flag，AP 模型改為 AP[2:1]；必須重新解碼 descriptor，不能沿用 AFE=0 的結論。

raw descriptors、domain、AFE 與 AP （續）

條件	PL0 read	PL0 write	理由
AFE=0、D3=Client、AP=010	成功	Permission fault	AP 對 PL0 授予 read、不授予 write
AFE=0、D3=No access	Domain fault	Domain fault	AP 檢查尚未成為成功證據
AFE=0、D3=Manager	不因 AP 被拒	不因 AP 被拒	Manager 不檢查 AP；其他架構 fault 另計
AFE=1、Client、同一字 `0x40862`	Access flag fault	Access flag fault	AP[0]=0；本例採軟體設定 Access flag
AFE=1、Client、改成 `0x40872`	成功	成功	AP[0]=1，AP[2:1]=01；此模型給 PL0 read/write，與 AFE=0 的 AP010 不同

raw descriptors、domain、AFE 與 AP （續）



表項與 domain 有效後才能判定權限；不要把位址轉換成功當作可寫。

權限 trace、mapping 移除與 TLB

假設 frame `0x40000` 僅有這兩筆映射，沒有其他引用或 pin；初始 mapping count=2。兩張 L2 entries 都是 `0x40862`，其 parent domains 都是 Client。P1、P2 分別使用尚未重用的不同 ASID 1、2，且 `TTBCR.PD0=0`。TLB 起初沒有這個 translation。

狀態與操作	translation	domain/AP	結果	frame 或 mapping count
S0：P1 read	`0x00101000→0x40000`	Client；AP=010 允許 PL0 read	success	C；mapping count 2
S0：P1 write	translation 有效	Client；AP=010 禁止 PL0 write	permission fault	C 不變
S0：P2 read	`0x00202000→0x40000`	Client；AP=010 允許 PL0 read	success	C；mapping count 2
S0：P2 write	translation 有效	Client；AP=010 禁止 PL0 write	permission fault	C 不變
S1：P1 descriptor absent	P1 L2 entry invalid，已完成對應 TLB maintenance	不再檢查資料 AP	translation fault	P2 仍映射；mapping count 1

權限 trace、mapping 移除與 TLB （續）

狀態與操作	translation	domain/AP	結果	frame 或 mapping count
S2：P2 read after P1 removal	獨立 P2 tables 仍有效	Client/read allowed	success	mapping count 1
S3：P2 descriptor absent	P2 L2 entry invalid，已完成對應 TLB maintenance	不再檢查資料 AP	translation fault	mapping count 0，所有舊 translation 已失效後 frame 才可回收

translation fault 與 permission fault 是不同證據。S1 的 P1 沒有效 mapping，因此不能說「頁存在只是不能寫」；S0 的 write 已有有效 mapping，且在 Client domain 下 AP 拒絕 PL0 write。

更新 descriptor 後，要依架構規則對所有可能持有舊 translation 的相關核心完成對應 VA／ASID 的 TLBI 與必要 barrier，確認失效完成後才能回收 frame。本例 nG=1，translation 屬於各自的 ASID；ASID 不決定 domain 或 AP。stale TLB entry 可能保留舊 translation 或權限，因此上表的 unmap 結果以失效操作完成為前提，不能在清除 PTE 後立即回收仍可能被舊 translation 存取的 frame。

先驗 storage layout，再談共享 frame

下列配置即使能算出 entry 位址，仍不構成合法硬體配置：

錯誤配置	問題	判定
P2 TTBR0=`0x9000`，N=0	`0x9000%0x4000=0x1000`，未達 16 KiB 對齊	invalid root
P1 root=`0x8000`、P2 root=`0x9000`	ranges `0x8000–0xBFFF` 與 `0x9000–0xCFFF` 重疊	invalid ownership
P1 L2=`0xA000`	落在 P1 root `0x8000–0xBFFF` 內	table storage 重疊
P2 L2=`0xB000`	同樣落在 P1 root 內	table storage 重疊
只為 16 KiB L1 配置 4 KiB	4096 個 4-byte entries需要完整 16 KiB	storage 不足

修正後，P1/P2 各有獨立 roots 與 L2 tables，只有最後的 `frame 0x40000` 被共享。page identity、各層 storage、domain、AP、XN、frame reference count 與 TLB state 必須一起核對。

若 P1 descriptor 真的可寫，P1 的 write 會修改 P2 看見的共享 frame；這是 shared writable page 的語意，不是 read-only code。若系統要讓 P1 修改而 P2 保留舊內容，應使用 copy-on-write 或新 frame，不能只改其中一份 descriptor。

本章收束

- page mapping 與權限是同一個 entry 契約的兩部分
- internal/external fragmentation 必須分類
- EMAT 是模型，需列出 cache 與 fault 假設

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 8. Intel 64 and IA-32 SDM, Vol. 3A; Arm Architecture Reference Manual ARMv7-A/R.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：當工作集可能大於實體記憶體時，缺頁、替換與配置政策如何維持正確性並控制延遲？

前置知識。 paging 位址轉換、page protection fault、儲存裝置延遲。

- 追蹤缺頁服務時序
- 比較 FIFO、OPT、LRU 與 clock
- 推導 page fault rate、frame allocation 與 thrashing

virtual reservation、commit/backing承諾與resident是不同量。合法VMA可先保留位址；是否已承諾足夠RAM/swap由系統策略決定；resident只算目前在RAM的頁。因此「virtual空間大於RAM」本身不表示所有頁可同時寫入成功，也不表示記憶體永不配置失敗。

位址合法、內容來源與實體駐留各問一件事

固定4 KiB pages、兩個process A/B，兩者虛擬位址配置相同。表中frame名稱只在這個例子使用，沒有以相同virtual page number推論相同physical page。

virtual範圍	合法性與權限	內容來源	A目前映射	B目前映射
[0x0000,0x1000)	合法、read/execute、不可write	同一程式code	共享F0	共享F0
[0x1000,0x2000)	合法、read/write、私有	zero-fill-on-demand	尚未resident	尚未resident
[0x2000,0x3000)	合法、read/write、私有	各自data	F1	F2
[0x3000,0x4000)	guard，無合法mapping	無	無	無

每個process有3個合法virtual pages、2個resident mappings；全系統4個resident mappings卻只占3個不同frames，因code F0共享。把每個process的resident數直接相加會重複計共享frame。

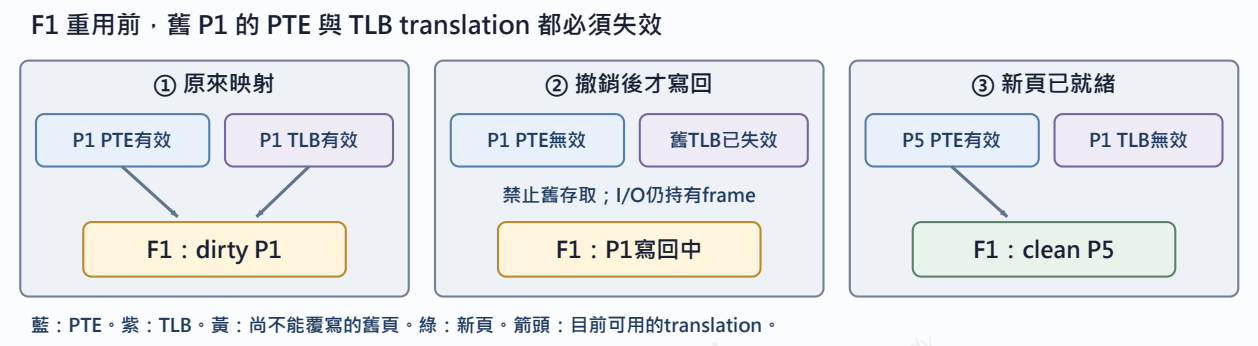
A首次讀0x1004時，可配置清零頁後重啟；A寫0x0004則是權限違反；讀0x3004是guard存取，不能因有free frame就建立原本未授權的mapping。page table的nonpresent狀態必須結合VMA/backing metadata判斷，單看一個bit無法區分三種結果。

Demand-paging fault：一次存取如何完成

固定 4 KiB pages、三個 frames 全部佔用；P5 位於合法 VMA 的 backing object，但尚未 resident。目前 frames 是 F0→P0、F1→dirty P1、F2→P2，替換政策選 F1。這是合法的 read fault，不是非法位址。

step	可觀察狀態	操作	此時 P5 可否被讀
0	P5 invalid；F0/F1/F2 已滿	CPU 存取 P5，hardware trap	否
1	VMA 合法、backing object 存在、read 權限通過	分類為 nonresident fault	否
2	victim F1 dirty	撤銷 P1 PTE，完成舊 TLB translation 的失效；禁止其他 CPU 再寫 F1	否
3	F1 為 P1 writeback，仍由 fault/I/O 路徑持有	等 I/O 完成，不能覆蓋 F1	否
4	F1 顯示 P5 loading	從 backing store 讀入 4096 bytes	否，mapping 未完成
5	P5→F1；必要的 TLB entry 已失效	重啟 faulting instruction	是

Demand-paging fault：一次存取如何完成 （續）



此模型計入 1 次 page fault、1 次 4096-byte disk write、1 次 4096-byte disk read，並有一個「撤銷舊 P1 翻譯」的 TLB 維護階段。單 CPU 模型只需本地失效；多 CPU 若曾快取該翻譯，須等待相關 shootdown 完成。安裝 P5 後是否另需失效或 barrier 由架構決定，不能把「所有架構恰好一條 TLB 指令」當結論。若 read 與 write 各 8 ms，且忽略其他開銷、不重疊，fault service 是 16 ms；clean victim 省去 writeback，為 8 ms。

缺頁率與有效存取時間

令 m 為正常參考的總成本、F 為發生 fault 的參考從開始到重試完成的總成本；0≤p≤1。以下門檻推導假設 F>m，且 EAT 上限不小於 m，避免重複計入重試存取：

$$\mathrm{EAT} = (1-p)m + pF, \qquad p \le \frac{\mathrm{EAT}_{\max} - m}{F - m}.$$

令 m=200 ns、clean fault F=8,000,000 ns，要求 EAT≤400 ns 得 p≤200/7,999,800≈2.5001×10^-5；若每次 fault 必須先寫 dirty victim 再讀入，F=16,000,000 ns，門檻降為約1.2500×10^-5。fault rate 相同的應用，dirty-writeback 比例仍可令成本翻倍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

合法缺失、保護錯誤與 prefetch 的分流

fault handler 依序回答：

1. 位址是否落在合法 VMA 或 backing object？
2. 是否已有合法 mapping 的軟體描述，只是 PTE 目前 nonpresent？nonresident 不代表有有效的硬體 translation。
3. requested operation 是否符合 read/write/execute 權限？若只因 COW 暫時關閉 write bit，另走 COW 分支。
4. 是否有 free frame？沒有時選哪個 victim、是否 dirty/writeback/pinned？
5. 完成 I/O 後 mapping 與 TLB 是否一致，原指令可否安全重啟？

重用過早的反例。 CPU0 清除 P1 PTE，CPU1 卻仍有 P1→F1 的 writable TLB entry。若 CPU0 立即把 P5 載入 F1，CPU1 對 P1 的 store 會改壞 P5。故「寫 PTE」與「舊 translation 已不可使用」不是同一事件。反向地，若 handler 完全不安裝 P5 PTE 就返回，原指令會再次 fault。正確不變量是重用 frame 前舊 mapping/translation 不可再存取它，重啟前新 mapping 的內容與權限已就緒。

prefetch/read-ahead 可在一次 I/O 帶入 P5、P6；它減少後續 fault 次數，但 P6 仍必須合法、有 backing data、取得 frame 並安裝 mapping。不能因 batch read 成功就把非法相鄰頁自動加入 address space。

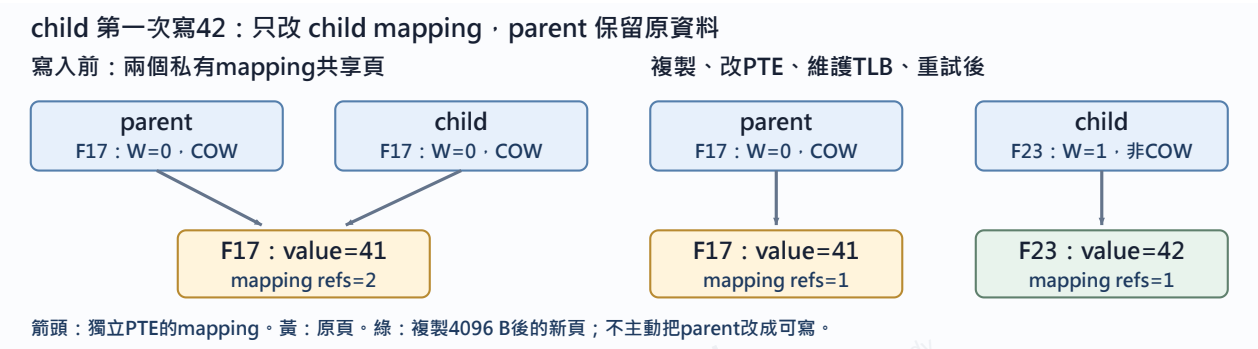
Copy-on-write：write fault 而非 missing-page fault

fork 後，parent 與 child 各自擁有 task 與 page-table metadata，但兩張 PTE 可暫時映射 F17；hardware 將兩者設為 read-only，OS 另記 COW。child 寫入時 translation 已存在，失敗的是 write permission，因此 fault handler 必須辨認 COW，而不能把它當成 backing-store page missing。

時刻	event	parent PTE	child PTE	F17	F23
S0	fork 完成	F17, W=0, COW	F17, W=0, COW	value=41, refs=2	不存在
S1	child 寫入	不變	不變	41, refs=2	不存在
S2	配置 F23，copy 4096 B，尚未公布新 PTE	F17 read-only	仍指 F17	41, refs=2	41，由 handler 持有
S3	公布 child PTE，完成翻譯維護，重試寫 42	F17, W=0, COW	F23, W=1, COW=0	41, refs=1	42, refs=1, dirty

必要順序是：驗證 COW fault→分配 frame→複製舊內容→更新 child PTE/TLB→重試 write。只清除 parent 的 COW 或把兩邊都改成 writable，會讓 child 直接改到 parent 的 F17。

Copy-on-write：write fault 而非 missing-page fault （續）



這裡 refs 只計入使用者 mapping，不把 handler 的暫時保留混入同一欄。child 的 fault 不必主動走訪並更新 parent PTE；parent 可以繼續保持唯讀。其後 parent 首次 write 再 fault 時，若確認 F17 是可獨佔的私有 anonymous page、沒有其他共享或引用限制，可只提升權限而不複製。僅憑「mapping 數等於1」不能把 file cache 的共享頁任意改成私有可寫。若 fork 後立刻 exec，未被寫過的 COW 頁可直接解除映射，省下無用複製。

zero-fill-on-demand 的私有資料頁在被寫入前必須先清零，不能把舊 frame 的殘留資料暴露給另一 process。COW、zero-fill 與 demand paging 都會在 fault path 配置 frame，但觸發原因、資料來源與安全檢查不同。

四種替換策略的完整狀態差異

固定 frames=3，reference string 為 `1,2,3,4,1,2,5,1,2,3,4,5`。每一 row 是一次 page access，`H/F` 表示 hit/fault；resident 的次序是該政策的 metadata，不把它誤當成唯一正確物理 frame 編號。

step/page	FIFO	LRU	OPT	CLOCK
1/1	[1]F	[1]F	[1]F	[1]F
2/2	[1,2]F	[1,2]F	[1,2]F	[1,2]F
3/3	[1,2,3]F	[1,2,3]F	[1,2,3]F	[1,2,3]F
4/4	[2,3,4]F	[2,3,4]F	[1,2,4]F	[4,2,3]F
5/1	[3,4,1]F	[3,4,1]F	[1,2,4]H	[4,1,3]F
6/2	[4,1,2]F	[4,1,2]F	[1,2,4]H	[4,1,2]F

四種替換策略：第 7–12 次存取

step/page	FIFO	LRU	OPT	CLOCK
7/5	[1,2,5]F	[1,2,5]F	[1,2,5]F	[5,1,2]F
8/1	[1,2,5]H	[2,5,1]H	[1,2,5]H	[5,1,2]H
9/2	[1,2,5]H	[5,1,2]H	[1,2,5]H	[5,1,2]H
10/3	[2,5,3]F	[1,2,3]F	[2,5,3]F	[5,3,2]F
11/4	[5,3,4]F	[2,3,4]F	[5,3,4]F	[5,3,4]F
12/5	[5,3,4]H	[3,4,5]F	[5,3,4]H	[5,3,4]H

總結是 FIFO=9、LRU=10、OPT=7、CLOCK=9 faults。victim sequence：

policy	victim sequence（- 表示 free frame 或 hit）
FIFO	`-, -, -, 1, 2, 3, 4, -, -, 1, 2, -`
LRU	`-, -, -, 1, 2, 3, 4, -, -, 5, 1, 2`
OPT	`-, -, -, 3, -, -, 4, -, -, 1, 2, -`
CLOCK	`-, -, -, 1, 2, 3, 4, -, -, 1, 2, -`

四種替換策略：第 7–12 次存取 （續）

FIFO 只看載入時間，所以 hit 不改變 queue；LRU 在 hit 時把頁移到最近端；OPT 知道未來，選擇下次使用最晚或不再使用的頁；clock 以 circular queue 與 reference bit 清除第一輪 chance。這些 metadata 是不可互換的。

Belady anomaly。 同一 reference string 上，FIFO 在 3 frames 為 9 faults，在 4 frames 反而為 10；增加 frame 不保證每一種 policy 的 fault count 單調下降。精確 LRU 的駐留集合有包含關係，因此缺頁數不隨容量增加而上升。OPT 的最小缺頁數也不會上升，但不能據此宣稱任意平手處理都會產生巢狀駐留集合；後文分別給證明。近似 LRU 不自動繼承精確 LRU 的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種置換策略：逐次核對（1–10）

同一參考序列 `7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1`，三個 frame 初始為空；只在缺頁時載入，沒有預取或額外失效。表中 slots 依固定實體槽位排列，– 代表空槽；H 是 hit、F 是 fault，F→x 表示 fault 並淘汰頁 x。

FIFO 淘汰最早載入且仍在記憶體的頁；LRU 淘汰最久未使用的頁；OPT 淘汰未來最晚使用的頁，永不再使用視為無限遠。OPT 若同分，選頁號較小者。

t	page	FIFO：slots；event	LRU：slots；event	OPT：slots；event
1	7	7,–,–；F	7,–,–；F	7,–,–；F
2	0	7,0,–；F	7,0,–；F	7,0,–；F
3	1	7,0,1；F	7,0,1；F	7,0,1；F
4	2	2,0,1；F→7	2,0,1；F→7	2,0,1；F→7
5	0	2,0,1；H	2,0,1；H	2,0,1；H
6	3	2,3,1；F→0	2,0,3；F→1	2,0,3；F→1
7	0	2,3,0；F→1	2,0,3；H	2,0,3；H
8	4	4,3,0；F→2	4,0,3；F→2	2,4,3；F→0
9	2	4,2,0；F→3	4,0,2；F→3	2,4,3；H
10	3	4,2,3；F→0	4,3,2；F→0	2,4,3；H

三種置換策略：逐次核對（11–20）

接續前頁；每格是**存取後**的三個固定槽位及本次事件。FIFO 命中不改載入順序，LRU 命中更新最近使用時間。

t	page	FIFO：slots；event	LRU：slots；event	OPT：slots；event
11	0	0,2,3；F→4	0,3,2；F→4	2,0,3；F→4
12	3	0,2,3；H	0,3,2；H	2,0,3；H
13	2	0,2,3；H	0,3,2；H	2,0,3；H
14	1	0,1,3；F→2	1,3,2；F→0	2,0,1；F→3
15	2	0,1,2；F→3	1,3,2；H	2,0,1；H
16	0	0,1,2；H	1,0,2；F→3	2,0,1；H
17	1	0,1,2；H	1,0,2；H	2,0,1；H
18	7	7,1,2；F→0	1,0,7；F→2	7,0,1；F→2
19	0	7,0,2；F→1	1,0,7；H	7,0,1；H
20	1	7,0,1；F→2	1,0,7；H	7,0,1；H

共 20 次引用：FIFO **15**、LRU **12**、OPT **9** 次缺頁。以第 8 次引用 4 為例：LRU 選最近一次分別為 t=4、7、6 的頁 2、0、3，故淘汰最久未用的 2；OPT 看未來，0 在 t=11 才再次使用，晚於 2 的 t=9 與 3 的 t=10，故淘汰 0。

OPT 與近似 LRU 的可用性

OPT 需要完整未來 reference，只能作離線比較下界。它證明的是一個 reference string 的最低 eviction 次數，不是線上系統能預知應用程式。

硬體 reference bit 可重建最近使用資訊：每個取樣 period 把 bit 移入歷史 byte，數值較小者通常較久未使用；clock 只保留一位 reference，遇到1時清為0並給第二次機會。改變 clock hand、period 或清除時機，就會形成不同近似策略，不能只寫「clock=LRU」。

本例 CLOCK 的 hand 初始指 slot0；每次 hit 設 R=1、不移 hand；有 free slot 時依0、1、2填入且設 R=1；滿後由 hand 逐格掃，R=1先清0，遇R=0替換、設新頁R=1並將hand前移。第4次 reference4 先把三個R清零，再換 slot0 的1；第7次 reference5 再清一圈，換 slot0 的4。這些初始化規則是重建表格所需的完整政策。

有效存取時間只以 fault count 代入仍不完整，因 dirty victim 多一次 write transfer：

$$T_{\text{fault}} = T_{\text{trap}} + T_{\text{swapout(dirty)}} + T_{\text{swapin}}.$$

同一 fault rate 下，若一半 victim dirty，平均 fault service 會介於單一 read 與 write+read 路徑，必須保存 clean/dirty 比例。OPT 在此 cost model 也不一定和最低 fault count 完全同序。

LRU 為何不會出現 Belady anomaly

將所有曾存取的頁依最後使用時間，由新到舊排列。每次存取頁 x，就把它從原位置移到最前方；若是首次存取，直接插入。這個順序與 frame 數無關。

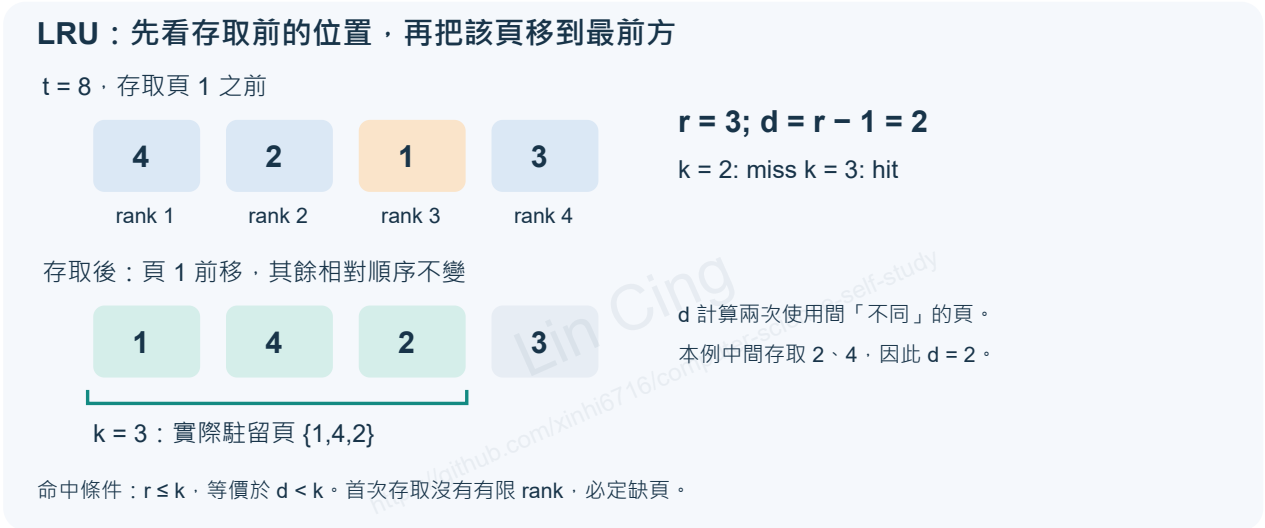
歸納不變量。 容量 k 的精確 LRU，其駐留集合恰為前 k 頁；不足 k 頁時則全部保留。初始為空。命中只改變前 k 頁的順序；缺頁時將 x 插入最前方，若已滿便擠出原本的第 k 頁，正是最久未使用的頁。因此每次操作後，不變量仍成立。

所以 $R_k(t) \subseteq R_{k+1}(t)$。若容量 k+1 在第 t 次存取缺頁，請求頁不在 $R_{k+1}(t-1)$，也不在其子集合 $R_k(t-1)$，容量 k 同樣缺頁。逐次加總可得 $F_{k+1} \leq F_k$。這是任意序列的證明；單一數例無法替代它。

FIFO 的載入順序會受先前缺頁影響，CLOCK 的掃描位置與引用位元也會隨容量改變；兩者不能直接使用上述證明。

用 stack distance 判斷 LRU 命中

固定等大頁、純 demand paging、初始為空、沒有預取、釘選或額外失效。所有已見頁依最近使用由新到舊排列；存取前的 1 起算位置為 r。



兩次使用同一頁之間，不同頁的數目稱 reuse distance d，此處 $d = r - 1$。有些文獻把 stack distance 定義為 r；使用公式前先核對是否從 0 起算。

一條序列，同時算出兩種容量的缺頁數

t	頁	存取前 r	存取後順序：新 → 舊	LRU 2	LRU 3
1	1	首次	1	F	F
2	2	首次	2,1	F	F
3	1	2	1,2	H	H
4	3	首次	3,1,2	F	F
5	1	2	1,3,2	H	H
6	2	3	2,1,3	F	H
7	4	首次	4,2,1,3	F	F
8	1	3	1,4,2,3	F	H

缺頁數：容量 2 為 6；容量 3 為 4。 F 表示缺頁，H 表示命中。第 8 次存取前頁 1 位於第三位，因此容量 2 已淘汰它，容量 3 仍保留它。

為什麼命中條件是 d < k？

上次使用頁 p 後，它位於第一位。此後每遇到一個不同的其他頁，就有一頁排到 p 前方；重複使用已在前方的頁不會再增加 p 的位置。因此再次使用前 $r = d + 1$。精確 LRU 的 k 個駐留頁恰是此順序的前 k 頁，故命中等價於 $r \le k$，亦即 $d < k$。首次引用則必定缺頁。

工作集視窗數的是存取次數；LRU 容量數的是不同頁。 在 t=7，四次引用涵蓋四頁，WSS=4；三個 frame 的 LRU 只能保留 {4,2,1}，頁 3 仍在工作集中卻已被淘汰。不能把 WS(t,4) 直接當作四個 frame 的駐留集合。

OPT 的貪婪選擇與計費模型

固定離線 reference string、相同初始 resident set、每次 miss 成本1、沒有 dirty 寫回差別。一次滿 frame 的 fault 上，OPT 移除「下次被要求最晚」的頁 q，永不再用視為 $+\infty$。

交換引理。 假設某最優排程第一次在此處不同，移除 (p)。修改它，先移除 q、保留 (p)。在下一次要求 (p) 之前，不會先要求 q，因 q 的下次使用不早於 (p)。故兩個排程可維持除 (p,q) 外相同的頁；原排程若先主動移除 q，差異可立即消除。否則到 (p) 被要求時，修改後排程已有 (p)，省下原排程的一次 fault；可用這次省下的傳輸載入 q 並移除原排程此次移除的頁，讓 resident set 對齊，總傳輸不增加。若兩頁都不再用，則差異不影響成本。

上述對齊使用一次可提前的讀入；把任何提前讀入延至首次要求該頁，成本不增，也得到合法 demand-paging 排程。因此存在與 OPT 此步一致的最優排程。逐次消除第一個分歧，得到 OPT 最優。此證明的成本是等權 page-in 次數；不能直接推論有不同 page sizes、非均勻 I/O 或 dirty page-out 時仍最優。

額外 frame 的最優排程可模擬少一個 frame 的排程、暫不使用多出的空間，所以 OPT 最小成本也不隨容量增加而上升。這個模擬證明不要求不同 tie-breaking 的 OPT 實作逐步具有相同的 resident nesting。

LFU、MFU 與歷史頻率失效

頻率政策還須定義計數生命週期與同分規則。本例從空的3 frames 開始，載入後 count=1，hit 加 1，eviction 即丟棄該頁計數，同分時選最早載入者。

輸入為 `1,1,1,2,3,4`。前五次後 resident={1,2,3}，counts={1:3,2:1,3:1}。第六次要求4時，LFU（least frequently used）移除2，結果{1:3,3:1,4:1}；MFU（most frequently used）移除1，結果{2:1,3:1,4:1}。二者並非 LRU 或 OPT 的別名。

LFU 的3次舊命中會讓1長時間留下，即使 locality 已轉移到{2,3,4}；可定期衰減計數以減輕舊歷史，但衰減週期本身又是一個政策參數。MFU 假設低頻頁可能剛載入、未來還有使用機會，這是工作負載假設，並無普遍最優保證。

CLOCK每一步的hand與R bits

此表把先前省略的實體slot、R bits與下一次掃描起點一併列出；hit只設對應R=1，不移hand。初始裝入free slots也不移hand，故前三次後hand仍0。

t	page	slots0/1/2	R0/R1/R2	下一個hand	H/F；victim
1	1	[1]	[1]	0	F；—
2	2	[1, 2]	[1, 1]	0	F；—
3	3	[1, 2, 3]	[1, 1, 1]	0	F；—
4	4	[4, 2, 3]	[1, 0, 0]	1	F；1
5	1	[4, 1, 3]	[1, 1, 0]	2	F；2
6	2	[4, 1, 2]	[1, 1, 1]	0	F；3
7	5	[5, 1, 2]	[1, 0, 0]	1	F；4
8	1	[5, 1, 2]	[1, 1, 0]	1	H；—
9	2	[5, 1, 2]	[1, 1, 1]	1	H；—
10	3	[5, 3, 2]	[0, 1, 0]	2	F；1
11	4	[5, 3, 4]	[0, 1, 1]	0	F；2
12	5	[5, 3, 4]	[1, 1, 1]	0	H；—

FIFO容量增加卻多fault的完整比較

FIFO的queue次序由各容量自己的fault歷史決定。第7步後3-frame set含1，4-frame set卻不含1，故第8步大容量反而fault；不具有LRU的包含性。

step/page	3-frame queue	4-frame queue	累計faults 3/4
1/1	[1]F	[1]F	1/1
2/2	[1,2]F	[1,2]F	2/2
3/3	[1,2,3]F	[1,2,3]F	3/3
4/4	[2,3,4]F	[1,2,3,4]F	4/4
5/1	[3,4,1]F	[1,2,3,4]H	5/4
6/2	[4,1,2]F	[1,2,3,4]H	6/4
7/5	[1,2,5]F	[2,3,4,5]F	7/5
8/1	[1,2,5]H	[3,4,5,1]F	7/6
9/2	[1,2,5]H	[4,5,1,2]F	7/7
10/3	[2,5,3]F	[5,1,2,3]F	8/8
11/4	[5,3,4]F	[1,2,3,4]F	9/9
12/5	[5,3,4]H	[2,3,4,5]F	9/10

Frame allocation：equal、proportional 與 rounding

equal allocation 若 total frames=F、process 數=m，先給每個 process $\lfloor F/m \rfloor$，餘數的分配規則必須明定。proportional allocation 依 process 大小或權重：

$$a_i = \left\lfloor F \frac{s_i}{\sum_j s_j} \right\rfloor,$$

再以最大餘數、priority 或固定順序分配剩下 frames。

固定 F=64，process sizes A=100、B=200、C=300，總大小600：

process	raw share	floor	remainder	final
A	10.667	10	0.667	11
B	21.333	21	0.333	21
C	32.000	32	0.000	32

floor 合計63，剩1 frame 依最大餘數給 A，得到 `11/21/32` 且總和64。若不保存 rounding 規則，同一個「proportional」百分比可產生不同配置。

任何 process 的 quota 仍要有 correctness 下界：執行中 instruction、page-table walk 與 interrupt/fault 路徑所需 pages 不能被任意壓到零。fault handler 核心記憶體是系統成本，不應被直接算成每個 process 的負擔。

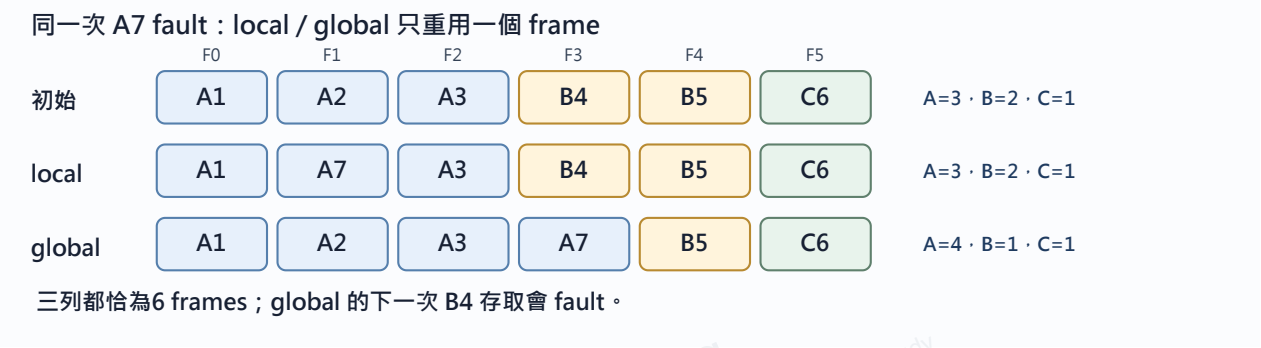
Local 與 global replacement 的同事件分歧

初始 resident sets 是 A={1,2,3}、B={4,5}、C={6}，共6 frames。last-use 時戳是 A1=10、A2=5、A3=8、B4=2、B5=7、C6=9；A 在沒有 free frame 時 fault page7。

policy	victim 範圍	victim	結果
local	只在 A 的 frames	A2（A 內最久未用）	A={1,7,3}；B/C 不變
global	所有 process 的 frames	B4（全域最久未用）	A={1,2,3,7}；B={5}；C={6}

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Local 與 global replacement 的同事件分歧 （續）



global 的 A fault 搶走 B4 的 frame，A 的 resident 數由3升4，B 由2降1，總數仍為6；既沒有刪掉 A3，也沒有憑空載入 B8。B 下一次讀4便再 fault，可能把局部壓力擴散成跨 process paging storm。local 保留隔離，但 A 無法借用其他 process 暫時閒置的 frame。

allocation 與 replacement 是兩個決策：allocation 決定每個 process 的 quota/上下界，replacement 決定 fault 時 victim pool。把二者寫成「global 就是沒有上限」會遺漏 residency set 與 admission control。

最小frame數不是working-set大小

架構允許一條指令同時碰多少code、source與destination pages，及fault後如何重啟，會決定執行所需的最小user frames。若指令不可部分完成，載入最後一個operand時卻逐出同指令必需的前一頁，便可能反覆fault而無法完成。這是correctness/progress下界；WSS則是所選時間窗的效能需求，兩者不可當同一常數。

本章proportional的11/21/32例子未加入額外minimum約束。若每個process另有下界 l_i，必須先查 $\sum_i l_i \le F$。可先給下界，再將餘額 $F - \sum l_i$ 依權重與明定rounding分配；直接算比例後各自取 `max(share,l_i)`，總和可能超過F。

例如F=6，三個process均需至少3個不可共享frames，總下界9>6；任何公平權重或replacement算法都無法同時滿足三者。必須暫停至少一個process、改變執行策略或增加frames。page-table/fault-handler的核心記憶體與裝置pin預留另計，不能把F的全部容量都先分給user quota。

工作集隨視窗移動，不是固定大小的快取

令第 t 次存取的頁為 p_t，視窗長度 $\Delta = 4$。

視窗長度 Δ = 4：計算引用次數，不是不同頁數

綠框內恰有四次引用；WS 只保留其中不同的頁。t < 4 時使用已有的前綴。

$WS(t,4) = \{p_j : \max(1, t-3) \le j \le t\}$，$WSS = |WS|$。例如 t=8 的視窗為 [1,2,4,1]，只有三個不同頁；工作集大小不是 4。

移出一次引用，不一定移除那個頁

同一頁可能在視窗出現多次。維護每頁的引用計數：加入新引用時計數加一，移出最舊引用時減一；只有計數降為 0，才從工作集移除該頁。

t	新頁	移出的舊引用	視窗內容	工作集	WSS
4	3	無	1,2,1,3	{1,2,3}	3
5	1	第 1 次的 1	2,1,3,1	{1,2,3}	3
6	2	第 2 次的 2	1,3,1,2	{1,2,3}	3
7	4	第 3 次的 1	3,1,2,4	{1,2,3,4}	4
8	1	第 4 次的 3	1,2,4,1	{1,2,4}	3

t=5 的頁 1 不能刪除：移出的是一次引用，視窗內仍有兩次頁 1。t=8 則移除頁 3 的最後一次引用，WSS 才減少。

Working set、頁面需求與 thrashing

以下仍取最近四次存取作為視窗。三個行程目前的存取序列如下：

process	recent references	working set	WSS_i
A	1,2,3,1,2,4,5,1	{1,2,4,5}	4
B	6,7,6,8,7	{6,7,8}	3
C	9,9,9	{9}	1

三個行程沒有共享頁，總需求為 $D = 4 + 3 + 1 = 8$ 個 frame，可用容量卻只有 6。若這些頁仍會被頻繁使用，系統無法讓三個工作集同時駐留，反覆換頁的風險便會上升。若行程共享同一個實體頁，應依對應的實體內容取聯集，避免重複計算。

需求超過容量是壓力訊號，不是量測結論。視窗太短會漏掉稍後重用的頁，太長則保留過時的需求。判斷 thrashing，還要檢查缺頁率、有效 CPU 使用率與換頁 I/O 佇列；等待鎖或單次慢速 I/O 也會造成 CPU 閒置。

PFF controller 與 hysteresis

page-fault frequency（PFF）直接用固定觀察窗的每頁存取 fault ratio。設定 upper=0.4、lower=0.1：

window	最近5次 F/H	PFF	決策
過載	F,F,F,F,F	1.0	若可取得 frame，分配給該 process
分配後	F,H,H,H,H	0.2	介於上下界，保留
持續局部命中	H,H,H,H,H	0.0	有記憶體壓力時可考慮回收一 frame

兩個門檻形成 hysteresis：只有 high 才加 frame，只有低於 low 才減 frame，避免在單一門檻附近反覆搬頁。若高 PFF 但沒有 free frame，系統應降低多程式度、換出一個 process 或以 admission control 拒絕新工作；繼續提高多程式度只會把其他 process 推入 thrashing。

PFF controller 與 hysteresis （續）

PFF 控制器有一條可重建偽碼：

```
for each process every sample_window:
    if accesses_in_window == 0:
        continue
    rate = faults_in_window / accesses_in_window
    if rate > upper and free_frames > 0:
        grant_one_frame(process)
    else if rate > upper and free_frames == 0:
        choose one process to suspend
    else if rate < lower and memory_pressure and quota > minimum_required:
        reclaim_one_spare_frame(process)
```

window 太短會受 burst 雜訊影響，太長則反應慢；因此 threshold、window、suspension policy 必須一起報告。

Memory-mapped file：mapping、dirty 與 durability

file page 先進入 page cache，再由兩個 process 的 PTE 映射到相同 physical frame。shared mapping 的 ordinary load/store 操作 cache page；private mapping則在 write 時以 COW 建立私有 frame。頁面 cache hit 只表示不用讀 disk，不代表資料已持久。

以下是兩個**各自從 value=7 開始**的實驗，沒有把 shared write 與 private mapping 混成同一保證。

shared 實驗	P1 mapping	P2 mapping	cache F40	backing file
S0 初始	F40	F40	7，clean	7
S1 P1 寫8，並用同步通知 P2	F40	F40	8，dirty	7
S2 P2 在同步後讀取	F40	F40	讀8；尚未持久	7
S3 寫回完成且滿足指定 flush 契約	F40	F40	8，clean	8

private 實驗	P1 mapping/value	P2 mapping/value	page cache	backing file
V0 初始	F40/7，COW	F40/7，COW	F40/7	7
V1 只有 P2 寫9	F40/7，未複製	新 F53/9	F40/7	7
V2 P1 再寫8	新 F52/8	F53/9	F40/7	7

Memory-mapped file：mapping、dirty 與 durability （續）

V1 只複製4096 bytes，V2 累計8192 bytes。不能在 P2 單獨寫入時，把尚未寫入的 P1 也列成新私有 frame。Linux `MAP_PRIVATE` 不保證 mapping 建立後檔案的外部變更是否可見；本實驗明定沒有其他檔案寫入，故不把 private mapping 說成自動建立快照。

shared mapping 的跨 thread visibility 仍要符合記憶體同步；一般 ordinary write 與讀取之間不能只因有 mmap 就自動形成任意順序。`msync` 或其平台對應 API 觸發 writeback 與必要的 ordering，但其 durability contract 仍依 API、文件系統、儲存 cache 與 flush 規則。

private map 的 COW page 修改不應寫回原 file。頁面被 reclaim 時，dirty shared/file page 必須先安全寫回；dirty private file-backed COW page 寫到 swap 或 backing store，而不是覆蓋原始 file。writeback 失敗要保留 errored state，不能把頁標成 clean 後重用。

mmap 與 read/write 路徑的相同與差異

`read(fd,buf,n)` 需要 system call、copy data 到 user buffer，並更新 file offset；mmap 首次存取靠 page fault 建立 file page→PTE mapping，之後 load/store 由 MMU 完成。兩條路徑都可受益於 page cache，但 fault、copy、TLB 與 page-dirty tracking 的成本不同。

固定冷 cache、4 KiB page：

path	system calls	data copies	TLB events	durability after write
read一次	1	cache→user 4096 B	normal	read 不承諾
write一次	1	user→cache 4096 B	normal	需 fsync/writeback/commit 才具指定持久性
mmap首次 read	先前1次 mmap；此次為 fault/trap	0次 cache→user copy；磁碟仍讀4096 B至cache	install/必要維護	不涉及寫入持久性
mmap shared write	0 until flush	0 byte CPU copy in model	access/dirty	msync/flush 後才有指定寫回承諾

表中固定冷 page cache、無 read-ahead、完整一頁且呼叫成功；disk→cache 的 I/O 與 cache↔user 的 CPU copy 分開。page fault 是 exception，不是使用者主動呼叫的 system call。`write()` 與 `mmap` store 都可能先讓 cache 變 dirty，仍需另行達到指定持久性契約。

Buddy system：以 21 KiB 請求重建配置

固定可用連續區域256 KiB，最小配置單位為4 KiB；下表位址以 KiB 為單位、使用左閉右開區間。buddy allocator 將非 power-of-two 請求上取整；21 KiB→32 KiB：

step	available split	配置結果
0	`[0,256)` free	尚未配置
1	`[0,256)`→`[0,128)`,`[128,256)`	兩個128 KiB buddies
2	`[0,128)`→`[0,64)`,`[64,128)`	兩個64 KiB buddies
3	`[0,64)`→`[0,32)`,`[32,64)`	兩個32 KiB buddies
4	配置 `[0,32)`	21 KiB request 使用32 KiB；內部浪費11 KiB

此時未配置的 buddies 是 `[32,64)`、`[64,128)`、`[128,256)`，合計32+64+128=224 KiB。free bytes 足夠，但分屬不同 power-of-two blocks；若下一個請求需要64 KiB，可合併 `[0,64)` 上的兩個 buddies（前提是已配置的32 KiB先釋放），否則仍需另找適合區域。

allocator invariant 是每個免費區塊屬於唯一的 block size 與 buddy pair，配置範圍不重疊；release 後只在 buddy 也 free 時合併。只把 free bytes 相加無法判斷請求能否滿足。

Buddy system：以 21 KiB 請求重建配置 （續）

若區域起點對齊256 KiB，大小 2^k 的 block（此處以 bytes 計）位於對齊位址 a，buddy 位址為 $a \operatorname{xor} 2^k$。釋放 `[0,32)` 後，先與 `[32,64)` 合成64，再與 `[64,128)` 合成128，最後與 `[128,256)` 合成256；每一步只查同 order 的那個 buddy。樹高 $\log_2(256/4) = 6$，分裂/合併最壞走6層，並非任意遍歷整個 free list。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Slab：物件重用與 fragmentation 的分母

固定 objects=256 bytes、slab page=4096 bytes，若完全不計 slab metadata，一個 page 可放 `4096/256=16` objects。實際 slab 會保留 page/slab 管理資訊，因此不能把16宣稱成所有實作的精確上限。

模型配置2個 objects、釋放1個：

指標	數值	意義
live objects	1	仍由 kernel 持有
free objects	15	可立即重用
live bytes	256	不包含 cache/slab metadata
slab page bytes	4096	已留在 cache 的 frame
process RSS	不由這個 object count 推得	分母是 process mapping

slab cache 把空物件留在 cache，能省 allocation、初始化與 page allocation；但 footprint 不會隨 live objects 立即下降。NUMA 系統還要保留 node locality，remote reclaim 與跨 node allocation 的延遲必須放在成本模型。

Page size、prepaging 與 pinning

對1 MiB virtual range：

page size	page/PTE 數量	TLB coverage	可能浪費
4 KiB	256	一個 TLB entry 只覆蓋4 KiB	每物件尾部較小
64 KiB	16	同樣 entries 可覆蓋16倍範圍	每物件尾部與 read amplification擴大

大 page 可減少覆蓋固定範圍所需的 PTE，通常提高每筆 TLB entry 的 coverage，但 fault 數沒有普遍單調關係；單頁 I/O、internal fragmentation 與 read amplification 也可能增加。

prepaging 在第一次使用前載入預期頁。固定 workload 有8 pages、locality{P1,P2,P3}，prepage三者成本3 reads，之後前三次 accesses0 faults；pure demand paging也做3 reads但三次 faults。兩者 data I/O 相同，差異在 trap、latency 與猜錯風險；多載入的P4若永不使用就是額外成本。

反例：大頁不一定較少 fault。 配給此工作負載的總容量固定為 256 KiB，純 demand paging、初始為空、精確 LRU，不預取且無其他競爭。依序循環存取五個互不重疊的 64 KiB 區域，各區域只碰其中同一個 4 KiB 頁。4 KiB 頁有 64 個 frame，五個頁首次各 fault 一次，以後皆 hit；64 KiB 頁只有 4 個 frame，五頁循環每次都 fault。兩種 page size 的載入單位和 frame 數不同，因此須先固定容量與存取模式再比較。

I/O interlock 與 pinned page

裝置 DMA 使用 frame F40 時，F40 的 page count／pin state 必須阻止 reclaim。事件表：

時刻	F40 owner	pin/DMA	reclaimable
t0	user buffer	pin=0	是
t1	device queue	pin=1, DMA pending	否
t2	device完成	DMA=0，pin尚未清	否
t3	completion handler清 pin	pin=0	回到 LRU，才可選為 victim

若 t1 錯把 F40 reclaim，device completion 會寫入已被其他 page 使用的 frame。t2 即使 DMA 已完成也不能在 completion protocol 未完成前假設 ownership 已歸還；pin 必須有明確 release/error path。

lock bit 與 mlock 是不同層次的例子，實際語意由 API 定義；共同不變量是「仍在使用的 frame 不得被重複配置」。長期限 pin 會減少 evictable pool，單看 free count 可能高估可用資源。

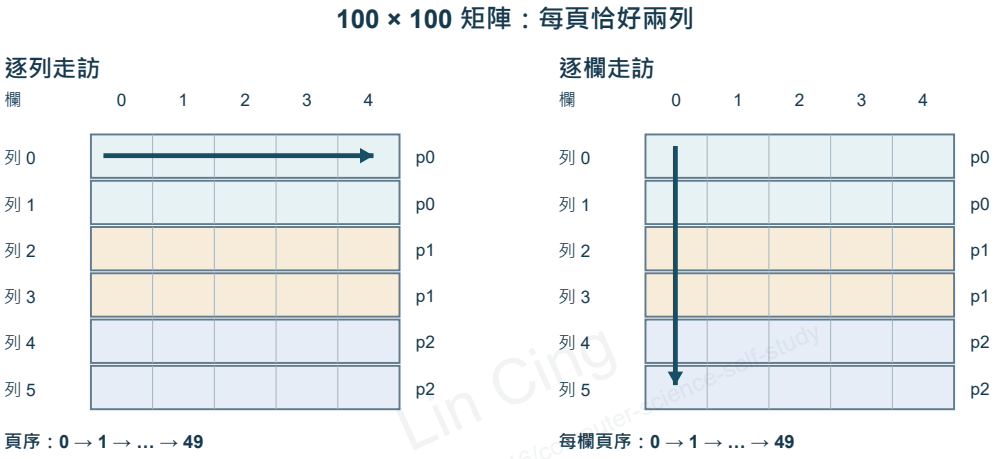
程式存取次序如何改變page locality

固定C/C++ row-major `int A[4][1024]`，int=4bytes、起點4KiB對齊、page=4KiB，每個row剛好一頁。只計此array的page references，2個frames、精確LRU、初始皆空；code/stack與cache-line成本另計。

走訪順序	page reference型態	fault推導	總faults
row外、column內	0重複1024次，再1、2、3各1024次	每row第一次fault，同行其餘1023次hit	4
column外、row內	0,1,2,3重複1024輪	兩frames保不住四頁；每頁再次使用前已被LRU淘汰	4096

column走訪前8次狀態（從最舊到最新）為 `[0]F,[0,1]F,[1,2]F,[2,3]F,[3,0]F,[0,1]F,[1,2]F,[2,3]F`，之後週期重複。相同16KiB資料與相同運算次數，存取排列便可改變page fault數；這不是硬體上每種loop都必然相差同一倍數，因真實OS可能分配更多frames、預取或改用其他政策。

矩陣走訪：同一資料，不同 page 次序



示意前 6 列、前 5 欄；同色兩列屬同一頁。

矩陣有 100×100 個元素，每元素 1 byte，採 row-major 儲存；起始位址與 200-byte page 對齊，所以每頁正好放兩列，共 50 頁。只計矩陣資料，其他程式與資料不競爭這些 frame。

表中的「第幾頁」從 0 起算；第 i 列（i=0…99）所在頁為 `floor(i/2)`。每種走訪都從空的 frame 集合開始。

為何逐欄會缺頁 5,000 次？

走訪方式	資料 frame、政策	可核對的頁面次序	fault 次數
逐列	2、LRU	每頁連續使用 200 次後才進下一頁	50
逐欄	2、LRU	每欄依序訪問頁 0,1,…,49；每頁連續兩次	100×50＝5,000
逐欄	50、LRU	第一欄載入全部頁；其後全命中	50

逐欄時，下一欄重用頁 0 前已依序接觸其餘 49 個不同頁；兩個 frame 無法保留它。每頁在該欄的兩列相鄰，首次 fault、第二次 hit；重複 100 欄，故每欄 50 fault。50 個 frame 則足以保留整個矩陣。缺頁數依 page 大小、對齊、可用 frame 與政策而變，不能僅由走訪方向決定。

Linux 3.2 reclaim 的觸發、掃描與逐頁處置

本頁固定 Linux v3.2、order-0 的非同步 reclaim 路徑。allocation slow path 在 zone 的 free pages 低於 low watermark 時可喚醒 `kswapd`；回收目標是恢復 watermark 所要求的可用頁，而不是「只要 free<low，每次掃描都必定成功」。`kswapd` 可掃 inactive anon/file lists，並依 active/inactive 平衡進行降級。`page_check_references()` 使用 PTE reference 與 PageReferenced 兩類狀態，`shrink_page_list()` 再依 unmap、dirty、writeback、mapping/refcount 決定結果。這是 Linux 3.2，沒有現代 multi-gen LRU。

固定本輪為 `current_is_kswapd=true`、`priority=9`、`DEF_PRIORITY=12`、`may_writepage=true`、允許進入檔案系統與必要的 I/O/unmap；page lock 成功，無並行重新引用或修改，P2/P3 的 mapping/refcount 可合法移除，P3 的 pageout 成功啟動非同步 I/O；本輪 P1 非 lumpy reclaim，P3 的 PageReferenced=0。P4 是已隔離候選，mlock 已設但尚未完成移入 unevictable；正常穩態 mlocked 頁不應視為一般 inactive 候選。這些條件使下表確實走到所列分支。

固定輸入為五個已隔離候選：P1、P2、P3、P5 來自 inactive LRU，P4 處於上述 mlock／cull 過渡。所有列的 PageReferenced 在此例均設為 0；PTE referenced 與 PageReferenced 是兩種不同狀態。

Linux 3.2 reclaim 的觸發、掃描與逐頁處置 （續）

Linux 3.2：五個候選頁的回收結果

教學快照：order-0、kswapd、非同步寫回；P4 已隔離，尚待移入 unevictable

頁面	決策條件		本輪動作	釋出 frame
P1	匿名頁；PTE referenced=1	→	移入 active LRU	0
P2	檔案頁；未引用、clean；可解除映射	→	回收並釋出 frame	+1
P3	檔案頁；dirty；PageReferenced=0	→	送出寫回，尚未回收	0
P4	mlocked；已隔離、尚待 cull	→	移入 unevictable	0
P5	PageWriteback=1	→	延後處理	0

P3 的 pageout 已成功啟動；寫回完成後仍須重新檢查 dirty 與引用，才能重用。

只有 P2 在這一輪成為可重用 frame。P1 是 anonymous 頁且 PTE 確認被引用，故 activate；P3 已是選中的回收候選，但尚未回收，writeback 完成前不可重用；P4 因 mlock 進入 unevictable；P5 已在 I/O 中。單純額外的 DMA pin/refcount 不等同 mlock：它可能仍在一般 LRU，最終因 reference 無法移除而留下，不能一概畫成 unevictable。

後續狀態、版本界線與反例

下一輪掃描前，P3 的 writeback 可能仍在進行，也可能完成。完成只清除這次 writeback 的狀態，**不保證沒有並行的新寫入使 dirty 再度為1**。必須重新觀察 dirty=0、writeback=0、mapping/refcount 可移除，才可重用。P4 須合法解除 mlock/unevictable；等待時間不能自行解除鎖定。

後續狀態、版本界線與反例 （續）

固定狀態轉移可寫成：

current event	required observable	next state	不得做的事
free below low watermark	kswapd wake / scan progress	scan	未掃描就增加 free count
referenced	PTE/page reference	active	當 unused 立即丟棄
clean unreferenced	mapping/refcount removable	free	忽略 pin
dirty	writeback submitted	writeback	在 I/O 完成前重用 frame
writeback complete	重新檢查 dirty=0、writeback=0，mapping/refcount 可移除	可回收	把 completion 當作永不再 dirty
mlocked	lock state cleared	evictable LRU	因 waited long 而解 pin

把 Solaris 的 `lotsfree/desfree/minfree`、Windows 的 working-set minimum/maximum 與 Linux watermarks 混成一個 threshold，會得到不存在的通用參數。本章保留這三個版本的機制名稱與責任邊界；數值只在明確標示的產品案例中出現。

改一個條件，為何回收結果就不同

改動	Linux v3.2 分支	結果
P1 改為 file 頁，只有首次 PTE reference，原 PageReferenced=0	設 PageReferenced，回傳 PAGEREF_KEEP	留在 inactive，並非無條件 activate
P3 的 PageReferenced 改為 1、PTE referenced=0	non-swap-backed file page 走 PAGEREF_RECLAIM_CLEAN	若仍 dirty，保留而非依原例啟動 pageout
P3 的 priority 改為10	`priority >= DEF_PRIORITY-2`	標記 reclaim 後保留，由其他 writeback 路徑處理
P3 改為 direct reclaim、不是 kswapd	file-cache dirty 限制	本分支不直接寫 file page
P3 的 may_writepage=false	shrink_inactive_list 可用 ISOLATE_CLEAN 在隔離前排除 dirty 頁；若已隔離而進入 shrink_page_list，則其 may_writepage 檢查阻止 pageout	此路徑不寫回 P3，也不能因此增加 free count

改一個條件，為何回收結果就不同 （續）

改動	Linux v3.2 分支	結果
P2 有額外 pin/reference	最後的 mapping/reference 移除失敗	保留，並非移到 unevictable 的同義詞
P5 writeback 完成但又被寫 dirty	重新遇到 dirty 分支	還不能當 clean frame 重用

因此不能用單一 `if referenced then activate; if dirty then writeback` 冒充此版本完整規則。reference 的來源、page 類型、scan context 和最後的所有權檢查都會改變結果。

教材 Windows working-set trimming 的操作例

採 OSC 9e §9.10.1 所述的歷史 Windows working-set 模型，與第19章 Windows 7 案例對讀；這是該書的簡化政策，非現行 Windows 所有修補版本的實作規格。working-set minimum/maximum 控制駐留頁範圍，超過 maximum 的細節由實作決定；free list 不足時，automatic trimming 可移除超過 minimum 的頁。

固定示範參數 threshold=3 frames、目前 free=1，三個 processes A/B/C 的 `(minimum,resident,maximum)` 分別為 `(2,4,4)`、`(2,3,4)`、`(2,2,4)`。本例待釋放頁全 clean、未鎖定，選 A→B→C，恢復到 free>3 停止。

動作	A resident	B resident	C resident	free	原因
初始	4	3	2	1	低於 threshold
A trim 兩頁	2	3	2	3	A 超過 minimum 的2頁可移除
B trim 一頁	2	2	2	4	free 恢復至 threshold 以上
C 不動	2	2	2	4	已在 minimum；不因本輪 trim 再減

後續 A fault 時，若尚在 maximum 以下且 free pool 足夠，可取得一頁；若在 maximum 且模型要求 local replacement，就在 A 的 working set 內選 victim。clustered demand paging 還可能連同鄰頁載入，所以「fault 一次恰增加一 resident page」只屬先前單頁模型。

教材 Solaris two-hand clock 的時間視窗

採 OSC 9e §9.10.2 的歷史 Solaris pageout 快照。free<lotsfree 開始 scanner，壓力加大時 scanrate 從 slowscan 提高；desfree/minfree 對應更積極回收或換出，不能把三者改成 Linux 同一個 low watermark。

front hand 清 R bit，back hand 延後檢查。若兩手相距 h pages、穩定掃描率 s pages/s，兩次觀察間隔為 h/s 秒。固定示範 $h = 4, s = 100$，為0.04秒；此數值是例題參數，不是產品預設值。

page	front hand 經過	中間事件	back hand 看到	處置
Q1	R←0	再次讀取，R←1	R=1	保留 locality
Q2	R←0	沒再用，clean	R=0、clean	解除映射後可回收/放 cache list
Q3	R←0	沒再用，dirty	R=0、dirty	寫回完成前不可重用 frame

把 s 提高到400而保持 (h=4)，間隔縮為0.01秒。同一頁更容易在短視窗內沒有再引用而被選中；壓力與 locality 的判斷會隨 scanner 速度一起改變。cache list 中尚未覆寫的乾淨頁可被重新取回，free list 的無效內容則不能當作仍保留原資料。

合法缺頁與非法存取的共同判斷順序

判斷	必須確認	不能直接當成什麼
virtual range／mapping	此位址是否有合法映射與backing	所有not-present都是合法page-in
權限／COW	requested read/write是否允許；COW是否適用	禁止write就無條件配新frame
resident／PTE	已有frame、standby或需storage取入	每個fault都固定一次disk I/O
victim／writeback	frame可否解除映射、dirty是否已安全處理	dirty／pinned也可直接重用
恢復執行	內容、mapping及權限一致後重試	同一page以後不會再被淘汰／refault

同一physical frame可以被多個PTE共享；frame容量按不同physical frames計，不能把各process的resident-mapping數直接相加當成frame用量。配置／淘汰及COW都要追到最後reference與TLB／I/O使用者。

用同一成本模型比較fault，不用單一比例替代效能

固定本章12次reference、3frames、normal hit100ns，每fault總服務8ms（已含該次存取）。

policy	faults	hits	總成本
FIFO	9	3	72ms＋300ns
LRU	10	2	80ms＋200ns
OPT	7	5	56ms＋500ns
本章CLOCK	9	3	72ms＋300ns

OPT使用future作比較下界；線上replacement仍要符合各自history規則。若fault服務時間或預取／clustering策略不同，不能只按fault個數套此表。

working set、frame allocation、PFF與scan壓力共同決定是否有足夠駐留資源。評估時同時追fault服務成本、有效工作／吞吐及等待，避免fault減少卻因其他瓶頸更慢。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 9. Linux v3.2 vmscan.c; Linux v3.2 memory.c; mmap(2).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：在磁碟、網路儲存與 SSD 等不同裝置上，如何安排 I/O 與冗餘以符合延遲、吞吐與故障保證？

前置知識。 block 與 sector、I/O interrupt、機率與平均延遲。

- 建立 disk service-time 模型
- 比較 FCFS、SSTF、SCAN、C-SCAN
- 分析 RAID level 0/1/4/5/6 的可靠性與寫入成本

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Media機制決定I/O成本，介面數字不等於實測

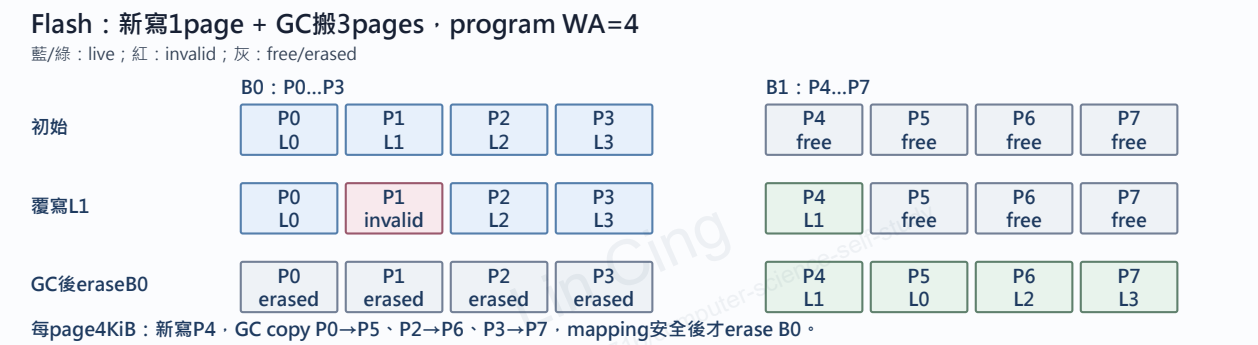
HDD以磁性表面、旋轉platter與移動head存取；SSD以nonvolatile semiconductor媒體配合controller/FTL；tape適合串流與較長定位成本的備份/封存。Host看到logical blocks，不直接控制每個physical位置。比較需固定request size、random/sequential、read/write比例、queue depth、cache與同步契約。

指標	明確分母	不能混用
Bandwidth	完成bytes / elapsed seconds	media/interface peak不必等end-to-end
IOPS	完成requests / seconds	4KiB和1MiB request的IOPS不可直接比吞吐
Service time	裝置實際服務區間	不含尚未開始的queue wait
Response latency	arrival至完成/回報	視契約可含排程、queue、copy
Durability	授權成功後可承受的故障	I/O completion不一定已離開volatile cache

固定每秒1000個4KiB完成requests，bandwidth=3.90625MiB/s；改為1000個64KiB則62.5MiB/s。相同IOPS不代表相同bytes/sec，也不能由上述算術推論任何實際HDD/SSD產品速度。

Flash out-of-place update與GC的真實工作量

固定NAND模型page4KiB、erase block4pages=16KiB。初始B0的P0/P1/P2/P3分別存L0/L1/L2/L3；B1的P4…P7皆erased。覆寫L1時寫新P4、mapping L1→P4，舊P1變invalid，不能直接在未erase的P1任意覆寫。



若回收B0，仍live的L0/L2/L3需分別copy到P5/P6/P7並更新mapping，確認live資料與必要mapping安全後才能erase B0。Host新寫1page，physical program=1new+3GC copies=4pages，這個crash-free計數模型write amplification=4。若L2先被discard且不再需要，GC只copy2pages，program總3pages、WA=3；不計mapping寫入等其他開銷。

Discard/TRIM告知某些logical資料不再需要，不保證立刻physical erase或可當安全抹除。Wear leveling分散erase負擔；GC搬移、over-provisioning與controller cache也會改延遲。若加入power loss，還要證明FTL mapping與資料commit順序，不能只靠這張正常路徑圖。

Cylinder/sector與LBA分屬不同抽象

傳統HDD的同一半徑tracks集合稱cylinder，seek改半徑，rotation使目標sector到head下方；sector/track幾何可因zone與缺陷remap不同。LBA是一維logical block編號，不應把LBA直接當永遠固定的physical cylinder；SSD更沒有移動磁頭可供SSTF最佳化。

固定sector512bytes，LBA4096對應logical byte offset4096×512=2MiB。若介面改4KiB logical block，同一編號對應16MiB，單位不可省。32-bit無號LBA可表示2^32個512-byte blocks，容量2^41bytes=2TiB；這是位址空間算式，不是所有partition/API限制的共同答案。

若上層一次寫512bytes而底層原子/physical單位4KiB，可能需要read-modify-write或裝置轉換；alignment影響效能與crash粒度。實際保證必須查device/driver contract，不能由filesystem block size推定atomic sector size。

磁碟容量：把幾何與單位逐層展開

假設磁碟組有 10 片雙面 platter，最上和最下各一個外表面不存資料，因此可用面數為 `10×2−2=18`。每面有 2,048 條 track，每條 track 有 4,096 個 sector，每個 sector 為 8,192 bytes；所有面採相同幾何，不另扣備援或格式化空間。

$$18 \times 2{,}048 \times 4{,}096 \times 8{,}192 = 1{,}236{,}950{,}581{,}248 \text{ bytes}.$$

同一數量除以 2^{30} 是 **1,152 GiB**；除以 2^{40} 是 **1.125 TiB**。若使用十進位 `1 GB = 10⁹ bytes`，則約 **1,236.95 GB**。標示單位才能比較數字；這是給定幾何下的容量，並非格式化後一定可用的容量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從轉速與每圈資料量推算時間

假設 6,000 rpm、每條 track 1,024 個 sector、每 sector 4 KiB；相鄰 sector 連續可讀，忽略 sector gap、controller、快取與排隊成本。

步驟	計算	結果
每秒圈數	6,000 ÷ 60	100 圈／秒
每圈時間	1 ÷ 100	0.01 秒 = 10 ms
每圈資料	1,024 × 4 KiB	4 MiB
理想循序速率	100 × 4 MiB／秒	400 MiB/s
平均旋轉等待	目標相位均勻，半圈	5 ms

若 seek 固定為 3 秒，接著讀取在同一 track 上連續排列的 4 MiB，從所需 sector 開始讀完恰需一圈，即 `4 MiB ÷ 400 MiB/s = 0.01 秒`。平均總服務時間是 `3 + 0.005 + 0.01 = 3.015 秒`。這是指定模型的算例；實際磁碟的幾何、gap、快取與其他開銷都可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7200RPM與64KiB讀取：每段時間分開算

明定教學HDD參數：seek5ms，7200RPM且目標相位均勻，transfer128MiB/s，request64KiB，無cache hit與額外controller成本。每圈60,000/7200=25/3ms；平均rotation=25/6ms；transfer=64KiB/(128MiB/s)=125/256ms。

Service=5+25/6+125/256=7415/768≈9.65495ms。Queue wait若15ms，總response≈24.65495ms；單request串列、無重疊時IOPS≈1000/9.65495=103.574。不能把多queue並行、cache或同一track連續requests套入此倒數。

循序傳1GiB的純transfer下界為1024MiB/(128MiB/s)=8s，定位與queue另加。隨機小I/O的定位比重與大串流不同，正是不能僅拿介面bandwidth評估體感latency的原因。

Host-attached、NAS與SAN：哪裡解析檔案

路徑	Client送出的抽象	Filesystem metadata在哪裡處理	額外故障邊界
Host-attached block	本機driver/device block requests	本機host filesystem	controller/device
NAS	遠端file/directory操作	file server	network、server、cache/重試
SAN block	網路/存儲fabric上的logical blocks	使用該block device的host	fabric、target、multipath

SAN不等於遠端file API；NAS也不是只把本機磁頭接長線。若兩hosts把同一SAN LUN當獨占本機磁碟同時mount不具協調的filesystem，metadata可能互相破壞；shared block可見不等於已提供distributed locking/一致性。

固定4KiB read：NAS client送file handle/offset/length，server查filesystem再送block I/O；SAN client先在本機filesystem把file offset轉LBA，再由block transport送target。兩者可能讀同一實體array，卻在pathname、cache與失敗重試上有不同責任。

Multipath與共享故障域

兩條fabric路徑若都接同一target/controller，能改善部分link failure，不能假設有兩份獨立資料。固定paths A/B，A斷而B與target存活可切路；target失效則兩條路都不可用。Retry必須遵守request identity與write completion契約，不能在不知先前是否提交時任意重做非冪等上層操作。

同樣地，remote mount成功只證當時能完成相關服務，不證所有後續I/O可用或持久。第17章處理network partition/replication語意；Latency由request size、protocol、network、cache與queueing共同決定，不由local/NAS/SAN名稱唯一決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

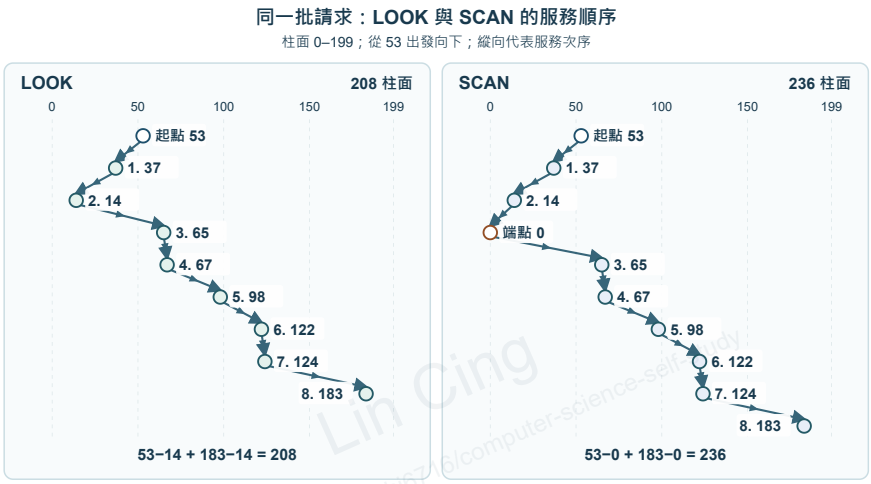
相同batch的磁頭移動：先固定端點與計費

固定cylinders0…199，head53，所有requests於t0到齊：98,183,37,122,14,124,65,67。成本只算相鄰服務/轉向位置的絕對cylinder差，直到最後請求完成；不含rotation、transfer或最後空跑至另一端。SSTF同距離取較小cylinder。

Policy	含轉向/回繞的路徑	總移動
FCFS	53→98→183→37→122→14→124→65→67	640
SSTF	53→65→67→37→14→98→122→124→183	236
LOOK往下	53→37→14→65→67→98→122→124→183	208
SCAN往下	53→37→14→0→65→67→98→122→124→183	236
C-SCAN往上	53→65→67→98→122→124→183→199→0→14→37	382
C-LOOK往上	53→65→67→98→122→124→183→14→37	322

C-SCAN/C-LOOK回繞也算physical移動，但不在回程服務requests；若別題明定不計回繞成本，必須另列計費而不是偷換本表數值。

SCAN的0與LOOK的14不能混成208



0 是 SCAN 的實體端點，不是請求；兩條路線都在最後請求 183 完成時停止。

LOOK下行到最後請求14就折返：|53−14|+|183−14|=39+169=208。SCAN本題必先到physical端點0再折返：|53−0|+|183−0|=53+183=236。SSTF恰好也236只是此輸入的巧合，規則與路徑並不相同。

FCFS逐差為45+85+146+85+108+110+59+2=640。SSTF每步距離12/2/30/23/84/24/2/59，加總236；每一決策只挑當時pending最近者，不保證整個batch總移動全域最小。LOOK208已是本例反證。

相同請求，起始方向會改變路線

固定 cylinder 0–199、起點 53；前一位置為 50，並明定本例初始朝較大的 cylinder 前進。八個請求均在開始前到齊：98，183，37，122，14，124，65，67。每段費用為移動距離，直到最後一筆請求完成即停止；不含旋轉或資料傳輸。

方法	路徑（199 是轉向點，不是請求）	各段距離	總移動
SCAN 向上	53→65→67→98→122→124→183→199→37→14	12、2、31、24、2、59、16、162、23	331
LOOK 向上	53→65→67→98→122→124→183→37→14	12、2、31、24、2、59、146、23	299

SCAN 在此約定到實體端點 199 才反向；LOOK 在該方向最後一筆請求 183 即反向。這兩條路徑都不是循環掃描，沒有額外回繞。若改以本章另一例的**向下**方向起步，SCAN 與 LOOK 分別是 236 與 208；同一組請求的數值不能脫離起始方向、端點及停止條件使用。

等待與效能保證依arrival/service模型

有限固定batch沒有無限starvation，SSTF的starvation風險來自新requests可持續在附近抵達，使遠端一直不被選。SCAN/LOOK方向掃描降低此偏好，但有限等待仍依bounded device range、每圈服務量與新到達的admission/每track處理規則；無限湧入同一位置不應被默認可在有限時間服務完。

Real device可在controller內重新排程、request merging與parallel queues；SSD沒有seek半徑，所以最小cylinder移動不直接代表最佳latency。選policy還要看deadline/fairness與throughput；表中成本只是明列的HDD定位抽象。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partition、logical formatting與mount是不同操作

Low-level/physical格式安排媒體sector及錯誤偵測等device細節，常由device製造/管理；partition在logical disk切位址範圍；logical formatting建立filesystem metadata/free-space structures；mount把可識別filesystem接到namespace。格式化不等於把所有舊資料安全抹除，也不等於自動mount。

固定1024MiB教學disk、sector512bytes，保留[0,1)MiB作抽象boot/partition metadata；A=[1,101)MiB長100MiB，B=[101,301)MiB長200MiB。A first LBA2048、sector count204800、exclusive end206848；B first206848、count409600、exclusive end616448；範圍不重疊。

這不是完整GPT layout規格，只驗證partition單位、左閉右開邊界與不同filesystem對象。A格式化/資料驗證失敗，不因B能mount而變成功；mount到同一路徑也不證底下是相同device/UUID。Bootloader能找kernel與kernel能mount root還需第2章的早期driver依賴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bad blocks、remap與修復不改物件身分契約

Controller可把壞physical sector重新對應spare，host仍看到同LBA；這可能保住之後可用性，卻不保證已無法讀出的舊bytes可恢復。資料checksum與冗餘可協助偵測/重建，但錯誤必須向上保留，不能把「已換到spare」當作檔案內容已正確。

Boot block受損可能讓loader找不到kernel；filesystem metadata受損可能讓檔名/配置不可重建；data block損壞則可能只影響部分內容。Recovery/backup要對到相應層次與已知版本，而不是看到某個partition存在就直接宣稱整顆disk已修復。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swap space是頁面backing，不是通用持久檔案

Anonymous頁面可能需要swap backing；clean file-backed pages可從原file重讀，未必另寫swap。Swap partition與swap file是配置/存取路徑選擇，實作可預建extent mappings降低正常filesystem查找成本；不能把歷史swap-file路徑成本當所有新版本的必然結論，也沒有無條件RAM兩倍的容量公式。

固定4KiB頁與swap slot S7，內容41，兩個private-COW邏輯頁暫以S7作backing。下表只算本模型未消失的swap-PTE references，不冒充Linux實際swap_map編碼或所有swapcache refs。

事件	P1	P2	S7仍需服務的swap引用
初始	swap S7	swap S7	2
P1讀回並私有寫42	private resident F23=42	swap S7，應讀41	1
P2讀回	private F23=42	resident F24=41	0
無in-flight I/O/cache保留引用	各自resident	各自resident	才能回收S7

P1新值不能覆寫仍被P2引用的S7=41；必要時為P1的新版本另配backing。此處不固定Linux的fault次數或共用swapcache page策略，只驗證引用/內容/lifetime的必要性。

Slot回收必須等最後使用者，不只看某個process退出

如果還有swap PTE、in-flight device request或保留的cache ownership，提早把slot給另一頁可讓後續fault讀到別人的資料。Refcount模型需把計數範圍、誰增減、何時I/O停止與哪些旗標另計明列；單說「有兩個process，所以count=2」不是所有實作的規格。

Swap資料一般用於當次系統執行的記憶體延伸，不是備份；hibernate映像則有額外恢復契約。加swap可改善commit餘裕，但若active working set超過physical frames，可能造成thrashing與長latency，不能用容量取代第9章的locality/回收分析。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Striping、mirroring與parity的容量/故障條件

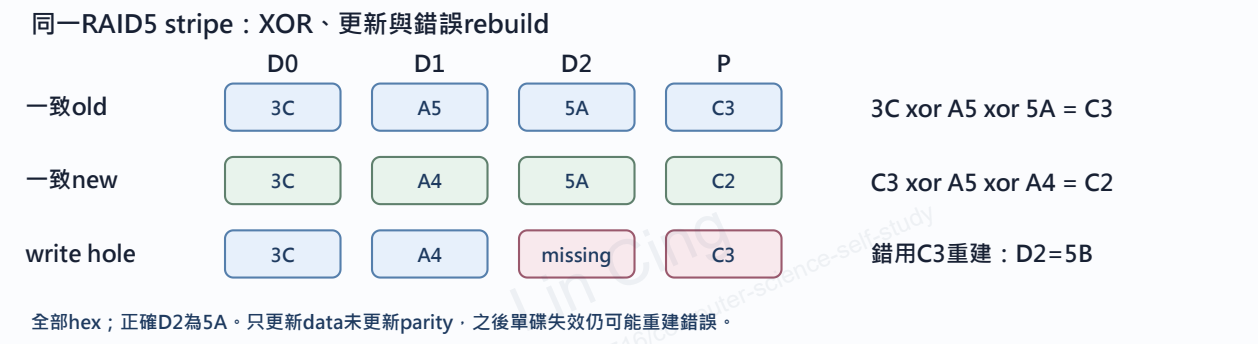
固定N顆同容量D，忽略metadata/spares；stripe是資料配置，mirror/parity提供特定故障下冗餘。RAID不是backup，刪除、惡意加密與共同controller錯誤仍可能同步破壞多份。

Level	組合概念	可用容量	固定完好array的device-loss保證
0	stripe無冗餘	ND	無冗餘，任一所需disk失效可失資料
1雙副本	pair mirror	每pair D	每pair可少1；不同pair與同pair雙失敗不同
2	bit striping與ECC	依code/check disks	依所選ECC與故障假設，不亂填通用公式
3	fine-grained stripe+dedicated parity	(N−1)D	1disk；多disk常參與同request
4	block stripe+dedicated parity	(N−1)D	1disk；parity disk可成write瓶頸
5	block stripe+distributed parity	(N−1)D	任意1disk，parity位置逐stripe分散
6	兩個獨立parity relations	(N−2)D	合格code/layout下任意2disk

RAID1可由多mirror組或一組多副本構成，容量先看拓樸。4disk的兩個2disk mirrors合計2D；一個四副本mirror只有D，不能共用同一容量答案。

RAID5一個stripe的XOR與小寫入

固定一個stripe有3個data bytes D0=3C、D1=A5、D2=5A，parity P=3C xor A5 xor 5A=C3（全部hex）。D1失效可由C3 xor3C xor5A=A5重建；單一XOR只提供一條關係，若兩個data同時未知則不足以唯一決定。



將D1改A4，P'=P xorD1 xorD1'=C3 xorA5 xorA4=C2。Cold uncached小寫入的read-modify-write需讀old D1與P、寫new D1與P'，共2reads+2writes；不含journal/cache額外I/O。完整3data stripe寫入可直接算P'並做4writes，無需為這個parity先讀old stripe。

Write hole會讓之後重建出錯

若new D1=A4已落盤但parity仍C3，此時stripe內仍能直接讀到D0/D1/D2，卻不再滿足parity。接著D2失效，錯用old parity重建得C3 xor3C xorA4=5B，而正確D2是5A。能容忍「任意一disk failure」的前提包含冗餘一致，不包含任意中斷造成的dirty parity再加disk loss。

可用具合適持久保護的write-back cache、parity/journal協議或copy-on-write等方式保護更新；其commit/恢復規則必須真正實作。僅加mirror、checksum或一個RAM flag不自動修復。重建期間要讀其餘devices，會帶來額外負載與再次failure風險，不能把rebuild speed當原始foreground bandwidth。

RAID6的第二條關係為何足夠

在適當finite field上，P=ΣD_i，Q=Σα_iD_i，兩失效位置a/b的係數α_a≠α_b；此處加法為field加法，在binary field可用XOR。扣除已知項後有A+B=P_missing，α_a A+α_b B=Q_missing。

代入A=P_missing+B，得(α_a+α_b)B=Q_missing+α_a P_missing。因係數不同，α_a+α_b非零可逆，故B唯一，再得A。這是兩個獨立parity relations的代數理由；若第二份只是複製同一XOR，兩式相依而仍不足。實際code要明定field多項式、layout與錯誤/erasure模型。

具體選GF(2^8)、modulus0x11D、係數1/2/4，資料3C/A5/5A。Q=3C xor(2・A5) xor(4・5A)=3C xor57 xor75=1E。失D1/D2後P_missing=FF、Q_missing=22，(2+4)B=22 xor(2・FF)=22 xorE3=C1；6的inverse為7A，B=7A・C1=5A，再A=FF xor5A=A5。乘法為此field的carryless/reduction操作，不是普通整數乘法。

RAID 6：逐類檢查兩個失效位置

固定一個 stripe，每個符號位於不同磁碟；資料符號為 D_i，$P=\oplus D_i$，$Q=\oplus(\alpha_i D_i)$。運算在 GF(2^8) 中進行，加法就是 XOR，係數 α_i **非零且兩兩不同**。下表假設失效位置已知、其餘符號正確，且同一 stripe 最多失去兩個符號。

失效符號	使用的資訊與恢復方法
D_a、D_β	從 P、Q 各扣除已知資料，解兩個方程；矩陣行列式 $\alpha_a \oplus \alpha_\beta$ 非零，解唯一。
P、D_a	從 Q 扣除其餘資料，得 $\alpha_a D_a$，再乘以 α_a^{-1}；最後重算 P。
Q、D_a	從 P 扣除其餘資料，得到 D_a；最後重算 Q。
P、Q	所有資料仍在，直接重算兩個 parity。

非零係數確保「P 與一筆資料同失」可由 Q 恢復；係數相異確保「兩筆資料同失」的兩個方程獨立。這是已知位置的 erasure 保證，不能據此推論能辨識任意靜默損壞，或容忍超出模型的共同故障。

P 與 D_1 同時遺失：用 Q 找回資料

採用多項式 `0x11D` 定義 GF(2^8)，各運算元以兩位十六進位表示。資料 D_0=`3C`、D_1=`A5`、D_2=`5A`，係數依序為 `01`、`02`、`04`，原本 `P=C3`、`Q=1E`。

P 與 D_1 遺失，但 D_0、D_2、Q 可讀。先從 Q 扣掉已知資料貢獻；在此 field 中扣除等同 XOR：

$$02D_1 = Q \oplus D_0 \oplus (04D_2) = 1E \oplus 3C \oplus 75 = 57.$$

因 `02` 的乘法反元素是 `8E`，故 D_1=`8E·57=A5`；最後重算 `P=3C⊕A5⊕5A=C3`。檢查 `02·A5=57`、`04·5A=75`，即可核對 Q。乘法採 carryless 運算並對 `0x11D` 約簡，不是一般整數乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0+1與1+0不能只看有一半冗餘

固定4disks。RAID0+1先stripe(A,B)、stripe(C,D)，再mirror兩stripe；RAID1+0先mirror(A,B)、mirror(C,D)，再stripe兩pairs。A失效後，0+1只剩完整(C,D)stripe可用；1+0則B供應第一pair，C/D供應第二pair。

此拓樸下再失C：0+1兩stripe皆有缺件而失效；1+0兩pairs仍各有B/D而可用。再失B則相反：0+1的(C,D)仍完整，1+0的(A,B)pair全失。兩種布局的容錯集合不同，不能不畫分組就說任意兩碟都安全。

固定同期間每disk獨立loss機率p、無repair，雙mirror存活率$1-p^2$；四disk RAID5為(1−p)^4+4p(1−p)^3。Common power/firmware/環境與rebuild會破壞獨立無repair假設；公式不是裝置實測壽命保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checksum樹與pool配置：偵測/冗餘/配額各有責任

本書ZFS案例在parent metadata存child地址與checksum，沿樹驗證metadata/data；若只在同一壞block裡自存一個也可被一同改壞的checksum，不能等同已知可信上層的end-to-end驗證。Checksum可偵測指定模型的損壞，只有另有正確冗餘來源才可修復，且checksum不等於對惡意者的authentication。

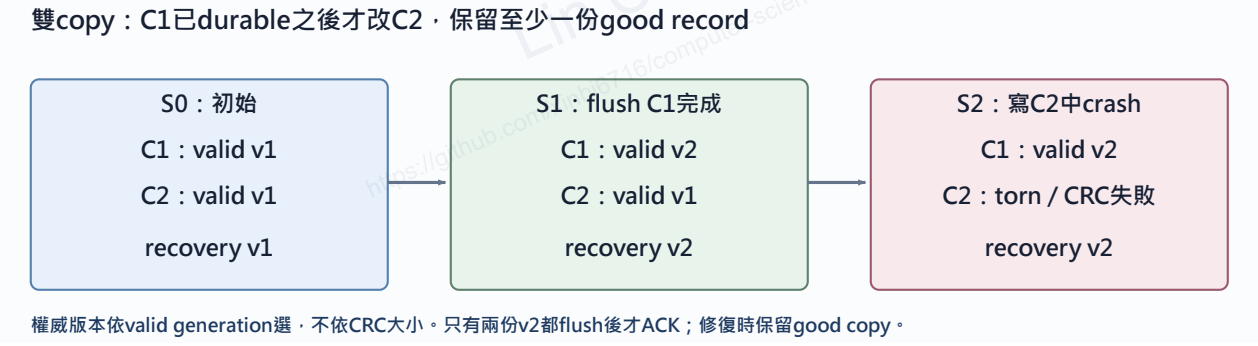
Storage pool把底層配置供多filesystem動態取用，quota限制某filesystem增長，reservation保障指定份額。固定抽象100GiB pool，A用30、B用20、free50；A quota60，B reservation40尚需保護20，則未受保留free30，A最多再用min(quota餘30,未保留30)=30。這是排除snapshot/dedup/compression的配額模型，不冒充所有ZFS accounting細節或無容量上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable storage是故障模型下的保證

理想stable storage不遺失資訊；實作只能在明列故障範圍下近似。固定serialized logical512-byte record，兩份device copies，generation不wrap，CRC可偵測本例注入的torn writes；durable flush可信。每一時刻只覆寫一個copy；本模型的單次crash／torn write只影響正在改寫的copy，另一未被改寫的copy及已完成flush的copy保持有效。同時破壞這些未寫／已flush copies的共同故障不在本次不變量證明內。CRC不是對任意corruption無碰撞證明。

Record格式為4-byte big-endian generation、504-byte payload、4-byte CRC32（覆蓋前508bytes）。v1 payload為ASCII V1加502zero bytes，CRC=BF4CA2E8；v2同法V2，CRC=4A112E80。Recovery只比較通過checksum的records，再選最高generation；不能拿CRC數字大小當版本順序。



每個crash prefix留下哪個完整版本

Crash邊界	copy1	copy2	可恢復結果
初始	valid v1	valid v1	v1
寫copy1途中、未確認flush	v1/v2/torn	valid v1	至少v1；若完整v2可取v2
copy1 v2已flush，尚未寫copy2	valid v2	valid v1	v2
寫copy2途中	valid v2	v1/v2/torn	v2，不是v1
copy2 v2也flush，之後ACK	valid v2	valid v2	ACK的v2可恢復

寫copy1時copy2保留old valid；第一份new durable之後才動copy2，所以第二階段copy1保留new valid。這個不變量由初始兩valid歸納維持至少一份valid。Recovery先保留所選good copy，再修另一份且flush；即使修復中再crash，good copy仍在。若兩份同時更新、flush不可信或雙份共同lost，前提破壞，不能套此證明。

Logical operation在ACK前crash可能恢復old或完整new，是未回覆操作的允許結果；不能把「未ACK」解釋成必然沒執行。單record的保證也不自動讓多record filesystem transaction atomic。

Flush、FUA、排序與atomicity分開

Volatile write-back cache可在資料未落nonvolatile媒體前回completion。Flush要求指定先前寫入達持久層；FUA要求該request的完成符合持久承諾；兩者是否由device原生支援、模擬及如何跨remapping layers傳遞，依driver契約。現代Linux文件的REQ_PREFLUSH/REQ_FUA是具體例，不能當成OSC9所有歷史裝置都有相同flag名稱。

CPU memory barrier不等於device cache flush，write ordering不等於單次write不會tear；單record雙copy亦不同於RAID5 parity一致性。必須把host buffer、device cache、persistent media與ACK逐層連起，才知道哪種power loss可以恢復。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一份資料的定位、冗餘與持久完成

HDD排程的53→14或53→0改變定位成本；SSD FTL把logical L1映到新physical page並搬移live資料；RAID用mirror/parity在指定故障下重建；stable-write協議決定更新途中crash留下何版本。四者可能同時存在，不可用其中一層成功替代另一層保證。

本章數例已分出SCAN236與LOOK208、RAID5 C3/C2及錯誤重建5B、雙copy更新第二階段必保留v2。每個結論都可由表內狀態/位址/資料重新計算，不只記policy名稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

評估成本與恢復時保持同一分母

將queue wait、seek/rotation、transfer與scheduler delay分開；計IOPS時同時標request bytes；計write amplification標host與physical bytes；計RAID可用容量先畫拓樸；談可靠度標故障域/相關性與repair；談ACK標flush/atomicity前提。

將HDD改SSD時，排程應重新考慮queue、並行度與flash搬移成本；改RAID拓樸時，重新列每個failure set能否重建；改flush或cache策略時，重新檢查ACK前後可恢復的版本。資料錯誤偵測、重建來源與歷史備份都須各自存在。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 10.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：檔案系統介面如何以可組合的物件、名稱與權限，表達持久資料的存取規則？

前置知識。 block 儲存、process 與 file descriptor、基本權限模型。

- 區分 file metadata、資料與 handle 狀態
- 比較 sequential、direct、indexed access
- 分析 mount、share 與 protection 的邊界

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

檔案、名稱、descriptor：三種不同身分

採 Linux/POSIX 一般本機 regular file 的一般讀寫 open，排除 O_PATH、裝置特殊行為及額外 LSM 撤權。物件身分用 (filesystem,inode)，例如 (F1,42)，不是字串 `/data/a`。同一物件可有兩個 hard-link 名稱；同一路徑也可先後指向不同 inode。此識別只涵蓋物件存活期間；回收後 inode 號碼可被重用，不能以這對數值跨任意時間保證同一物件。

層	保存的狀態	可共享者
directory entry	name → inode；名稱在此	hard link 增加另一個 entry
inode／file object	size、owner、mode、資料定位、link count	所有指向同一物件的 opens
open file description，OFD	current offset、access mode、status flags	dup／fork 繼承的 descriptors
process descriptor table	小整數 fd → OFD；FD_CLOEXEC	各 process 有自己的 table（本例 fork）

權限在 open 建立已授權 OFD；後續 read/write 仍檢查 descriptor 的 access mode。chmod 改 DAC 不會自動讓本機已開 fd 失效。其他保護機制另有規則。

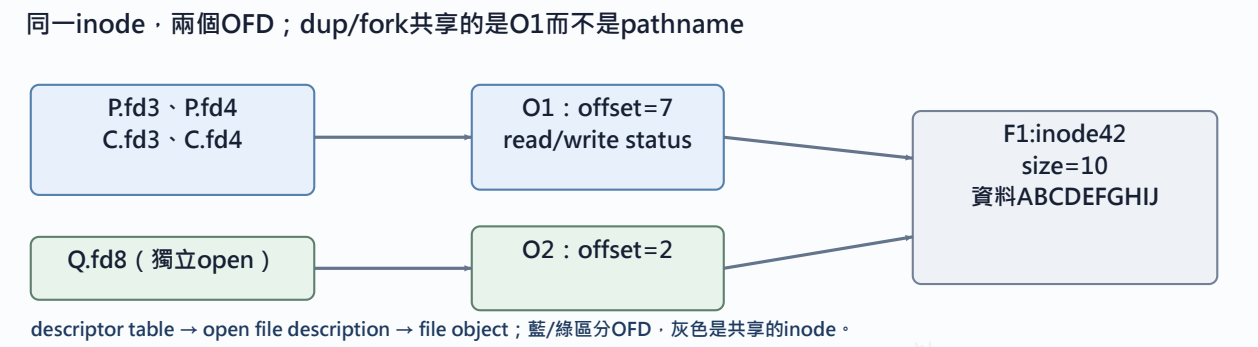
共享 offset 的每一步：資料為 ABCDEFGHIJ

P 先 `fd3=open(F)` 得 O1，再 `fd4=dup(fd3)`；Q 獨立 `fd8=open(F)` 得 O2。接著 P fork C，C.fd3/4 同指 O1。下列為指定的序列執行，避免把歷史 Linux 並行 offset 更新視為本例保證。

步驟	回傳資料	O1.offset	O2.offset
初始化	—	0	0
P read(fd3,3)	ABC	3	0
P read(fd4,2)	DE	5	0
Q read(fd8,2)	AB	5	2
C read(fd3,2)	FG	7	2
P pread(fd3,2,offset=1)	BC	7	2

每次成功 read 長度 k，使對應 OFD 的 offset 加 k；pread 使用自己的 offset、不改 O1。兩次獨立 open 的 offset 不會因 inode 相同而合併。

共享 offset 的每一步：資料為 ABCDEFGHIJ （續）



unlink、truncate、rename 的存活對象

普通檔 F1:42 初始有 `/a`、`/b`，link count=2；O1 已 open。unlink(a) 後 count=1；unlink(b) 後 count=0，但 O1 仍可讀。最後 close 且無其他持有者，才能回收。一般模型須同時區分名稱引用與開啟引用；實際還可能有 mmap 等引用。

truncate(F,0) 保留物件與名稱，size 變0；既有 OFD 的 offset 不必歸零。若 offset=7 再讀已空檔，回傳0，不是把 offset 改成0。

在同一掛載點、權限與類型允許、tmp 和 target 是不同物件且 rename(tmp,target) 成功時，呼叫原子替換名稱：新 lookup(target) 指向新 inode，已 open 舊 target 的 fd 仍指舊 inode。若兩名稱已是同一 inode 的 hard links，Linux rename 成功但不改變任何名稱；跨不同 mount 即使底層是同一 filesystem 也可能以 EXDEV 失敗。原子名稱變更只解決執行期間的可見性；斷電後能否保留，另需持久化協定。

副檔名是應用慣例，不能證明內容格式或允許執行。Linux 執行還檢查權限、格式／interpreter 等；名稱改成 `.exe` 不會把任意 bytes 變成合法程式。

sequential、direct、indexed：邏輯介面而非實體位置

方法	指定什麼	需要維持什麼
sequential	下一段	current position；rewind／seek 規則
direct	相對 record／block number	固定長度或可換算的 record 邊界
indexed	key	key→record／block 的索引與更新一致性

固定長度 L、record 編號從0開始，record j 起始 byte=jL。相對 block number 也從檔案開頭算；不能直接當磁碟 LBA，實體映射由第12章處理。

例 L=32B、block=1024B，每 block32筆；record77 的 byte offset=2464，file block2、block內 416。1000筆共32000B，需要ceil(32000/1024)=32 blocks，末 block有8筆、256B，內部碎片 768B。此例不含 sparse/compression/inline data。

sparse index 的範圍判斷與 I/O 計數

1000筆依唯一遞增整數 key 0…999 排序，仍用每 block32筆。索引32 entries，entry=(first key,block pointer)，各16B，共512B。查 key77，找最大 first key≤77，得到64→block2；讀 block2 後在64…95中找77。

一般若第 i block 的鍵值區間為 [k_i,k_(i+1))，鍵排序與索引正確就保證只需查該候選 block；落在 gap 仍須比對實際 key，不能把 predecessor 命中視為資料存在。

索引已在 RAM 時，binary search 比較次數至多 floor(log2 32)+1=6，另1次 data-block I/O；索引不在 RAM 且整個索引位於1 block，則最多1次 index +1次 data I/O。這不是每個現實查詢固定2次，更不是 O(1) CPU 搜尋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

短讀寫：不變量是已完成前綴，不是呼叫次數

要求寫6000 bytes；依指定回傳值2500、1500、2000：

呼叫	buffer起點	本次請求	成功回傳	累計done／下一offset（初始100）
1	0	6000	2500	2500／2600
2	2500	3500	1500	4000／4100
3	4000	2000	2000	6000／6100

迴圈不變量：0≤done≤N，buffer[0:done]已依序交付；下一次只送 [done:N]。成功 k>0 後 done+=k，保留前綴且縮短剩餘量；若永遠有正進展，最多 N 次1-byte成功就完成。錯誤並不保證終止成功；EINTR（未傳資料）可重試，EAGAIN 應等就緒，回傳0不可無限空轉。

read 到 EOF 可得到短資料，下一次 request>0 回傳0；count=0 的零回傳不證明 EOF。read error 後 offset 未必可推知。本表固定成功結果，沒有替錯誤亂加 offset。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

append 的原子單位與 positioned I/O

自行 lseek(SEEK_END) 再 write 不是單一原子步：P/Q 同時見 size100，皆寫 offset100，後者可覆蓋前者。O_APPEND 將「取 EOF＋該次 write」合為受介面規範的步驟；依檔案系統／協定的實作範圍判斷，尤其不能假定 NFS 模擬追加等同本機。

即使每次 append write 原子，一筆 record 分成 A1、A2 兩次呼叫，仍可能交錯 A1,B1,A2。需要整筆 record 的 framing、鎖或單 writer 才能建立更大的承諾；regular file 不是 pipe，不能套 PIPE_BUF 定理。

pread/pwrite 把 offset 作參數，可避免 lseek＋read 被別的 thread 改共享 offset；它們不自動讓重疊資料寫入互斥。本節的 pwrite 情境排除 O_APPEND；Linux 的 O_APPEND＋pwrite 有追加行為的特例。

directory 結構的能力與代價

目錄是 name→object 的映射；同一目錄名稱唯一，跨目錄可重複。搜尋／create／unlink／rename／列舉是介面，不要求底層必用線性表。

結構	可表達	需要面對
single-level	所有人同一名稱域	名稱衝突、無分組
two-level	各 user 有自己的目錄	跨 user 分享不自然
tree	路徑與子樹分組	單一父系下無直接共享子物件
DAG	多個名稱／路徑到同物件	引用計數、重複走訪
general graph	允許 cycle	終止、回收與環偵測

成功的 hard link 在同一 mounted filesystem 中新增同 inode entry，資料只有一份；Linux 跨不同 mount 可能以 EXDEV 失敗，即使底層是同一 filesystem，權限與物件類型仍須允許；symlink 本身是存路徑的另一物件，可能跨 filesystem、dangling 或循環。一般使用者不能任意 hard-link 目錄，避免破壞父子結構；概念 DAG 不是允許任意 Linux 目錄 hard link。

reference count 為何無法回收 cycle

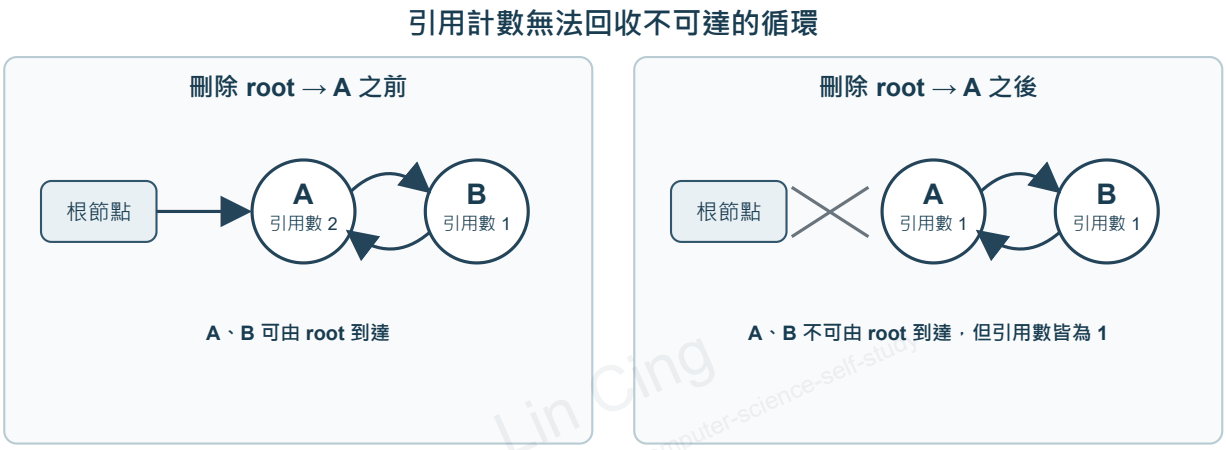
只考慮名稱／邊的抽象物件圖，不含持有中的 opens。A→B、B→A 且 root→A，計數 A=2、B=1。刪 root→A 後 A=1、B=1，兩者都不可達卻不會變0。

對有限 DAG：不可達子圖仍無 cycle，必有入度0頂點。它沒有可達頂點的入邊（否則可達），所以實際引用數為0；刪它再歸納，所有不可達頂點可被回收。cycle 沒有這個入度0起點。

traversal 用 visited set 可避免重複拜訪已遇過的物件，有限 V/E 的 DFS 為 O(V+E)；它不等於回收算法。mark/sweep 則從 roots 標記可達、回收未標記，必須固定／協調圖在觀察期間的修改。symlink 的循環限制只保證單次解析終止，不證明資料都可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

reference count 為何無法回收 cycle （續）



只計圖中的邊；不含 open、mmap 等其他引用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

路徑不能先作字串消去

固定 `/a/link` 的 symlink 內容為絕對 `/x/y`，所有目錄存在、可搜尋；輸入 `/a/link/../b`。

步	目前物件	待處理成分	動作
0	root	a,link,..,b	由 process root開始
1	/a	link,..,b	查 link 得 symlink
2	root	x,y,..,b	絕對target重設root，保留原suffix
3	/x/y	..,b	解析x、y
4	/x	b	從已解析物件處理..
5	/x/b	空	完成

先刪 `link/..` 會誤得 `/a/b`。相對 symlink 的 target 以「含 link 的目錄」為基準，並不是 process cwd。每次查下一成分要對目前目錄有 search(x) 權限；final symlink 是否跟隨則依操作。

路徑不能先作字串消去 （續）

TOCTOU：檢查的名字不等於操作的物件

P 先檢查 `/tmp/out` 是安全 ordinary file；攻擊者在檢查後將它改為指向敏感檔的 symlink；P 再 open 同字串，可能解析到另一物件。問題是兩次解析中間名稱圖可改，不是所有 symlink 都非法。

以已取得的可信目錄 fd 搭配 openat 可固定相對起點；O_NOFOLLOW 只處理 final symlink，不能因此聲稱中間各成分都可信。Linux 新版 openat2 的解析限制可約束整條路徑，屬後續介面，不回填為 Linux3.2已有。

操作應沿同一受保護 reference／受限解析完成。取得 fd 後的 fstat 檢查的是已開物件；仍需規定可接受的 owner/type 與 open 本身允許的副作用，不能先 O_TRUNC 再事後判斷是否安全。

mount 改的是名稱解析邊，不是把資料複製過去

F0 的 `/mnt` 原本有 `old` 且沒有 `new`；F7 root 有 `new` 且沒有 `old`。所有路徑祖先與涉及的目錄均可搜尋，且沒有其他阻擋解析的權限限制。掛 F7 到 /mnt 後，新 lookup `/mnt/new`→(F7,new)，`/mnt/old`→ENOENT；F0的old仍在，未被刪除。卸載後原名稱重新可見。

觀察	mount前	mount後
新解析 /mnt/old	F0:old	F7無old，ENOENT
新解析 /mnt/new	ENOENT	F7:new
事先open(F0:old)的fd	F0:old	仍F0:old

namespace 選擇哪張 mount map；相同字串在不同 namespace 可到不同物件。這不是 namespace 複製資料。圖中 blue=F0，green=F7，灰=被新 lookup遮住的舊內容。

mount crossing、.. 與 busy reference

在本章 Linux式模型，解析到 mountpoint 時換成掛上去的 root；在掛載 root 處解析 `..` 回到該掛載點的父目錄，而非任意停在底層filesystem root。process root 本身的 `..` 仍留在 root。

例 `/a/link` 的 target=`../b`，F7掛在 `/b`；輸入 `/a/link`：root→a→讀link→a/..=root→b→cross到F7root，至此結束。沒有輸入 filename，就不能再憑空加 `/file`。

正常 unmount 可因 cwd/open file/子掛載等活躍引用回報 busy，依系統規則判斷。Linux lazy detach 是先從 namespace 解連，物件最後引用消失再清理；不等於立刻讓所有 fd無效、也不保證 pending I/O成功。

將 mountpoint 當成 symlink 是錯誤模型：mount由namespace表決定，symlink是目錄中的一個有target字串的物件；兩者能在同一次pathwalk先後發生，但不能混為同一entry種類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享要分開指定 identity、ordering、visibility 與 durability

同一 user name 在兩台主機不必有相同 UID；AUTH_SYS式數值UID映射若不一致，可能對錯人授權。網路位址／UID宣告本身也不能代替可信驗證。書中的 NIS／LDAP／Active Directory 屬命名與身分基礎設施案例；LDAP是目錄協定，不能只因使用LDAP就推得連線加密或正確驗證。

檔案協定需分開回答：誰有權操作、兩個操作如何排序、對方何時重讀到新值、何時扛得住 crash。NFSv3基本檔案協定不靠OPEN狀態定位每次READ/WRITE，但仍有cache與穩定寫入語意；NFSv4加入stateful機制。不能把「stateless」推成沒有任何狀態或自動安全。

同路徑可在不同mount看到不同file；同file也可因cache／session規則暫時看到不同內容。成功write、成功lock、其他client看見、媒體持久化是四個不同事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一歷史，比較 UNIX可見性與session semantics

固定初值0、A/B都先open。A寫1；B在A close前讀；A close；B在自己既有session再讀；C此時才open讀。每步依序，忽略錯誤／data race；「UNIX欄」是本章單一一致file image的抽象，不宣稱所有網路cache已符合。

同一段歷史：即時可見與 session snapshot

事件	單一共享 image	Session snapshot
A、B open	A、B 見 0	A、B 各存 0
A write 1	image 變 1	A 私有 1，B 仍 0
B read	讀到 1	讀到 0
A close / 發布	image 仍為 1	公開版本變 1
B read（不重開）	讀到 1	舊 snapshot 仍讀 0
C open / read	讀到 1	新 snapshot 讀 1

範例固定依序執行；session 的多寫者衝突另定，不代表所有網路檔案系統。

session模型讓已開instance不被後來close改動；多writer的衝突／last-close規則須另定。immutable shared file則禁止內容與其唯一名稱被修改／重用，降低cache一致性問題，代價是更新以新物件／名稱表達。

advisory lock 的保證是有條件的

A對byte區間[0,100)持exclusive lock；B要同區間shared/exclusive須等，C對不重疊[100,200)可進。半開區間重疊當 max(start)<min(end)；[0,100)與[100,200)不重疊。

t	A	遵守協定的B	未拿lock的C
0	取得[0,100) X	—	—
1	寫version8	申請同區間，等待	可直接寫同資料（advisory不阻止）
2	fsync成功	仍等	可能破壞A/B預期
3	unlock	取得lock，依cache協定重驗／讀	—

互斥證明依賴「所有衝突操作先取得相容鎖」：若兩writer同時在區間內，將同時持互斥X鎖，與鎖服務規則矛盾。C不拿鎖時前提失效，證明不能套用。單獨鎖不保證cache invalidation或durability；必須先指定共享檔案系統契約。

發布新版本：atomic rename 不等於 durable rename

固定Linux一般本機filesystem，支援同掛載點的 rename 原子替換及 directory fsync，裝置遵守 flush；tmp 與 target 在同目錄且為不同物件，並假設權限、類型及其他條件允許替換。寫新版本時保留舊target：

1. 建立唯一tmp並取得fd，依短寫規則寫完；設置需要的metadata。
2. fsync(tmpfd)成功，讓新file內容與必要metadata持久。
3. rename(tmp,target)成功；之後新lookup見新inode，已open舊inode者繼續讀舊版。
4. fsync(parent-dir-fd)成功後，才承諾新名稱在crash後持久；每步錯誤都需處理。

第3步的atomicity避免執行中出現「target暫時消失」；第4步關心持久名稱。尚未完成第4步時，本章不對所有filesystem宣稱固定回到old或new。兩步fsync也不是跨filesystem交易，rename跨filesystem可失敗EXDEV。

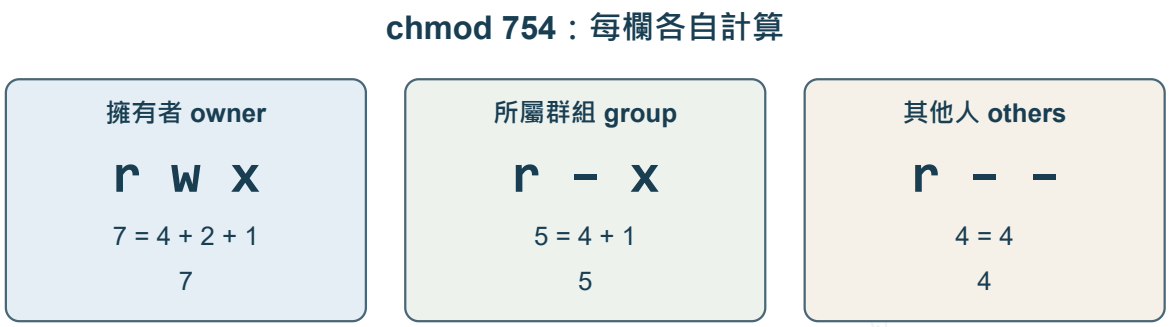
file rwx 與 directory rwx 的操作對象不同

忽略root/capability、ACL、sticky與其他保護，先看mode bits。Linux 以 filesystem UID（fsuid）、filesystem GID（fsgid）與 supplementary groups 作身分比對；本例假設 fsuid/fsgid 分別等於有效 UID/GID。依 owner 匹配、否則任一檔案系統群組匹配、否則 other 選「一個 class」，不把三class相加。bits r=4,w=2,x=1，要求位元req都在chosen中才准許。

權利	regular file	directory
r	讀取資料	列舉名稱
w	寫／改資料	新增、刪除、改名entry（通常另需x）
x	請求執行且格式等條件符合	search／穿越至已知名稱

D mode0711、F mode0644：非owner可穿越D找已知F並讀，不能列D全部名稱。D mode0733時other有wx，可新增／刪名，卻無r列名。刪F主要檢查父目錄wx，不是F的w；sticky directory另限制刪除者身分。

chmod 754：九個位元分成三組



`chmod 754 target` 將三組基本 mode bits 設為 owner `rwx`、group `r-x`、others `r--`。每組的 `r=4`、`w=2`、`x=1`：例如 group 的 `5=4+1`，所以有讀取與執行位元而沒有寫入位元。

權限檢查依適用的 owner、group 或 others 類別進行，不能把三欄任意相加。下例先假設 Linux 本機檔案、一般非特權使用者、沒有額外 ACL、LSM 或掛載限制；ACL、特殊權限和特權程序可能改變結果。

相同的 rwx，檔案與目錄代表什麼？

對象	r	w	x
普通檔案	允許讀取內容	允許寫入內容	可請求執行；格式、interpreter 等仍須符合
目錄	可列出名稱	可修改目錄項目，通常還須 x	可搜尋該目錄並解析其中名稱

假設 `/work` 是 754 目錄，沿途上層目錄均可搜尋。符合 group 類別者有 `r-x`，可列名稱、搜尋並走訪，但不能只靠這些權限在該目錄新增名稱；others 只有 `r--`，可讀取目錄項目名稱，卻缺少搜尋權，不能僅憑此解析 `/work/a`。owner 有 `rwx`；建立或刪除仍須符合其餘相關條件，例如 sticky bit 與掛載限制。

另一個普通檔案在 `chmod 754` 之前已由程序成功以可寫方式開啟。若後來修改 mode 移除該程序的寫入位元，Linux 本機一般檔案的已開 descriptor 不會因此自動關閉；新開啟是否成功則按當時權限判斷。此例不含額外 LSM 撤權、網路協定或裝置特殊行為；已開 fd 的 access mode 仍限制後續操作。

POSIX-style ACL：mask和選擇順序

以下為 Linux access ACL，非 Windows deny ACL；忽略 privilege/LSM。本例 filesystem UID/GID（fsuid/fsgid）分別等於 effective UID/GID，群組比對另包含 supplementary groups。owner=alice、filegroup=dev：user::rw-，user:bob:-w-，group::r--，group:qa:-w-，mask::rw-，other::r--。

1. fsuid 等於 owner，直接以user::判斷，不受mask。
2. 否則有matching named-user，要求其entry與mask都包含本次全部req；不通過便拒絕，不落到groups/other。
3. 否則匹配owning/named groups；至少一個matching group entry包含全部req，且mask包含req才准。已匹配卻不足就拒，不能落到other。
4. 沒有任何前述匹配，才用other；other不受mask。

Bob即使也是dev，只有w，不因dev而補r；把mask改r後，Bob完全無權，不回退other的r。Alice仍rw。ACL有mask時，顯示mode的group-class bits對應mask，不一定等於group::entry。

多group與複合請求：不能任意union後授權

令Carol屬dev及qa，沿前頁ACL，dev entry=r、qa entry=w、mask=rw。read(req4)可由dev通過；write(req2)可由qa通過；一次要求read+write(req6，如O_RDWR)卻沒有任何matching group entry同時含rw，因此拒絕。

主體／請求	命中路徑	判定
Alice／rw	owner rw	准；不套mask
Bob／r	named user w	拒；不回退groups
Bob／w	named user w & mask rw	准
Carol／r	dev r & mask rw	准
Carol／w	qa w & mask rw	准
Carol／rw	兩group各缺一項	拒
無matching身份／r	other r	准

將mask改r後，Carol的write也拒絕。此算法針對本次requested set，不是把所有顯示entry任意OR；named-user優先也不是一般化deny entry。

預設ACL、umask與既有handle

無default ACL時，create請求mode0666、umask0027，初始mode=0666 & ~0027 =0640；umask移除權利，不會把未要求的x加進去。

若父目錄有default ACL，子物件繼承為access ACL，再以create傳入mode限制對應owner/group-class/other權利；不能又無條件套一次umask。default ACL決定新物件初值，不會追溯修改舊file，也不是每次查詢的替代規則。

先open可寫fd，之後chmod0000：新open會被DAC擋，但一般Linux regular-file既有可寫fd通常仍可write。要撤回既有authority須定義不同機制／關閉或撤銷reference；修改畫面權限不等於回收所有已授權reference。ACL完整檢查規則見acl(5)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一檔案的四條追蹤線

追蹤線	必須記錄	常見錯誤
名稱	root/cwd、每個component、symlink suffix、mount map	先作字串link/..消去
物件	filesystem/inode、links、open references	unlink最後名稱就立刻free
存取	OFD offset/access mode、每次回傳長度	dup後假設offset獨立
授權	目錄search、物件ACL class、mask、授權時點	entry有w就無條件可寫

應用若要atomic publish再加持久線：tmp資料完成→file fsync→rename→directory fsync。名稱圖和持久儲存的提交時點不同，不能只用單一「成功」標籤表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從反例選擇正確操作

需求	不足的做法	能建立承諾的條件
指定offset讀且不影響共用游標	lseek後read	pread；仍處理短讀／錯誤
整筆append record不交錯	拆成多次O_APPEND write	record framing＋整筆鎖／單writer
刪除已知名稱	檢查file的w	父目錄wx、sticky與其他政策
限制解析到可信子樹	只檢查final不是symlink	可信root/reference及全路徑約束
跨client一致更新	只拿advisory lock	所有writer遵守＋cache/visibility協定
crash後新版本仍可找到	只rename成功	支援所述flush契約並完成file與dir持久化

本章的身分、名稱與handle契約，是第12章實作目錄／inode／cache時必須維持的介面；實作不能把物理block或cache狀態當成另一套使用者語意。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 11. Linux open(2); rename(2); acl(5); path_resolution(7); credentials(7).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：檔案系統如何以有限 block、metadata 與故障恢復協定，維持可存取且可恢復的持久物件？

前置知識。 file interface、邏輯/實體 block、journaling 概念。

- 追蹤 inode、directory、open table 與 page cache
- 比較 contiguous/linked/indexed allocation
- 分析故障時 metadata 與資料的一致性

從 logical file 到可恢復的 disk structures

固定教學 volume 的 block size 為 4 KiB：

block	persistent structure	關鍵內容
0	superblock	block size、volume block count、inode/free-space 位置
1	inode bitmap	inode 是否配置
2	data-block bitmap	data block 是否 free
3	inode table	file type、size、mode、direct/indirect pointers
4	root directory	`home→inode 3`
5	`/home` directory	`u→inode 5`
6	`/home/u` directory	`notes.txt→inode 7`
8–10	file data	`notes.txt` 的三個 4 KiB blocks

basic file system 把 logical block 轉成 device block；file-organization module 管理 free-space 與 logical-to-physical mapping；logical file system 管理 directory、FCB/inode 與 API；I/O control 與 driver 完成 block transfer。cache 可放在 upper layer，但 commit 的 durability point 必須由系統契約明確決定。

從 logical file 到可恢復的 disk structures （續）

一次 append 必須維持：

$$\text{name} \to \text{valid inode} \to \text{self-consistent block set}, \qquad \text{allocated} \cap \text{free} = \varnothing.$$

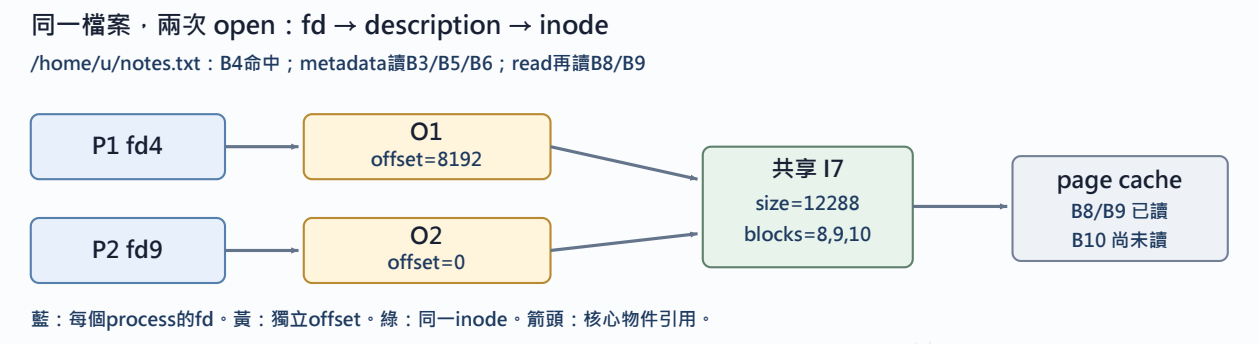
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Path walk、open 與 read 的完整狀態

固定 namespace `/home/u/notes.txt`，初始只快取 root directory block4。cold metadata path：

step	查詢	讀取或命中	得到
1	`/` 中 `home`	root directory block4 hit	inode 3
2	inode 3	inode table block3 cold read	directory block5
3	`/home` 中 `u`	directory block5 cold read	inode 5
4	`/home/u` 中 `notes.txt`	directory block6 cold read；inode table 已快取	inode 7
5	open inode7，權限通過	不新增 disk read	O1→I7、P1 fd4→O1、offset0
6	P1 read 8192 B	data blocks8、9	data加載；P1 offset=8192
7	P2 另一次 open 同 file，權限通過	inode/cache hit	新 O2→I7、P2 fd9→O2、offset0

Path walk、open 與 read 的完整狀態 （續）



這一條 cold path 有3個 metadata block reads（inode table、directory5、directory6）與2個 data block reads，共5次 block read。全部 cached 時是0次 disk read。directory cache、inode cache、system-wide open-file table與 page cache 是不同層，任何一層 hit 都不能直接推出資料持久。

本章區分三層。**fd table** 是每個 process 的整數fd→description引用，可含 close-on-exec 等 descriptor flags。**open-file description** 是 open 建立的核心物件，保存 current offset、access/status flags 及 inode 引用。**inode/FCB** 保存檔案身分、size、權限、block mapping，與共享 page cache 關聯。兩次獨立 open 產生 O1/O2，不能因 inode 相同就把兩者的 offset 合併。

Path walk、open 與 read 的完整狀態 （續）

OSC 9e 的教學圖把部分欄位畫在 per-process table，另有 system-wide FCB 表；若採該抽象，必須另說明共享 offset 所屬的物件。本例採用 Linux/POSIX open/dup 的三層表示，避免把「同一 inode」誤等同「同一 open description」。現在有2個 descriptions 指向 I7；並非同一個 description 的 reference count 等於2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

獨立 open 與 dup 的 offset 不變量

沿用 P1 fd4→O1、P2 fd9→O2，I7 size=12288。每次 read 完整成功、不受 signal 中斷，沒有其他 writer。

事件	fd 關係	O1 offset	O2 offset	回傳範圍
兩次 open 後	P1:4→O1；P2:9→O2	0	0	—
P1 read(fd4,8192)	不變	8192	0	bytes [0,8192)
P1 fd6=dup(fd4)	P1:6 也指 O1	8192	0	不讀資料
P1 read(fd6,4096)	fd4/fd6 共用 O1	12288	0	bytes [8192,12288)
P2 lseek(fd9,4096,SEEK_SET)	只改 O2	12288	4096	不改檔案內容
P1 read(fd4,1)	O1 已在 EOF	12288	4096	回傳0，不再前進

成功讀 r bytes 的一般規則是 $o' = o + r$，其中 $0 \le r \le \min(n, \max(0, size - o))$。dup/繼承同一 description 的 `fork` 分享此 o；獨立 open 不分享。關閉 fd6 只減少 O1 引用，fd4 仍可用；最後 description 關閉也不等於移除目錄名稱，unlink 與最後 open reference 是不同生命週期。

Mount 與 VFS 的具體分派

固定 mount table 將根檔案系統 A 的 `/mnt` 掛上檔案系統 B 的 root inode2。解析 `/mnt/log/x` 時，先在 A.root 找到 mountpoint；再切到 `(B,inode2)`，由 B 的 lookup 找 log、x。inode number2 在 A/B 可以重複，故識別至少要包含 filesystem/mount context 與 inode，不能單靠數字。

VFS 先查 dentry/inode 與 mount 關係，對 regular file 的 read 再呼叫該 filesystem 的 operations；local filesystem 解析 block mapping 後送 block I/O，NFS 則形成 RPC。共用介面不是所有下層都以同一個磁碟 index block 實作。權限、symlink、`..` 與 mount traversal 必須在各元件解析時處理，本例的5次讀取模型明定沒有 symlink、mount crossing 或額外權限資料 I/O。

Directory：linear list、hash 與可驗證查找成本

假設 n 個現存名稱被等機率查詢，且每次名稱比較成本相同，linear list 的成功搜尋平均為 $(n+1)/2$ 次比較；若第 i 個名稱查詢機率為 p_i，平均比較次數為 $\Sigma_i\, p_i i$。hash table 把 name 映射至 bucket，再比較 bucket 內 entries。固定 hash：

$$h(name) = \left(\sum_i code(char_i) \right) \bmod 8.$$

七個 names 的分桶：

bucket	entries	chain length
0	—	0
1	—	0
2	delta, epsilon, eta	3
3	gamma	1
4	beta, zeta	2
5	—	0
6	alpha	1
7	—	0

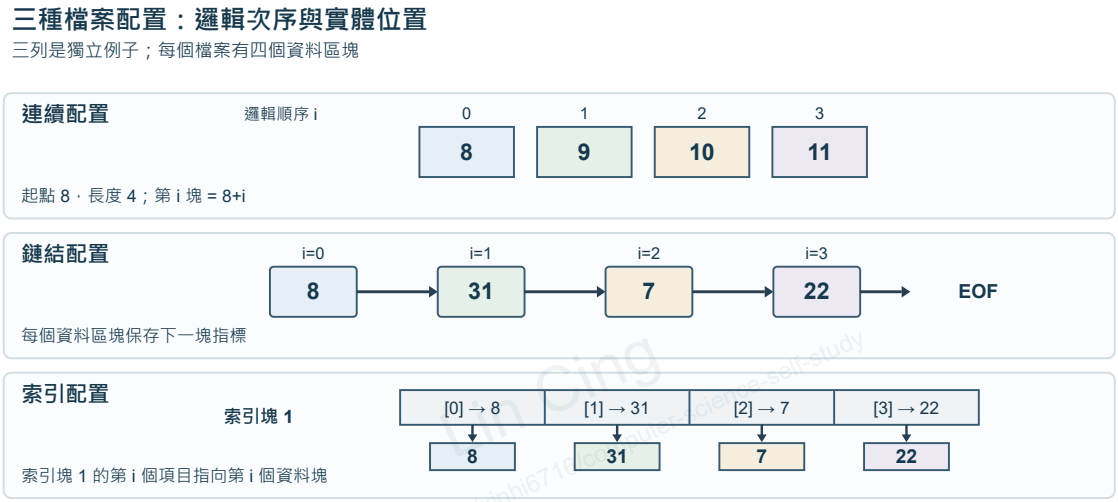
Directory：linear list、hash 與可驗證查找成本 （續）

load factor $\alpha = 7/8 = 0.875$。假設七個現存 keys 等機率被查詢，固定這組 buckets 的成功比較平均是 `(1+1+1+1+2+2+3)/7=11/7=1.571`。這是對查詢目標取平均，沒有對 hash function 取期望。

若另假設每對相異 keys 獨立、等機率碰撞到 m 個 buckets，鏈結位置依固定插入順序排列，則第 i 個插入者的期望成功比較為 $1 + (i-1)/m$。對 $i = 1 \ldots n$ 平均，得 $1 + (n-1)/(2m)$；本例 n=7,m=8 為1.375。常寫的 $1 + \alpha/2$ 是忽略有限 n 校正的近似。觀測1.571與期望1.375的分母/隨機性不同，不能說前者算錯。若所有 keys 碰撞，最壞仍為 $O(n)$。

chaining 的8 buckets 可存超過8個 names，不會因第9個 name 就必然裝不下；可依 load factor 與延遲政策選擇 resize。open addressing 則每 slot 至多一 entry，容量限制不同。rehash 的中間版本若讀者可見，可能漏掉舊 bucket 或出現同名兩筆；正常協定需用鎖、版本或增量搬移規則保證每次查找有一致的可見結果。

三種資料區塊配置



Allocation methods：直接、鏈結與索引的成本

固定 block=4 KiB、pointer=4 bytes，因此每個 pointer block/indirect block有1024 entries。以 logical block2（0-based）為例：

method	mapping state	取得 logical block2 的 disk reads	主要代價
contiguous	start=11, length=3	1 data read	external fragmentation、成長需搬移
linked	11→12→13	3 block reads	random access要追鏈
indexed，index cached	inode/index entry→data13	1 data read	index space
indexed，index cold	read index block，再讀 data13	2 reads	大型 index memory/TLB

contiguous block i 的映射為 `start+i`；linked 必須從 chain head 走；indexed 可用第 i pointer，代價取決於 direct、single、double、triple indirect 層級。

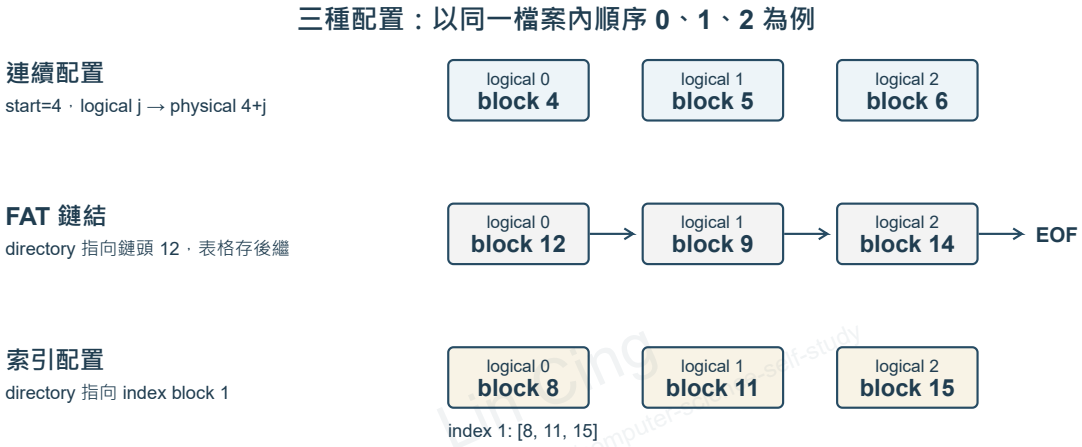
Allocation methods：直接、鏈結與索引的成本 (續)

12 direct pointers、1024-entry indirect blocks的四個範圍：

區間	logical block覆蓋	data bytes
direct	0–11	48 KiB
single indirect	12–1035	4 MiB
double indirect	1036–1,049,611	4 GiB
triple indirect	再下一段	4 TiB

若 offset=5 MiB，已超出12 direct+single indirect範圍，需要 double indirect；sparse hole 不必配置 data block，read 回零，但 logical size 可涵蓋該 offset。把「inode pointer 為0」解成 file 損壞會誤判合法 sparse file。

三種配置：從檔案內位置找到實體區塊



這是三個獨立的配置狀態；圖中的數字為 physical block 編號。

三個例子互不接續，各有 3 個 data blocks，logical block 從 0 開始。連續配置的 `start=4, length=3` 將 logical block 2 算成 physical block `4+2=6`。FAT 的 directory 記錄鏈頭 12，查 FAT 得 `12→9→14→EOF`；要找 logical block 2，沿鏈取得 12 的後繼 9，再取得 9 的後繼 14。索引配置的 directory 指向 index block 1，其三個 entry 依序記錄 8、11、15，所以 logical block 2 指向 physical block 15。

兩層索引：地址轉換與 I/O 分開計算

固定每個 data block 與 index block 均為 4 KiB，每個 block pointer 為 4 bytes，因此每張索引有 $N=4096/4=1024$ 筆。兩層索引最多選到 $N^2=1{,}048{,}576$ 個 data blocks；logical block j（從 0 起算）使用 $outer=floor(j/N)$、$inner=j \bmod N$。

例如 $j=1280$：outer=1、inner=256。先從第一層 entry 1 找第二層 index block，再從其 entry 256 找資料區塊。若 inode 已在記憶體、兩層 index blocks 和目標 data block 均不在相關快取、每個都需一次讀取，且不計其他 metadata 與 device cache，這次讀取模型是 **2 次 index-block reads + 1 次 data-block read = 3 次**。若第一層已快取，則少一次 index read；若 data 亦已快取，所需 I/O 會再減少。這是指定模型的讀取次數，而非任何實作一律固定三次。

inode 索引樹的容量上限

假設 inode 有 12 個 direct pointers，另有各 1 個 single、double、triple indirect pointer；block 大小 4 KiB，每個 pointer 4 bytes，所以一張 index block 可放 $N=1024$ 個 pointers。每個 direct pointer 指一個 data block；其餘三種依序可指向 N、N^2、N^3 個 data blocks。

路徑	可定位 data blocks
12 個 direct	12
single indirect	1,024
double indirect	1,048,576
triple indirect	1,073,741,824
合計	**1,074,791,436**

容量為 $1{,}074{,}791{,}436 \times 4096 = 4{,}402{,}345{,}721{,}856$ bytes，約 **4.00391 TiB**。這是本索引樹可定址資料區塊的結構上限；尚未扣除索引 metadata 的實體儲存需求，亦不代表檔案系統的 file-size 欄位、block address 格式或實體容量一定允許如此大的檔案。若檔案有 hole，在檔案大小內讀取 hole 得零值資料；在 EOF 之後讀取則回傳 0 bytes，不能混為一談。

由 byte offset 解出 indirect 索引

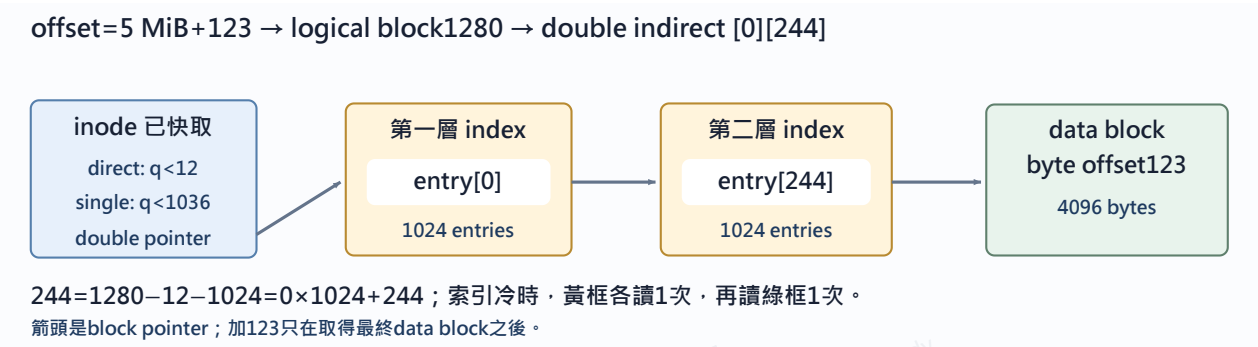
令 $q = \lfloor offset/4096 \rfloor, d = offset \bmod 4096$。前12個 logical blocks 是 direct；其後1024個是 single；再其後 1024^2 個是 double；最後 1024^3 個是 triple。邊界使用左閉右開區間，避免最大索引多算1。

固定 `offset=5 MiB+123=5,243,003` 且該位元組位於檔案 size 以內，有 $q = 1280, d = 123$。先扣 direct 的12，再扣 single 的1024，得 double 範圍內編號244。用1024進位表示：

$$244 = 0 \cdot 1024 + 244.$$

inode.double→第一層 entry0→第二層 entry244→data frame，再加 byte123。若 inode 已在記憶體、兩層 index 與 data 都冷，讀取恰為2個 index blocks+1個 data block；若第一層 index 已快取，則少一次 metadata I/O。

由 byte offset 解出 indirect 索引 （續）



總 data addressability 為 $4096(12 + 1024 + 1024^2 + 1024^3)$ bytes。這沒有把 index blocks 自己算成使用者資料；實際最大 file size 還受 size 欄位、block number 範圍及 filesystem 規格限制。在檔案 size 以內，sparse hole 的 pointer=0 可回傳零而不配置 data block；讀回的零不證明磁碟上存在全零頁。若讀取起點已在 EOF，則一般 read 回傳 0 bytes，不會無限回傳零。

配置不變量須容許合法的 **sparse holes** 與 **reflink/copy-on-write sharing**。hole代表沒有data block，不能判成失去指標；reflink讓不同inode暫時引用同一受COW保護的block，另以引用或版本資料追蹤，在任一方修改時才分離。沒有共享協定卻把同一block當成兩個互相獨立的可寫配置，才是重複配置錯誤；一份inode停止引用也不能在其他snapshot/reflink仍使用時就把block加入free set。

FAT 與嵌入式 linked allocation 的差別

固定 chain `11→12→13→EOF`。若 next pointer 就放在每個4 KiB data block，4-byte pointer 留下 4092 bytes payload；要求第3個 chunk 要先讀11、12取得鏈，最後讀13，共3個 block reads。不能同時把每塊當成4096 bytes user payload 又忽略嵌入指標。

FAT 把 next pointer 集中到獨立表，`FAT[11]=12,FAT[12]=13,FAT[13]=EOF`。FAT 全在 memory 時仍需2次 pointer lookup 才定位第3塊，但不用讀前兩個 data blocks，所以是1次 data I/O；FAT 冷時應另計它所跨的 table blocks。FAT 改善的是 chain metadata 的 locality，不會把追鏈的 CPU 步數從 $O(q)$ 自動改成 $O(1)$。indexed allocation 才能用第 q 個 index entry 直接定位。

extents 將一段表示成 `(logical_start,physical_start,length)`，在段內以差值運算；例如 `(0,8,6)` 直接把 logical4映到physical12。成長若遇到相鄰區域已占用，可新增 extent 而非一定搬移整個檔案，但 extent tree 的查找與 metadata 成本必須另計。

Free-space bitmap：位元約定與實際配置

OSC 9e 的 free-space bit vector約定 **1=free、0=allocated**。若 bitmap 覆蓋16 blocks，bytes/bit order 固定為 block0先：

```
block:  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
bits:   0 0 0 0 0 0 0 1 0 0  0  1  1  1  1  1
```

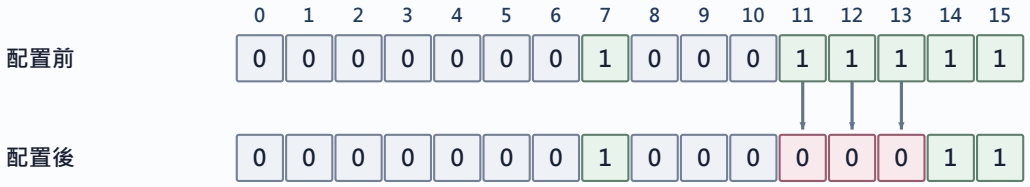
初始字串是 `0000000100011111`，free blocks={7,11,12,13,14,15}。inode7已有 data blocks8–10；append12 KiB需要三個 blocks。若要求單一連續 extent，first-fit從block11起選11、12、13，不選block7只因下一個block8已配置。

配置後：

```
bits: 0000000100000011
free: {7,14,15}
```

Free-space bitmap：位元約定與實際配置 （續）

first-fit配置三個連續blocks：11、12、13由free變used



inode7：blocks [8,9,10] → [8,9,10,11,12,13]；size 12KiB → 24KiB

綠1=free。灰0=原已配置。紅0=本次claim；剩餘free集合{7,14,15}。

順序必須維持 已配置 `block` 不在 `free set`；有效 `inode` 指向的 `block` 必為 `allocated`。先在volatile memory選位、再在disk bitmap與inode pointers先後更新，若中間 crash，可能造成double allocation或 lost/free orphan。記憶體 lock只解決並行 race，不提供斷電原子性。

bitmap overhead 可精算：

$$\#bits = \#blocks, \quad \#bytes = \left\lceil \frac{\#blocks}{8} \right\rceil.$$

1 TiB volume、4 KiB blocks有 2^{28} blocks，bitmap至少 2^{25} bytes=32 MiB；storage / filesystem I/O仍以 blocks 為單位。

同一 free set 的四種表示與更新

沿用 free set `{7,11,12,13,14,15}`。bitmap 保留16 bits；linked free list 可為 `head7→11→12→13→14→15→null`。若只配置一塊，可直接取 head7並令head=11，並非每次配置都要掃完整串列；找連續3塊才需要另外搜尋/整理。

Grouping 把數個 free addresses 放在一個管理 block，中間不必為每個 next 都讀一個磁碟 block。若每組存3個 addresses，前兩個為可配置頁、第三個連到下一組，則可用一筆讀取取得兩個候選；指向下一組的管理頁何時釋放，必須由組格式規定。

Counting 用 `(start,length)`，本例為 `(7,1),(11,5)`。配置連續3塊時，找到 `(11,5)`，回傳 `[11,14)`，剩下 `(14,2)`；若日後釋放 `[11,14)`，與相鄰的 `(14,2)` 合併回 `(11,5)`。不變量是各 free intervals 不重疊、相鄰者合併，union 恰為 free set。以平衡搜尋樹保存可做 $O(\log r)$ 的插入/合併，r 為 free runs 數；若還要按長度找足夠區間，需另外的 size index 或 subtree maximum，不能只靠 offset tree 保證所有查找都是對數。

ZFS space-map 的教材模型再把配置/釋放記為有序 extent log。初始`(11,5)`，重放`alloc(11,3)`後為`(14,2)`，再重放`free(12,1)`為`(12,1),(14,2)`。以 checkpoint+剩餘 log 重建，避免每次都在原位置改整片 bitmap；重放仍需正確的事件次序與交易識別，不能把同一 alloc 重放兩次當成兩次新配置。

Efficiency：cache、read-ahead 與 writeback

沿用 cold path：3 metadata reads+2 data reads=5 block reads。若 directory/inode cache全hit且data已在page cache，則本輪是0 disk reads；cache hit率必須按層報告，不能用「全部I/O減少80%」替代。

至少三個成本模型：

1. **cache hit**：省 disk transfer，但 lookup、copy、locking與 memory仍要成本。
2. **read-ahead**：把 block8、9接續載入，減少 seek/latency；猜錯則多讀。
3. **async writeback**：`write()`可先回傳，資料仍在 dirty page cache；完成與 crash durability 另由 flush/commit記錄。

固定10,000個檔案**每個恰為33 KiB**、無壓縮/dedup/sparse，對比4 KiB與64 KiB allocation units。4 KiB時每檔需要9 units、配置36 KiB、tail waste3 KiB；64 KiB時需要1 unit、配置64 KiB、tail waste31 KiB。後者每檔多浪費28 KiB，合計280,000 KiB=273.4375 MiB。不能只給平均file size就對平均值取 ceiling，因一般 $E[\lceil S/B \rceil] \neq \lceil E[S]/B \rceil$。

在「每 unit 另外一個4-byte mapping entry」的簡化模型，entry 數9→1，只省32 bytes/file、全體320,000 bytes；這與28 KiB資料空間差額不是同一單位。真實 inode/extents/index blocks 的 metadata 要依結構另算，不能把少配置的資料 bytes 稱為省掉的 pointer bytes。

Pointer size、統一 page cache 與同步寫入

4-byte pointer 可表示 2^{32} 個 block位置；在4 KiB block下，單層 pointer範圍只是 $2^{32} \times 4KiB = 16TiB$ 的理論 addressability，實際 file size 受 direct/indirect結構、block count與filesystem欄位限制。8-byte pointer擴大地址範圍，但 index block能存放的 pointers減半，因此並不一定在同容量下更省空間。

unified page cache讓 mapped file與 read/write共用 page；一致性要靠 PTE、dirty tracking、writeback與 file locking協定。synchronous write 必須等指定 durability point；asynchronous write 可能在 cache 回傳，若之後斷電，成功 `write()`不足以證明內容持久。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Journal recovery：從故障點重建唯一結果

固定交易 T7 在原始檔案 size=12 KiB 的狀態追加 12 KiB：自選定起直到 commit 或 abort，blocks 11、12、13 由分配鎖或預留機制獨占，不供並行交易使用；舊資料塊不被覆寫。崩潰後先完成 journal recovery，才恢復分配或對外提供新狀態。

```
metadata: bitmap 11,12,13: free→used
          I7.blocks += 11,12,13
          I7.size = 24 KiB
data:     block11,12,13
```

本例使用 redo-only、no-steal 的 ordered metadata journal。descriptor、metadata after-images 與 commit 使用可辨識 transaction ID；假設完整 commit record 可原子辨識，torn/incomplete record 不算提交。持久寫入次序為 descriptor→data completion→metadata records durable→commit durable→checkpoint durable→clear transaction。commit 前不把未提交 metadata 寫回 home locations，且 flush/barrier 真正要求的持久順序不能被裝置 volatile cache 任意顛倒。故障前後：

Journal recovery：從故障點重建唯一結果 （續）

crash point	redo見到的狀態	recovery action	外部可見
descriptor前	無T7	discard	old，size12 KiB
descriptor後、commit前	未提交	丟棄	old
commit後、home未完成	已提交	冪等重放	new，size24 KiB
home完成、transaction未清	已提交且可能已套用	重放/辨識已完成	仍為new

「已提交」不能只看 data block寫了；commit record是 atomic transaction的判準。為避免metadata指到尚未寫完的data，本模型要求 data完整寫回後才寫commit（ordered metadata journal）。若只用metadata journal，user data的內容不自動具有同一個原子性或durability保證。

fsck可從directory/inode/bitmap掃描找出orphan block或重複配置，但無法推回一個從未持久寫入的使用者寫入；consistency與durability分開驗證。

Journal recovery：從故障點重建唯一結果 （續）

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何任一 crash prefix 都只恢復成舊或新

metadata 狀態用三元組 $M = (bitmap, pointers, size)$。舊狀態 M_0 的 pointers=[8,9,10]、size=12288；新狀態 M_1 的 pointers=[8,9,10,11,12,13]、size=24576。

commit 前。 no-steal 保證 home=M_0，新 data 雖可能已在11–13上，尚未由有效 inode引用；忽略未提交 log 後仍是 M_0，free bitmap可重新配置那些 blocks。

commit 後、checkpoint 未完成。 完整 after-images 已先持久，故任何部分 home 更新都可由 log 覆寫成 M_1。例如 home bitmap 已新而 inode pointers仍舊，不能直接對外開放此中間狀態；先 redo三個欄位，再 mount。因為 redo 是 `size←24576` 與 `pointers←完整新陣列`，重放兩次仍是 M_1，不是再 append一次變成36 KiB。

log 清除後。 清除只允許在 checkpoint durable 之後，故沒有 log 時 home 已完整 M_1。三分支涵蓋每個合法 crash prefix，證得原子 metadata 可見性。若允許 commit 前改 home，就需 undo 或其他協定，不能仍以「丟掉未提交log」證明恢復成功。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Journal、snapshot 與 backup 解不同故障

journal 解決中斷更新的結構一致性，不能恢復使用者合法刪除後已被覆寫的內容。snapshot 保留舊可達 blocks，可回到先前版本，但與原 volume 共用裝置，裝置完全損毀仍可能一起失去。backup 另外保存資料副本，恢復所需的來源必須實際存在。

固定星期日 full backup B0，週一 incremental B1只含週日後變動，週二 B2只含週一後變動。週三損毀要回到週二狀態，必須依 B0→B1→B2恢復，還要包含刪除/改名紀錄；只套 B0+B2會漏掉僅在週一改動的檔案。若 B2 是 differential，保存「自B0後所有變動」，才可 B0+B2。mtime 不可信或備份中來源仍在改寫時，需要 snapshot/一致性點與更可靠的變更記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFS：file handle、cache validation 與 retry

固定 **NFSv3（RFC1813）**，不把NFSv4的OPEN/session state混入。client先從mount取得遠端root handle，再以`LOOKUP(directory_handle,name)`逐段取得opaque file handle；READ明列handle、offset與count。NFSv3沒有遠端OPEN/CLOSE程序，client的fd/offset管理不等於server保存一個POSIX open description。handle識別的是物件，不是可以任意解析的檔名。

例如已取得 `/export` 的H0，`LOOKUP(H0,"u")→H1`，`LOOKUP(H1,"notes.txt")→H7`，再`READ(H7,offset=4096,count=4096)`。相同READ可重送而不重複修改server狀態，但若期間其他client寫檔，重讀內容未必等於第一次回覆；「副作用冪等」不等於「結果永不改變」。REMOVE回覆遺失後，另一client若在同名建立新物件，再把舊REMOVE當新請求執行便可能刪錯物件。

固定 retry trace：

step	client	server	保存證據
1	send create X id=12	建立 X，回覆遺失	server request id/effect/result
2	timeout，重送 id=12	找到舊紀錄	回傳原結果，不再建立

這需要 duplicate-request cache或對應版本的 session/transaction機制，且保存 duration 與 crash behavior要明示。有限 cache過期後，不能虛稱永久 exactly-once。client cache要驗證版本/mtime/file handle；cache hit不是server commit。

UNSTABLE write、COMMIT 與 server 重啟

NFSv3的WRITE回覆含實際承諾的穩定程度與write verifier。client要求UNSTABLE時，server可先只保留volatile資料；client在需要持久性時發COMMIT，並檢查verifier與先前WRITE一致。若server重啟遺失volatile資料，verifier改變，client必須重送仍未確認穩定的資料。

事件	server狀態	client尚需保留什麼
WRITE offset0,data=A，回UNSTABLE,v7	A可能只在RAM	A與未提交範圍
server重啟，回覆verifier改v8	A可能遺失	不能以先前WRITE成功視為持久
COMMIT回v8，與v7不同	不能證明A已保存	重送A，再要求穩定完成
重送WRITE後成功穩定完成，verifier一致	A已達協定穩定儲存承諾	可解除這段重送義務

client attribute/data cache還可能暫時陳舊。close-to-open或timeout validation是特定client的快取政策，不能從NFSv3名稱推成全域線性一致。遠端mount也不自動繼承server其餘mounts；namespace要依client自己的mount table解析。

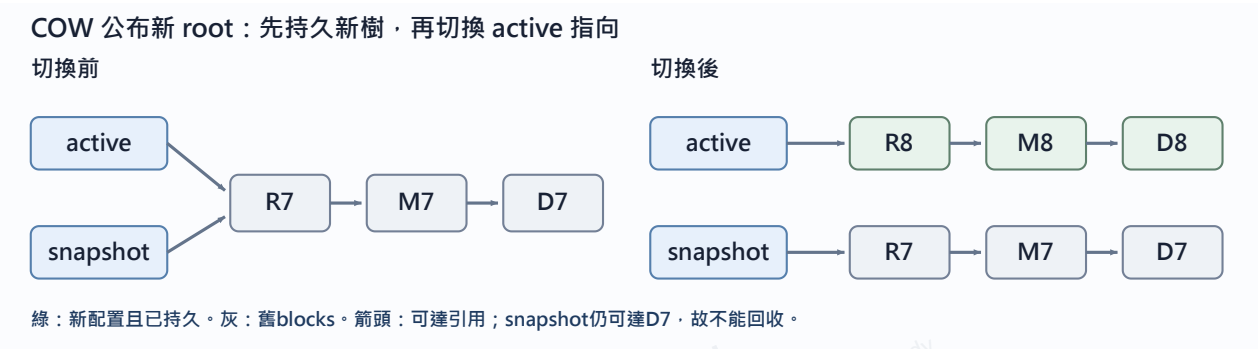
WAFL COW：root 切換與 snapshot 共享

固定 version7 使用 root R7→metadata M7→data D7，active root與snapshot root都可到達舊樹。下表的「2個保留root可達」是教學用的可達性計數，**不是斷言WAFL對每個block實作整數refcount=2**；教材描述的snapshot bitmap可記錄哪些快照仍使用該block。修改 D7 的流程：

stage	active root	新 blocks	舊樹的可達性
v7	R7	—	active與snapshot都可達R7/M7/D7
prepare v8	仍是R7	D8,M8,R8	R8由交易保留，尚未公布；舊樹仍可達
durable	仍是R7	D8/M8/R8已持久	crash before switch仍讀v7
atomic switch	R7→R8	切換後	active可達新樹，snapshot仍可達舊樹

COW寫入把新資料放到新 block，所有新data/metadata先持久，最後以可復原的atomic root publication切換可見版本；不要求真實裝置任意大小的root更新都天然原子。snapshot保留R7時，D7仍可達而不能回收；刪掉snapshot且沒有其他root引用後才具備回收條件。準備中未公布的blocks也須由交易的allocation管理保留，不可因尚無public root就被別人重用。

WAFL COW：root 切換與 snapshot 共享 （續）



這增加 write amplification：一個 data block更新可能同時需要新 data、metadata與root blocks。WAFL的 free-space collector需同時追蹤 active/snapshot refs，不能只因 block標成「舊」就回收。

從名稱到block：需要維持的四組對應

對應	一般不變量	失敗例
path／directory→inode	依namespace解析至實際物件	symlink suffix被字串先消掉
fd→OFD→inode	offset／reference歸屬正確	dup後誤以為offset獨立
logicalblock→physicalblock／hole	實際配置與資料位置一致	hole被當作disk block0
pointer／size↔free-space metadata	已配置block不被第二owner重配	bitmap先free但舊pointer仍可用

memory lock維持同時操作的互斥；crash後的多項metadata一致性另需journal／COW等更新協定。cache valid、dirty、writeback及device completion也不能用一個「已寫」合併。

journal與COW的提交時點回到同一個問題

T7 使用 redo-only／no-steal；commit durable 後可重放完整 after-images。

crash prefix	no-steal redo模型的恢復
commit尚未durable	home仍old，不能把未提交更新先寫home
commit durable、home部分更新	重放全部after-images得到一致new
home全部durable、checkpoint完成	按已證安全截斷條件回收log

after-image賦值可冪等重放；若允許未提交home先落盤，便須額外undo等機制，不能沿用本表。COW則先持久化新data／metadata，再以可恢復的atomic root publication切換；old snapshot仍可達的blocks不能先回收。

metadata 一致不等於完整資料交易、備份或 NFS 跨 client 一致性。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 12.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：作業系統如何以標準介面控制異質裝置，同時控制延遲、吞吐、隔離與錯誤？

前置知識。 interrupt 與 DMA、memory mapping、block 與檔案。

- 追蹤 application request 到裝置完成
- 比較 blocking、nonblocking、async 與 buffered I/O
- 定位 I/O bottleneck 而不只看 CPU

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I/O 的三條時間線

一次 `read(fd,buf,4096)` 可能只碰 page cache，也可能啟動裝置。必須分開追蹤：①呼叫 thread 的 running/blocked/ready；②request 的 queued/in-flight/completed；③buffer 的 CPU/device 所有權。三者不是同一個狀態機。

假設單一 blocking thread、cache miss、單次 DMA 成功：t=0 進 syscall，t=2 µs 完成檢查並提交；t=3 µs thread 睡眠；t=102 µs DMA 完成；t=104 µs handler 標記完成並喚醒；t=110 µs thread 被排程，t=112 µs 回到 user。裝置服務約 100 µs，呼叫延遲 112 µs，醒來到實際執行另等 6 µs。

不變量： thread 看到成功結果前，資料必須已可由 CPU 讀取；thread 變成 ready 並不表示已返回 syscall。排程、完成通知與資料可見性各有自己的同步條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一介面，不同操作契約

`fd` 代表已開啟的物件與存取權，不是硬體位址。regular file 可以 seek；pipe/socket 是 byte stream，通常不能 seek；terminal 可能有 line discipline；裝置控制透過型別特定的 `ioctl`。統一入口不保證所有操作都可用。

需要分辨五種觀察點：資料被接受進 user/kernel buffer、driver 提交、controller 完成、媒體持久化、遠端應用處理。它們並非所有 I/O 共通的線性次序；本地檔案可能依提交、裝置完成及同步契約達成持久化，網路操作則須依協定區分傳送完成與遠端應用處理。`write` 返回 4096 不普遍代表這五層全部成立。持久化要使用檔案系統與裝置都支持的同步契約；網路傳送成功也不能推出收件應用已提交交易。

本章用明確的教學裝置模型推導機制；不把該模型的數字當作某台真機量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Port、MMIO、controller 與匯流排

device 是周邊本體；controller 解碼命令、保存狀態並傳資料。CPU 可以用獨立 I/O 指令存取 port，也可透過受保護的虛擬映射讀寫 MMIO register；MMIO 不是一般 RAM，不能任意合併、快取或重排存取。具體順序由架構與 driver 的 I/O accessor/barrier 契約決定。

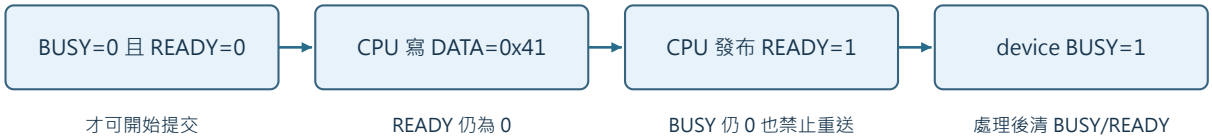
例：教學 controller 有 STATUS、COMMAND、DATA 三個 register。STATUS.bit0=BUSY，bit1=ERROR；COMMAND.bit0=READY。driver 必須先確認 **BUSY=0 且 READY=0**，再寫 DATA=0x41、設定輸出操作與 READY。若先 READY 再 DATA，裝置可能讀到上一筆值。多個 CPU 提交還需要 driver 的 lock/queue；只有硬體 BUSY bit 不是互斥鎖。

bus 是共用傳輸仲裁點；菊鏈位置與 bus topology 影響存取/中斷路徑，不代表 OS 可以省掉權限檢查。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polling：完整握手與循環不變量

PIO 輸出 0x41：命令發布晚於資料準備



can_submit = !BUSY && !READY；發布後至完成前，下一筆均不得覆寫 DATA。

步驟	執行者	BUSY	READY	DATA	可進行的下一步
0	初始	0	0	舊值	driver 可提交
1	CPU 寫資料	0	0	0x41	發布命令
2	CPU 設 READY	0	1	0x41	controller 接收
3	controller 接收	1	1	0x41	輸出資料
4	完成後清命令/狀態	0	0	0x41	CPU 檢查 ERROR

提交的循環前提為 can_submit=(BUSY==0 && READY==0)。READY=1 但 BUSY=0 是「已發布、尚未接受」窗口，仍不得覆寫。單一 producer 也可能在步2立即嘗試第二筆，故只檢BUSY會把DATA=0x41改成0x42，裝置尚未讀便遺失第一筆。

Polling：完整握手與循環不變量 （續）

正確兩筆trace：初始(0,0,舊值)→CPU發布(0,1,0x41)→第二次submit被拒、DATA保持0x41→裝置接受(1,1,0x41)→完成清為(0,0,0x41)→CPU才發布(0,1,0x42)。在單producer、上述兩bit檢查及存取順序保障下，從發布到完成之間can_submit永遠false，故DATA不被下一筆覆寫。若裝置永不完成，polling不會終止；timeout也不表示裝置已停止使用buffer。

Interrupt：事件通知不搬運全部資料

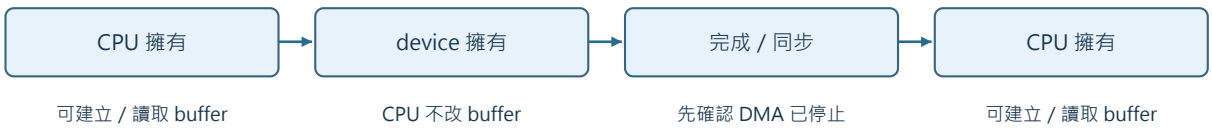
controller 發 IRQ；CPU 在架構允許的邊界保存必要狀態、依 vector 找 handler。handler 判定來源/狀態並 acknowledge，更新 request、排 deferred work 或喚醒 waiter，再返回。共用 IRQ 要檢查「是否本裝置造成」，不能收到 IRQ 就任意完成 queue 首項。

可遮罩的裝置中斷、不可遮罩的 NMI、同步 exception 不是同一分類；不能把整段低 vector 都稱為 NMI。短 handler 與可睡眠的後續處理分開；具體分工依 OS。interrupt 使 CPU 保存必要控制狀態並轉入 handler；若原本在 user mode，才會跨越特權模式邊界。只有排程換 thread 時才另有 thread context switch。

遺失喚醒反例： waiter 先檢查 `done=false`，IRQ 設 true 並 wake，waiter 隨後才睡眠，可能永遠睡。正確流程在同一 lock 下「檢查條件→入 wait queue→釋放 lock 並睡眠」，醒後以 while 重查；IRQ 在同一同步規則下發布 done。通知不是可累積的完成證據，done predicate 才是。

DMA：地址、資料一致性與 buffer 壽命

4096 B 接收：地址映射與 ownership 不能省略



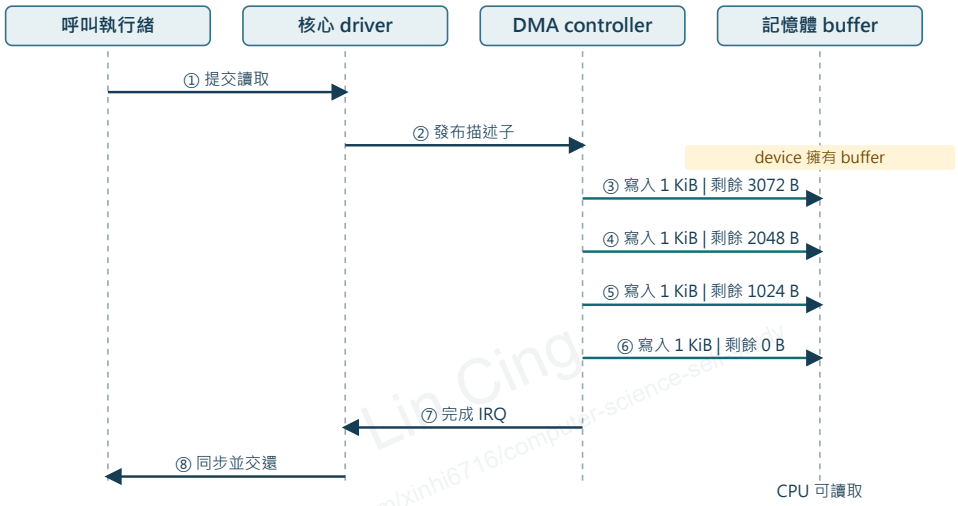
submit 前：map / cache 維護 → descriptor / ownership → doorbell

terminal 後：完成必要同步 → 發布結果 → unmap / 釋放（僅在裝置已不再存取後）

CPU 建 descriptor `{device_addr, dma_addr, length=4096, direction=FROM_DEVICE}`，準備映射/必要 cache 維護後發布；controller 搬資料，最後以 completion 通知。DMA address 可以經 IOMMU 轉譯，不必等於 user VA 或直接等於 physical address。IOMMU 限定 device 可 DMA 的範圍，不能取代 fd 存取權檢查。

本例 CPU→device 轉移 ownership 後，CPU 不讀寫接收區；完成且完成必要同步後才讀取。CPU 必須保留 descriptor、映射與 backing pages，直到確知裝置不再存取。釋放 user VA 不代表 DMA 自動取消。非 coherent 平台尤其要使用指定 DMA API；CPU memory barrier 不會自動完成 cache invalidate。

一次 DMA 讀取：控制與資料分開走



例中讀取 4096 B，分四次寫入各 1024 B，完成後發一次 IRQ；分塊與 IRQ 次數是模型假設。Driver 先映射並發布描述子；DMA 期間 buffer 由裝置使用，完成及必要同步後才交回 CPU。喚醒不代表執行緒立即執行。

PIO、interrupt、DMA 成本分開算

教學模型：4 KiB，PIO 每次搬 4 B，CPU 每次 80 ns，共 1024 次，因此資料搬運 CPU 時間 81.92 µs。改 DMA，提交 2 µs，IRQ/收尾 1 µs，CPU 成本 3 µs；裝置傳輸仍需 100 µs，並非 3 µs 完成。

若採週期性檢查，每次檢查 100 ns、每 5 µs 檢查一次，且兩次檢查之間 CPU 執行其他工作或睡眠，100 µs 服務期內最多約 20 次、成本 2 µs，但完成偵測另有 0–5 µs 延遲（均勻相位假設下平均 2.5 µs）。每完成一次 IRQ 成本 1 µs 時，2000 次/s 的 IRQ CPU 佔用為 0.002 s/s=0.2%；固定 5 µs 週期檢查卻需每秒 200000 次、2%。此數值只算檢查本身；若在間隔中持續忙等，核心佔用會接近 100%。

DMA 仍消耗 memory/bus bandwidth；CPU 可並行計算不表示傳輸沒有系統成本。小筆傳輸可能由 PIO 較省 setup；是否較快必須固定資料量、延遲與 CPU 分母比較。

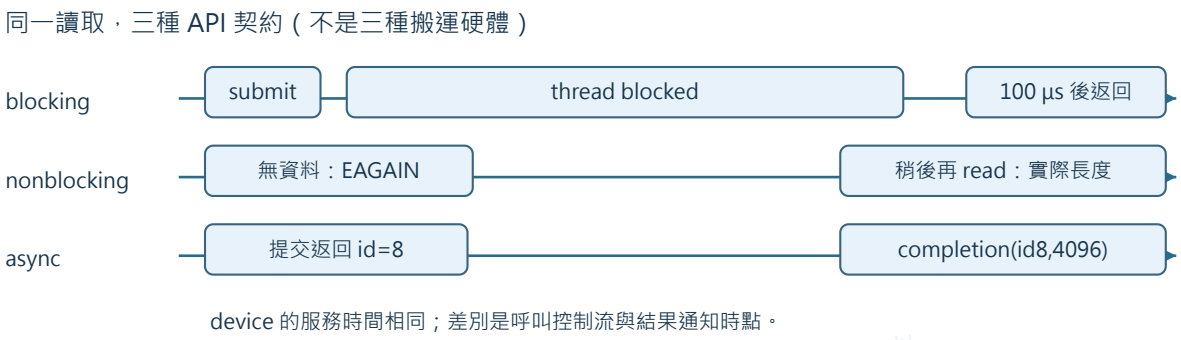
Block、character、network 與 memory mapping

block device 按 block 定位、可支援 random access；character device 提供序列資料，但介面是否保留 record、能否 seek 都依裝置契約。file `mmap` 把檔案內容映進位址空間，由 page fault/page cache 處理；controller MMIO 是寄存器存取，不能混同。

raw/direct I/O 旨在繞過部分快取/檔案層服務，仍受權限、buffer alignment、生命週期與隔離限制。network socket 分出 endpoint、連線與傳輸；stream 不保留 write 邊界，datagram 則有自己的 message/truncation 契約。readiness API 告知可嘗試操作，不替應用搬資料，也不保證另一個 thread 沒先消耗資料。

timer 裝置提供時鐘、中斷與計時。虛擬 timer 可用按 deadline 排序的 queue：在 now=7 ms 時存在 deadline 10、13、13 ms，硬體只設 10；到期取出 10，再設 13，到 13 時取出兩項。精確同時性不保證 callback 當下立即執行。

Blocking、nonblocking、asynchronous 三種觀察



blocking：呼叫可睡眠，結果返回時代表該次操作結束，但仍可能 short read。nonblocking：以要求正長度的非阻塞 pipe 或 stream socket 為例，暫無資料、寫端仍開啟且操作原會等待時，返回 EAGAIN；若已到 EOF 則可返回 0。regular file 的 O_NONBLOCK 語意依檔案類型及系統契約，不可套用此規則。先 poll/select 再 read 仍須處理其他讀取者搶先消耗資料後的 EAGAIN。asynchronous：提交返回與完成分離，完成項包含 id、status、actual length；非成功提交可能根本沒有 request。

stream 有資料 200 B 而要求 512 B，合法結果是 200；更新 `done=200`、`remaining=312`、`dst=buf+200`。下一次回 112 則 done=312、remaining=200；再回 0 代表 EOF，應把 312 B 的短結果交給上層或回報協定截斷。回傳 -1 時不把負值加進 done；EINTR/EAGAIN 的處置依 API 與已完成進度。

向量 I/O：部分結果如何跨 iovec

`writev` 有三段長度 [100,200,300]，總計 600 B。若此次成功傳 250 B，先完整消耗第 0 段 100，再消耗第 1 段 150；重試向量應是 `[base1+150,50]`、`[base2,300]`，共 350 B。再次從原向量重送會重複前 250 B。

向量介面減少組成連續暫存區的 copy 與 syscall 次數，仍不保證裝置零 copy，也不保證跨所有檔案/網路類型的任意長度原子性。原子寫入、寫入順序與耐久性是三個獨立規格。

延遲非同步 I/O 使用的 user buffers 必須活到 completion；若改用內核 copy，成本和生命週期改變。取消只要求停止尚可停止的工作；API 說取消失敗/已完成時，不能立刻釋放仍可能被裝置使用的記憶體。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel I/O：排程、buffer、cache、spool 各解什麼問題

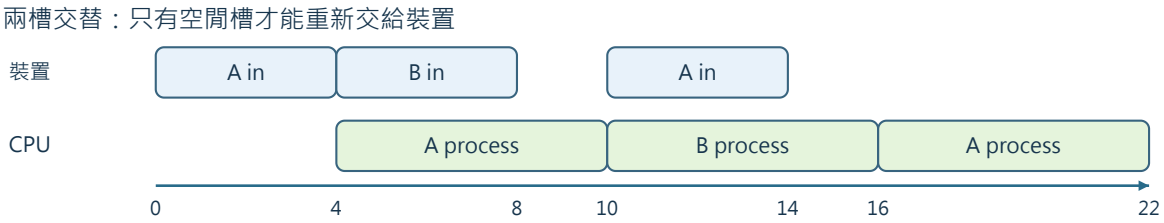
scheduling 決定待服務 requests 的次序；buffering 吸收速率/單位不匹配並保存 copy semantics；caching 保存可重用資料；spooling 將完整工作排入較獨占的裝置 queue。單一記憶體區可兼具 buffer/cache 角色，但目的與正確性條件不同。

三個 read 依序到達 block 40、10、41；若排程改成 10→40→41，可減少某些 HDD 移動，但不得越過必要的依賴/barrier，也不能把這套 seek 模型當作 SSD 機械延遲。公平性、deadline 與吞吐可能衝突。

write-back cache 的 dirty copy 可能是最新資料唯一副本；「cache 一定只是別處已有相同資料」在 dirty 狀態不成立。buffered write 已接受、device write 已完成、durable 都不能混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Double buffering：明確時序與可用槽位



ms；每塊輸入 4 ms、CPU 6 ms。B 在 8–10 ms 等 CPU；A 在 14–16 ms 等 CPU。

0–4 填滿；4–16 重疊；16–22 排空。3 塊完成於 22 ms，並非 18 ms。

每塊裝置輸入 4 ms、CPU 處理 6 ms；單 buffer 每塊先輸入後處理，3 塊共 30 ms。雙 buffer：A 輸入 0–4、處理 4–10；B 輸入 4–8、等 8–10、處理 10–16；A 再輸入 10–14、等 14–16、處理 16–22，3 塊共 22 ms。穩態週期仍是 max(4,6)=6 ms。

4–8 ms 兩單元可重疊，但到 8 ms 不能覆寫仍由 CPU 使用的 A。對 N 塊、可完全重疊且兩槽足夠的本模型，總時長 $t_{in}+t_{cpu}+(N-1)\max(t_{in},t_{cpu})$。初始化/排空不能從有限 N 的公式中刪掉。

Copy semantics、spooling 與錯誤界線

copy semantics：應用寫入 buffer="ABC" 後立刻改成 "XYZ"，若 write 契約在返回前已複製/完成讀取，後續輸出仍應為 ABC；若零 copy 非同步直到完成才釋放，應用不能過早修改。必須寫出是哪一種契約。

spooler 將 job J17 的 3 頁寫入持久 queue 後開始印；printer 已印第 2 頁但 ack 遺失。主機重送整份可能出現重複頁，直接刪 job 可能少印。持久 job id 只能避免主機重複建工作，不能證明不可回復的實體列印 exactly-once；需要裝置本身可查進度/去重契約，否則向使用者回報「結果未知」。

短暫錯誤可重試，但非冪等操作不能盲重送。回傳值通常同時承載長度與失敗資訊；errno 只有在 API 以失敗返回時才有對應意義。controller 的細分 sense/error logs 可供診斷，不可用上一筆殘留 errno 推定本次成功操作失敗。

權限與 kernel data structures 的逐層責任

本例 P 的 fd=4 → per-process fd table → open-file object {offset=8192, readable=true, ops=regular_file_ops} → inode {logical block2→physical block70} → block device/driver queue。兩個 fd 可能共享 open-file offset；另一個獨立 open 不一定共享 offset。

syscall 先驗 fd/mode/range/user pointer；VFS dispatch 按物件型別呼叫 `read`。page cache hit 直接 copy；miss 才進 block/driver。對 socket，ops 指向 socket receive，不能套 inode block70 的流程。名稱解析與 I/O dispatch 是兩階段，不是每次 read 都重新查完整路徑。

I/O port/MMIO/control register 受特權與映射權限保護；user buffer 也須驗證/適當 pin。授權使用某裝置不能推出可 DMA 任意記憶體。資源保留可能造成死結：A 持 scanner 等 printer、B 持 printer 等 scanner；需要一致取得順序/一次申請/回復策略之一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從名稱到 controller：不跳過中間映射

固定教學檔案系統：mount `/data` 對應 device (8,2)，`/data/log.bin` 的 inode=37；open 建立 fd=4。read 從 offset=8192 取 4096 B，block size=4096，故 logical block=⌊8192/4096⌋=2、offset-in-block=0；inode 將 logical2 對應 device block70。若裝置 sector=512 B，請求是 LBA=70×8=560、8 sectors。

device (major8,minor2) 選 driver/partition，假設 partition 起始 LBA=2048，實際整碟 LBA=2608。controller queue 的 descriptor 再放入 DMA mapping，不把 fd、inode number 或 user VA 當 LBA。mount/partition/VFS 細節依 OS；以上是完整固定模型，不聲稱所有 UNIX 逐字如此實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一筆讀取的位址換算

同一筆讀取：從檔案位置到整碟扇區

區塊 4,096 B · 扇區 512 B · 分割區起點 LBA 2,048

1	開啟檔案	fd = 4；讀取 offset = 8,192 B
2	檔案系統	inode 37；logical block = 8,192 ÷ 4,096 = 2
3	區塊映射	logical block 2 → device block 70
4	分割區扇區	70 × 8 sectors/block = LBA 560
5	整碟扇區	2,048 + 560 = LBA 2,608（起始扇區）

讀取長度 4,096 B → 8 個扇區；整碟 LBA 2,608–2,615。

完成、逾時與取消競爭：誰能釋放 request

狀態為 NEW→QUEUED→IN_FLIGHT→DONE 或 ABORTED；每筆 request 至多發布一次 terminal 結果；若裝置永不回應，未必能發布。IN_FLIGHT 超時先標 timeout 並要求 abort，不可直接視為 ABORTED。只有正常 completion 或具明確停止 DMA 保證的 abort ack／hardware quiescence 到達，並完成平台要求的同步後，才解除 DMA mapping、釋放 pages。

例 id8：t=0 提交；t=100 timeout；t=101 發 abort；t=102 正常 completion 抵達；t=103 abort 回 already-completed。lock/CAS 保護 terminal 狀態，t=102 贏得唯一完成，t=103 只清理 abort bookkeeping，不再第二次喚醒或解除buffer映射。timeout 可以是提前的狀態通知，而非 terminal completion。若 DMA 直接使用 user buffer，API 必須要求呼叫者在最終 completion／quiescence 前保留並且不存取或重用 buffer；只 pin pages 並不足以阻止重用。另一種設計是讓 DMA 使用 kernel-owned bounce buffer，使 user buffer 不再受後續 DMA 存取。錯誤通知、terminal completion 與資源回收應分別定義。

一般推理： 只有能把 state 從非 terminal 原子改為 terminal 的分支取得發布資格；解除 buffer 映射還須先確定硬體已停止存取並完成必要同步。所有其他回報看到 terminal 就不再重做。這證明 at-most-once completion，沒有證明外部副作用 exactly-once，也沒有保證壞裝置最終一定回應。

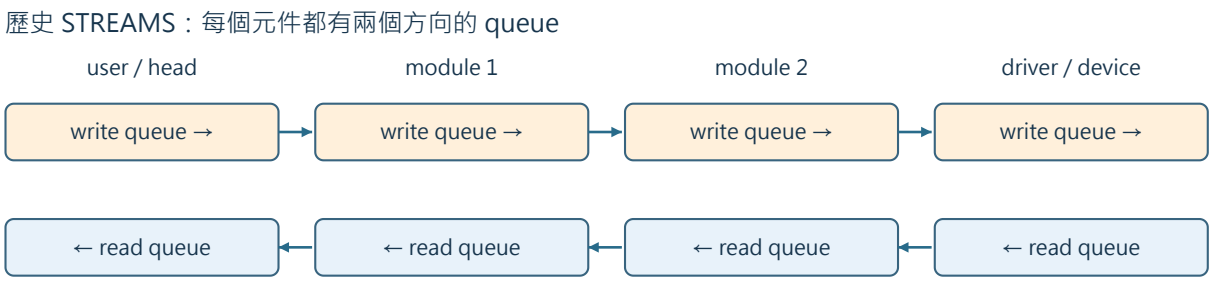
Buffer 與 request control block 的回收時點不同

裝置已停止DMA時，可依完成路徑同步並解除buffer映射；但timer、IRQ、abort callback仍可能存取request的state。故terminal winner不能直接free control block。每個已排入/尚未退出的callback先持有一個穩定reference，退出時release；取消timer若無法證明其callback已退出，也不能提早丟該reference。

延續id8，t=101時有owner、IRQ與abort三個引用，refcount=3。t=102 IRQ取得terminal資格、清DMA buffer後退出，refcount=2；owner交付結果並退出後refcount=1；t=103 abort callback讀到DONE、清自己的bookkeeping後退出，refcount=0，這時才free control block。若另有timer在途，還要再等該引用退出。所有取得reference的入口必須受同一生命期保護；不能先讀裸pointer再對已free物件增ref。

這個reference-count不變量保證「只要任何callback仍可能讀控制狀態，就有正引用維持物件存活」；配合terminal檢查，才能同時避免double completion與use-after-free。若使用request slot/id重用，另需generation/stable event record避免晚到事件誤認成下一筆request。

STREAMS：雙向 queues，不是單列函式呼叫



固定歷史範圍：UNIX System V/Solaris 傳統 STREAMS。stream head 接 user；driver end 接 device；中間可由 ioctl push/pop 模組，每個元件都有 read queue 與 write queue。read data 向 head 上行、write data 向 driver 下行。訊息可含 data/control，不能把 module 一律當成重新 copy payload 的函式。

put procedure 接到訊息後可立即處理/往下送，也可排入自己的 queue；service procedure 在可運行時取 queue。read/write 是 byte-stream 介面；getmsg/putmsg 可保留訊息控制/資料部分。head 的 user syscall 可 blocking；模組之間的訊息排隊不等於 user thread 逐層同步等待。

背壓的真數值：容量不會因畫箭頭而增加

教學 write queue Q 容量 1000 B，高水位 800、低水位 300，訊息每筆 200 B。初始 Q=600；接受一筆後 Q=800，設 FULL，上游停止再送。driver 每次取 200：Q 依序 600、400、200；降到低水位以下後 back-enable 上游，才容許排隊服務繼續。這是滯後控制，避免每跨單一門檻就反覆開關。

若上游另存兩筆，停住時只是移到上游 queue，系統總緩衝不是消失；上游也滿才回頭阻塞 head 的 writer。入口若是無法阻塞的網路硬體，buffer 全滿可能 drop，不能在 IRQ 中無限睡等。高優先控制訊息與實際 STREAMS byte accounting 有其規則；本數例只模型化普通固定 200 B data messages，不假稱是所有實作的完整 queue 規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

吞吐瓶頸：CPU、bus 與裝置分別有上限

固定每 request=4 KiB；裝置最多 100000 requests/s；CPU 每筆 5 µs，單核心上限 200000/s；記憶體可供本工作負載 1 GiB/s，每筆總 traffic=12 KiB（DMA 4 KiB，加一次 memory copy 的讀4+寫4），故上限 1 GiB/12 KiB=87381.33/s。整體上限 ≤min(100000,200000,87381.33)=87381.33/s；這是資源界，不保證實際達到。

若真正省掉該 copy，traffic 降到4 KiB，上限升至262144/s，裝置100000/s轉成瓶頸。若「零 copy」仍有其他 memory traffic，須另計。加核心不會突破本例記憶體上限。

batch/coalescing 減少 IRQ 次數，但可能增加排隊延遲。例如 8 筆合併、每筆到達間隔10 µs，首筆需額外等70 µs、末筆0；均勻一批平均35 µs。每IRQ1 µs從每筆1 µs降為每筆0.125 µs，是以延遲換CPU成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延遲、queue depth 與 Little 定律的適用條件

對穩態、有限平均、相同邊界的觀察，L=λW。若完成率80000/s、從進入系統到完成平均W=250 µs，平均在系統 request 數 L=20。W 必須含同一邊界內的 queue+service；不能用平均 queue 長度配總 response time。若裝置 service=100 µs，平均 in-service=8，平均 queue=12，queue wait=150 µs。

最小延遲100 µs與100000/s上限意味維持該吞吐至少需要平均10筆在途；不是queue depth一設10便必定達標。飽和後增加queue常把W拉長，不能無限增加λ。若到達率110000/s而最大完成100000/s、無丟棄/節流，queue持續成長，不能把有限穩態L硬代入。

本頁 Little 關係是本章延伸的流量守恆模型；它既不是 exponential service 假設，也不是 M/M/1 的 W=1/(µ−λ)。沒有額外分布假設不得套後者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨層推導：同一筆讀取的資料與控制

fd4、offset8192→inode37的logical block2→partition LBA560→整碟LBA2608→DMA4096 B。thread blocked，但 request in-flight、device 持 buffer。completion 先確立資料CPU可見、發布狀態，再使thread ready，最後由scheduler執行返回。

cache hit 可避開最後幾層；nonblocking 可在尚未取得資料時返回 EAGAIN；async 可先返回提交結果。這些變更不會自動消除短讀、錯誤、權限或buffer lifetime。

要判定實作是否正確，沿著「命名/授權→排隊→交付裝置→完成發布→資料使用→資源回收」逐條核對前後條件，而不是由一個成功 callback 推論所有外部效果已持久化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

代價與可觀察保證

PIO/DMA 是搬運方式，poll/IRQ 是偵測方式，blocking/async 是呼叫契約；它們可以組合，不能互相作同義詞。buffer/cache/spool 解不同層的問題；複本可能提高效能，也可能讓「返回」早於「耐久」。

數值模型需同時交代 bytes/request、CPU time/request、bus bytes/request、service time 與觀測邊界。只有同一分母的成本才能比較；有限批次保留填滿與排空時間，穩態公式明列穩定條件。

本章練習分別驗證握手、遺失喚醒、short I/O、timer、double buffering、取消資源壽命、STREAMS 背壓與瓶頸。詳解保留中間狀態，不把測試少數 trace 當作一般正確性證明。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 13.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：如何用可強制執行的資料結構，限制主體對物件的能力，並在撤銷後維持可證明的一致性？

前置知識。 mode bit、access control 基礎、graph 與 matrix。

- 區分 subject/object/domain/right
- 比較 ACL 與 capability
- 分析 revocation 對已開 handle 的影響

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

保護的問題是「誰可對哪個物件做什麼」

主體（subject）是發出請求的執行者，例如行程；**保護域（domain）**是執行者當下可行使的物件與權限集合。object 是資源，right 是可執行的操作。同一主體經授權切換 domain 後，能行使的權限可以改變；不能把 subject 與 domain 當成同一個概念。

政策例：「助教可讀作業，只有任課教師可改成績」；ACL、capability 與受保護的系統呼叫是實施政策的機制。完整仲介（complete mediation）要求每次使用都受到有效授權約束。若在 open 時檢查 ACL、往後以受保護 handle 的權限檢查 read/write，也必須明定 ACL 變動後既有 handle 的撤銷語義。

reference monitor 必須防止繞過、保護自身狀態，並足夠小而可檢查。硬體 mode 與位址空間隔離是部分支撐，不能單獨證明應用政策正確。安全（security）還涉及認證、攻擊、故障、稽核與信任；難猜的路徑不是存取控制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

基底與界限：政策如何交給硬體執行

機制：比較 base ≤ address < base + limit；不符合時觸發 fault。

政策：作業系統決定行程可存取的範圍，並安全設定 base 和 limit。

本例以 byte 為單位，檢查實體位址是否落在 `[base, base + limit)`；假設加法不溢位。作業系統決定行程可使用的範圍並保護設定；硬體比較位址並在越界時觸發 fault。實際硬體也可能先檢查相對位址再做重定位，不能把本圖當成所有架構的實作。

最小權限同時限制範圍與時間

備份工作只需讀 `/course/submissions`、寫備份目標，不應同時有修改成績或載入核心模組的權限。若整個程式一直用全權帳號，解析一個惡意檔案的錯誤就可能取得所有權限。拆成低權解析器與小型授權服務，服務仍須驗參數與物件範圍。

權限集合 R(t) 要包含當下必要權限 N(t)，並盡量縮小 R(t)−N(t)。這不是單純「權限數量越少越好」：只有 read 但可讀所有學生個資，仍可能比可寫單一暫存檔風險大。細粒度 scope、持續時間與可委派性都影響損害界。

fail-safe defaults：未找到明確授權就拒絕。稽核記錄 subject、object、operation、decision、時間與關聯 id；有 log 不代表攻擊被阻止。separation of duty 可要求兩角色批准成績發布，但若同一使用者能自行同時啟用兩角色，政策仍可能失效。

Domain 是權限集合，不等於 CPU mode

D={⟨F1,{read}⟩,⟨F2,{read,write}⟩}。同一 user 的兩個 process 可以有不同 domain；同一 process 的受保護呼叫也可能切 domain。user/kernel 兩個 mode 無法單獨分隔所有同為 user mode 的行程，還要有位址空間與個別授權。

靜態 domain 把各階段需求聯集：compile 需讀source、link需寫binary，若全程持兩權就擴大每階段暴露。動態 domain 可在狀態切換時改權限，但切換本身也必須授權、保存/還原正確 caller context，並檢查參數。

傳統 UNIX setuid 執行檔可在成功 exec 的指定條件下改 effective UID 為檔案 owner；不是把 real UID 一律改掉，也不是單純呼叫一般函式就改 UID。例 realUID=1001、owner=0：執行受允許的 setuid 程式後 euid 可為0，realUID仍1001；程式需正確降權、驗證環境與輸入。具體抑制條件依 OS，這裡不把歷史機制當成任意提權方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MULTICS rings：受控進入，不是內圈任意入口

書中歷史抽象模型：ring0較高權；access bracket (b1,b2)，gate limit b3≥b2；caller ring i 在 [b1,b2] 可直接執行。i>b2 時，只有 i≤b3 且入口是指定 gate 才可轉入較高權；i<b1 轉往較低權時，參數不得讓低權程式任意引用其不可見區域。

例 bracket=[1,3]、limit=5、gate={entryA}：i=2 呼叫一般入口可保持ring2；i=4 呼叫entryA可受控進入ring3，呼叫entryB拒絕；i=6即使entryA也拒絕。返回要恢復 caller ring/stack。此例限於書中 ring model；完整 MULTICS 還有 read/write brackets 等硬體細節。

純巢狀權限無法表達互不包含的需求：D甲只需F、D乙只需G，若強制D甲⊆D乙或反之，至少一方會多得不需的物件。一般 access matrix 能表達這種非階層分隔。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一 access matrix 的初值

授權矩陣與 domain-switch 可達性

domain	F	G	P	D2	D3
D1	r*	ø	ø	switch	ø
D2	w	r	ø	ø	switch
D3	ø	ø	print	ø	ø

switch 可達：D1 → D2 → D3

目前 row 不會自動聯集

r* = read + 同物件授權傳播；ø = 沒有權利。右圖只表示 switch，不代表直接入口任意可用。

令 A[D,O] 是操作集合；空格就是∅，不表示「尚未決定」。目前process只使用當前domain那一row。D1可讀F，不能寫F；D2可寫F且可切D3；D3可印P。D1能切D2再切D3，代表可能取得後續domain的行為能力，不能只看D1原始row就斷言它永遠無法印P。

domain-switch 本身當作對 domain object 的 switch 操作。切換後不是把兩row聯集；若需要受控服務，還應限制入口與參數，避免任意直接進入高權程式中段。

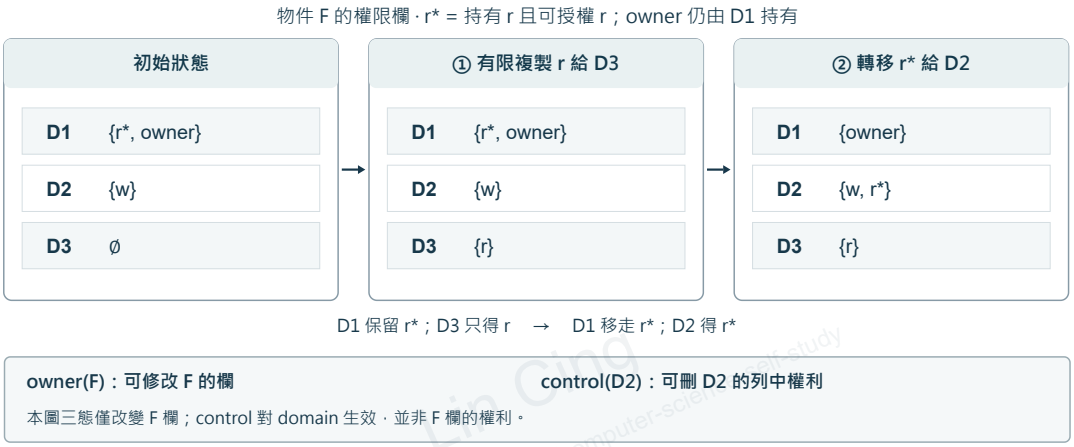
copy、transfer、limited copy、owner、control

模型中的 r* 表示持有 r，且可在**同一 object column** 傳播 r；不能用 F 的 r* 給 G 加 r。普通 copy 保留來源；transfer 將本例的 r* 從來源移到目標；limited copy 僅給 r，目標不獲傳播權。

獨立初態。 此例不是前一張矩陣的延續：A[D1,F]={r*, owner}、A[D2,F]={w}、A[D3,F]=∅。D1 對 D3 執行 limited copy 後，D3={r}，不能靠這項權利再轉授。D1 作為 F 的 owner，可在 F 的 column 中增加或刪除指定權利，例如刪除 D2 的 w；owner(F) 不代表擁有所有物件。

control 是對 domain object 的權利。若 control∈A[D1,D2]，本模型允許 D1 從 D2 的 row 移除權利，不允許任意新增。刪除 D2 對 D3 的 switch，會中斷原來 D1→D2→D3 的路徑；其他路徑是否存在須另行檢查。

用同一欄追蹤授權的複製與轉移



此例使用獨立初始狀態：D1 持有 F 的 `r*` 與 `owner`，D2 持有 `w`。有限複製只給 D3 一般 `r`；轉移再把 D1 的 `r*` 移給 D2。`owner(F)` 管 F 欄；另有 `control(D2)` 時，才可依模型刪除 D2 列的權利。

保護權利不等於禁止資訊流

即使D3只有read(F)且沒有copy(F)權利，只要它能read資料並write(G)，就可把內容逐byte寫到G。copy right限制的是授權的傳播，不是資訊的複製。已讀進user記憶體的資料通常也不能透過撤銷F的read「收回」。

可用不變量證明受限系統安全：若唯一授權操作是同column、只能授出來源已有權利的attenuation，且沒有owner/建立新權利規則，則任一後繼狀態的權利都源自初始某個domain對同object持有的權利。歸納：初始成立；每一步新增r必有來源r，沿來源鏈回溯到初始。此結論不保證機密資訊不外流，也不能套到允許owner任意造新權利的模型。

稀疏矩陣的四種實作

表示	索引方向	回答什麼最快	更新/保護責任
global triples (D,O,R)	全域或雙索引	任一pair	保護全域表，避免每次線性掃全表
ACL(O)=[(D,R)]	object column	誰可用O	可集中刪O的授權；須定義matching/default規則
capability list(D)	domain row	D目前持有哪些能力	cap不可偽造；傳遞/attenuation由機制控制
lock/key	object locks與domain keys	是否有matching key	key/lock清單本身受保護，換key可撤銷

capability 是受保護的物件參考＋權利；數字fd是本行程表的索引，不能靠猜別人的fd就取得其capability。不可偽造可由kernel table、tagged hardware或適當的受保護表示達成；不是「所有capability都必須永遠不能出現在user位址空間」。

同一權限的 row／column 轉換與檢查

F的ACL=[D1:r,D2:rw]，G的ACL=[D2:r,D3:w]。轉成cap lists：D1=[F:r]；D2=[F:rw,G:r]；D3=[G:w]。兩者在同一時點表示相同矩陣，但委派/撤銷的更新成本不同。若採hash索引查pair可期望O(1)，線性list查長k則O(k)，不能把表示名稱直接當作固定O(1)。

hybrid：open時檢ACL，給kernel保存的handle{F,read};後續write查mode拒絕。若此後ACL刪read，舊handle是否仍可read依撤銷政策；「改ACL必立刻撤所有已開啟fd」並非UNIX的一般保證。新增每次generation檢查可改此語義，但需處理mapping與in-flight操作。

教學ACL採只有allow且union/default deny；真系統若有deny ACE/順序/群組規則，需要另一套明確求值演算法，不能把此簡化算法套到Windows DACL。

RBAC：user→role→權限與任務的映射

歷史案例固定為 Solaris 10。其RBAC把roles、rights profiles、authorizations與process privileges分開：role聚合管理任務，profile可指定命令與執行屬性，authorization可由特定應用檢查，privilege由kernel對操作檢查。不是所有角色權利都只是某個UID，也不是authorization字串本身直接替代kernel privilege。

原創模型：user U可assume BackupRole；該role可執行BackupCmd，執行時只帶所需讀取/目標寫入權；不能改GradeDB。user V可assume GradeRole，但不能替自己核准發布。啟用角色的認證與稽核要另做，單看user-to-role配置不能推出所有請求均允許。

形式化簡化RBAC：UA⊆Users×Roles、PA⊆Roles×Permissions、session activeRoles⊆assignedRoles(U)，當且僅當某active role含所需permission才允許。若有互斥roles，啟用時再驗constraints。此式是教學RBAC模型，不等同Solaris全部privilege集合運算。

撤銷有四個獨立維度

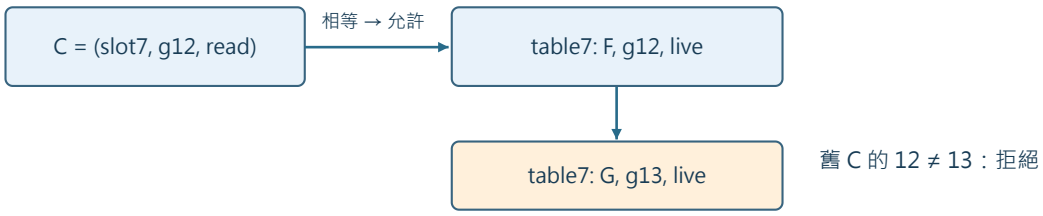
immediate/delayed 表示何時生效；selective/general 表示影響哪些持有人；partial/total 表示撤銷哪些操作；temporary/permanent 表示未來是否能再授權。「即刻」仍須定義邊界：阻止之後的新操作，不代表已開始的 I/O 自動取消，也不能收回已讀出的資料。

ACL 可集中刪除 entry，但已核發的 handle 或快取可能延後效果。分散的 capability 可採重新取得（reacquisition）／租期、back-pointers、indirection 或 key/version；back-pointers 要追蹤全部複本，indirection 則把查驗收斂到可撤銷的 slot。更換共用 master key 會撤銷使用該 key 的所有能力；選擇性撤銷需要分組 key 或代理。

租期範例。 使用一致且可信的時間基準；授權從 t=0 起有效至 t=10 秒，且每次新操作都先檢查未過期。t=3 撤銷後，授權者拒絕續租；即使撤銷通知未立即送達，t≥10 的新操作也因租期到期而拒絕，最長延遲為 7 秒。若允許無限快取、過期後使用或撤銷後續租，便不能聲稱這個延遲上限。已在途操作是否完成，由另外的取消契約決定。

Indirection 撤銷：generation 避免 ABA

同一 slot 重用：不能只比較索引 7



撤銷 / 重用時換 generation；resolve 的檢查與取得物件 reference 必須同步。

cap C={slot=7,generation=12,rights=r}；table[7]={obj=F,g=12,live=true}。每次resolve先驗live與generation相等，再查r，並在同一同步保護下取得obj存活reference。撤銷設live=false；重用slot存G時generation升13。舊C仍是(7,12)，對(7,13)拒絕，不能錯指G。

一般證明條件： 不重用仍可能存在的generation，且check→取得穩定reference不可被撤銷/重配插入。則接受C必對應它核發時的物件版本。若generation可wrap且舊C永存，終會ABA；必須不重用或先確保舊tokens全消失。若只驗slot數字，F撤銷後G占slot7就會讓舊cap取得G，是確定的權限漏洞。

Hydra：抽象操作與受限 amplification

歷史Hydra讓user-defined type有auxiliary rights。原創例 Account物件A的client cap只含deposit，不含代表資料的raw read/write。呼叫受信任deposit(A,10)時，機制確認型別與操作權，受保護procedure暫時獲得實作A所需的representation access，更新balance=40→50；返回後client仍只有deposit。

amplification不是「caller自選要升權」；只由已認可的procedure、指定型別與受限規則觸發。若任意routine都能把readonly參數升成write，就破壞caller的保護需求。因此互不信任subsystems需要控制可放大的權利、可保存的參考及參數範圍；不能只說封裝就自動安全。

Cambridge CAP：資料能力與軟體能力

data capability由machine/microcode強制read/write/execute段權限；software capability的自定義意義由受保護procedure解釋，底層保證不能任意偽造/篡改。seal/unseal式權限讓只有對應型別與環境的procedure能檢視受保護表示。

與Hydra共同點：client只拿抽象操作能力，implementation可能需要更低層存取；邊界是受保護的typed procedure。差別是權利解釋與支援庫的設計，不是「CAP能執行任何user程式就一定安全」。若Bank子系統的procedure錯把balance寫壞，Bank語義仍壞；隔離只把損害限制在其保護環境，不能證明Bank業務不變量。

本節按原書歷史架構比較，Account數值為原創抽象模型，不冒充Hydra/CAP可直接執行的原始API。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

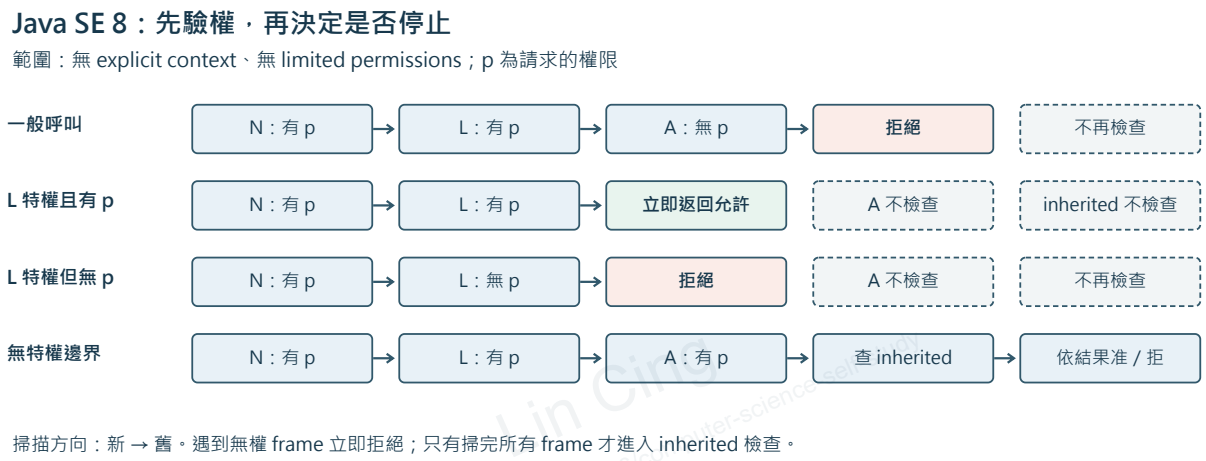
編譯期、load-time、run-time 各能保證什麼

language把存取宣告放在type/module/interface邊界；compiler可靜態排除非法操作，runtime檢查動態index/type/capability。若執行者可改機器碼、偽造reference或透過不受管native入口繞過檢查，單靠原始碼private欄位不足以隔離敵對程式。

type safety保證類型/記憶體規則，不代表業務授權。能合法取得Account reference的程式，仍可能錯誤地允許任意提款；type-correct也可能資訊外洩。kernel隔離粗到process，language可細到同process內的class/object，但其runtime、loader、native interface進入trusted computing base。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Java SE 8 的 stack inspection：privileged 也先驗權



限 Java SE 8 傳統模型，doPrivileged 不帶 explicit context 或 limited permissions。由新到舊 N→L→A，p=connect(proxy.example:80)；N、L 有 p，A 沒有。普通檢查在 A 拒絕。若 L 使用 doPrivileged，N、L 均通過後立即允許，A 與 inherited context 不再檢查；若 L 無 p，仍在 L 拒絕。

只有沒有命中提前返回的 privileged 邊界，且所有 frame 均通過時，才檢查 thread 的 inherited context。帶其他參數的版本另有規則；本例不是現代 Java 沙箱建議。

Confused deputy：在高權框架驗證實際目標

Loader為了讓低權Applet讀取代理服務，可在受控privileged區塊連到固定proxy。若Loader把Applet給的任意addr原封不動放入doPrivileged，會讓低權caller借Loader的網路權掃內網。正確例先把目的地固定在已驗證的proxy、限制操作與回傳資料，不把整個user callback都納入高權區塊。

stack inspection的歸納條件：每個已檢查的frame都具有p；遇到privileged邊界時，其本身也具有p才停止。由此可保證「本次授權不超過邊界caller已有的p」，但仍須信任邊界caller選擇了正確object/參數，不能據此證明整體服務政策正確。

從矩陣到實作的等價條件

access matrix只指定當前允許關係；ACL/capabilities是不同索引，dynamic operations再改關係。實作要保存object identity、right的型別與domain context，並確保不可偽造、不可繞過。若查驗結果被cache，必須把撤銷語義一起定義。

owner操作column，control操作domain row的刪除；switch改目前domain；copy/transfer/limited-copy依規則改同object授權。這些操作的組合會改未來可達domain，所以單看靜態row不能推出整段程式的權限上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自學時可檢驗的三類結論

①授權安全：每次操作是否有相應權利；②狀態安全：撤銷、重用、競爭後是否錯指物件；③資訊/業務安全：即使所有權限檢查都通過，是否仍可藉合法read/write外洩或濫用高權服務。三者需要不同證據。

一般證明必須先限定允許的狀態轉移。cap attenuation的歸納不適用於有任意owner加權的系統；generation證明不適用於無界舊token搭配有限wrap；Java privileged邊界也不免除參數驗證。練習以完整矩陣、slot版本與call stack把這些界線具體化。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 14. Oracle: Java SE 8 AccessController. Oracle: Solaris 10 RBAC.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：在身分、程式與網路都可能被攻擊的環境中，哪些 cryptographic 與 system mechanisms 能提供可驗證的安全保證？

前置知識。 protection、網路基本概念、機率與雜湊概念。

- 區分 confidentiality、integrity、availability、authentication
- 分析對稱/非對稱加密與 hash 用途
- 判斷防禦措施在攻擊模型下的實際覆蓋

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先定資產、對手能力與安全目標

confidentiality限制未授權讀取；integrity限制未授權變更；availability要求授權服務能在規定條件下運作。威脅是可能破壞目標的條件，vulnerability是可被利用的弱點，attack是實際嘗試。三個目標可能同時受損：竊取後再加密刪除資料不只是一種分類。

原創情境：成績服務保護成績、帳密與可用性。對手可控制網路、提交任意表單，但不能直接改核心或取得伺服器私鑰。若後來假設私鑰已被偷，原來的網路認證推論就不能沿用。安全結論必須連同對手能力、信任根、金鑰與端點完整性前提一起讀。

physical、human、OS、network都會影響此邊界。TLS保護線路，不保護已被鍵盤側錄程式取得的輸入；ACL限制未授權使用，不保證持有效帳號的惡意操作符合業務規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Masquerading、重放與中間人不是同一問題

Masquerading 是冒用身分；replay 是重送曾有效的訊息；中間人（MITM）控制兩端互相觀察的內容。來源 IP 不是密碼學身分；收到公鑰也不表示它確屬目標服務。

例：有效訊息 `transfer(A,B,100,id17)` 帶正確 MAC，對手即使看不懂仍可原封重送。在 MAC 達到所需的不可偽造性、金鑰未洩漏且持鑰者符合信任假設時，驗證成功提供「內容由某持鑰者產生」的計算性證據，並非新授權的證明。將 session、序號、nonce 或操作 ID 納入受驗內容，並讓接收方保存使用狀態；若重試要回傳舊結果，去重紀錄須持久化並與交易原子連結。

「不能偽造新操作」與「同一授權不被重複執行」是不同目標；加密本身不能同時保證兩者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

程式威脅：執行權限與傳播方式分開

Trojan horse以看似正當功能濫用被執行時的權限；backdoor/trap door繞過正常入口；logic bomb由特定條件啟動。virus需依附並感染host檔/程式，worm可透過服務/網路自我傳播；分類可重疊，不表示病毒只可能出現在某一家OS。

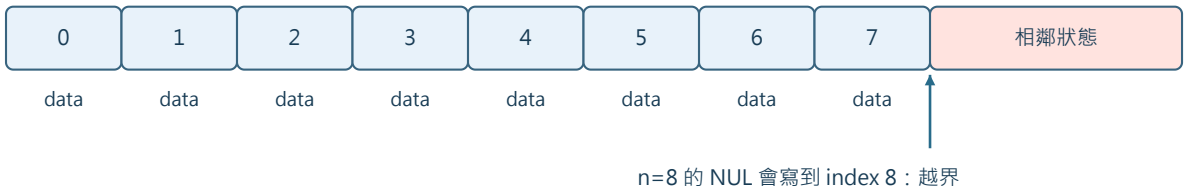
PATH含不可信工作目錄時，同名工具可能取代預期程式；受害者執行後攻擊程式取得的是受害程式實際持有的權限，不會因稱作Trojan就自動取得root。編譯器本身若被植入惡意行為，僅檢查應用source不足，因為產物不一定對應source。

保護邊界包含可執行檔、loader、compiler與更新來源。檔案/boot/macro/source virus描述感染位置；polymorphic/encrypted/stealth/armored描述規避分析方法；multipartite可感染多位置。辨識byte特徵與判定執行行為互補，不能據未命中特徵斷言無感染。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Buffer overflow：先證明界限，再談控制流

抽象 byte 配置：字串終止字元也占一格



容量 8 → 完整 C 字串最多 7 data bytes + 1 NUL；binary buffer 改用顯式長度。

固定byte buffer容量8，合法indices0–7。C字串若資料長n，還需一個NUL，因此完整複製的前提n+1≤8。n=8時資料恰占0–7，NUL落index8已越界；不是「小於等於8字元都安全」。實際stack布局由ABI/compiler決定，圖只示範相鄰狀態可能被破壞，不提供特定平台位址。

安全copy有兩種不同契約：reject過長，或明確截斷。若截斷，取k=min(n,7)，copy前k bytes，再寫buf[k]=0；只做`strncpy(...,7)`未必自動補尾端NUL。若是binary資料，不應用strlen判長，而要攜帶顯式長度並檢查範圍與整數溢位。

多層防禦的保證各不相同

bounds/type/lifetime檢查針對非法記憶體操作的成因；stack canary嘗試在返回前偵測部分覆寫；NX/W⊕X限制資料頁執行；ASLR使地址較難預測；control-flow約束限制可接受目標。NX不阻止越界寫，也不能單獨排除重用既有可執行程式片段的攻擊。

即使沒有改return address，覆寫相鄰的`is_admin`、length或object pointer也可能破壞授權。故「攻擊沒有執行stack上的code」不等於安全。最小權限再限制已受害component可造成的損害，仍不能替代修正越界來源。

受管理語言的safe subset可消除某些記憶體錯誤類型，但unsafe/native code、runtime錯誤及業務授權仍是邊界；不得把「type-safe」解讀成任何安全目標都已證明。

Worm 傳播：局部漏洞與全網負載的連鎖

1988 年的 Morris worm 利用服務弱點與遠端信任尋找可達主機；同機器重複感染也消耗資源。安全事件不以竊取資料為必要條件。

在離散模型中，每輪每個感染主機**最多**感染兩個不同的尚未感染主機，且無清除，故 $I_{t+1} \le 3I_t$。若每輪恰好各感染兩台、對象互不重複且易感主機足夠，從 $I_0=1$ 才得到 1、3、9、27。封閉 100 台網路滿足 $I_{t+1} \le \min(100,3I_t)$；這是理想上界，不是 Morris 的實測數據。

Port scan 只能觀察可達服務；開放埠不等於存在漏洞，banner 也未必可信。授權檢查、修補與停用不必要服務可縮小暴露範圍，但關閉一個入口不能修好其他服務的輸入錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DoS：資源守恆與防禦副作用

教學服務有 1000 個 pending slots，每筆未完成握手占用 10 秒，無提前清除。假設系統初始為空，從 t=0 到 t=9.99 秒每隔 0.01 秒接受一筆，首次到期前已接受 1000 筆，恰在 t=9.99 秒占滿。這是指定到達時序的有限案例，不代表平均到達率相同的所有流量都會同時占滿。

穩態 Little 定律 $L=\lambda W$ 的 λ 是同一觀測邊界內**實際接受率**；當有限容量導致拒絕時，它不必等於外部 offered arrival rate。若穩態平均接受率為 100/s，平均占用時間為 10 s，才可得到平均占用 $L=1000$；平均滿額也不保證所有後續請求都能接受。

合法負載 80/s 加攻擊 40/s，高於 100/s 服務上限，至少 20/s 無法在穩態消化。Cookie、限額與延後配置改變狀態成本，不能創造無限上游頻寬；按 IP 限流也可能誤傷共用 NAT 的合法使用者。密碼錯誤鎖帳可抑制猜測，也可能被利用來拒絕他人服務。DDoS 與合法流量暴增的因應須考慮誤封代價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

密碼學工具：目標、key 與可驗證性

工具	保密	完整性/來源	誰能產生與驗證
對稱加密	適當scheme下提供	單純加密不保證	持同secret key者
public-key encryption	適當scheme下提供	不直接證sender	任何人可encrypt，private key可decrypt
hash	不提供	只偵測與可信digest不符	任何人可計算
MAC	不提供	持key者來源/完整性	持同key者可產生及驗證
digital signature	不提供	private signer來源/完整性	private產生、public驗證

正確性是$D_k(E_k(m))=m$；安全性是特定對手模型下難以區分/偽造等計算性保證，並非「不知道key在邏輯上不可能猜中m」。演算法公開，key保密；公開key不需保密，但其身分綁定需要完整性/認證。

Symmetric cipher：nonce 重用會洩漏什麼

stream模型$c=m\oplus s$、$m=c\oplus s$。例m_1=0x41、m_2=0x42、s=0xA6，得到c_1=0xE7、c_2=0xE4；同keystream重用時$c_1\oplus c_2=0x03=m_1\oplus m_2$。已知$m_1$即可求s=0xA6，再解$m_2$。這是代數恆等式，不需破解底層cipher。

block cipher處理固定長度block，長訊息還需正確mode與nonce/IV規則。AES block128 bits、keys128/192/256 bits是不同參數，不能互換。DES/3DES/RC4/WEP為原書歷史脈絡；本章不提供把它們部署為新系統的建議。

保密與驗證應由完整受分析的scheme提供；自製「加密後再附公開hash」仍不能阻止對手改ciphertext並重算未受保護hash。

RSA 原創小數例：完整模冪

只示範數論，非安全加密：p=5、q=11、n=55、φ(n)=40，e=3，d=27，ed=81=1+2φ(n)。public=(55,3)，private=(55,27)。m=12：12^2 mod55=34；12^3 mod55=34×12 mod55=23，所以c=23。

解密平方表：23^1→23，23^2→34，23^4→1，23^8→1，23^{16}→1；27=16+8+2+1，所以23^{27} mod55=1×1×34×23 mod55=12。不能只說「用私鑰解回」而省掉mod值。

raw RSA確定性且具代數結構，不是可直接部署的encryption/signature scheme；實際編碼/padding/金鑰長度與用途分離屬協定規格。本例小質數沒有任何保密強度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA 正確性：包含不互質的明文

對任意m∈{0,…,n−1}，因ed=1+k(p−1)(q−1)。模p：若p ∣ m，m^{ed}≡0≡m；若p∤m，Fermat給m^{p-1}≡1，因此m^{ed}=m$(m^{p-1})^{k(q-1)}$≡m。模q同理。p、q相異質數，CRT推出m^{ed}≡m modpq。

這證明decrypt(encrypt(m))的數學正確性，不證明RSA假設、抗chosen-ciphertext安全或實作無side channel。只對gcd(m,n)=1援Euler會漏掉可被p或q整除的合法m；上面分支補齊邊界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash、MAC 與 signature 的差別

n-bit digest只有2^n種輸出，長訊息空間更大，collision由鴿籠原理必存在。collision resistance指有效找任意$m \neq m'$且$H(m)=H(m')$很難；second-preimage指給定m再找m'很難，兩者不同。digest相等是計算性證據，不是$m=m'$的數學定理。

公開傳$(m,H(m))$，對手可改成$(m',H(m'))$，所以沒有來源認證。MAC使用secret key，驗證者也能產生MAC，因此不能單靠MAC向第三方證「只有另一方簽過」。signature由private key產生、public驗證；仍需可信public-key binding、完整訊息編碼與key保護，不能把簽章抽象成「任意加密反過來用」。

HMAC是帶key的標準構造，不等同隨意$H(key \mathbin{/\!/} m)$。認證內容應包括context、角色、session與序號，避免跨用途搬用一個有效tag。

Key distribution 與中間人

公鑰不必保密，但不能讓對手任意替換身分綁定

未驗綁定：A 用 K_M 加密 → M 能解；cipher 正確，對端身分仍錯。

可信 certificate chain + 名稱 / 用途 / 有效性 + 握手證明持 private key，才能建立相應身分。

A欲連B，若只從網路收「我是B，公鑰K」，M可改成K_M；A用K_M加密，M可解。加密演算法完全正確，身分卻錯。certificate把subject/service identity與key綁定，CA簽章需沿可信鏈驗證，另查名稱、用途、有效期與相關撤銷政策；不能只看certificate存在。

共享對稱key若每對users各一把，N人需$N(N-1)/2$對；$N=6$為15，新增第7人需另6把。KDC可減少每個user長期key數，但把信任集中到KDC。public-key機制降低secret配送需求，仍不消除public-key認證與private-key保護。

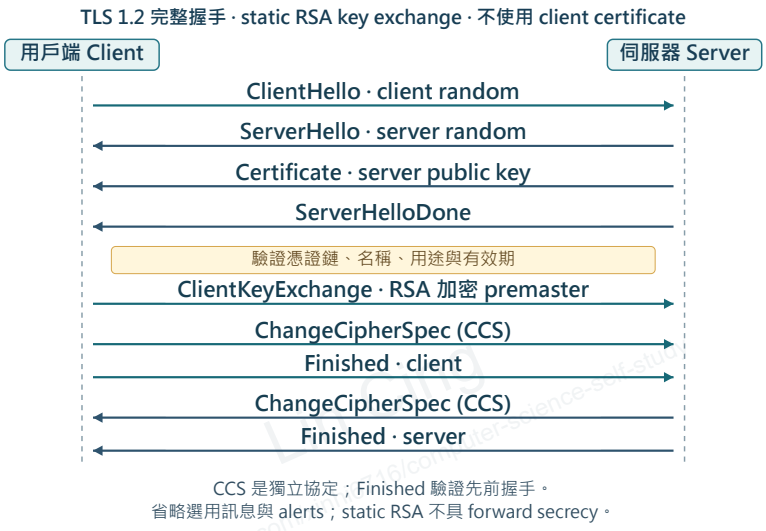
放在哪一層：endpoints 決定可見範圍

IPsec可保護兩個network endpoints間的IP流量；TLS保護其session endpoints間的資料；mail內容加密可跨越多個store-and-forward hops。若TLS在reverse proxy終止，proxy到backend的另一段不是自動被第一段TLS覆蓋；proxy可見解密內容。

link/network層保護不直接證明最終登入者的身分；應用認證仍需獨立契約。disk encryption保護離線媒體，不防已登入且有read權的程式讀明文。安全層數不能相加成無條件安全；每層要列誰持key、誰看明文、誰是TCB。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歷史 TLS 1.2 RSA handshake 的必要步驟



依 RFC 5246，固定歷史 TLS 1.2 完整握手、static RSA key exchange，非 TLS 1.3：client 傳 ClientHello(n_c)；server 依序傳 **ServerHello(n_s)**、獨立的 **Certificate** 與 **ServerHelloDone**。client 驗證鏈、名稱、用途與時間後，以 server 公鑰加密 48-byte premaster secret（2-byte 版本加 46-byte 隨機值），放入 **ClientKeyExchange**。雙方以 PRF 導出 master secret 和雙向 traffic keys。

歷史 TLS 1.2 RSA handshake 的必要步驟 （續）

client 的 ChangeCipherSpec 後送 Finished；server 驗證後亦送 ChangeCipherSpec 與 Finished，client 再驗證。圖中若省略 ChangeCipherSpec，並非表示協定不需要；不顯示的 client certificate 等視所選情況而定。Finished 驗證先前 handshake transcript；證書簽章本身不證明連線當下的對端持有私鑰。協定驗證依賴正確名稱綁定與端點假設，也不排除即時轉送。

Static RSA 交換不具 forward secrecy：若後來 server 私鑰洩漏且攻擊者錄有 ClientKeyExchange，先前 premaster 可能被解出，即使 traffic keys 已刪除。SSL 3 與 TLS 版本並非同義；RFC 7568 於 2015 年禁止 SSL 3。此處為歷史教學模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

認證與防重放：nonce、sequence 與原子提交

原創協定：server發一次性nonce n=42，pending[n]=(U,expiry)；client回 MAC_k("login" ∥ U ∥ n ∥ serverID)。server在同一原子操作中驗tag、驗pending/expiry，並把n改used。第一次可接受；第二次tag仍正確，但n已used所以拒絕。

推理： 正確MAC不能阻止複製舊tag；nonce狀態才能區分舊授權。只有驗證成功後才消耗nonce，且check與consume必須原子，否則兩個並行replay可同時看到pending。隨機nonce需要足夠熵及不重用；counter需跨重啟保存/換session。若是付款等操作，nonce consume與業務效果還要用交易或可恢復去重狀態連結，才不會crash後遺失/重做。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

User authentication 與 session 是兩個邊界

knowledge、possession、inherence分別是知道、持有與生物特徵。兩個password仍是同一類factor；MFA指兩類或以上，並非一定要三類。認證成功後產生的session token若被偷，後續請求可能繞過原先factor，因此session保護、失效與重新認證仍重要。

password verifier不應是明文。抽象記錄為{user,salt,cost,verifier=KDF(password,salt,cost)}；登入時用同salt/cost重算並比較。salt可公開，讓不同記錄不能共用同一個預算好的字典表；不阻止對特定記錄猜測，也不保證相同password在數學上永不hash collision。verifier外洩仍是offline guessing風險，存取控制仍需保護。

歷史UNIX八字元限制只適用特定舊scheme，不能當成所有UNIX規則；本章不把舊password政策當現代部署指引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

猜密碼的分母與成本

固定均勻隨機四位**十進位PIN**，包括0000，共10^4=10000種。無重複、無先驗偏差的猜測順序，平均排名(10000+1)/2=5000.5；每次1 ms則平均5.0005 s。這不是任意四字元password的空間，也不適用於人選偏態字典。

有1000筆不同salt記錄，字典100000候選：攻擊者不能只算100000個hash就與所有salt直接比較；逐record約需1000×100000次KDF。若cost讓每次5 ms，單工作單元總500000 s≈5.79天；可平行的資源、記憶體限制與已知偏態會改成本，不能把單核時間直接當實際攻擊時間。

online rate limit不會自動約束偷走verifier後的offline guesses；強迫鎖帳號也可能形成DoS。防禦需要分清攻擊在哪個邊界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OTP 與 biometrics 的誤差模型

OTP/challenge-response把新鮮性加入認證。一次性是server只接受一次的狀態規則，不是畫面顯示不同數字就成立；TOTP的時間窗需處理已用碼/重放範圍，且可被即時釣魚轉送，不能自動推得phishing resistance。

biometric以threshold判matching score，false accept rate (FAR)以impostor attempts為分母，false reject rate (FRR)以genuine attempts為分母。教學資料：100000次冒用，FAR0.1%意味100次誤接受；10000次本人，FRR2%意味200次誤拒絕。提高threshold通常改兩種錯誤取捨；數字依測試人口/裝置/環境而定。

生物特徵不是可隨意更換的secret；template保護、活體/攻擊偵測、遺失設備與fallback政策都影響安全。多factor若共用同一已受害端點/復原流程，失敗風險可能相關，不能把兩個失敗機率無條件相乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

政策、弱點評估與 defense in depth

政策指定資產、允許操作、責任與處置；assessment檢查configuration/程式/暴露服務是否符合，penetration test在授權模型下檢驗可利用路徑。通過一次scan只支持該時間、scope、工具與已知檢查範圍，不能證明無漏洞。

defense in depth要避免共同失效：兩層都依賴同一被偷key或同一個錯誤身分標頭，不代表獨立兩層。patch、least privilege、輸入/記憶體安全、segmentation、認證、監控與回復分別降低不同失效的機率/損害；不能只增加工具名稱而不指定誰阻止哪條攻擊路徑。

原創風險模型：單次損失10萬元、每年事件機率0.02，期望損失2000元/年；控制若把機率降到0.005則剩500元，期望減少1500元。它不描述尾端災難、相關事件或法規義務，不能只按平均值決定所有安全政策。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

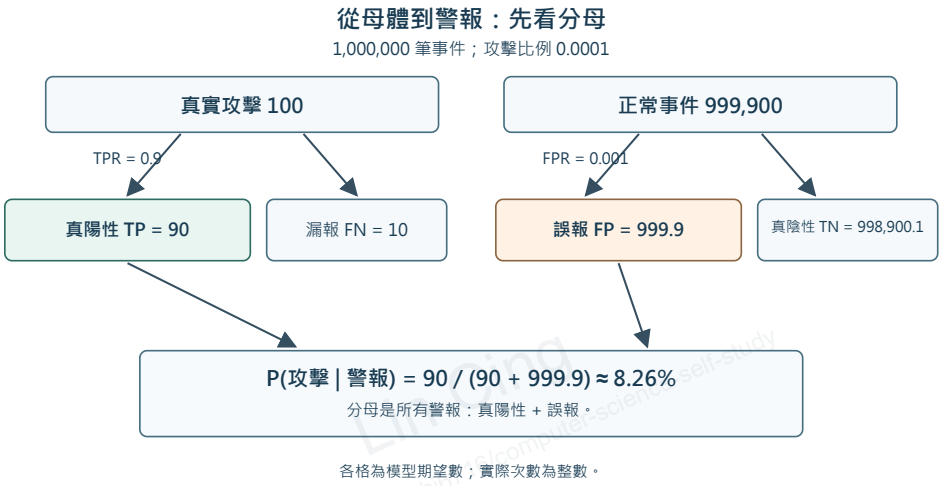
IDS／IPS：base rate 不能省略

signature辨識已編碼的可疑特徵；anomaly辨識偏離baseline；前者不保證所有未知攻擊都漏、後者不保證所有異常都惡意。IDS通常告警，IPS可介入阻擋；阻擋錯誤會直接影響availability。

原創資料：1000000事件中100是真攻擊；TPR=0.9，FPR=0.001。TP=90、FN=10；normal=999900，因此FP=999.9（期望）、TN=998900.1。
P(attack|alarm)=90/(90+999.9)≈8.26%，不是90%。TPR的分母是真攻擊，precision的分母是全部告警。

Bayes式：precision=π・TPR/[π・TPR+(1−π)・FPR]。固定π=0.0001、TPR0.9，若要precision≥0.5，需FPR≤0.000090009…。baseline被入侵資料污染、概念漂移與相關事件聚合均會影響估計，不能拿開發資料回稱獨立驗收。

IDS 告警：先分母再算比例



在題設模型中，警報來自 90 個期望真陽性與 999.9 個期望誤報。以同一母體的條件率計算，警報中攻擊比例約 8.26%；小數是期望數，並非實際事件次數。

Integrity monitoring、antivirus 與可恢復性

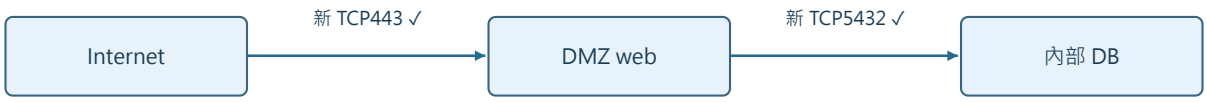
可信baseline記錄檔案內容digest與metadata，重新比較可發現變動；baseline與checker若同被攻擊者修改，對比就失去根。hash matching的前提是使用適當抗collision算法與受保護原始digest，並非絕對同一內容的數學保證。

Tripwire式監控要分開authorized update與unexpected change；頻繁變動log不能因「本來會變」而完全放棄audit，要使用適當遠端/append-only/受保護記錄與關聯分析。honeypot用於觀察，不得成為可任意攻擊內網的新跳板。

antivirus特徵、行為、sandbox各有偵測範圍；clean scan不證明合法供應鏈產物無惡意。發現問題後還需隔離、保全證據、修補來源、由可信備份恢復及輪換受影響credentials。audit record應帶subject/object/action/decision/time/correlation id，並保護其完整性與可用性。

Firewall：先畫允許的連線，才寫規則

安全區不是全信任：只放行列明的方向與服務



Internet → DB 新連線：拒；web → DB22：拒；返回封包另依既存 state 驗證。

web 被攻陷後仍有 DB5432 這條必要通道；DB 必須另做認證與最小查詢授權。

原創stateful policy：Internet→DMZ web TCP443允許；DMZ web→內部DB TCP5432允許；內部→Internet TCP443允許；其餘**新連線**default deny；只讓符合既存state的返回封包通過。沒有一條「DMZ可去所有內網」規則。即使web被攻陷，對DB5432的既有必要權仍可被濫用，所以DB應另做認證/查詢權限。

stateless filter看每包header；stateful追connection state；application proxy可終止並理解協定；host firewall限制本機入口/出口。TLS未在proxy終止時，單純packet filter看不見加密payload的應用語義；允許443不代表443內所有內容安全。

規則求值與邊界

規則2–4分別實現Internet→web、web→DB及內部→Internet三種新連線；規則1處理符合既存允許連線state的雙向後續封包。TCP服務比對的是目的port。

次序	比對條件	動作
1	符合既存允許連線state的後續封包，雙向均可	允許
2	新TCP，Internet→DMZ web，目的443	允許
3	新TCP，DMZ web→內部DB，目的5432	允許
4	新TCP，內部→Internet，目的443	允許
5	其餘封包	拒絕

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

規則求值與邊界 （續）

封包情境	第一條命中規則	結果
新Internet→web TCP443	2	允許
新web→DB TCP5432	3	允許
新內部→Internet TCP443	4	允許
新Internet→DB TCP5432	5	拒絕
新web→DB TCP22	5	拒絕
DB→web，且確實符合既存連線的返回state	1	允許
web→DB，既存允許連線中的後續資料	1	允許

source/destination IP、zone、目的port、direction、state都要比對，不能只看到5432或ACK bit就放行。只宣稱established而沒有state-table證據的封包不命中規則1。

如果把「deny all」放第1且仍採first-match，其他新連線規則不可達。真產品可能有不同chain/priority語義，須按其規格；此模型只是清楚展示規則順序與狀態的作用。

firewall不修復允許協定中的程式漏洞，不保證host身分只靠IP就可信，也無法在上游鏈路已塞滿時憑本地drop恢復頻寬。tunneling/出口控制與DNS等依賴需另列，不得以有DMZ圖就宣稱內網不可達。

歷史 TCSEC：功能要求與 assurance 分開

固定為美國DoD歷史TCSEC/Orange Book分類，不是今日所有產品的一條通用安全分數。D為未達較高類別；C1/C2重視discretionary access與accountability，C2加個體控制、audit與object reuse等要求；B1–B3逐步加mandatory labels、較完整的結構/分析要求；A1在B3功能上增加形式化設計/驗證與開發管理保證。

B2的covert-channel要求是識別/分析可能繞過正常資訊流的通道，不是「支援秘密外洩通道」。TCB是支撐政策強制執行的硬體/軟體/韌體集合；不在TCB的component錯誤不應破壞該政策，否則邊界畫錯。

認證結論限制在受評版本、配置、政策與環境。A1不代表業務永不出錯、所有攻擊都不可能或今日任何新版本自動繼承同保證；形式化證明也只對其模型、假設與規格成立。

Windows 7：token、security descriptor 與 operation

固定歷史Windows7核心模型。process primary token帶user/group SIDs及privileges；server thread impersonation時可使用client context。object security descriptor含owner、DACL與SACL；DACL決定允許/拒絕，SACL可含audit與mandatory integrity label。已取得handle的granted access與後續操作也要對照，不是每個API都重跑同一open流程。

下例限制為普通specific access bits，無owner/privilege例外、restricted-token附加檢查或object ACE；用來逐步演示DACL：remaining=requested；依ACE順序，matching deny若與remaining相交則拒絕；matching allow移除其granted bits；remaining=∅可成功；掃完仍非空則拒。不是先把所有allow union後再把deny全部相減的通用算法。

container inheritance與copy/move/既存object傳播依檔案系統/API選項而異；建立object時需檢查最終descriptor，不能假設更改parent ACL必自動以同方式重寫所有已開handle。

DACL 的實際 mask trace

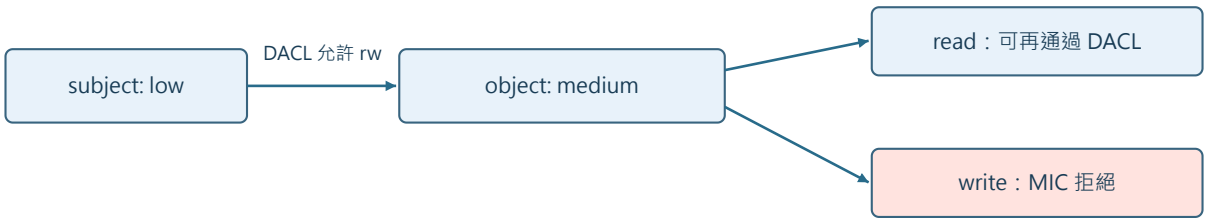
request={read,write}，token含user=U與enabled group=Students。DACL依序：deny(Students,write)；allow(U,{read,write})。初始remaining={r,w}；第一ACE命中且交集={w}，立即拒。若U不是Students，第一ACE略過，第二移除r,w，remaining空則允許。

若故意使用非canonical順序把allow(U,rw)放前，會先滿足全部請求而成功；所以不能說「任何位置的deny永遠覆蓋所有allow」。系統偏好的canonical順序將explicit deny置於相關allow前，inheritance等細節還需額外排序規則。

NULL DACL（無DACL）與存在但零ACE的empty DACL不同：前者在本普通access-check模型允許，後者沒有grant而拒絕。兩者都不表示可繞過另一層mandatory policy或特別privilege規則。

MIC 與 UAC：不是低完整性一律不能讀高物件

同一 DACL grant · mandatory policy 仍分 read / write



前提：mandatory label 只有 NO_WRITE_UP；若加 NO_READ_UP · read 分支也拒絕。

low subject對medium object，DACL允許r,w。若mandatory policy只有NO_WRITE_UP，write仍拒絕，read可在DACL與其他檢查通過後允許；若另設NO_READ_UP，read也拒。integrity主要保護不被低信任來源修改，不等於confidentiality clearance的完整模型。

Windows7 UAC下管理員的一般程式可使用filtered medium token；經受控elevation才取得相應較高token。UAC提示不是給任意程式創造使用者從未有的所有權限，也不能把UI確認本身當作惡意程式辨識器。

可組合的安全推理

認證綁定key或credential到subject；授權限制subject的operation；完整性/保密scheme保護訊息；freshness與receiver state防重放；transaction/recovery把授權連到一次可恢復效果。缺任一層都不能用另一層的名稱補上。

實際例：正確MAC的重放仍需拒；合法TLS但proxy已被攻陷不保證end-to-end秘密；DACL允許write仍可能被MIC拒；NX啟用但越界改資料仍可能破壞授權。結論應列前提、失效後果與可觀察驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般證明、數值例與配置證據

RSA以Fermat/CRT證全域正確性，模冪表只核一個輸入；重用keystream的洩漏由XOR恆等式推出，不是靠挑某個cipher破解；IDS以Bayes連接不同分母，不能由單一TPR推precision。

歷史協定/OS案例需保留版本、endpoint與省略範圍。測試一次nonce重放、跑一個DACL trace或scan一次網路只支持相應觀察；不替代對所有state transition、威脅模型與安全規格的論證。原創練習用這些區分檢查是否真正理解機制。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 15. RFC 5246. RFC 7568. Microsoft: DACL. Microsoft: MIC.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：如何在隔離多個 guest 的同時，保持指令語意、裝置介面與資源上限？

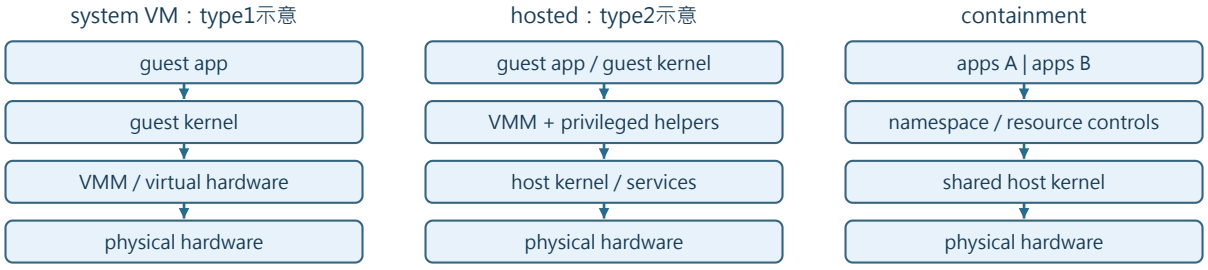
前置知識。 OS 結構、CPU trap、位址轉換與 I/O。

- 區分 type 1/type 2 hypervisor
- 分析 trap-and-emulate 與 binary translation
- 說明 memory/I/O 虛擬化的安全與成本

Lin Cing https://github.com/xinhi6716/computer-science-self-study

VM、host、guest 與 VMM 的邊界

控制資源的層不同：不能把 VM 與共用 kernel 的 container 混同



system VM向guest OS提供CPU/memory/device介面；VMM/hypervisor管理實體資源並實現虛擬介面。guest內的application仍呼叫guest OS，guest OS再控制虛擬device/CPU；不是所有guest syscall都必須立即進host kernel。

vCPU保存guest可見的register、PC、狀態與虛擬interrupt等，排到pCPU才實際執行。兩個VM都可見「physical page5」，卻由各自第二階段映射到不同host frame；同名不等於同一資源。VMM必須管理共享CPU、記憶體、裝置與管理介面的隔離。

system VM、process VM（如JVM）、cross-ISA emulator、OS-level containment是不同抽象。分類應問「對上提供哪個介面、誰持kernel、誰控制physical resource」，不只背type號碼。

歷史動機與三個設計目標

IBM VM/370（1970年代）的minidisk把實體disk的不同區段呈現為guest獨立disk，讓多個OS共享一台mainframe。此種抽象需要保存地址範圍與隔離，而不是把完整實體disk register任意交給所有guest。

fidelity：除了明定差異，guest可見行為符合虛擬機器規格；resource control：guest不能奪走VMM控制的資源；efficiency：大部分可安全執行的工作可直接執行。效能目標不表示每種工作負載固定只慢某個百分比，也不表示時間觀察完全相同。

對傳統trap-and-emulate的常見充分條件是需要VMM仲介的sensitive instructions能可靠被攔截；失去此條件可改用binary translation/paravirtualization/硬體輔助。完整emulation可在軟體中解釋guest ISA，不需要CPU先有專用VM指令；專用硬體主要改可用技術與效率界線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Isolation、consolidation 與共同失效

VM隔離方便OS開發、測試多版本與整合低利用率server；若hypervisor/device emulator有漏洞，或多VM共用同一故障host，仍可能共同失效。資源隔離也須有CPU/memory/I/O限額，不能由「地址分開」推出鄰居不會拖慢自己。

原創capacity例：host8cores，VMM預留1core，A/B/C的峰值需求分別3/3/2cores；總8>剩餘7，即使平時平均各只用1，也不能保證三者同時峰值不排隊。overcommit依賴工作負載與服務目標，並非免費增加運算能力。

VM網路/共享磁碟可解決guest間分享，但重建了新的信任邊界。管理平面能clone/snapshot/讀disk image，本身是重要TCB，不因guest彼此隔離就可忽略管理者權限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Suspend、snapshot、clone 與 backup 不同

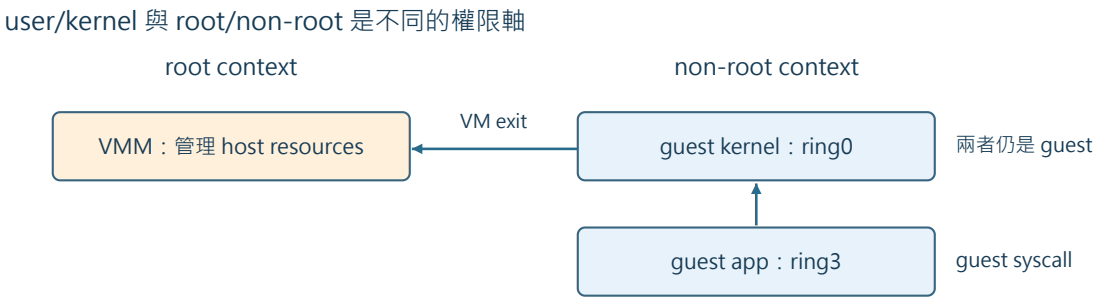
suspend保存可恢復CPU/RAM/device狀態；disk snapshot保存磁碟某點的block版本；clone建立可獨立運行的副本。只copy正在寫入的多個disk files不一定得到一致時間點，guest RAM與disk也可能不匹配；需停機/一致snapshot或guest application quiescence等契約。

crash-consistent snapshot相當於某時刻斷電後可恢復的disk state，application-consistent還需應用的flush/transaction協調。外部mail/payment/database effects不在同一snapshot內，回滾VM不會自動撤回它們；clone也須處理重複IP/MAC/machine identity與credentials。

snapshot若與source disk位於同一故障範圍，不等於獨立backup。template提高建立一致性，但template中的漏洞/帳密也會被複製；cloud/VDI把資源建立變成可管理API，仍受同樣的隔離與外部狀態限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trap-and-emulate：虛擬模式與實體模式分開



傳統無額外VM mode的模型：VMM持physical privileged mode，guest kernel與user都在較低實體權限執行，但VMM保留guest虛擬mode。guest kernel的特權操作trap，由VMM驗證guest原本是否有虛擬權，再emulate；guest user違法操作應得到guest exception，不能因到了VMM就被授權。

例guest kernel要求關interrupt：VMM把vCPU.virtualIF設0，延後此guest可遮罩interrupt的注入；不應讓它永久關掉整台host服務其他VM的真實interrupt。VMM保留host控制權，並保存guest可見狀態；之後guest重新開interrupt才按規格注入pending事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不會 trap 的 sensitive instruction 與 translation

若某instruction在低權執行時靜默忽略部分狀態改變，guest kernel以為已修改虛擬flags，VMM卻未得到控制，單靠trap-and-emulate就失真。x86歷史POPF的特定flag/權限行為是這種問題的例子；不能由「未trap」推「對guest無害」。

binary translation讀guest instruction blocks，將需仲介的操作替換為等價、安全的VMM操作，其餘轉成可直接執行的host code，並cache翻譯結果。需要處理indirect branches、自修改code、page權限變更與exception-visible狀態；修改原code後不能沿用語義已變的translation cache。

paravirtualization另讓合作guest呼叫hypercall取代不易虛擬化操作；它與binary translation可解決相似問題，但對guest ABI/修改要求不同。硬體輔助也不表示device emulation、scheduler或I/O成本都消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hardware-assisted guest execution

root/non-root或host/guest mode提供比傳統user/kernel更多的控制維度。guest OS可在non-root ring0，guest app在non-root ring3；VMM以root context管理。VM entry載入guest state；特定event產生VM exit，保存guest狀態、提供exit reason，由VMM處理再resume。

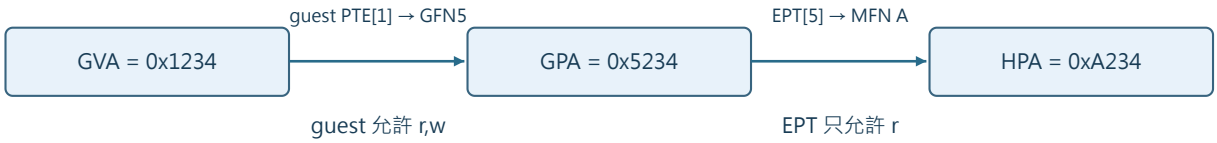
哪些instruction/event exit由架構與controls決定，不是每條guest instruction都exit，也不是每次guest syscall都exit。VMM要正確設定intercepts、保存狀態與事件注入，才能同時滿足fidelity/resource control。

DMA的IOMMU與interrupt remapping解決device端隔離/路由，與CPU instruction virtualization不同；不應把CPU page table或EPT誤當作自動約束所有device DMA的機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shadow page table 與硬體兩階段轉譯

地址與權限逐層組合；不是只串兩個名詞



page size 4096 B；offset 0x234 在兩階段都保留。read 通過；write 第二階段拒。

Shadow table 可直接組合 VPN1 → MFN A，但需同步 guest PTE 更新與 translation invalidation。

guest page table：GVA→GPA；VMM第二階段：GPA→HPA。**hardware two-stage**（如EPT/NPT）由硬體在page walk中組合兩層；**shadow table**則由VMM維護GVA→HPA的組合映射，讓一般MMU直接使用shadow，並攔截/追蹤guest page-table更新以保持一致。兩者不能只因都有額外表就混成同一演算法。

本例page4096 B，GVA0x1234=(VPN1,offset0x234)，guestPTE[1]=(GFN5,rw)，GPA=0x5234；EPT[5]=(MFN0xA,r)，HPA=0xA234。read需兩層都允許，write被EPT拒；guest以為可寫不代表有權寫host page。若guestPTE本身deny，應按guest paging規格處理guest fault；第二階段違規由VMM的相應機制處理，不可一律當同一fault。

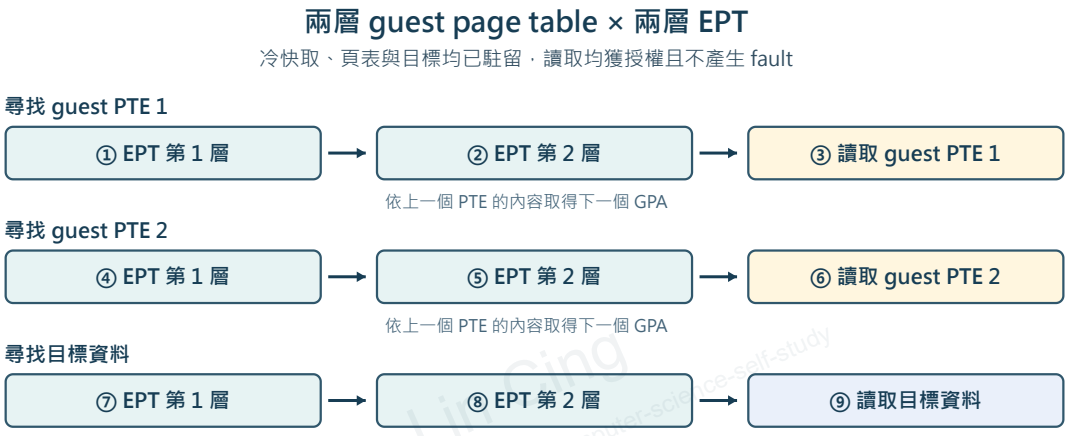
無cache的 nested walk：為何不只「兩倍」

假設 guest 有 m 層、第二階段有 n 層，頁表和資料已駐留且權限通過，沒有 TLB、page-walk cache 或 large pages。每讀取一個 guest PTE，先以 n 次第二階段 PTE 讀取轉譯其 GPA，再讀 guest PTE 本身，共 m(n+1) 次；最後資料的 GPA 還需 n 次第二階段 PTE 讀取。

因此共 m(n+1)+n=(m+1)(n+1)−1 次**頁表讀取**，再加一次資料讀取。m=n=4 時為 24 次頁表讀取及 1 次資料讀取，不是 4+4；這是冷快取模型，並非每次 load 都要 25 次。TLB、walk cache 和 large pages 可減少實際讀取；shadow 的硬體走訪較短，但 guest 更新頁表時需要 VMM 維護。

這裡只計讀取，假定 accessed／dirty 位元不需另寫；真實硬體若要更新狀態，可能產生額外寫入。映射或權限變更後須按架構規則失效 translation cache；切換 VM 時也須正確標記或 flush，避免錯用其他 VM 的映射。

逐次追蹤兩階段 page walk



表項讀取：2 + 1 + 2 + 1 + 2 = 8 次；資料讀取：1 次。A/D 位元更新等寫入未計入。

假設兩層 guest page table 與兩層 EPT，沒有快取、所有映射有效且頁面已駐留。每個 guest PTE 的 GPA 都先走兩層 EPT，接著才能讀取該 PTE；資料 GPA 也需再走 EPT。共 8 次表項讀取與 1 次資料讀取，未計 accessed／dirty 位元更新等寫入。

VM lifecycle 與 type0/1/2 的責任

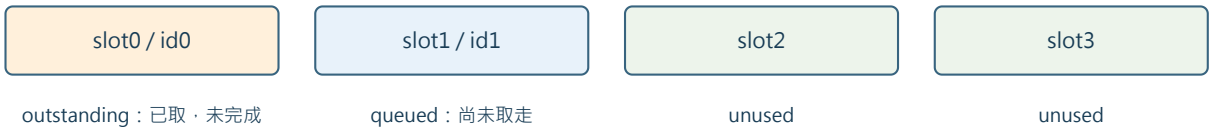
建立VM先驗CPU、memory、disk、network需求與policy，再配置vCPU、地址空間與device state，載入guest；running可pause/resume、snapshot、遷移，delete前先停止device/事件並回收資源。只刪管理紀錄而留活DMA或網路身份不是正確銷毀。

type0是firmware/hardware partition類型，可dedicate或分享部分資源；type1直接管理host資源，含某些general-purpose kernel中的VMM；type2依host OS提供process/device/memory服務。type2常仍有特權helper/driver與硬體輔助，不能一概說host完全不知虛擬化或永不能用VT-x。

分類不是對效能/功能的無條件排名；nesting是否支援由VMM、CPU與設定決定，不是邏輯上只有type0可嵌套。管理範圍、TCB與resource path才是可檢驗的差異。

Paravirtual I/O：descriptor、發布與回應

四槽順序完成模型：backend 取走不等於 frontend 可重用



req_prod=2，req_cons=1，rsp_prod=0，rsp_cons=0；used=2，free=4−2=2。

完成 id0：rsp_prod→1；frontend 消耗 response 後 rsp_cons→1，才釋 slot 容量。

固定歷史參考為Xen SOSP2003的合作guest、hypercalls與shared-ring設計；下面是明確簡化的單producer/consumer模型，不冒充所有Xen版本的完整ABI。counter單調遞增，slot=index mod4；req_prod=2、req_cons=1、rsp_prod=0、rsp_cons=0，因此queued=1、outstanding=1、response=0、used=2、free=2。

backend完成id0後rsp_prod=1，outstanding變0、response變1；frontend消耗此response後rsp_cons=1，used=req_prod−rsp_cons=1、free=3。不能在backend剛取走request時就重用descriptor slot，因為它可能仍等completion。

Shared ring 的同步與驗證

producer 先填 descriptor，再以 release 發布 req_prod；consumer 以 acquire 看見新 prod 後才讀 descriptor。response 亦先寫結果再發布 rsp_prod；frontend 消耗 response 並推進 rsp_cons 後，slot 才可重用。event-channel／doorbell 是通知，不取代 counter predicate。

只「重新檢查後睡眠」仍有競爭：consumer 最後看見空 queue，producer 隨後發布並通知，但 consumer 尚未登記等待；接著 consumer 睡眠，通知已失去。以教學 lock／condition 為例：producer 在同一把鎖下發布狀態並喚醒；consumer 持鎖用 while 檢查 predicate，若仍為空，**原子地登記等待並釋鎖睡眠**，醒後重查。若跨 domain 不能共用此鎖，須以事件計數器等機制保證檢查與睡眠窗口的發布可被觀察；不能只依普通通知。

在不溢位的簡化順序完成模型，0≤req_prod−rsp_cons≤N 且 rsp_cons≤rsp_prod≤req_cons≤req_prod，四個區間不重疊。backend 仍須驗 ID、operation、block 範圍、長度與 buffer grant／ownership；shared memory 不授權 guest 任意 DMA 到 host。亂序完成需要另一套 ID／slot 管理。

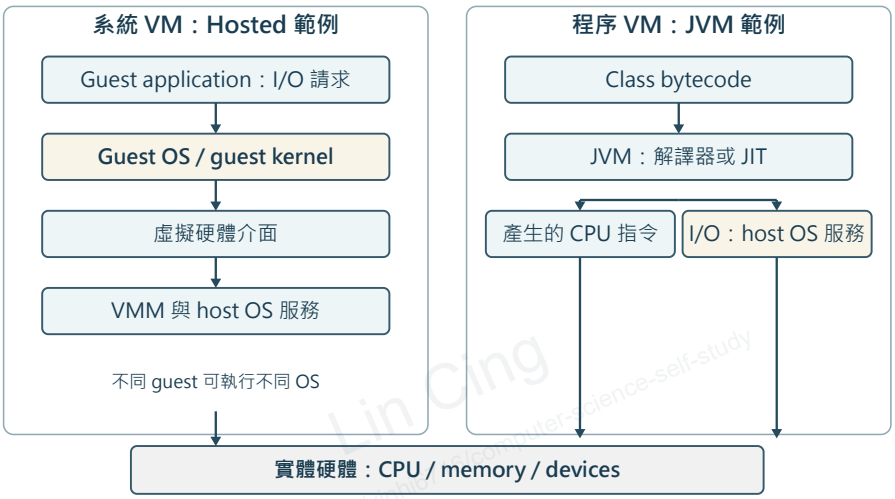
Process VM、emulation 與 containment

JVM/.NET式process VM提供language bytecode/runtime介面，不是完整guest OS；cross-ISA emulation解釋/翻譯不同ISA的instruction與machine state，可能慢但能執行不相容二進位。兩者都不能直接用「是否有自己的hardware ring0」作唯一判準。

OS containment（書中Solaris10 zones、BSD jails等）共用host kernel，隔開namespace/resource/security context；不是每個container有獨立kernel。它能有不同userland/network view，但kernel bug的共用故障範圍與VM不同。Solaris10的global zone管理non-global zones，系統服務與resource controls需具體配置，不宣稱每個zone獨立實作整套kernel scheduler。

選擇抽象取決於需求：測試另一kernel需system VM或emulator；同kernel下隔開服務可用containment；執行bytecode需對應runtime。這是機制適配，不是單一方案永遠最佳。

系統 VM 與 JVM：介面與核心的差異



圖示 hosted 模型；隔離也依賴管理面與資源控制。

系統 VM 向 guest OS 提供虛擬硬體；JVM 提供 bytecode 執行環境，仍使用 host kernel。圖中的 host OS 路徑是 I/O 等服務，不代表每條指令都進行 system call。隔離還取決於 VMM 與管理面，CPU、記憶體和 I/O 共用也可能互相干擾。

CPU scheduling：guest 排thread，VMM 排vCPU

兩個pCPU；A0/A1與B0/B1共四vCPU，host overhead先忽略。0–10 ms排A0/A1，10–20排B0/B1，20–30再A，30–40再B。A某thread需20 ms CPU，從t0開始在A0於0–10與20–30執行，牆鐘t30才完；B對應thread在10–20與30–40執行，t40完。平均各得一半pCPU，不代表每個20 ms budget都剛好40 ms latency。

guest ready thread可能因其vCPU未獲pCPU而等待；guest內lock holder被deschedule時，另一vCPU spinning可能浪費其份額。virtual clock、wall clock與實際vCPU runtime要分開；virtual timer到期後也可能等vCPU才能處理。real-time保證需要host admission/隔離/排程條件，不能只在guest改priority。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Memory overcommit 與 balloon 的真正方向

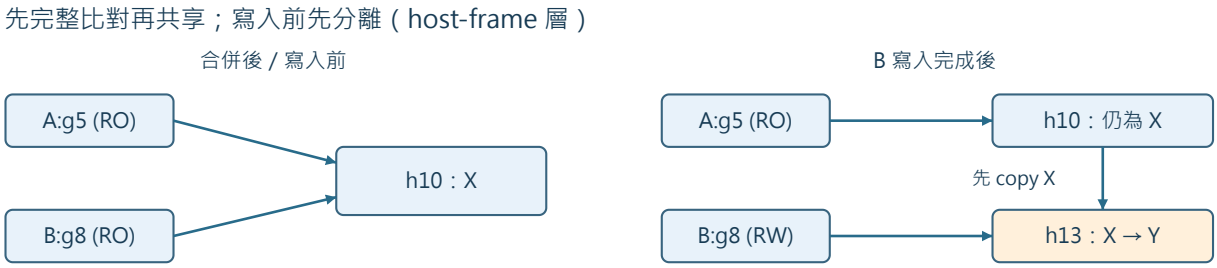
歷史參考固定 Waldspurger 的 ESX Server OSDI 2002 設計。balloon driver 在 guest 內配置／保留 pages 並報告 PFN；VMM 可回收對應 host backing，guest 依自身策略釋放或換出其他頁。inflate 減少工作負載可用記憶體，不會增加實體 RAM。

例：host 4096 MiB，VMM 預留 512，餘 3584；A、B 各要求 2304 MiB 常駐 backing，總需求 4608，超出 1024。令 A balloon 256、B balloon 768，目標 backing 分別為 2048、1536，合計 3584；加 VMM 512 剛好 4096。這是需求與配置目標的守恆，不表示物理 RAM 曾同時裝下 4608 MiB。guest nominal 配置可不變，但不能無代價使用全部原需求頁。

balloon pages 由 driver 保留，不應由正常 guest 工作負載使用。為說明隔離，另假定本模型對未 backed 映射觸碰時會 fault，由 VMM 拒絕或重建安全 backing；這不是所有 balloon 協定對錯誤存取的通用保證。deflate 前須確保頁可安全使用；driver 不合作或回收太慢時可能需要其他 paging／限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Page sharing、COW 與 double paging



B 的 write fault：先配置 / copy → 改 B 映射與 translation 失效 → 再放行寫入；A 映射不變。

A:g5→h10、B:g8→h12 都含 X。hash 僅篩 candidate：合併前先鎖定相關狀態、移除兩頁所有可寫映射，完成必要 TLB／EPT 失效，並暫停或排除可能修改頁面的 DMA；在保護持續有效期間逐 byte 比較。相等才將兩映射切成對 h10 的唯讀共享並維護 reference，防止比較後、切換前的寫入讓資料失配。h12 必須確定無剩餘 reference、pin 或在途存取，才能回收。

B 寫 Y 時觸發 COW fault：先把 X 複製到私有 h13，將 B:g8 改指 h13，按架構規則失效舊 translation，再放行寫入；A 仍見 X。不能先放行 write 再 copy。VMM swapping 可能與 guest virtual swap 形成 double paging；balloon 讓 guest 選 victim 可改善資訊落差，但不保證總是較快。頁面掃描與同步也有成本，跨租戶 sharing 應受隔離政策約束。

I/O 與 virtual networking 的三條路

emulated device：guest driver操作虛擬register，VMM/device model驗證並映到host driver；paravirtual device：guest與backend用合作queue；passthrough：device直接服務guest，仍需IOMMU/address isolation、interrupt路由與安全的reset/ownership transfer。

若guest DMA address0x5000映到host0xA000，IOMMU須只容許配置的範圍；guest CPU的EPT限制本身不等於DMA限制。passthrough減少部分仲介，可能讓snapshot/migration需要額外device state支援，不能只搬RAM就無條件遷移。

bridge讓VM接入相應L2網路；NAT由host把內部位址/port映到外部tuple。例A10.0.2.15:5000與B10.0.2.16:5000分別映host:40001/40002，可共用外部IP；所以「每個VM必須有不同公網IP」不成立。firewall規則仍要界定guest間及對外通道。

Storage image 與 snapshot 的 block 追蹤

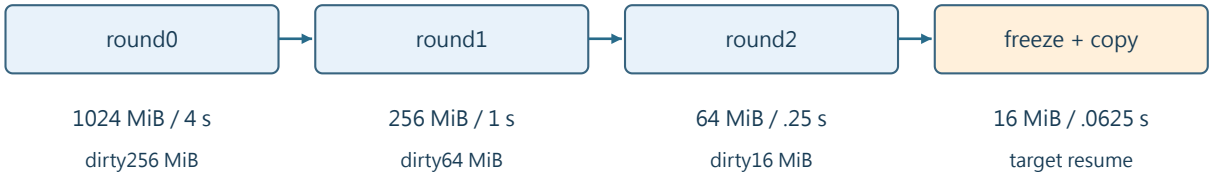
guest disk的LBA只是虛擬disk位置，再由image mapping/host filesystem到實體儲存；也可能直接用block volume，不保證永遠只是一個普通file。thin provisioning只延後配置，不保證最大邏輯容量同時寫滿仍有空間。

原創snapshot chain：base存blocks{0:A,1:B,2:C}；建立snapshot後active delta寫{1:B′}。read1先查delta得B′，read2落base得C；回到snapshot選base可讀回B，但外部已寄出的mail不會撤回。clone若共享base，base不能在仍被引用時刪除；完整backup須包含必要chain/metadata與一致時間點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pre-copy migration：每一輪都能核算

正常路徑 pre-copy：總時間與 downtime 分開



B=256 MiB/s · d=64 MiB/s；總 1360 MiB / 5.3125 s · pause .0625 s（未含控制開銷）。

假設同VMM/兼容CPU、shared storage、無不可遷移device，網路有效copy B=256 MiB/s，guest每秒產生d=64 MiB不同需重送頁、忽略協定/CPUstate成本。R_0=1024 MiB，t_0=4 s，期間dirty R_1=256；下一輪t_1=1 s、R_2=64；下一輪t_2=.25 s、R_3=16。此時freeze，copy最後16需.0625 s；總copy1360 MiB、5.3125 s，其中pause .0625 s。

一般模型$R_{i+1}=(d/B)R_i$，若d/B<1幾何收斂；d≥B時此模型不縮小，需限制輪數/降dirty rate/延長pause或改方法。真實dirty頁可重複、頻寬波動且RAM有限，本式是明確假設的模型，不是所有migration的精確預測。

Stop-and-copy 與單一活動副本

freeze不只停user code，還需quiesce可改guest狀態的device/DMA與交付中的事件；保存vCPU/device/dirty RAM及必要network state，讓target取得一致狀態。轉移ownership前source不可再對外執行guest，target可運作後再清source資源。

若source只因沒收到ack便重新運行、target其實已啟動，就可能split brain。安全遷移協定須有明確commit/ownership與故障恢復規則，不能以「等target ack」代替所有failure case證明。本節簡化正常路徑只證pause時的consistent handoff，不宣稱任意network partition下仍透明可用。

網路需維持可達地址/MAC等對外身份與路由更新；TCP可容忍短停頓不表示pause=0。shared-storage是此pre-copy模型前提，其他系統也可做block migration，因此不能由本模型推出「任何live migration都不能搬disk」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Workstation 與 Xen／ESX 的歷史對照

Workstation採書中2013年前後的hosted架構示意：guest app→guest kernel→virtual device/虛擬CPU→VMM/host kernel服務→hardware；普通guest instruction可直接執行，I/O可能經host服務。此處不把示意稱為某release的完整程式碼或量測。

固定原始研究例：Xen SOSP2003以合作guest/hypercalls/shared ring減少不易虛擬化的介面；ESX OSDI2002以balloon/content sharing等管理overcommit。二者的介面與研究年代不同，不能把早期PV guest規則宣稱為所有後來Xen的唯一模式，也不能從paper某個benchmark泛化固定效能比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

JVM：bytecode verifier 與執行引擎

以Java SE7時期的一般JVM抽象為歷史比較：class loader載入class，驗證型別、stack使用與合法控制流；execution engine可interpret或JIT；runtime提供heap/GC及安全機制。JVM不包含另一個通用guest kernel，仍依host OS取得memory、thread與I/O。

bytecode例`iconst_2; iconst_3; iadd; istore_1`：operand stack依序[]→[2]→[2,3]→[5]→[]，local1=5。若初始[]直接iadd，需兩個int卻沒有，verifier可拒；不是等硬體stack underflow才算錯。這不保證業務邏輯正確。

JIT不一定第一次呼叫就編完整method，可能依hotness/tiering；GC也不是單純把「不再使用」的語義自動判斷，而是依runtime的可達性/collector規則回收。語言規格與實作策略應分開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VM 的兩個必守條件

fidelity要求每個guest-visible state transition符合虛擬機器規格；resource control要求guest不能逃過VMM對CPU、地址與device的管制。trap/translation/hypercall/hardware assistance是達成方式，不是保證本身。

GVA→GPA→HPA的權限交集、shared ring發布/重用順序、COW寫入前分離、migration單一活動副本，都在保護「此guest以為自己的狀態」與「host實際持有的狀態」一致。單次demo成功不能代替所有轉移條件。

vCPU數不等於pCPU容量；nominal RAM不等於當下host backing；logical disk容量不等於已配置bytes；snapshot不是外部世界回滾；pre-copy總傳輸時間不是downtime。每一個數字都要標在哪一層。

練習以mode、地址、walk次數、ring四counter、balloon容量、COW與migration遞迴重建機制；計算前先標出guest、host與device各自的狀態與存取條件。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 16. Xen SOSP 2003. ESX memory management OSDI 2002. JVM SE7 verifier.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：在沒有共享記憶體、時鐘不一致且節點可能失效時，系統如何提供可理解且可恢復的服務？

前置知識。 network stack、RPC、process 與 file-system interface。

- 比較網路型 OS 與 distributed OS
- 分析 naming, routing, RPC 與 failure semantics
- 檢查 DFS cache consistency 與 availability

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Message-passing 系統的目標與限制

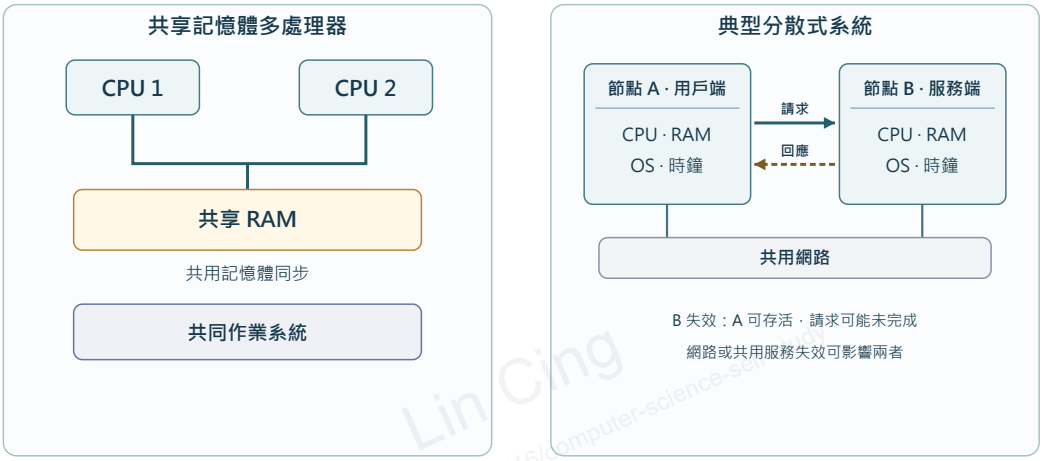
本章以典型 message-passing 模型為例：每個 node 有本機 memory 與 OS，透過訊息通信；不假設共享實體記憶體或完美共同時鐘。local 與 remote 是相對於請求者而言。分散式共享記憶體可作為另一層抽象，時鐘也可同步，但同步仍有誤差。資源分享、計算加速、可靠度與通信是設計動機，不是分散後自動得到的性質。

固定速度模型：單機 100 s 工作中 20 s 串行、80 s 可平均分給 4 nodes，另加 5 s 協調；$T_4 = 20 + 80/4 + 5 = 45$ s，加速比 $100/45 \approx 2.222$。忽略協調的 Amdahl 上限為 $100/20 = 5$；加入 node 不能消除串行部分，網路或協調成本增加還可能使系統更慢。

在**固定的一次讀取時刻**，三個可互代副本各自不可用的事件獨立且機率 $p = .01$；假設網路及 metadata 服務可用，任一可用副本皆能讀取有效資料，則讀取不可用的機率為 $p^3 = 10^{-6}$。若共用電源、rack 或依賴同一 metadata leader，此獨立模型可能不成立。期間內三個副本曾各自失效，並不代表它們同時不可用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享記憶體與訊息傳遞



兩者是模型：效能與可靠度取決於瓶頸、負載及故障域。

共享記憶體模型以同步機制協調 CPU；分散式模型中各節點透過網路交換請求與回應。服務端失效或網路中斷時，用戶端可能無法判斷請求是否已執行。共享網路與服務也可能成為共同故障點；分散式共享記憶體是另一種可建立的抽象。

Network OS 與 distributed OS 的透明範圍

network OS允許明確連遠端host或copy檔案，使用者知道機器邊界；distributed OS/中介軟體可提供更一致的命名、存取與遷移介面。透明性不是把遠端latency/failure從物理上消除，而是把定位/調用方式包進介面並定義失敗語義。

remote login在遠端執行session，file transfer建立另一份copy，DFS則提供遠端共享檔案語義；三者不能互稱。SSH/SFTP可保護對應session，仍需驗host身分與user授權；一般FTP本身的傳輸/認證不應被當作加密通道。

匿名file service需獨立限制namespace與操作，不能由名為anonymous就假定隔離。存取命名、帳號映射、網路失敗與多份copy的更新衝突都必須有規則。

Data migration 與 computation migration 的實算

資料1 GiB、結果1 MiB、有效吞吐100 MiB/s、單次RTT20 ms。把全資料搬來至少1024/100+.020=10.260s，再做local compute；把計算送去，若remote compute=.500s且request可忽略，回結果需.010s，含一次RTT共.530s。比較須含remote CPU queue、啟動/序列化與結果大小，不能只說「大資料一定搬計算」。

whole-file caching將首次成本前置，後續local access便宜；on-demand block caching只搬所需區塊。若只read 1 MiB，整個1 GiB的資料搬移多1023 MiB；若讀全檔且逐block各一RTT、又沒有pipeline，很多小RPC可能更慢。

process migration要轉移或重建PC/register、memory、open files/socket等外部references與執行環境；只把程式碼傳到另一機啟動，屬remote execution/computation placement，不等於帶完整執行狀態的process migration。跨ISA/ABI更需要相容層，不能直接搬register值。

LAN／WAN：範圍不是單一速度規格

LAN常在較小管理/地理範圍，WAN跨站點；真實capacity/latency取決於link、distance、queue與負載。switch、router、wireless AP有不同職責；現代switched Ethernet不應一概描述成所有host共用一條collision bus。LAN可能較擁塞，WAN backbone也可能有高bandwidth，不能只按名稱排序吞吐。

latency≈serialization L/R + propagation + queue + processing。教學例1500B packet、1 Gbit/s，serialization=1500×8/10^9=12 μs；單向propagation5ms遠大於serialization。把link升10倍只把12μs降1.2μs，不能把5ms propagation也除10。

bytes、bits與rate單位分開：100 Mbit/s=12.5 MB/s（十進位，尚未扣headers）；MiB/s使用2^{20} B分母。wireless名目rate也不等於application goodput；本章數字皆為指定模型，不列產品當前最高規格。

Bandwidth-delay product 與在途資料

link100 Mbit/s、RTT40ms，BDP=100×10⁶×.040=4×10⁶ bits=500000B≈488.28KiB。若可靠stream最多容許64KiB未確認資料，理想window bound=65536×8/.040=13.1072 Mbit/s，即使link有100Mbit/s也可能填不滿。

這是無loss、固定RTT、無其他瓶頸的上界模型；TCP的congestion window、receiver window、ACK行為、slow start與retransmission還會影響實際結果。增加send buffer只可能解除一種限制，不保證消除congestion或server CPU瓶頸。

低latency RPC與大檔stream的效能需求不同：一個只回16B的RPC可能主要花RTT，不是16B的serialization。這解釋為何資料/計算擺放與cache對分散式OS至關重要。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Naming：domain name、IP、port、object id 分開

`store.lab.test` 是 domain name，不是 IP address；解析可得 IPv4 如 198.51.100.20 或 IPv6。IP 位址不直接等於 user 或永久 machine identity；service 還需 transport protocol、port 與應用層名稱。TCP connection 通常由來源／目的 IP 與 port 及 protocol 區分，port 不是 process PID。

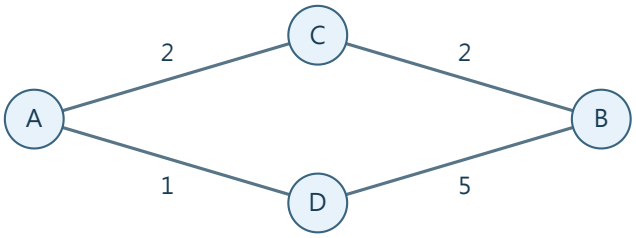
DNS 使用 resolver、query、cache 與 delegation，不要求每個 label 都有獨立 name server。假設 root 委派 test、test 委派 lab.test，resolver 在 cache miss 時由已知 root hints 沿 referrals 查詢權威 A record。以下限定遵守 TTL 的教學 resolver，沒有 serve-stale 或本機固定 override；此限定不是所有實作的通則。

若 t=0 取得 A=198.51.100.20、TTL=600 s，t=100 剩 500 s；權威在 t=200 改為 .30 時，舊 cache 仍可於 t=600 前使用 .20。到期後若重新 query 成功且記錄未再改動，才取得 .30；查詢失敗不能稱其為已驗證的新值。DNS 資料完整性與連線對端認證另需相應機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Routing：用同一加權圖追下一跳

同一張非負加權圖：distance 更新不跳步



從 A：D=1 · C=2 · B=∞

確定 D：B=1+5=6

確定 C：B=min(6,2+2)=4

確定 B：A → C → B · cost4

若 C–B 失效 · 重算走 A→D→B · cost6；動態協定仍有偵測與收斂時間。

原創graph：A–C=2、C–B=2、A–D=1、D–B=5。Dijkstra從A：初始dD=1、dC=2、dB=∞；取D後dB=6；取C後改dB=4；取B確定。最佳A→C→B，cost4。若C–B失效，重新算後走A→D→B，cost6。

static routing按預設規則/管理更新；virtual circuit固定某session路徑；dynamic routing由協定更新可達性/metric。dynamic不等於每packet一定挑當下最空link，也不保證收斂期間無loop/drop。沿同一路徑僅在link/queue等保持相應次序時保序；end-to-end stream ordering仍由transport協定實現。

control plane計算路由，data plane按forwarding table處理packet；SDN可分開這兩者，仍需更新一致性、失敗處置與保護管理通道。

Packet 與 switching：有狀態連線不等於專用電路

packet/frame/datagram可有可變長度，各層各有header與大小限制；一個application message可拆多segment/packet，也可多message共一TCP read。TCP connection不保留專用physical link，UDP也可由應用另建重傳/可靠訊息協定。

circuit switching預留相應路徑資源；message switching整個message store-and-forward；packet switching可把message分段pipeline與統計多工。兩串行links各1Mbit/s、忽略propagation/header/queue：4000B整message走兩hop需2×4000×8/10⁶=64ms；拆4個1000B packet，每packet每hop8ms，pipeline完成需(2+4−1)×8=40ms。

若每packet另加40B header，最後一個packet到達需5×1040×8/10⁶=41.6ms；整message若只加一次40B則需64.64ms。這是固定store-and-forward模型，不能用其公式直接推所有cut-through/circuit網路。

OSI 是職責模型，不是每層一定一顆硬體

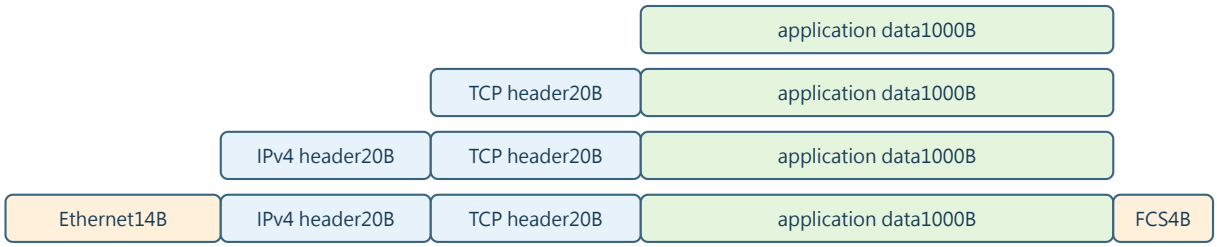
physical傳signal/bits；link在相應link傳frames；network做logical addressing/forwarding；transport提供process endpoints間服務；session管理dialog；presentation處理表示/encoding；application定義服務操作。真實TCP/IP常把session/presentation功能置於application libraries/protocols，不能據層數少就定理式推出更有效率。

TCP提供ordered reliable byte stream與flow/congestion control，不保留application message邊界；UDP保留datagram邊界但不內建同樣的交付/排序保證；IP是best-effort datagram。NIC offload可把部分checksum/segment工作移到硬體，所以「下三層一定全硬體、上四層一定全軟體」不是通用規格。

序列化需定欄位長度、byte order、encoding與版本；不能把C struct含padding/pointer原樣傳到異構機器就當跨平台協定。

Encapsulation：同一份 payload 的長度守恆

每層加入自己的 header / trailer；長度逐層核對



欄寬非比例；IP total1040B · MAC frame1058B；preamble/SFD與IFG另計。

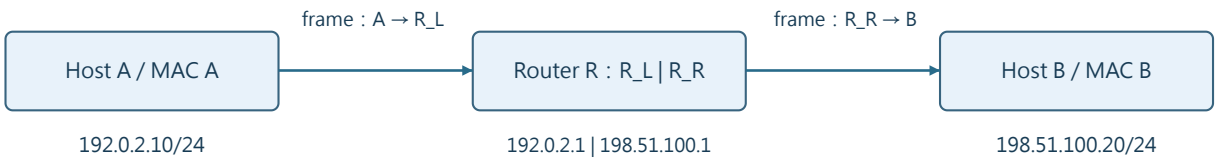
固定IPv4無options20B、TCP無options20B、application data1000B：TCP segment1020B，IP total length1040B，EthernetII header14B+IP1040+FCS4=1058B。未含preamble/SFD8B與指定模型的IFG12 byte-times，若連續傳送的wire占用為1078 byte-times。

IPv4 MTU1500時，本例1040可放一frame；MSS通常從可用IP payload扣TCP header，不能把1500全當application bytes。加VLAN/options/tunnel/security headers後重新核算，不延用上述數字。

TLS或其他認證/加密有明確endpoints；router一般不因轉送就成為application session的認證對端。lower-layer checksum主要偵測傳輸錯誤，不是對惡意變更的secret-key authenticator。

IPv4 跨兩個 Ethernet LAN 的完整路徑

無 NAT 的跨 LAN 封裝：每 hop 的 MAC 不同



A 對 next-hop 192.0.2.1 做 ARP，不在左 LAN 對遠端 B 做 ARP。

IP：192.0.2.10 → 198.51.100.20 保持；router TTL64→63，IPv4 checksum 與 link FCS重算。

固定無NAT/tunnel：A=192.0.2.10/24、MAC A；router R左介面192.0.2.1/MAC R_L，右介面198.51.100.1/MAC R_R；B=198.51.100.20/24、MAC B。A查DNS取得B地址，/24比較發現非local，route選next-hop192.0.2.1。

A沒有R_L的ARP entry時，向本LAN broadcast「誰有192.0.2.1」，R回答R_L；A送Ethernet dst=R_L/src=A，內含IP dst=B/src=A。不能在A的LAN直接ARP遠端B。router查route、TTL64→63並更新IPv4 header checksum，查/ARP右LAN B的MAC，再封裝dst=B/src=R_R，IP endpoints仍A→B。

B解封裝後交IP，再按TCP/UDP port/connection送給對應socket。ARP broadcast通常不由router轉送到別的L2廣播域；ARP回答也不是密碼學身分證明。

哪些 header 改，哪些通常不改

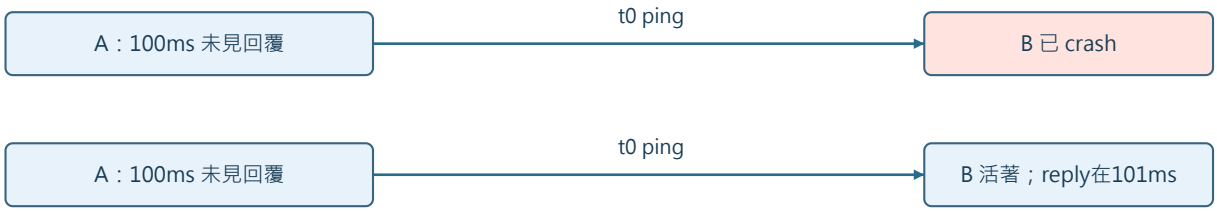
上例每hop Ethernet src/dst與FCS重建；IPv4 TTL/checksum變，無NAT/tunnel且無其他轉換時src/dst IP保持；TCP端點與payload保持。故不能說跨router時「只有Ethernet header變，其他headers全部不變」。若TTL到0，router不能繼續正常forward；錯誤回報也可能被網路遺失/過濾。

普通無tag EthernetII採6B destination MAC、6B source MAC、2B EtherType、46–1500B data/pad、4B FCS，從destination到FCS最小64B。data不足46需pad，但IP total length不把pad當IP資料。preamble/SFD在此frame長度分母外。

原創短TCP payload4B：IPv4 20+TCP20+data4=44B，需2B link pad，frame=14+46+4=64B；加8B preamble/SFD與本例96-bit IFG，共84 byte-times，在100Mbit/s占6.72µs。IFG是空閒時間不是附進IP packet的12B資料；本例不泛化到所有高速PHY的gap容差。

Failure detector：timeout 只能先產生懷疑

相同本地觀察，兩種不同遠端狀態



100ms觀察窗內，A 的 event history相同。沒有delay上界，timeout不是crash證明。

在沒有已知訊息／排程延遲上界的 asynchronous 模型，A 於 t=0 發 ping，t=100 ms 仍無 reply，不能分辨 B crash、link partition、message loss 或 B 只是很慢。兩條執行可讓 A 的本地觀察完全相同：① B 已 crash；② B 仍活但 reply 延到 101 ms。任何只依這段觀察便斷言 crash 的算法，都會在②誤判。

另考慮有界教學模型：B 持續運作，heartbeat 的**名義排程時間**每 H=1 s 一次，每次實際發送比其名義時間遲 0–s，s=.2 s；每筆訊息都送達，單向延遲在 0–δ，δ=.1 s。接收處理與計時不增加延遲或漂移。前一筆可在名義時刻立即到達，後一筆最晚在下一名義時刻再加 s+δ 到達，故到達間隔至多 H+s+δ=1.3 s；從上一次到達開始的 1.5 s timeout 在此模型不會誤報活節點。若發生丟包、partition、延遲超界或計時假設不成立，便不能套用此保證。

Reconfiguration：選新 leader 還要擋舊 leader

A 原持 shared storage 寫權；partition 後 B 因 timeout 自稱新 leader，而 A 仍在運作，雙方寫同一份資料會造成 split brain。心跳與本地宣稱「我是 leader」不能保證 mutual exclusion。

教學 fencing 模型中，協調服務正確核發單調 epoch；storage 安裝 currentEpoch=43 後才讓 B 操作。**每筆 write 的 token 驗證與其提交，必須和 epoch 安裝在同一線性化順序中原子執行，或由等價序列化機制保證。** 若 A 的 epoch 42 write 在安裝 43 前已線性化提交，它屬於舊時期；若排在安裝 43 之後，儲存端必須拒絕。只在提交前某個時點檢查一次並不足夠：舊寫入可能先通過檢查，延後到安裝新 epoch 後才落盤。

此保證還要求每條寫入路徑執行驗證、核發與安裝正確，不能讓不同 partition 各自遞增數字就宣稱取得合法權限。經確認的硬體或存取隔離也可擋舊節點。恢復 A 時先同步已提交狀態、成員／epoch 與未完成操作，再作 follower；request id 或 generation 可防止重啟後錯配舊回應。

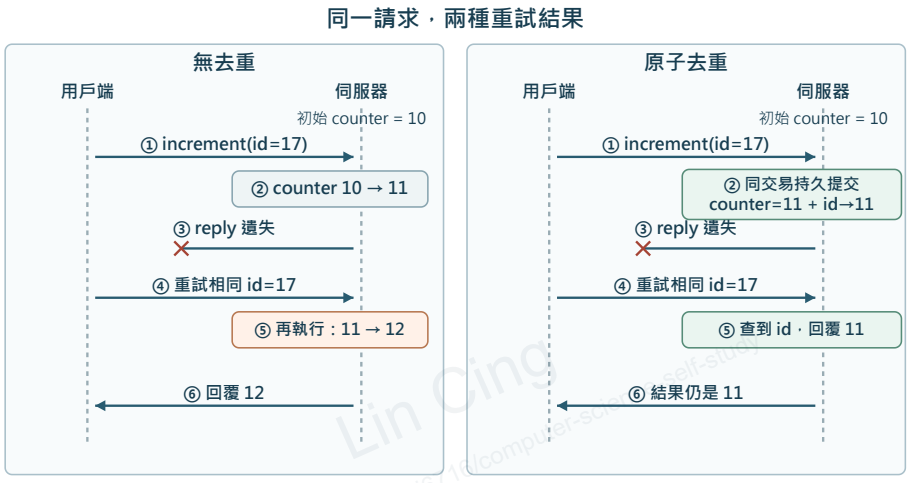
RPC retry：遺失 reply 不等於沒有執行

原創counter=10，client呼叫increment(id17)。server改成11後reply丟失；client timeout重試若直接再執行變12。TCP重傳提供同connection的byte交付，不替應用跨重連去重，不能由用了TCP推exactly-once RPC。

明確的教學at-most-once模型：server交易中查durable table[id]，若存在回保存result；否則把counter++與table[id]=result**原子持久提交**再reply。重試id17得到11而不再加。若先改counter後crash、尚未記id，重試仍可能加第二次；若先記id後未改counter，又會遺失效果。

此模型保證在table與業務狀態同交易/恢復域、id唯一且未過早忘記的前提下最多提交一次。client在network永久中斷時仍可能不知道結果；發電郵/外部付款不在同交易內時，需要額外協定，不能直接沿用本例結論。

RPC 重試：回覆遺失與去重



前提：ID 在重試範圍內唯一且記錄持續保留；外部效果不屬此交易，須另設協定。

同一事務持久提交計數更新與 `id→result`；重試沿用唯一 ID，伺服器返回已存結果。去重表必須保存至所有可能重試結束；郵件或付款等交易外效果需要額外協定。

Fault tolerance 的容量與故障域

兩台active-active servers各可100 requests/s，平時合計負載160；失一台後容量剩100，至少60/s不能穩態消化。只有replica可接手不代表failover仍達原SLO，必須預留容量、load shedding或降級。

stateless worker容易重啟，但輸出副作用仍可能重複；stateful service還要log/recovery/replication/鎖。RAID處理部分disk failures，不等於跨站點服務可用或logical corruption有backup；cluster/replica需要配合獨立故障域與一致性機制。

recovery先重建durable committed state，辨識/回復in-flight operations，再重新接流量。不能把「還沒收到commit reply」一律當成「未commit」，重試策略須依查詢/去重契約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transparency 與 scalability 要寫可觀察規格

location透明隱藏在哪裡，migration透明讓搬動後名稱/操作仍可用，replication透明隱藏多副本；failure透明不能無限制隱藏永遠不回覆的remote operation。user mobility還需身分、home/environment與應用session一致，不只是同名帳號存在。

原創資源上限：每worker100requests/s，10workers共1000；metadata coordinator每request花2ms CPU、只有一core，上限500/s。第11個worker不提高這個500/s界；cache/批次/partition metadata若改每request資源量才可能解除瓶頸。增加replicas的同步/重建traffic也要算入容量。

HDFS歷史案例固定Hadoop1.2.1：NameNode管namespace與block locations，DataNodes保存/傳輸blocks，client資料路徑不經NameNode。大block/streaming/較簡化的寫入模型適配大量資料，不能把它當作完整POSIX共享檔案系統或任意低延遲交易資料庫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

HDFS 1.2.1 的明確副本與資料流模型

本例自訂block64 MiB、replication factor3；檔案130 MiB分64、64、2共3blocks，logical130、payload副本總390 MiB（不含metadata、checksum、padding/配置額外）。最後2 MiB不是強制占滿64 MiB資料流；metadata仍須為該block定位。

client向NameNode取得block與DataNode位置，再直接向DataNode讀；寫入可沿DataNode pipeline複製。DataNode heartbeat/block report讓metadata知道狀態，缺副本時安排re-replication；失效偵測仍有延遲，重建也消耗network/disk。

本版本的Secondary NameNode主要協助checkpoint，不等於自動接替active NameNode的hot standby。副本分散到故障域能改善data availability，但NameNode/metadata與client可達性仍要納入，不由factor3推無條件高可用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS naming：路徑、穩定身份、位置分三層

location transparency是名稱不暴露物理位置；location independence是搬位置後名稱仍不需改。`serverA:/x`把server編進名稱；mount遠端目錄可讓client路徑較透明，但各client可掛到不同位置；global namespace則再統一命名/管理規則。

原創兩級映射：`/course/a.txt`→FID(volume7,file12,generation3)；volume7→serverA。搬volume到serverB只改第二層，不改path/FID。file12刪後再配置需換generation，否則舊FID可能錯指新檔。快取path→FID與volume→location有不同失效條件，不可一起永久保留。

replicated/cached location map更新仍需一致性/版本/失效規則；多一層indirection沒有消滅更新問題。diskless boot在DFS client尚未載入前，需firmware/初始boot protocol先取得kernel，再啟DFS；不能用尚不存在的DFS程式去載入自己。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Remote service 與 cache 的完整成本例

100次4KiB reads；無cache每次remote cost5ms，共500ms。現在10miss各5ms、90hit各.1ms，共50+9=59ms；payload network bytes10×4KiB=40KiB，相對無cache400KiB。這個例已指定hit/miss與remote cost，未把實測工作負載命中率當作定理。

若每次還要1ms server validation，總另加100ms→159ms；「有cache」不自動避開network round trip。cache granularity大可能read-ahead改善後續hit，也可能單byte讀造成大量多餘transfer；cache總容量固定時，大單位還會減少可存blocks。

client memory cache快但斷電易失；disk cache能保存已durable內容，仍需恢復metadata與驗證新鮮性。server自己也有cache；client寫到server RAM不等於已到stable storage。write-through/write-back描述何時向下一層推送，需另定那一層返回的持久性。

Cache consistency：以時間線說出保證

server新版本與client cache可見性是不同時間線

只定期驗證會有stale窗口；強callback/lease模型須在write完成前收回衝突讀權。

client-initiated：C1在t0讀v0並驗valid；C2於t3完成v1；C1若只在t10再驗，t4仍可能讀cache v0。這不是「每read都看最新提交」。在open才驗，則必須界定同一open期間的可見性；在每access驗則增加network成本，且validation與實際read仍需協調避免競爭窗口。

server-initiated：server追蹤誰持cache/何種讀寫權，以callback失效/回收許可，或衝突時停cache轉remote service。若要強一致教學模型，write完成前須確認所有衝突read權已收回/失效，或其受控lease已到期；partition時不能一邊讓舊client無限讀舊值、一邊宣稱新write已對所有人立即可見。

書中AFS/OpenAFS的callback避免每次都驗server，但應用已讀進自己的buffer不因callback自動改值；save/close與cache manager取新資料的時點影響使用者觀察。不同DFS的session/close-to-open/更強語義不能互當同義詞。

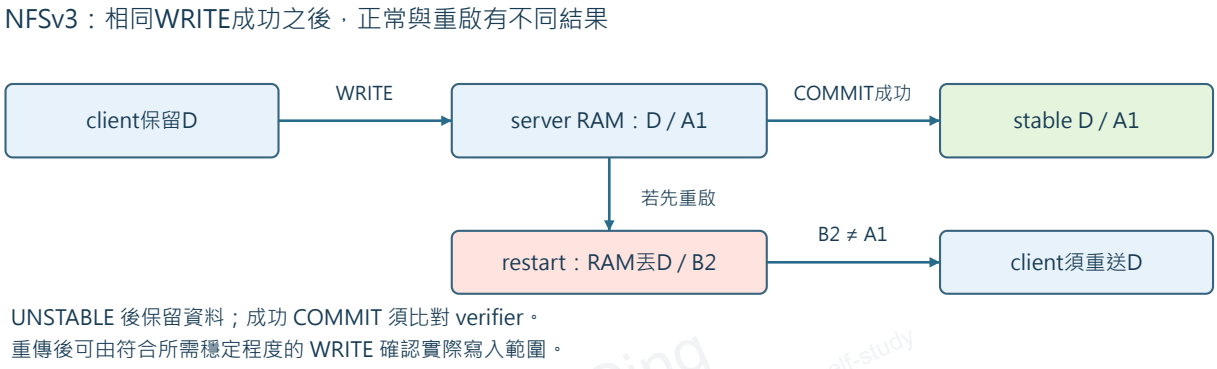
NFSv3：stateless 不表示沒有 cache，也不表示每次 WRITE 已落盤

固定RFC1813 NFSv3：READ/WRITE攜filehandle、offset、count等，核心檔案I/O不依賴server為每次open保存同等session state；server仍可有cache、duplicate-request state，其他鎖/協定另有責任。client permission/authentication與filehandle生命期仍要處理。

WRITE要求/回報有UNSTABLE、DATA_SYNC、FILE_SYNC等穩定程度；client不能把UNSTABLE成功當作所有資料已durable。延遲寫入若需後續COMMIT與verifier比對，必須保留可重送資料。weak cache consistency attributes有助判定變更，不單獨保證所有clients嚴格線性一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFSv3 unstable-write 的重啟 trace



原創4KiB資料D：t0 client送WRITE(offset0,D,UNSTABLE)，server RAM接收，回committed=UNSTABLE、verifier=A1；client保留D。正常路徑t1 COMMIT成功且verifier仍A1，server已按契約保存到stable storage，client才可清這批重送狀態。

故障分支：t0之後server重啟丟RAM未提交D，新的service instance用B2。client後續COMMIT/WRITE看到B2≠A1，不能假稱原UNSTABLE資料已保留；需重送尚未保證stable的D，再取得相應穩定結果。不是簡單地「COMMIT回success就不必看verifier」。

這個協定保障依賴client仍有重送資料、server遵守stable/verifier契約及正確處理範圍；不證明不受限concurrent writers會有應用想要的交易語義。durability、cache visibility與RPC去重是三個不同問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把一個遠端操作接回所有層

service name→DNS address→route next hop→ARP MAC→link frame/IP packet/transport endpoint→RPC id→服務授權與durable commit→client cache。每層的成功只支持該層契約：能送frame不代表RPC完成，收到TCP bytes不代表業務只執行一次，NFS UNSTABLE不代表落盤。

分散式系統的「透明」是在明列邊界內提供一致介面；latency、partial failure、資料副本與外部副作用不能用介面名稱消除。正確設計要在觀察不足時保留unknown/suspected，而不是把timeout當作邏輯證明。

速度用serial/parallel/coordination拆分；網路用bytes/bits、headers、RTT與window對分母；可靠度先畫故障域；RPC重試以request id與原子記錄推導；DFS把name/object identity/location、visibility/durability分開。

一般保證取決於允許的state transitions：heartbeat在無界delay不能準確斷crash，fencing依賴資源端強制epoch，at-most-once交易依賴去重紀錄不被過早忘記。原創練習把反例、正常路徑與重啟後中間狀態都列出，讓讀者可逐步驗算。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 17. RFC 1034, RFC 826, RFC 1813. Hadoop 1.2.1 HDFS. OpenAFS.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

把一個遠端操作接回所有層 （續）

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：一個以 UNIX 傳統為基礎的核心，如何組合程序、資源與模組介面？

前置知識。 process/thread、virtual memory、檔案系統與 module。

- 區分歷史 Linux 設計與現行系統
- 追蹤 fork/exec、module load、page cache 路徑
- 分析書中 CFS 與其前身、page reclaim 及 file system 的版本限制

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel、system libraries與distribution不是同一層

本章固定Linux v3.2核心，依OSC9e的歷史案例讀機制。核心提供task、virtual memory、VFS、IPC、network與drivers；GNU工具、shell、libc與套件管理器大多在user space。distribution把核心、工具、設定與安裝/更新流程整合成產品，不能從distribution名稱反推所有kernel設定。

固定兩台教學機均用v3.2、同一x86 ISA，執行的ELF檔案含`PT_INTERP=/lib/ld-linux.so.2`。A有此interpreter，B沒有。兩者都找得到可執行檔，但B會在取得interpreter時失敗；這不表示「同一kernel version保證所有user binaries可執行」。靜態連結、動態連結、ISA與ABI、library symbol版本是不同相容性條件。

元件	執行權限/責任	失敗的可觀察結果
kernel image及其CONFIG	核心模式；提供syscall與裝置支援	缺driver可使裝置無法使用
ELF interpreter/libc	通常user mode；載入共享libraries與API包裝	interpreter缺少可使exec失敗；symbol缺少可使loader失敗
shell/utilities	user processes；解析命令與使用服務	某命令不存在不代表kernel沒有fork/read
distribution policy	套件與預設設定的整合	不同設定可產生不同權限/啟動行為

Kernel、system libraries與distribution不是同一層 （續）

Linux從1991年的初期核心逐步擴充至SMP與多種架構；可載入模組、細粒度鎖與標準介面回應了硬體多樣性、多人多工及相容性需求。歷史脈絡解釋設計取捨，具體行為仍由所指定原始碼與設定支持。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Modular monolithic的邊界在哪裡

Linux把scheduler、memory、VFS及多數drivers放在同一kernel protection domain。模組可獨立載入，但不是各自有硬體隔離的user servers；合法function pointer呼叫即可進入driver，不因跨module就必須切換address space。

固定單一同步buffered read：user以syscall進核心，VFS呼叫filesystem，再呼叫block/driver。只跨user/kernel邊界會發生mode transition；核心內函式呼叫不等於mode switch。若等待I/O時scheduler改跑另一task，才另有context/address-space切換，不能把所有read都固定計成同一個切換數。

隔離與維護性的取捨由此產生。一個driver用錯pointer可破壞同kernel內的資料；module ABI、型別檢查或簽章驗證不會把它變成另一個故障隔離domain。反過來，將檔案服務移到user process可利用page-table protection，但必須設計IPC、資料傳輸與server失敗後的協定。第2章用固定配置比較兩種read路徑，本章則把Linux核心內部的物件接起來。

Linux的UNIX/POSIX介面讓程式操作file descriptors、processes與signals；API相容是可觀察語意的契約，與核心內部是否採某一份資料結構不同。相同read API可以由磁碟、pipe或socket提供，各自的blocking、seek與EOF條件仍不同。

模組載入是一個資源交易

以下是自行設計的「最後發布」協定，並非 Linux 3.2 對所有註冊入口的保證。固定 driver module M 需要exported symbols A/B並註冊device D。載入依序進行格式/版本檢查、配置與relocation、解析允許的symbols、執行init、公布註冊結果。若只有A可用而B未解析，不能公布一個已可呼叫D的半成品；若init取得buffer後註冊D失敗，須回收buffer與先前資源。

階段	M的可見性	取得的資源	失敗出口
resolve前	不可供一般呼叫	載入區域	驗證失敗即拒絕
resolve A/B	尚未LIVE	relocation後的code/data	未解析symbol時釋放載入資源
init	尚在建立服務	queue、buffer；尚未發布 device registration	反向撤銷已成功取得者
LIVE	新呼叫可取得合法引用	已註冊服務	每個呼叫遵守生命週期

Linux 3.2 的部分註冊入口可在 init 返回前對外可見；驅動必須在註冊前完成外界可見狀態的初始化，並在 init 失敗時撤銷入口、同步並行呼叫及回收資源。上表的「init」在本模型中僅指私下準備、尚未對外發布的階段。

M與kernel同權限。signature enforcement依版本與CONFIG決定，不能當成Linux3.2所有模組必備的前提；它與runtime引用安全解不同問題。

為何reference count等於0仍需停止新呼叫

固定安全卸載模型，每個open handle及每個queued request各持有一個M引用，初始3個handles+2個requests=5。狀態變化如下，表中的引用政策是本driver的明定規則。

階段	可新增引用	handles	requests	refs	code可釋放？
LIVE	可以	3	2	5	否
GOING/撤銷新入口	不可以	3	2	5	否
關閉3個handles	不可以	0	2	2	否
兩筆I/O完成或合法取消	不可以	0	0	0	尚須確認所有callback已離開
callback/worker皆靜止	不可以	0	0	0	才可釋放

若先看到refs=0，再撤銷入口，另一CPU可能在兩步間取得引用並開始執行即將被釋放的code。須由module狀態/註冊協定阻止新進入，再完成既有引用與工作。使用RCU的callback要等其相應grace period；使用workqueue、timer或DMA者要完成各自的同步取消，不能宣稱所有module等「一次RCU」就安全。

fork 複製哪一層：task、mm、頁表與資料頁

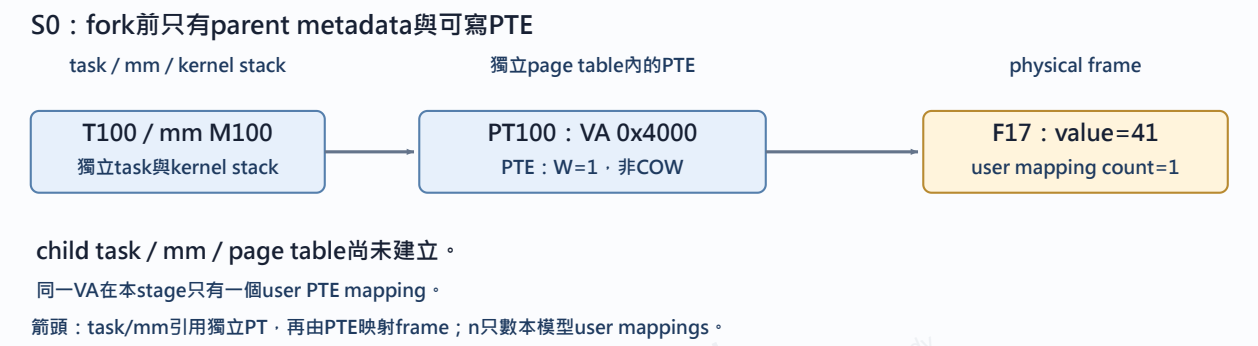
Linux 3.2 的 `copy_process()` 呼叫 `dup_task_struct()`，先配置 child 的 task 與 kernel stack，再初始化 PID、credentials、scheduler state 與其他 execution context。一般 `fork()` 未設定 `CLONE_VM`，`copy_mm()` 走 `dup_mm()`：child 有新的 `mm_struct` 與頁表。`copy_page_range()` 可以讓 private user pages 先共用 physical frame 並寫入保護；某些 VMA 的頁表複製可以延後到 fault 建立，並非所有頁表項都必須立即複製。write fault 需要隔離時才複製；若匿名頁已可安全獨佔，核心可能重用原頁並恢復可寫。整條路徑沒有「等到第一次寫 task metadata 才複製 `task_struct`」的步驟。

層次	fork 後父子關係	首次 private write 的作用
task_struct／kernel stack	不同物件，各有 PID、排程狀態與核心 stack	不以資料頁 COW 延後建立
mm_struct／頁表	一般 fork 為不同物件	child 更新自己的 PTE
適用且已駐留的匿名私有頁	可共用同一 frame；兩個 PTE 暫時禁止寫入	write-protection fault 需要隔離時才分離複製；安全獨佔時可重用
open file description	對應 descriptor 可以仍指向同一物件	offset 可共享，與 user page COW 不同

fork 複製哪一層：task、mm、頁表與資料頁 （續）

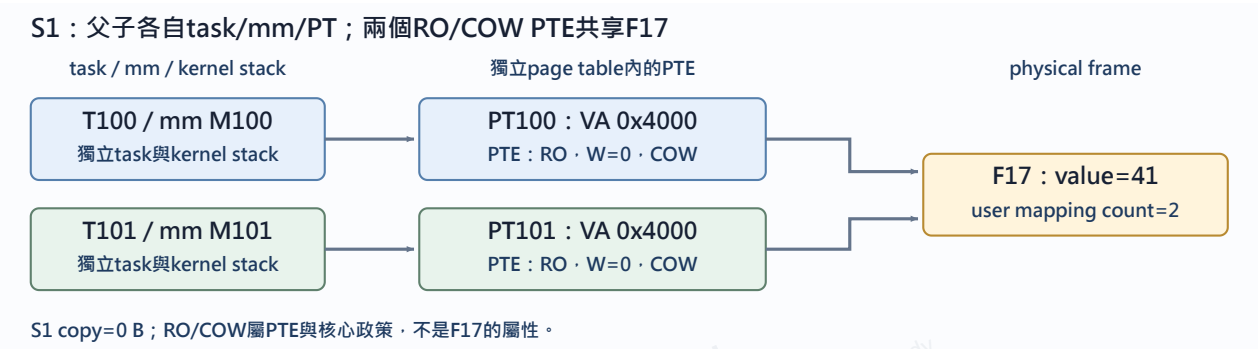
`clone()` 是資源分享的參數化入口。`CLONE_VM` 讓兩個 task 指向同一 mm；`CLONE_FILES` 共享 descriptor table；`CLONE_FS` 共享 filesystem context；`CLONE_SIGHAND` 共享 signal-handler table。這些 flags 可形成 thread-like task，但不能拿來證明一般 fork 共用 task metadata。

S0：fork 前只有 parent task



固定 parent 為 T100/M100/PT100，VA 0x4000 映射 frame17，x=41、W=1、mapping count=1；child task 尚未存在。此狀態定義後續比較的基準，不能先假設 COW 或 child metadata 已存在。

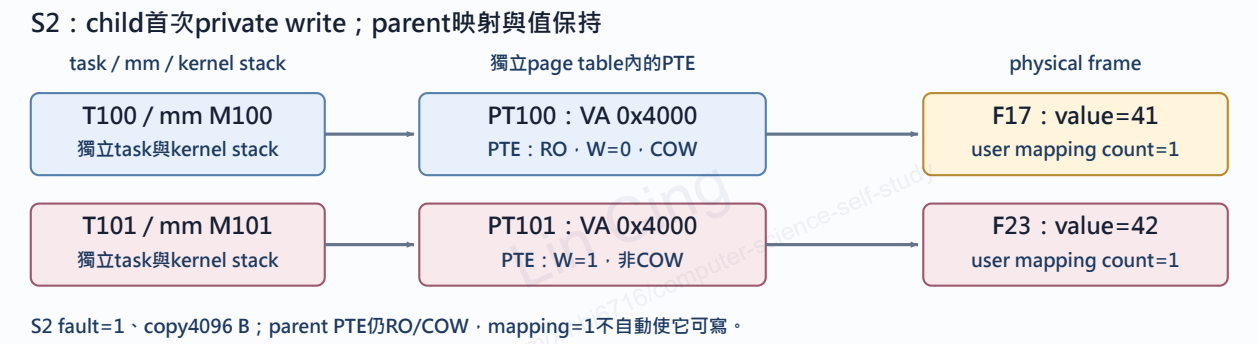
S1：fork 後 metadata 已分離，frame 仍共享



child 為 T101/M101/PT101，PID、task state、kernel stack、mm 與 page table 都是自己的物件。兩張 PTE 暫時都映射 frame17、禁止寫入並標記 COW；F17 內容仍為 x=41，user PTE mapping count=2，資料複製 bytes=0。

S2：child 第一次寫入才複製 private page

假設只有一個 4 KiB anonymous private page，VMA 邏輯上可寫；x 位於 VA 0x4000、頁內 offset 0，初值 41。無 huge page、額外 mapping、DMA 或同時寫入。表中 COW 是核心政策標籤，不宣稱 x86 PTE 有名為 COW 的硬體位元；計數只算此模型的 user PTE mappings，不等於 Linux page 的全部 kernel references。



S2：child 第一次寫入才複製 private page （續）

狀態	parent task／mm／PTE	child task／mm／PTE	frame 內容與 mapping 數
S0 fork 前	T100／M100／PT100: frame17,W=1	不存在	F17:x=41，n=1
S1 fork 後	T100／M100／PT100: F17,W=0,COW	T101／M101／PT101: F17,W=0,COW	F17:x=41，n=2；資料複製0 bytes
S2 child 寫 x=42	T100／M100／PT100: F17,W=0,COW	T101／M101／PT101: F23,W=1	F17:x=41,n=1；F23:x=42,n=1；fault1次、複製4096 bytes

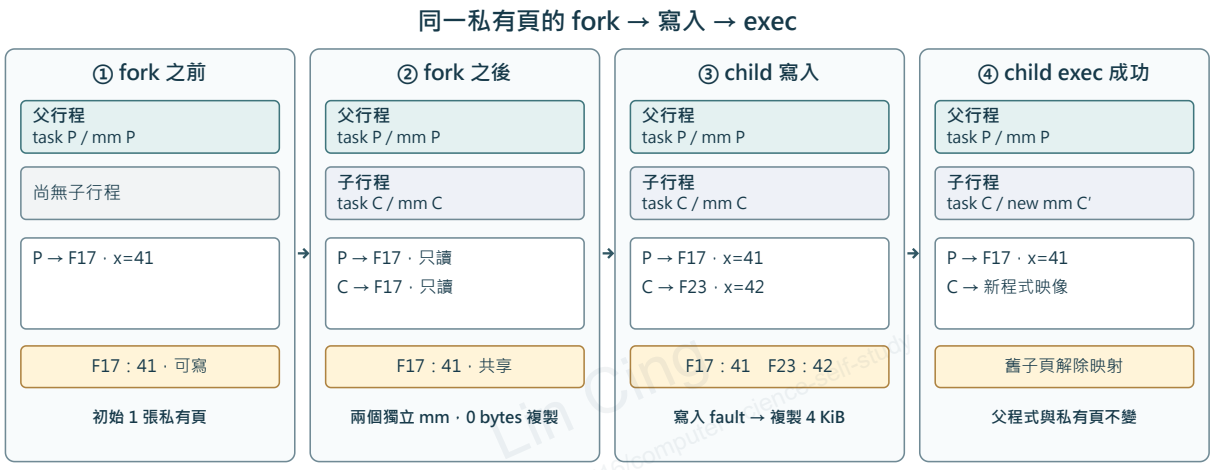
錯誤模型的可觀察反例與 clone 對照

若誤把 child task 設為 parent task 的同一物件，再把 child PID 改為101，parent PID 也變成101；兩個程序不再有獨立身分。正確模型中 child state 由 ready 改 blocked，parent state 仍是 ready。資料頁共享不能推成 task 共享。在本節single-thread child模型中，`exec()`成功後保持process PID而換入program image；一般在破壞舊image前失敗可回傳錯誤，但Linuxv3.2 `flush_old_exec()`之後已跨過point of no return，後續部分錯誤會送signal終止，不能無條件保證還原舊image。

`CLONE_VM` 是另一個明定的共享選項：兩個不同 task 可指向同一 mm／頁表。若初始 F17 可寫，child 寫42，parent 讀到42，而且不因本次寫發生 private-page COW fault。這個結果與上表一般 fork 的 parent=41、child=42 不同。

三態表與 CLONE_VM 對照只驗證本節固定模型的可觀察差異，不代表實際 Linux 核心測試。真核心行為仍須以對應版本的 source、fault trace 與 frame/PTE counters 驗證。

fork、寫入與 exec：先分清物件身分



固定模型：private anonymous page；無併發寫入、額外映射與 DMA

本例只有一個可寫的 private anonymous page。fork 後父子各有 task 與 mm，暫時共享 F17；當兩者仍共享時，child 寫入觸發 fault，複製後才寫 F23。成功 exec 替換 child 的程式映像，parent 保持自己的映像。

一般情況下，fault **必要時**才複製；若頁已可安全獨佔，核心可能重用。`MAP_SHARED` 不採本例的私有寫入語意；已開啟的 file descriptor 也可能繼續引用同一 open file description，與記憶體頁的 COW 是不同關係。

clone flags不是可任意排列的布林組合

建立模式	task/TID	TGID	mm	fd table	file descriptions
一般fork	新	新group	新mm，private pages可COW	複製table	原有fd仍引用同一descriptions
只加CLONE_VM	新	仍可新group	共享mm	可複製table	引用可共享
thread-like，含VM/SIGHAND/THREAD	新	共享thread group	共享mm	是否共享另看CLONE_FILES	依table entries引用

Linuxv3.2拒絕CLONE_THREAD卻沒有CLONE_SIGHAND，也拒絕CLONE_SIGHAND卻沒有CLONE_VM。因此一般pthread模型除了資源sharing flags，還要CLONE_THREAD等thread-group語意；只列VM/FS/FILES/SIGHAND四個flags就說成完全等同pthread，會漏掉TGID與signal/thread-group生命週期。

例如parent TID100/TGID100，普通fork child可為TID101/TGID101；同group thread可為TID102/TGID100。兩者都有獨立task與kernel stack，不能因CLONE_VM共享mm就把兩個TIDs當成同一個task物件。

CFS把實際CPU時間換成virtual runtime

固定Linux3.2的fair scheduling class。runnable entities依virtual runtime排序，權重較高者在相同實際時間內增加較少的virtual runtime，因此可獲較多CPU。以nice0的基準weight1024：

$$\Delta v = \Delta t \frac{1024}{w}.$$

兩個持續runnable tasks A(nice0,w1024)、B(nice5,w335)，固定目標period6ms、無其他負載。理想比例為A=1024/1359、B=335/1359；對應約4.521ms與1.479ms。兩者的virtual increments均約4.521ms，因B以較小weight換算。這是權重公平性的算式，不是宣稱每一個實際period都精確切成這兩段；tick、wakeup、min granularity及其他classes會改變逐段時序。

CFS的rbtree保存的是fair class的排序，running entity的入/出tree由調度路徑處理；不能把「最左者」簡化成每一時刻所有tasks都在同一tree。RT FIFO/RR有自己的priority規則，可優先於fair class；本章不把它套進同一個vruntime比較。第6章的HRTICK與periodic-tick兩份固定Gantt提供實際選取的中間值。

權重如何決定 CPU 份額與 vruntime

CFS 權重：從實際時間換算虛擬時間

單一 CPU，A、B 持續 runnable；理想共享期間 6 ms

總權重 1024 + 335 = 1359　　$\Delta v = \Delta t \times 1024 / 權重$

權重比例	實際 CPU 時間	虛擬時間增量
A：nice 0，權重 1024 1024 / 1359 ≈ 75.35%	6 × 1024 / 1359 ≈ 4.521 ms	6 × 1024 / 1359 ≈ 4.521 ms
B：nice 5，權重 335 335 / 1359 ≈ 24.65%	6 × 335 / 1359 ≈ 1.479 ms	6 × 1024 / 1359 ≈ 4.521 ms

這是理想份額的算術模型，並非每個排程週期的實際執行順序。

Linux 3.2 的理想模型中，兩者持續 runnable 且只考慮一顆 CPU。較小權重 B 的實際時間較少，以未四捨五入的時間經權重換算後的虛擬時間增量與 A 相同；實際排程還受 tick、喚醒與其他 class 影響。

local IRQ、preemption與SMP互斥各保護什麼

假設process context與本CPU的interrupt handler都會取得spinlock L。process持L時若未禁止該interrupt，handler可搶入並spin等待L；但持有者被handler打斷而無法釋放，形成同CPU自我死鎖。對這種共享關係須使用相應irq-save鎖定協定，離場恢復原IRQ狀態，不能一律無條件開interrupt。

機制	能阻止	仍不能阻止
禁止本CPU preemption	scheduler切換本CPU的task	硬體IRQ、其他CPU並行
禁止本CPU普通IRQ	對應本CPU IRQ handler搶入	其他CPU、NMI
atomic spinlock	遵守同鎖協定的其他CPU/contexts	未用同鎖者、錯誤DMA與不合規存取
irq-save spinlock	同鎖SMP互斥及指定本地IRQ重入	所有未納入協定的其他機制

不能睡眠的critical path不應取得可能阻塞的mutex；spinlock持有區應短，否則浪費其他CPU。NMI與不同softirq/interrupt配置須另作協定分析，不能把教材的優先層圖當作涵蓋所有kernel configuration的總排序。

固定x86-32配置後，zones才有具體位址

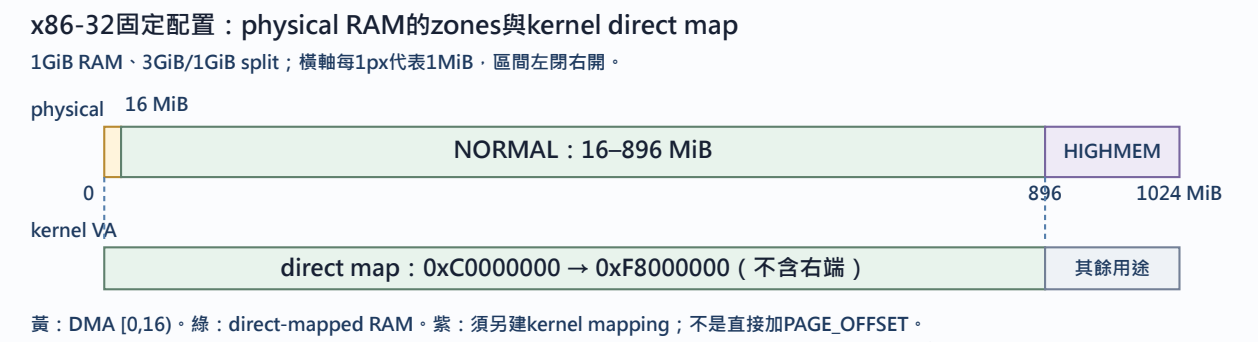
本例固定Linux3.2的x86-32、4KiB pages、3GiB user/1GiB kernel split，正常direct mapping覆蓋前896MiB physical RAM，示範機有1GiB RAM。這是此配置，不能搬到所有32/64-bit Linux。zones表示位址coverage，不表示所有頁都free；kernel與reserved pages仍占用其中空間。

physical interval	zone	用途/限制
[0,16MiB)	DMA	支援特定legacy ISA DMA位址限制
[16,896MiB)	NORMAL	一般可由kernel direct map存取的RAM
[896MiB,1GiB)	HIGHMEM	不在永久direct map，kernel需適當暫時映射

normal physical `0x01002000`可在這個模型以PAGE_OFFSET `0xC0000000`相加，得kernel VA `0xC1002000`。highmem physical `0x39000000`不能因此逕稱kernel VA=`0xF9000000`：那段kernel VA有其他用途，須先建立合法映射。user PTE仍可映射highmem frame；例如VA頁0x40000映至PFN0x39000，存取VA0x40000123得到PA0x39000123。

physical page allocator依zone/order約束配置；buddy處理連續power-of-two pages，slab再管理小物件。裝置能用的DMA位址還取決於device mask、IOMMU/bounce與DMA API，不是所有DMA都只能用前16MiB。

固定x86-32配置後，zones才有具體位址 （續）



ELF的file bytes與zero-fill不能混算

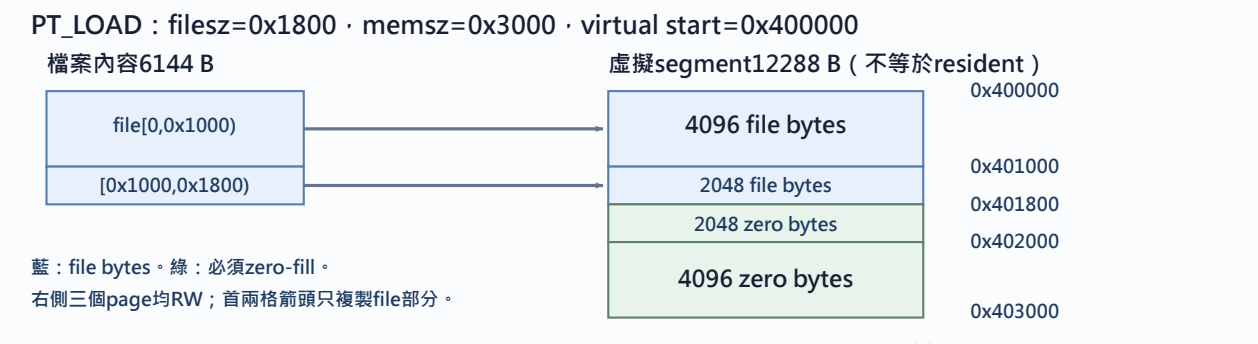
本段另採具NX支援的x86-64 ELF權限模型；地址只是例題指定值，與前頁x86-32 zones分開。固定一個PT_LOAD segment：p_offset=0、p_vaddr=0x400000、p_filesz=0x1800、p_memsz=0x3000、權限RW。p_memsz大於p_filesz的尾部必須讀回零，而非後續檔案垃圾。

virtual page	bytes來源	權限
[0x400000,0x401000)	file [0,0x1000)	RW
[0x401000,0x402000)	file [0x1000,0x1800)，再zero 0x800 bytes	RW
[0x402000,0x403000)	zero-fill 0x1000 bytes	RW

合計file-backed內容6144bytes、zero尾部6144bytes、virtual segment12288bytes；建立這個VMA不表示三頁都已resident。read-only code可由另一個RX PT_LOAD描述，private writes再依COW處理；由此連接18.4的fork與第9章fault path。

動態ELF的PT_INTERP指定user-mode interpreter。kernel建立program/interpreter mappings與初始stack並交出入口，動態linker再解析DT_NEEDED與symbols。不能把所有動態連結說成kernel為每次函式呼叫找symbol，也不能要求所有共享libraries永遠映到同一個VA。

ELF的file bytes與zero-fill不能混算 （續）



Page reclaim：保留後備資料與存活引用

回收時須分開追蹤映射、backing、dirty 狀態與是否可回收。若 CPU 或裝置仍有需要該 frame 的有效引用，就不能重用。乾淨的 file-cache page 只有在後備資料仍有效時才能重建；dirty page 可能是最新內容的唯一副本，必須完成所需 writeback，才能將其涵蓋的狀態標成 clean。

回收協定還須移除不安全的映射與使用者，並釋放必要引用；應用程式暫時沒有執行，不代表頁面不再被引用。writeback 完成也不自動等於整個檔案系統交易已持久化。

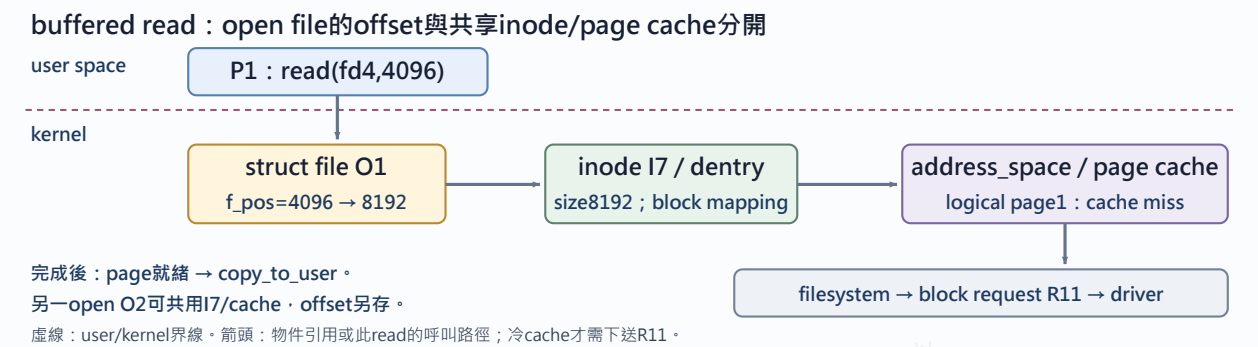
VFS四種物件接上buffered read

固定Linux3.2、一般ext3 buffered read、file size8192、page4KiB、P1 fd4已成功open；fd4指struct file O1，O1.f_pos=4096，引用檔案dentry與inode I7。I7的address_space管理共享file pages，superblock表示所屬filesystem。本例另假定讀取完整成功、資料可用、copy_to_user全部成功，且沒有並行 truncate 或共享 offset 的變更。

物件	保存/連結	不應混入
superblock	filesystem instance與operations	每次open的offset
dentry	parent/name與inode關係，可含negative lookup	檔案的唯一名字假設
inode	身分、mode、size、block mapping、address_space	所有open共用的current offset
struct file O1	本次open的f_pos、flags、operations與path引用	每個fd都必然獨立offset的假設

P1 read4096先以fd取得O1引用，驗證操作與user buffer，再走file operations；若logical page1不在page cache，filesystem把它的blocks送到I/O，等待page就緒後copy到user，O1.f_pos=8192。另一個獨立open有O2，但可共用I7的同一page cache；dup(fd4)則共用O1。cache hit可省block I/O，不省所有查找/copy成本。

VFS四種物件接上buffered read （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ext3、block groups與procfs的語意邊界

ext3以inode、data blocks、bitmaps與block groups配置資料，嘗試維持locality；鄰近配置是政策目標，不能當作「同一file永遠連續」的正確性不變量。檔案增長或碎片可迫使新block落到其他位置，索引仍須正確。

固定 `data=ordered` 的教材模型：write 6000 bytes 使兩個 page-cache pages 變 dirty，且此次 metadata transaction 要公開新配置的 block mapping。相關資料在公開該 mapping 的 metadata commit 前須按此模式的排序要求處理，以避免新 metadata 指向未就緒內容。普通write成功不等於fsync成功，journal的metadata原子性也不保證所有in-place user data修改都原子。`data=writeback`或`data=journal`是不同模式，不能共用同一句持久承諾。

procfs的`/proc/123/status`在read時由kernel狀態形成文字，不一定對應一塊持久disk data。PID123會在process結束後被重用，因此稍後同一路徑未必是同一個process；系統監測需結合啟動時間或其他身分證據。多個proc檔案分開讀取也不自動形成全系統同一時刻的snapshot。本章不採「PID固定16bit、inode高16bit永遠編碼PID」作Linux3.2通用規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

block、character與network三條I/O路徑

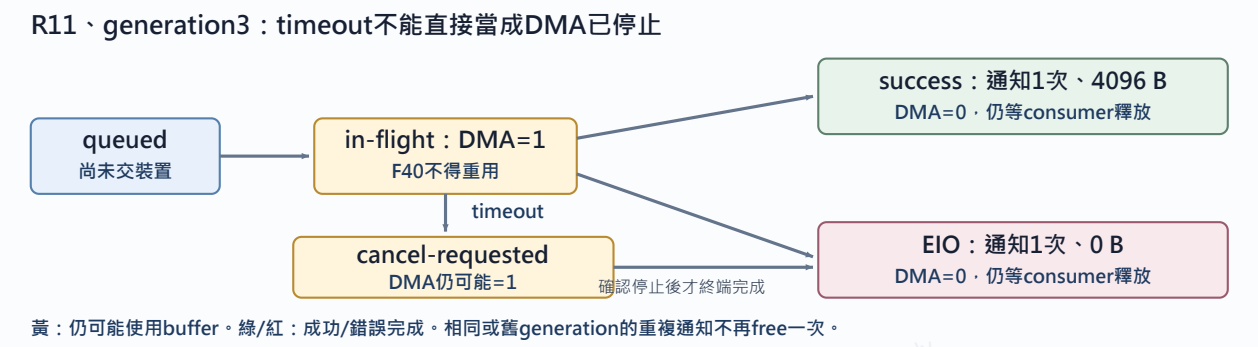
block device提供依block/sector定位的資料；filesystem透過block mapping及request layer送到driver。character device以該driver的file operations定義串流或其他操作，不保證都可seek。network device經socket/protocol stack送packet，不等同把所有網卡都當作可任意seek的device file。

固定同步read一頁、cache cold、user buffer已resident、無read-ahead，request R11指向一頁kernel buffer F40。state依序queued→in-flight(DMA持有F40)→terminal success或terminal error；完成後才能發布結果並按生命週期解除mapping。實際block layer可merge/split requests，本例一頁一request只是一個固定輸入。

結局	device/DMA證據	終端callback	app結果	F40何時可重用
success	I/O完成、ownership歸還	恰1次	本例讀4096bytes	copy/引用結束後
error EIO	本request已停止或失敗完成	仍恰1次	沒有先前成功bytes時回錯誤	DMA已停且引用結束後
timeout但裝置仍可能工作	尚未證明停止	不能先當成功/安全釋放	等取消/重置協定	尚不可

錯誤完成也是完成，不能把EIO當作「completion次數不增加」。反過來，timeout不是裝置已停止的證明；若過早free F40，遲到的DMA會改壞新owner資料。

block、character與network三條I/O路徑 （續）



duplicate completion與reset的狀態機

本例以`(request id=11,generation=3)`識別一次提交。第一個合法terminal事件以受保護狀態把in-flight→terminal並通知一次；再來同generation的重複事件只記錄/忽略，不再通知或free一次。下一次重用id11時generation=4，舊generation3事件不能完成新request。

這個generation只處理**軟體事件身分**，不是停止硬體DMA的替代品。reset/cancel路徑還須確認裝置不再使用舊DMA address，完成相應同步後才能解除mapping、回收buffer。driver需按裝置協定處理late interrupt與error，不可只靠一個timeout數字判定安全。

user buffer可被釋放的時點也依同步/非同步API不同：同步read返回前copy已完成；非同步提交後則要等completion契約。request queue、memory ownership與通知次數須連在同一張狀態表上。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

pipe的EOF由所有writer引用決定

固定pipe含read description R、write description W；parent fd3→R、fd4→W，fork複製descriptor table後，child fd3/fd4仍引用同一R/W objects。parent將讀交給consumer、child負責producer時，必須關閉各自不用的端點。

事件	parent writer	child writer	buffered bytes	read端結果
fork後	1	1	0	阻塞或nonblocking EAGAIN，並非EOF
child寫ABC、關fd4	1	0	3	可讀ABC
consumer讀完	1	0	0	仍未EOF，因parent誤留writer
parent關最後writer	0	0	0	後續read回0即EOF

同一stream保留bytes次序，但多個writers的操作原子性受PIPE_BUF與write大小/模式限制，不能從「是pipe」推出任意長write永不交錯。shared memory只共用資料，不提供queue/互斥；需另選locks、semaphores或atomic protocol。

signals、wait queues與原子semop

standard signals在blocked期間同一signal多次產生可合併成一個pending事件；不能用它當可靠計數器。real-time signals可排隊，sigqueue等介面可帶受限payload，故不能說「所有signals都不能傳資料」。處理函式可用的操作還受async-signal-safety限制。

kernel內等待I/O常用condition加wait queue。若consumer先查done=false，producer立刻設done=true並wake，consumer卻之後才入queue並sleep，就可能漏喚醒。正確等待協定先建立可被喚醒的等待狀態，再重查condition，若已true就不sleep；被喚醒後仍重查，因其他waiter可能先消耗資源。wake把task變runnable，不保證它立刻執行或condition永久成立。

System V semop可把同組多個semaphore操作合成原子批次。初值S0=1,S1=0，請求`S0-=1;S1-=1`在兩者都足夠前不部分扣S0；若先把S0扣成0再等待S1，就不是此原子批次語意。IPC權限、資源上限、刪除與錯誤返回仍需明定，不能只列介面名稱。

從socket到wire，資料量的分母各不同

固定IPv4/TCP、無options、MTU1500、無TSO/GSO等offload；application成功send6000bytes。MTU扣除標頭後容許最多1460 B，但實際分段還受對端MSS、路徑MTU、排程及窗口等影響。本例另指定有效MSS恰為1460 B、無重傳且先送四個滿載段與一個尾段，wire上的五個TCP payloads為1460、1460、1460、1460、160，合計6000；五組IP/TCP headers共200bytes，L3 transmitted bytes=6200，尚未計Ethernet overhead、ACK或重傳。

kernel socket buffer/sk_buff、TCP send queue、IP routing與network driver各保存不同狀態。若啟用segmentation offload，一個較大sk_buff仍可在硬體分成多個wire packets，因此sk_buff數不能直接當wire packet數。send成功只承諾API已接受的bytes，不等於remote app已讀、對方已ACK，更不等於磁碟持久。

routing例子：目的10.1.2.3，同時存在default /0、10.0.0.0/8、10.1.0.0/16及10.1.2.0/24，最長匹配是/24。FIB實際資料結構、route cache與firewall hooks依版本/CONFIG；本章保留可核對的longest-prefix語意，不把舊書某一hash/cache布局宣稱為所有Linux版本。

queue、retransmission與延遲的證據

固定一筆量測trace，以enqueue時刻0為基準：dispatch=2ms、device transmit完成=3ms、所需ACK到達=13ms。因此queue等待2ms、已派送到transmit完成1ms、其後到ACK10ms，這是分段timestamp支持的結論。

只有queue length20、retransmissions3、device errors1三個聚合計數，不能推出「長尾就是queue造成」；重傳可能來自網路丟失、重排或其他原因，device error也未必與該TCP流一致。需要同一request/flow、同一時間窗的enqueue/dispatch/completion、drop與ACK evidence，或明確干預比較，才能分辨因果。

TCP按sequence numbers處理重傳與out-of-order bytes，供應成功交付的有序byte stream；UDP保留datagram界線但不保證交付。IP/driver queue本身不因上層TCP就成為不會丟失或重排的通道。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Authentication與authorization各回答一個問題

authentication確認宣稱的身分；authorization判斷該credentials對特定object/operation有何權限。PAM把authentication/account/session等政策交給user-space模組組合，kernel仍依其credentials執行檔案與其他資源的檢查。密碼salt讓相同密碼不必有相同儲存值，但不增加密碼本身entropy；hash的成本參數與存取保護是另一部分，本章不把舊UNIX八字元機制當現行建議。

Linux credentials包含real/effective/saved IDs、filesystem IDs、groups與capabilities等。對檔案的普通DAC，先選owner/group/other類別，再看requested read/write/execute；capability、ACL/LSM、mount條件可進一步改變結果。成功open還可能形成持續可用的handle，日後只改path mode並不等於撤銷所有既有handle。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

root並不等於「mode沒有write就一定被拒絕」

固定Linuxv3.2、使用 generic_permission 的非 immutable regular file，owner1000、mode0400、rw mount、無ACL及其他LSM拒絕；路徑 traversal 已成功，caller current_user_ns 與 inode_userns 相同。每次獨立要求單一 read、write 或 execute 權限。下表capability指在對應user namespace中的有效DAC能力，忽略其他額外政策。

caller	read	write	execute	規則
fsuid1000，無DAC能力	允許	拒絕	拒絕	owner bits=100
fsuid2000，無DAC能力	拒絕	拒絕	拒絕	other bits=000
fsuid0，有CAP_DAC_OVERRIDE	允許	允許	拒絕	能越過read/write DAC；regular file仍須至少一個execute bit
fsuid0，但無相關DAC能力	拒絕	拒絕	拒絕	UID數字本身不補上缺少的能力
fsuid2000，只有CAP_DAC_READ_SEARCH	允許	拒絕	拒絕	regular file只補read，不補write/execute

v3.2 `generic_permission()`先做一般檢查，失敗後對regular file檢查CAP_DAC_OVERRIDE；涉及execute時還要求`S_IXUGO`非零。若mode改0401，有DAC_OVERRIDE者的execute DAC可通過；是否可真正執行仍受格式、mount noexec與其他政策限制。這直接反駁「root write因mode未開而必deny」，也避免把root說成無條件穿過所有安全層。

暫時降權與永久降權的差別

固定setuid-root程式的IDs為real1000、effective0、saved0。`seteuid(1000)`後real仍1000、effective1000、saved0；之後可依saved-ID規則恢復effective0。這是暫時放棄有效UID，不是永久移除全部特權。

若要永久降權，還須按相應API處理saved IDs、groups與capability sets，並關閉不應保留的privileged handles；僅改euid不能撤銷早已開啟的file descriptors。具體實作還受securebits、user namespaces與程式所需資源影響，所以本章表格只證明ID轉移，不把一段省略其他credentials的偽碼當成完整sandbox。

把read的一次成功與一次失敗接回同一機制

固定18.7的fd4→O1→I7、offset4096、read4096、page cache cold、user buffer已resident；沒有並行 truncate 或共享 offset 更新，I/O 和 copy_to_user 均完整成功。正向鏈為fd引用/操作檢查→page cache查找→filesystem mapping→R11 block I/O→device完成→page就緒→copy_to_user→O1.offset8192→回傳4096。等待中task可被切換；這與函式呼叫不是同一事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

錯誤完成、部分進度與物件生命週期

若R11以EIO終端完成且尚未copy任何bytes，read回錯誤，offset不因此前進；完成通知仍恰一次，buffer必須等DMA停止與references釋放才重用。若已有部分copy，API可能回傳成功bytes再於後續觀察錯誤，不能把所有read失敗都硬改成零進度。本例排除了先前成功bytes，才有確定的零進度結果。

fork共享資料頁不代表共享task，兩次open共享inode不代表共享offset，send接受bytes不代表遠端接收，write修改cache不代表持久；這些界線由物件身分、權限、生命週期與完成契約共同維持。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 18. Linux v3.2 fork, memory, module, DAC, ELF. fork(2), signal(7).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：分層 Windows 架構如何在 user/kernel 邊界上組織物件、程序、I/O 與服務？

前置知識。 OS 結構、protection domain、file/I/O。

- 辨識 user mode/kernel mode 元件
- 分析 object manager 與 handle 權限
- 分離 7 版本案例與當代實作

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows7是NT家族案例，不是所有Windows版本的總稱

本章採原書的Windows7／NT架構脈絡，區分user-mode subsystem／services與kernel-mode executive、kernel、HAL及drivers。不將早期DOS-based Windows、Mach microkernel或現代Windows新增功能回填為相同契約。

名詞	本章意思	不可直接推論
Win32 API	應用使用的程序／物件／I/O等介面	每個API都必然system call
NT kernel/executive	受信任排程、memory、object、I/O等實作	每個component獨立address space
compatibility	既有程式在指定環境可運行	所有架構／bitness／舊subsystem都支援
portability	讓硬體依賴集中且有移植路徑	每個driver binary跨CPU直接可用

歷史圖若出現OS/2、Win16或POSIX subsystem，須按其世代／版本理解，不能因此宣稱每一套Windows7都可執行所有舊binary。後續算例均明列執行、配置及故障模型。

從外部介面到受信任服務的追蹤

例應用ReadFile：library準備參數，受控kernel入口驗證handle與buffer；I/O manager／filesystem／cache與memory manager決定命中或送driver。若等裝置可使thread等待，完成後再依結果回到應用。

一個API名稱不足以決定是否阻塞、是否copy、是否有IRP或幾次context switch；fast/cached路徑可能不同。先記錄handle開啟旗標、檔案／裝置種類、cache狀態及完成模式，再比較路徑。

NT的模組劃分包含大量kernel-mode元件。稱它有layered design，不能推成每次file service都跨user-mode server或自帶獨立failure domain。

設計目標要連到具體強制機制

目標	可用機制	不能被目標名稱保證
security	tokens、security descriptors、handle granted access	kernel/driver沒有漏洞
reliability	保護邊界、reference lifetime、錯誤回報／恢復	一個錯誤不會影響共享kernel
performance	cache、async I/O、thread scheduling	全部工作都低latency
extensibility	object／driver interfaces	任意擴充都安全
portability	HAL、架構專屬層、較高階程式碼	移植零成本
internationalization	Unicode介面與地區資料	字元數等於byte數
energy／dynamic devices	power states、PnP生命期	移除裝置不會遇未完成I/O

HAL隔離interrupt／timer／平台細節的一部分；不是把所有硬體差異、memory model與driver責任消除。支援SMP也不使共享更新自動race-free。

Unicode、相容性與緩衝長度的真單位

Win32 的 W 介面使用 UTF-16 code units，A 介面涉及 code page 轉換；這些單位不能直接等同 UTF-8 bytes 或 Unicode scalar 數。

字串 A、U+1F600、B 編碼為 0041 D83D DE00 0042，共 4 個 16-bit code units、3 個 Unicode scalars、8 bytes 資料；若需終止 NUL，再加 2 bytes，共 10 bytes。4-byte buffer 無法容納完整資料與終止符。

以 wchar_t units 表示的容量不可傳入 byte 數；byte length 也不可直接使用 scalar 數。Unicode API 不保證所有檔名都已正規化成同一等價形式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

power／PnP的安全性依ownership，而不是只靠通知

固定狀態working→stop-pending→stopped→removed；這是生命期模型，不是完整PnP IRP表。stop-pending先擋新submit，再等待／取消既有requests並確認完成，才能釋放device context、DMA映射及buffers。

若仍有request引用context，先free後interrupt才完成，會use-after-free；若取消已提出但driver還沒終止DMA，也不能重用buffer。相反，device斷線可能使request以錯誤完成，應用必須接受failure而非永等成功。

可攜與節能policy都必須保留這個原則：任何能觸及資源的execution／device引用結束前，不能把該資源交給不相容的新owner。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

kernel、executive、HAL分工

元件	主要責任	典型狀態
kernel dispatcher	thread選取、dispatch、同步／trap基礎	ready queues、thread state
executive managers	object、memory、process、I/O、安全、cache等	handles、PTE/PFN、IRP
HAL	平台／interrupt／timer等硬體介面抽象	硬體路由與控制
drivers	特定device、filter、filesystem等路徑	device objects、queues、buffers
user services／subsystems	登入、背景服務、應用環境等	user address spaces與IPC

executive與kernel layer在此是實作責任名稱，兩者都在kernel mode；不要把同一thread進kernel時所用的kernel stack說成另創一個可獨立排程的「第二thread」。

thread priority和IRQL是兩個不同次序

Windows7 dispatcher選可在該CPU執行的最高priority ready thread；variable1…15、real-time16…31，0為idle特殊用途。variable boost及其decay不低於base、上限不進real-time範圍；具體boost與quantum受條件影響，不把本節q2當全系統固定值。

IRQL控制當前processor可處理哪些interrupt／kernel活動及允許的操作，並非另一個0…31 thread priority。

x86/x64常用IRQL	典型執行	限制
PASSIVE_LEVEL=0	一般thread／worker	仍須按API規則判斷可否等待
APC_LEVEL=1	部分kernel APC活動	有特定APC／呼叫限制
DISPATCH_LEVEL=2	普通DPC、部分spinlock區段	不可作非零時間的blocking wait；不可觸發pageable存取fault
device DIRQL	ISR／device同步	短處理，限制更嚴

在某CPU提高IRQL不會讓另一CPU停止存取共享物件；跨CPU互斥仍需同步。real-time priority也不保證hard deadline。driver規則。

固定priority模型：每次選取可回到queue規則

單CPU、無boost／decay、同priority q2、較高priority立即搶占；被高者搶占後保留本slice餘量且回同級前方。P1/P2 priority8，t0ready，bursts4/3；P3 priority10，t1ready、burst2。

區間	選取原因	結束時狀態
0–1 P1	同級先入	P3較高priority到達；P1餘burst3／slice1
1–3 P3	priority10	P3完成
3–4 P1	同級前方且保留slice1	餘burst2，slice用完回尾
4–6 P2	priority8 queue前方	餘burst1回尾
6–8 P1	同級前方	完成
8–9 P2	唯一ready	完成

completion P1/P2/P3=8/9/3，CPU工作9。此模型只隔離priority與quantum規則；真實系統的boost、quantum與processor條件不同時，須依相應規則另算。

Priority 搶占與 queue 追蹤

單 CPU：priority 搶占與 ready queue

教學模型：P1、P2 priority 8；P3 priority 10；quantum = 2

時間與事件	選擇前 priority 8 ready queue（左側先選）	
t=0	P1 → P2	P1 先入
t=1	P1 → P2	P3 搶占；P1 保留 slice 1
t=3	P1 → P2	P3 完成；P1 接著用完 slice
t=4	P2 → P1	P1 slice 用完，回隊尾
t=6	P1 → P2	P2 尚餘 1，回隊尾

完成時間：P1 = 8、P2 = 9、P3 = 3

固定規則範例；不代表 Windows 7 的實際 quantum 或完整排程行為。

P1 在 t=1 被 P3 搶占，保留 1 單位時間片；t=3 恢復後於 t=4 用完，回到 priority 8 queue 尾端。完成時間為 P1=8、P2=9、P3=3。這是固定教學模型，並非 Windows 7 的實際排程保證。

dispatcher objects：signaled狀態不是通用訊息queue

manual-reset event一旦signaled，維持到ResetEvent；可讓多個waiters通過。auto-reset event在滿足一個wait後回nonsignaled；若已signaled又SetEvent，不能累加成兩個獨立tokens。因此需計數的resource要用semaphore或自己的queue，而不是拿event當無限計數器。

mutex具有owner，持有者才能依協定release；Wait回WAIT_ABANDONED表示前owner終止等異常，取得者須檢查被保護資料一致性，不能等同資料已修復。mutex並不自動防兩個mutex的lock-order deadlock。

本章用可見semaphore count0…limit，count0的waiter睡眠，release可喚醒並handoff許可；不要套入第5章另一種內部signed counter而混淆外部count。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISR、DPC、APC與worker的執行契約

ISR先辨識／確認裝置來源，完成必要最小動作；普通DPC延後一部分處理，但仍在DISPATCH_LEVEL，不能因叫「deferred」就執行可blocking的filesystem I/O。需要sleep/pageable服務的工作送適當PASSIVE_LEVEL worker，並延長其引用的context生命期。

APC排給特定thread；一般user APC通常在該thread進alertable wait時執行，不是任意搶進所有user指令。Sleep與SleepEx(alertable=TRUE)的交付條件不同；本章不把所有特殊kernel APC／thread啟動路徑概括為相同規則。

例ISR排DPC後return，DPC再排worker；若在排worker前把request context free，worker將讀失效記憶體。正確順序保留reference→提交worker→worker完成並釋放最後reference。IRQL與ownership共同限制合法動作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

object manager：handle值只在指定table有意義

process A的handle0x40→objectF／grantedR；process B的0x40可能指eventE，不能把整數直接當跨process通用能力。正式sharing可經inherit、DuplicateHandle或以名稱再open，各自有授權／生命期規則。

步	A handle table	B handle table	object F
初始	0x40→F,R	空	有A引用
Duplicate SAME_ACCESS成功	仍0x40→F,R	0x88→F,R	兩handle同object
A close	空	仍0x88→F,R	仍不能free
B close但kernel I/O仍持reference	空	空	仍不能free
最後internal reference釋放	空	空	依物件規則可清理

SAME_ACCESS明定不增權；一般DuplicateHandle的desired-access行為與object type／呼叫權限另查API，不把所有dup都說成單純subset運算。source／target process handles須有PROCESS_DUP_HANDLE等所需權限。DuplicateHandle。

token、security descriptor與ordered DACL

token帶SID／groups／privileges等；有impersonation時還要選正確有效token。object security descriptor含owner、DACL／SACL等，DACL管允許／拒絕，SACL偏向稽核而非同一允許表。

以下簡化為enabled SIDs、普通allow/deny ACEs及已展開的specific rights，排除privilege、integrity、owner特例／object ACE；抽象R=1、W=2不是Win32實際mask。requested set=R|W，remaining初為3：

ACE順序	匹配？	remaining／結果
ALLOW Alice R	是	移除R，剩W
DENY Staff W	Alice屬Staff	W仍未授予，拒絕並停止
ALLOW Staff RW	不再執行	不能救回前段拒絕

順序有意義；若把最後ALLOW移到deny前，可先授完remaining而停止，不能無視順序稱「deny永遠蓋過一切」。系統有建議的canonical ACE排序。NULL DACL與empty DACL也不同：前者不以DACL限制，後者無entry可授權，本簡化一般請求會拒。AccessCheck。

VA保留、commit承諾及實際駐留分開

固定page4KiB、allocation granularity64KiB，一個private region基址0x100000，先reserve64KiB。reserved只占此process的VA區域；commit其中0x101000與0x102000兩頁，charge8KiB，尚不表示當下已分到兩個physical frames。

步	reserved VA	committed private	本例實際resident
reserve	64KiB	0	0
commit兩頁	64KiB	8KiB	0
首次觸及0x101123	64KiB	8KiB	4KiB，zero-fill頁
decommit第一頁且無其他引用	64KiB	4KiB	0
release整個region	0	0	0

VA保留、commit承諾及實際駐留分開 （續）

reserve/commit可失敗；commit受系統可承諾資源限制，不是把page file每頁立刻寫零。virtual memory也不是各process搶同一組VA數字。VirtualAlloc。

PTE、PFN與working-set trimming

PTE描述virtual page mapping／軟體狀態；PFN database描述physical frame、references與list狀態，兩者不是同一表。合法但未resident頁可能是demand-zero、file/section-backed、page-file-backed或可由RAM既有頁恢復，不把所有fault都當disk I/O。

固定VA0x401123=VPN0x401＋offset0x123；若PTE valid指PFN0x2A，PA0x2A123。若write遭RO限制且沒有合法COW條件，是access violation；不能任意補頁來繞過權限。

working set頁被trim不代表直接erase：clean且有可恢復backing者可到standby；modified需安排writeback才可供他人重用；仍pinned／I/O使用中的frame不能僅因age大就free。被trim頁若仍在standby可soft fault找回；backing不在RAM才可能要storage I/O。

Windows7用近似參考歷史／工作集heuristics管理，不把它說成知道未來的OPT或每次精確LRU。可用frame、page-table state與pending writeback要同步追蹤。

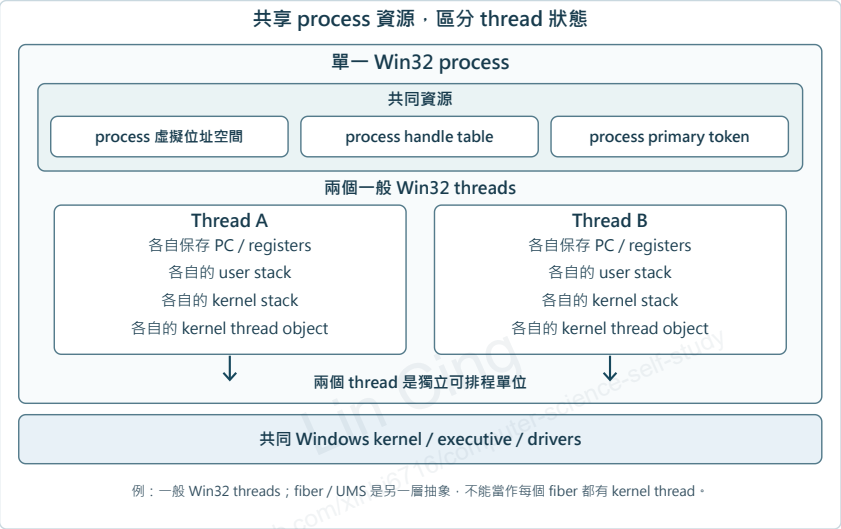
process、section與ALPC的authority

CreateProcess建立process環境並含initial thread；再CreateThread一次才是兩thread。process持address space、handle table與token等；thread有registers、user/kernel stack與執行狀態。關一個process handle不等於TerminateProcess；threadexit後object仍可供有handle者查詢／等待。

section object支援file-backed／page-file-backed映射；不同process可map不同VA而共享同一backing範圍，需各自rights及可變資料同步。ALPC／port式IPC支援client/server請求與authority傳遞；shared section可減少資料copy，但不免除buffer所有權與長度檢查。

例A VA0x100000、B VA0x500000都map section offset0的page，寫A+16可能使B+16見同一共享資料；但不把字面pointer0x100010直接交B就當有效pointer，應傳section-relative offset16和長度。

同一 process 的兩條 Win32 threads



一般 Win32 thread 各有核心可排程的 thread object 與執行狀態；同一 process 的 threads 共用位址空間和 handle table，但各有暫存器、user stack 與 kernel stack。圖中 token 是 process 的 primary token；thread impersonation 時還須考慮其有效身分。

Fiber 和 UMS 是其他使用者層排程抽象，不能把每個 fiber 說成一條 kernel thread；共用資源也需要同步。

IRP的向下dispatch與向上completion

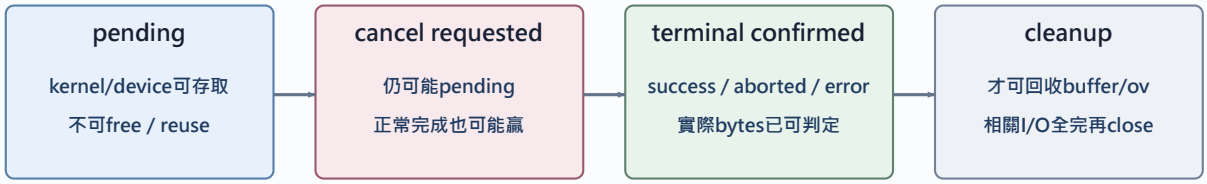
I/O manager可建立IRP，經filesystem/filter/device stack向下送；各device object有對應driver責任與per-stack狀態。底層完成時status／實際bytes沿completion路徑返回；有fast／cached路徑不必每次完全走相同IRP鏈。

時點	狀態／ownership	允許的動作
submit	caller提供buffer與唯一operation狀態	不得在pending時重用
pending	driver／device仍可能存取	取消是請求，不是立即free
hardware完成／cancel生效	status與實際bytes確定	沿completion規則發通知
caller確認terminal completion	知道success／aborted／othererror	才可回收buffer／OVERLAPPED

CancelIoEx可與正常完成競爭；其成功只表示取消已提出，ERROR_NOT_FOUND亦可能只是已無匹配pending。仍須收集原operation結果，不能只看取消回傳就結束生命期。取消規則。

IRP的向下dispatch與向上completion （續）

取消已提出 ≠ I/O已完成：buffer / OVERLAPPED須保留到terminal



圖為有提出取消的路徑；pending也可直接正常完成，取消成功回傳不代替原operation完成結果。
每個未完成operation有唯一ov / buffer；Win7 ReadFile需有效byte-count參數，最終以completion bytes為準。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

cache manager、memory manager與檔案讀取

cached read先在filesystem/cache映射找file範圍；已resident可copy到caller。若cache view缺頁，VM manager安排paging read，經儲存路徑填入frame，再滿足原read。cached I/O與paging I/O是相關而不同的request，不把一次cache miss重複當作兩份交給應用的資料。

固定讀4096B：cache hit需caller copy4096B；miss還需device→cache frame4096B（本例DMA）、最後caller copy4096B。memory copy成本、DMA bytes與storage讀取次數各有分母，不能都稱成「兩次CPU copy」。

lazy write允許dirty cache晚些寫回，改善合批但帶來durability時間差。log／data flush的次序由filesystem與storage契約建立；cache hit／write成功不自動代表斷電安全。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PnP、power、registry與boot的依賴

PnP協調device discovery／driver stack及移除，power manager協調狀態轉換；它們都要排除仍會碰硬體的pending request後再釋放相關資源。driver object、device object與實際硬體存在／可用性是不同生命期。

registry以hives／keys／values保存配置，也可能有volatile狀態；它不是一般file pathname namespace。更改配置何時對driver生效，依服務重讀／重啟或通知契約，不是寫key就保證所有process立即換設定。

Win7 boot分firmware/boot manager／OS loader、載kernel/HAL與boot drivers、kernel初始化、session/services及logon等階段；BIOS與UEFI路徑不同，不固定全部平台都經同一MBR檔案序列。載入page tables與儲存driver等相依資源必須先可用，才能讓後面的file／memory服務工作。

session、window station、desktop與process不是同一層

terminal services／fast user switching讓不同互動使用者保留各自session、桌面／輸入視圖及process。Windows7中service與一般互動session的安排必須與舊版混用假設分開；session隔離不把每個使用者變成獨立VM。

資源	不同sessions是否一定獨立？	理由
user address space	不同process各自有映射	不是因sessionnumber本身代替MMU
window station／desktop與部分named objects	有session相關scope	仍需object rights與指定namespace
kernel、drivers、physical RAM	共享	共同故障／資源瓶頸仍存在
NTFS files／network服務	可共享	取決於credentials與object policy

兩人同時登入並不表示本機產品允許無限制同時互動連線；功能／授權／部署限制不由kernel抽象表推論。

Local與Global名稱：scope不等於授權

固定S1/S2各建立Local\Ready event，兩者可對不同session-local objects。若依法取得Global\Ready，則可指同一global物件，但仍要通過其DACL與建立／開啟所需權利，不能以Global字樣略過access check。

handle值0x80在S1 process與S2 process可能完全不同物件；即使物件名相同也須先確定namespace與授權。session id不是可信user身分替代品。

切換前台使用者不會自動終止另一session所有process；也不能保證其持有的file鎖／CPU／memory消失。資源管理與隔離的契約應各自追蹤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NTFS：volume、MFT、attribute與extent

Windows7可支援不同filesystem，本章聚焦NTFS，不宣稱所有local／removable volumes必是NTFS。allocation以cluster為單位，LCN從volume起點計；若volume在磁碟partition內，轉disk byte offset須再加volume起點。

MFT保存file records／attributes，小attribute可resident在record內，大data可nonresident，透過VCN→LCN extents定位。非resident不是「沒有資料在RAM」，而是attribute內容不直接放在該MFT record。

假設record1024B、其他headers／attributes已占700，剩324；300B data在此固定預算可resident，512B放不下。真門檻依attributes／名稱／格式等，不能把300當全NTFS固定cutoff。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

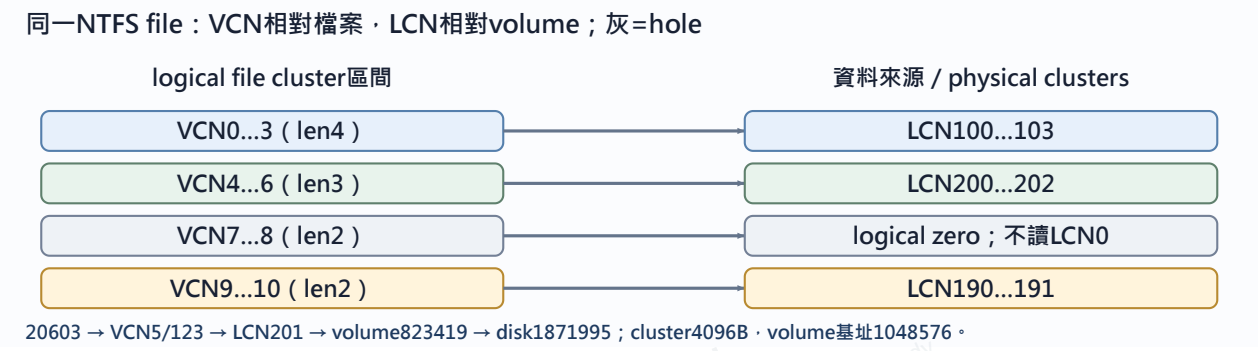
VCN到LCN：完整同一檔案數例

cluster4096B，volume起點disk byte1048576。file extents如下，區間均半開，None表示sparse hole：

VCN起點	長度	LCN起點	對應
0	4	100	VCN0…3 → LCN100…103
4	3	200	VCN4…6 → LCN200…202
7	2	None	VCN7/8讀出logical zeros，無data clusters
9	2	190	VCN9/10 → LCN190/191

file byte20603=5×4096+123，命中第二extent；LCN=200+(5−4)=201，volume byte=201×4096+123=823419，disk byte=1048576+823419=1871995。若落hole只填零，本例不把None算成LCN0。

VCN到LCN：完整同一檔案數例 （續）



file reference、目錄index與metadata files

本書NTFS file reference用48-bit MFT record number＋16-bit sequence，reuse時sequence變更，可偵測序號不符的stale reference。有限16bits不能被說成永不wrap的數學唯一身分；長期生命期仍須按實際系統規則處理。

目錄index以排序key及file reference定位；B+類索引的balanced path支持對數高度，不表示insert/split完全沒有重組成本。小index可在record／root中，長大需額外index allocation。列目錄cache的size/time欄不等於每次都已重新查最新file內容。

MFT、bitmap、log、volume與其他metadata以特殊files表達，有利共用cache／I/O機制。bitmap表示clusters的配置；其更新仍需與extent／size一致，不能各自成功就當整個allocation已atomic。

metadata WAL：先寫log，再允許home持久化

以下為單一 transaction、無其他交易競爭且新 block 被獨占保留的 undo／redo 教學模型，不代表完整 NTFS log 格式或並行 recovery。交易所有 before／after records 和完整性標記必須**全部 durable**後，才允許任何本交易的 home 欄位更新。完整標記與 commit 可辨識，flush 可信；steal 允許 commit 前寫 home，no-force 允許 commit 後 home 尚未全寫。

舊 metadata 為 size=4096、extents=[100]、bitmap[200]=0；新狀態為 size=8192、extents=[100,200]、bitmap[200]=1。完整 log 先 durable，再可更新 home；commit durable 後才宣布交易成功。

crash 時紀錄	home 可能狀態	recovery
完整 log 尚未 durable	全部 old	此模型禁止任何 home 更新
完整 log durable、無 durable commit	old 或混合	用 before-images undo 到完整 old
完整 log 與 commit 均 durable	old、混合或 new	用 after-images redo 到完整 new

恢復選擇整組 metadata。實際 NTFS 還處理 LSN、並行與 checkpoint 等。

恢復不變量、checkpoint及user data範圍

在上述模型，任何 home 更新之前，全交易 before／after log 與完整性標記均已 durable。無 durable commit 時，用 before-images undo 已更動欄位；有 durable commit 時，用 after-images redo 所有新值。以 assign old／new 重播具冪等性；log 須保留，且故障與 flush 符合模型假設。

checkpoint 不表示所有較早的 log 均可刪：活躍 transaction 的 undo、尚未落 home 的 dirty metadata 的 redo 須保留到安全截斷條件成立。

metadata 一致不保證 user file 內容構成完整應用 transaction。write 成功可能只在 cache；兩個業務檔也不因 NTFS 有 log 而自動跨檔原子。應用仍須設計 flush、發布或交易協定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ACL、EFS、BitLocker與repair處理不同威脅

NTFS連到Windows security descriptor／DACL模型；不是把POSIX owner/group/other mask算法套進來。平常有bypass-traverse privilege的情境與逐directory FILE_TRAVERSE檢查也不同，不能一概宣稱Windows不做路徑保護。

EFS在file內容層加密，BitLocker在volume／儲存保護層；它們各自有key與解鎖條件，不能單靠NTFS ACL保護被搬到另一系統直接讀取的未加密媒體。加密不自動提供backup或應用transaction。

bad-sector remapping更換位置；若舊data已無法讀又沒有有效mirror/parity等來源，重新配好cluster也不會創造原內容。RAID／spares、metadata recovery與錯誤資料修復是不同保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

sparse、compression、USN與snapshot分開追蹤

前述file共11logical clusters，其中2holes，非hole data配置9clusters，若不另compression則36864B；logical size可為45056B。hole讀零，將來寫入時另需allocation，不是把整個file都先保留9clusters大小的VA。

NTFS compression以compression units等規則表達，效果依內容及配置條件；不能宣稱任意file都固定壓縮比。USN change journal記錄可供追蹤／索引的變更，與metadata recovery log不同；journal可wrap，consumer遇歷史gap需重新建立狀態，不能視為永遠完整的backup。

VSS snapshot給一個時間點的storage視圖；若應用當時正更新多個相關欄位，仍可能需要writer協調／quiescence才有application-consistent狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

snapshot保留的是舊版，不是把新版誤當備份

copy-on-write snapshot簡化模型：live blocks X=41、Y=80，建立snapshot S。首次把live X改42前，先讓S保留old X41，再允許live指新X42；後續live X43，S仍讀41，不再改成42。Y沒被改，S/live可共用同old backing80。

時點	live X	S X	Y
snapshot後	41	41	live/S共用80
第一次寫後	42	41	仍80
第二次寫後	43	41	仍80

old backing只有在不再被任何snapshot／reader需要時可回收。snapshot若與live共用同一故障儲存，單次device全失仍會一起失去；不是單獨備份媒體。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

networking interfaces與protocol責任

user Winsock提供socket概念；NDIS處理網路adapter／protocol相接的driver模型。原書TDI屬歷史介面討論；官方對Vista及以後新kernel network modules引導使用WSK，不把TDI當Windows7新driver唯一途徑。這些是API／driver分工，不能直接對應為每個packet必經幾次process switch。

機制	處理什麼	不直接保證
TCP/IP	位址／封包／可靠byte stream等層級	應用訊息邊界、業務exactly-once
SMB redirector/server	remote file operations	localfile與remote failure完全相同
named pipe	本機／遠端pipe語意	byte與message mode可互換
RPC	把請求／回覆包成遠端呼叫	timeout表示server未執行
COM	物件介面與marshalling等	每個COMcall都跨機器

協定版本、認證／加密與錯誤語意要另外指定。HTTP/WebDAV、VPN等名稱也不能代替安全設定；歷史介紹不作當代安全建議。

remote file：UNC命名不抹掉partial failure

例應用開`\\server\share\report`，I/O路徑辨識remote名稱，redirector將操作編碼成network request，server經自己的filesystem及credentials政策處理，再回status／bytes。client kernel、network、server與server storage可各自失效。

client request送達，server寫入後reply遺失：client timeout不能推出「沒寫」。若重試非冪等append，可重複新增；需要協定／server request identity、重試規則與持久化語意。TCP可靠stream本身不解業務重複執行。

redirector使API可相似，但不同share／offline cache策略會改visibility及錯誤時點。成功open也不保證連線之後永不失效。

named pipe模式、domain與directory services

固定為 message-type named pipe，且本次接收 handle 使用 `PIPE_READMODE_MESSAGE`。若訊息為 500 B、buffer 容量 200 B，本例可取得 200 B 與 `ERROR_MORE_DATA`，再讀同訊息餘下的 300 B；FALSE 不一律代表零 bytes 的致命錯誤。Message-type pipe 的 handle 亦可使用 byte-read mode，不能把本例套用到該模式或 TCP。

domain 以共同 identity／管理協定整合資源；Active Directory 結合 directory 與認證服務，不只是「加密資料庫」。相同顯示名稱不代表相同 SID，server 應依有效 credentials、token 與 object policy 判定。

client-side caching 可供離線工作，但重新連線有版本及衝突問題；cached success 不自動等於 remote durable commit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CreateProcess、threads、fibers與UMS的排程單位

CreateProcess成功包含一個initial thread；process object與thread object分別可由handle觀察／等待。再建第二thread時可共享address space與某些process資源，但各自保存PC、stack、TLS等。可用句柄名稱不能替代reference lifetime。

fibers由user code在thread內切換；若承載thread執行blocking system call，普通fiber切換本身不會讓這個kernel thread同時替另一fiber執行。Windows7 UMS是特定平台／API條件下讓user scheduler與可管理的kernel執行狀態配合，不能簡化成所有阻塞都免費或所有Win32 API自動相容任意fiber。

thread pool以重用workers／queue工作減少反覆create成本，仍需管理工作資料生命期。單worker pool的job A等待同pool內尚未執行的job B，可造成pool starvation deadlock。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

handle sharing與authority不能只傳一個整數

inherit需要來源handle的inherit設定與CreateProcess的相應條件／指定handle list；DuplicateHandle需來源與目標process的合法操作權限；named open則在指定namespace重新依security policy取得handle。

明定DUPLICATE_SAME_ACCESS時，新handle保持原granted access；A有R但無W，不因複製給child就變W。若要取得不同access，必須按object type及API授權規則處理，不假設可任意增權，也不把本例subset模型冒稱所有DuplicateHandle行為。

對同一file object的duplicated handles可共享current file position；獨立CreateFile可得到不同file position。後續同步seek/read仍可能競爭，overlapped I/O用每個operation自己的offset與狀態。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

ReadFile三種起始結果與Win7參數

採regular file且handle用FILE_FLAG_OVERLAPPED開啟，每個未完成operation使用獨立OVERLAPPED與有效buffer；本例用manual-reset event做完成通知。Windows7要求ReadFile的lpNumberOfBytesRead非NULL，提供DWORD位置，但pending時以最後completion回報的bytes為準。

ReadFile起始結果	意思	生命期
TRUE	這次已完成，檢查實際bytes	可按完成結果處理
FALSE＋ERROR_IO_PENDING	已提交且尚未完成，不是一般失敗	buffer/OVERLAPPED/event保持有效
FALSE＋其他error	依file／pipe／device契約處理	先確認是否仍有未完成operation

pending可用GetOverlappedResult等相應完成機制確認。不要拿現代GetOverlappedResultEx範例回填為Win7必有；本章例程使用較早的GetOverlappedResult。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一file的6、4、EOF追蹤

固定 regular file 為 ABCDEFGHIJ，共 10 B；沒有並行寫入或 truncate。每次用 `FILE_FLAG_OVERLAPPED` handle 請求 6 B，並指定獨立有效的 OVERLAPPED、buffer 與 offset。R0、R1 均成功；起始呼叫可立即完成或先返回 pending。

operation	offset	request	最終結果	下一步
R0	0	6	成功 6 B：ABCDEF	terminal 後才重用狀態
R1	6	6	成功 4 B：GHIJ	已讀前綴共 10 B
R2	10	6	EOF：`ERROR_HANDLE_EOF`	停止

本例 overlapped 正長度讀取，R2 可由 `ReadFile` 立即回報 `ERROR_HANDLE_EOF`，或先 pending 後由 `GetOverlappedResult` 回報；立即失敗時沒有 pending 工作可等待。另一種非 overlapped 同步讀取到 EOF 可成功返回 0 B，不能混進 R2 的契約。pipe／device 有不同規則；取消可能輸給正常完成，須收集實際結果。

memory、heap、mapping與TLS的不同層級

VirtualAlloc管理VA reserve／commit及page protection；heap manager在其取得的區域內細分小物件，HeapFree不是自動release整個region。file mapping／section為file-backed或page-file-backed view，MapView取得各process內的VA；視圖大小／offset與實際file大小需一致。

TLS提供per-thread slot，保護的是索引／邏輯歸屬，不阻止同address space另一thread若有pointer就存取該記憶體。物件超出thread生命期時需明確owner／同步；TLS不等於獨立MMU頁表。

virtual commit、working set與heap allocation各有不同統計量。64KiB reserve、8KiB commit、4KiB resident可同時成立；不能拿一個數說明全部memory使用。

取消與cleanup的完整協定

本章的安全不變量：只要operation仍可能被kernel／device碰觸，buffer、OVERLAPPED、event及其context就必須存活。

對已pending R：CancelIoEx(h,&ov)提出取消→用原完成管道確認R terminal→處理normal／aborted／othererror與實際bytes→釋放資料／ov，最後在所有相關operations終止後關handle與event。

如果取消輸給正常完成，R可success；如果取消成功中止，R可ERROR_OPERATION_ABORTED；取消回ERROR_NOT_FOUND也不取代原R結果。不得在取消請求回來後立刻free，亦不得同一ov同時承載R1/R2。不同completion API／IOCP通知模式有自己的交付規則，需與所選模型一致。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一個read的各層狀態

層	需要追蹤	可能已成功但後面仍失敗的情況
authority	process-local handle、granted mask、有效token	有handle不代表有W或永久network可用
virtual memory	buffer VA、commit、resident、page rights	reserve成功但尚不可觸及未commit頁
I/O	IRP／operation state、pending、actual bytes	提交成功不代表完成
cache／filesystem	file offset、VCN/LCN、metadata一致性	write cache完成不代表user data durable
device／network	DMA ownership、連線、storage contract	取消已提出不代表硬體已停止

IRQL限制目前可做的kernel操作；thread priority決定何者在一般排程中先取得CPU。混為同一priority會讓ISR/DPC生命期與等待推理一起出錯。

本章應能重建的四個例子

1. 同priority q2且P3於t1以priority10到達：P1/P2/P3完成8/9/3；每次選取回到queue與剩餘slice。
2. region64KiB reserve／8KiBcommit／4KiBresident；decommit與release各改不同集合。
3. NTFS file byte20603→VCN5/123→LCN201→volume823419→disk1871995；hole不當成LCN0。
4. ReadFile offsets0/6/10，完成6/4/EOF；pending、cancel-request、terminal-completion分開，才有合法cleanup。

這些例子連到一般不變量：grant只允許持有right、已發生I/O引用不得早free、file配置與bitmap／size須一致、shared memory及其他processors不受單CPU IRQL自動互斥。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 19. Microsoft: IRQL, VirtualAlloc, AccessCheck. ReadFile, EOF, CancelIoEx, named pipes.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

本章路線

這章要回答的核心問題是：哪些運算需求與侷限推動作業系統功能演進，這些歷史設計如何轉化成今日抽象？

前置知識。 OS 導論、process/memory/file abstraction、computer architecture history。

- 用需求與限制比較歷史系統
- 辨識功能傳承與版本差異
- 避免把歷史產品直接等同當代系統

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

功能遷移不是原實作原封不動搬家

本章以歷史系統觀察「限制→機制→新增契約」，不將章節次序當成嚴格時間線。batch sequencing減少人工等待，spooling把裝置生產／消費解耦，time sharing加入互動與CPU回收，virtual memory把資訊身分與實體駐留分離。

原限制	機制	新增而非自動解決的問題
人工換job慢	resident monitor	如何辨識job終止與保護monitor
讀卡／列印慢	offline I/O、spool	queue、空間、錯誤／重試
單人占機久	time sharing	timer、隔離、response與公平
快memory昂貴	paging／automatic backing	fault重啟、mapping、replacement

Multics、UNIX及後續較小平台出現相似的多使用者、階層檔案與共享機制，能支持概念遷移的討論；若要斷言某段code直接衍生，仍須更具體的source／設計記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較系統先固定四個維度

同一「process」可能是靜態worker、動態建立的process或Mach task內的thread；同一「paging」可能只做relocation，也可能包含demand loading。

1. 執行代理是否動態建立？一個proxy對應一個job，還是多job重用worker？
2. memory address的單位是byte或word？page多大，table為何存在？
3. 服務跨越的是軟體layer、privilege、address space，或獨立機器？
4. fault／timeout／message buffer歸誰？何時可釋放與重試？

這四項不同時，不能因名詞相同就把現代API或效能保證套回歷史案例。本章給定數值的演示模型會明列其規則，不把它們當成當年實機量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從手動操作到resident monitor：減掉哪一段等待

早期job經compile、assemble、load、run等步驟，operator換媒體／觀察終止造成CPU空等。batch先把相似需求聚合，monitor再依control record自動選下一步；controller/driver仍處理實際裝置細節。

固定3jobs順序F,C,F，CPU各2分鐘；每次切compiler種類成本3分鐘，初次設定也算。照順序設定F/C/F共9，CPU6，總15；改F,F,C只設定F/C共6，總12。此改善依job允許重排，且未計等待回應與相依性。

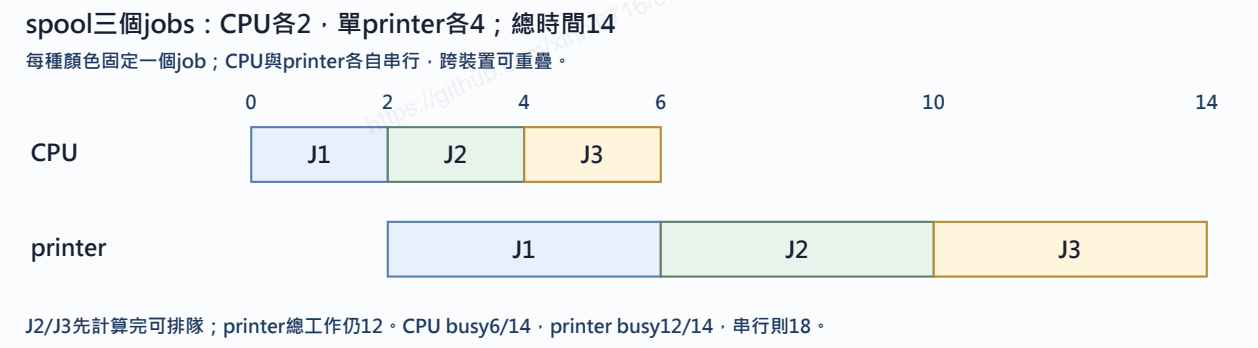
monitor須留在受保護狀態；user job若能覆寫它或直接halt全機，自動sequencing就不可信。control card與data的邊界也需要語法／長度／escape規則，不能單靠資料「通常不含那個字元」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

offline、buffering、spooling與multiprogramming分開

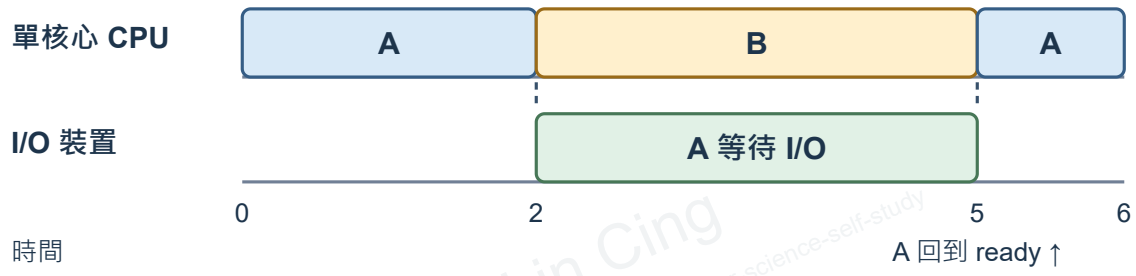
- offline I/O：別的設備先把卡片轉到tape，再帶去主機；省主機等待，但搬移／集滿媒體增加單joblatency。
- buffering：暫存資料以銜接生產者／消費者，容量不足仍會backpressure。
- disk spooling：獨立保存jobs的輸入／輸出與queue metadata，使裝置與不同job的計算可重疊；不能把同一tape想成兩端同時隨機存取。
- multiprogramming：OS讓一job等待時執行另一ready job；需要隔離與排程，spool本身不等同所有工作都已在RAM並行。

給3jobs、每jobCPU2與print4、CPU/單printer可重疊：CPU0–2 J1、2–4 J2、4–6 J3；print2–6 J1、6–10 J2、10–14 J3。總14、CPU6/14、printer12/14；完全串行CPU＋print需18。新增spool不會把printer12縮成6。



一顆 CPU 如何在等待時做別的事

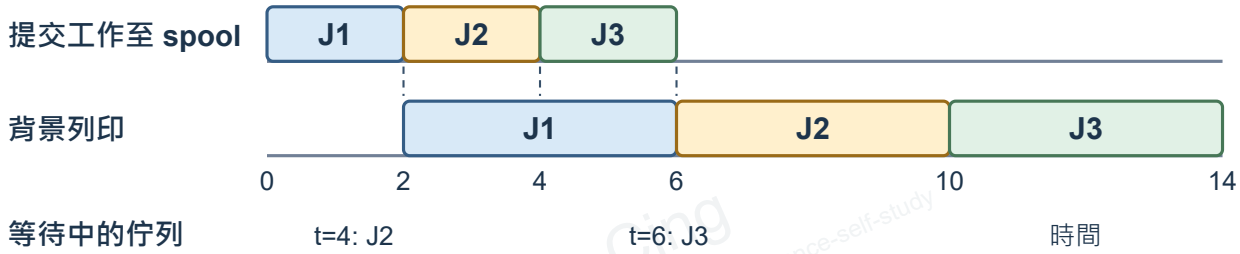
假設 A、B 同時就緒，忽略切換成本；A 執行 2 單位後發出耗時 3 單位的 I/O，B 執行至結束，A 再執行 1 單位。



時間 2–5，A 等待裝置，B 使用 CPU；CPU 和裝置可同時忙，但單核心不會同時執行 A、B 的指令。時間 5 A 回到 ready 並執行。若無其他 ready process，CPU 仍可能閒置。

Spool：送入佇列與實際印完是兩個時點

單 CPU 按序處理 J1–J3，各需 2 單位；提交至容量足夠的 spool。印表機按 FIFO 每份印 4 單位，列印與 CPU 可重疊，忽略其他成本。



時間 4：J2 已提交、等待列印，J1 仍在列印。時間 6：J1 印完、J2 開始，J3 等待；時間 14 全部完成。**提交成功不代表印表完成。** 佇列滿或裝置故障時，應依介面規定處理；buffering 與 spooling 依用途區分。

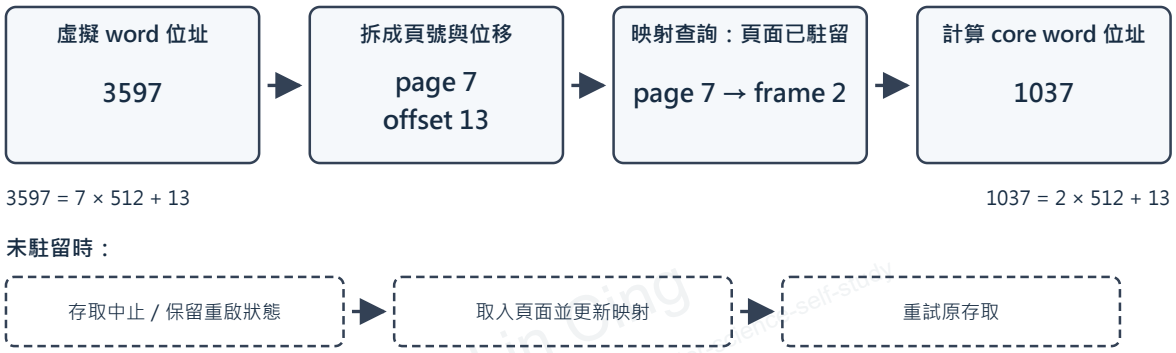
Atlas one-level store：保留資訊位址，自動換駐留位置

Atlas把快core與drum結合成程式看到的一層store；512-word pages與associative mapping，使不在core的block可由系統載入後繼續。原書所述16Kiwords core可分32frames；word不是byte，不把16Kiwords誤稱16KiB。

以縮小的4frames示意，不改512word/page：virtual word address=7×512+13=3597。若page7在frame2，core word address=2×512+13=1037；若不在core，先保存可重啟狀態、取入空frame並更新mapping，才能重試原存取。

one-level抽象不表示每次都像core一樣快，也不讓drum直接當CPU可任意讀的cache-freeRAM。需安排換出、保留空frame與進行中transfer，不能把仍被使用的frame交給另一page。

Atlas：從 word 位址走到 core frame



映射缺失表示頁面目前不在 core；若無空 frame，另需選擇 victim 並處理換出。

3597 = 7 × 512 + 13；若 page 7 映射 frame 2，位址為 1037。缺頁時先取得 frame；無空 frame 才選 victim，修改過的資料必要時先寫回。

Atlas learning replacement：三個分支都要有

Kilburn等1962原論文以t表示距最近使用的時間、T表示上一段inactive間隔；採樣時間依執行指令數，不是普通牆鐘時間。選擇依序：①任一t>T+1；②否則在t≠0者中選最大T−t；③若全t=0，選最大T。tie須另定，本例用表序。

案例	pages的(t,T)	分支／victim
A	(6,3),(2,5),(0,9)	①首page，6>4
B	(2,4),(1,6),(0,8)	②忽略t0；scores2/5，選第二
C	(0,2),(0,5),(0,3)	③全部剛用，選T5的第二

第二分支把過去間隔當下次使用預測；它不是知道future的OPT。若預測最晚的page下一步反而被讀，就會有額外fault。保留第三分支，才能對全部t=0的合法狀態選出victim。原論文。

XDS-940：paging可只做relocation，不必demand paging

依原書歷史配置，process virtual space16Kiwords、physical64Kiwords、page2Kiwords：每process有8virtual pages，physical有32frames。page table用registers保存，允許process頁分散擺放；必要時以整體image swapping增減在RAM的process。

例P的virtual page3→frame11，word offset17：VA=3×2048+17=6161，PA=11×2048+17=22545。這是word address，不乘成bytes；平台word位寬不在本例中額外假設。

可重定位不代表某頁能合法缺席後由demand fault補入。XDS-940本節的paging用於relocation；Atlas本節則自動按需取頁。兩者都有page table，不能據此推成相同memory政策。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享reentrant code與新增的控制邊界

兩個process各8pages，完全獨立需16frames。若其中2pages是可安全共用的唯讀reentrant code，則2shared+6private×2=14frames。兩個virtual映射指向同一physicalcode，private可寫資料仍分開。

reentrant不代表所有資料天然安全：若code偷偷修改同一全域buffer，唯讀codepages也不能讓該buffer自動成每process副本。需要區分指令頁與可變狀態。

XDS-930改成940時加入user/monitor、特權I/O/halt與受控system call，使資源操作可由OS中介。動態subprocess與共享memory另外帶來生命期、同步和終止清理；多個保護特性必須一起維持。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

THE分層依賴：上層只在下層已建立的抽象上推理

Dijkstra的THE是batch multiprogramming設計，以明確同步的sequential processes合作，不是以人機terminal互動為目標。

level	責任／對上隱藏的細節
0	processor allocation、timer；上層用sequential-process抽象
1	segment controller／automatic backing；上層用資訊segment身分
2	console message interpreter；上層不用分配實體console
3	I/O buffering／logical communication units
4	user programs
5	operator（不是其軟體實作的level）

這是抽象與依賴順序，不表示每layer都有獨立address space或每次call都有trap。segment是資訊單位、core/drum page是存放位置，兩者身分分開。EWD196。

layer契約的歸納與破壞方式

令第i層在假設下層契約C0…C(i−1)成立時，保證Ci且不破壞下層隱藏狀態。base C0由硬體／processor控制維持；歸納可推到每層。這不等於「分層就自動正確」，每個Ci及干擾範圍仍須證明。

例console level2的message文字放在segment，可使用level1取回，不必全永久留core。若level1轉而呼叫level2才能完成自己的基本取頁，會形成依賴cycle，原先自底向上證明不再適用。

同步也需不依執行速度：一個產生者尚未產生的segment，消費者不能先讀；printer同一時刻不能被兩個不相容owner使用。等待可調整進度，不可把「通常比較慢」當作正確性條件。

RC4000 nucleus把政策留給上面的process

RC4000的目標是提供可組裝不同系統的nucleus，而不是把單一batch／time-sharingpolicy全寫死。它提供process控制、保護與message primitives；上層process可表達不同服務。

原書message固定8words、共用buffer pool；每process有待收message queue。send-message／wait-message及send-answer／wait-answer組成request/reply；primitive原子性只涵蓋其內部狀態更新，不讓整個server操作自動變transaction。

裝置也透過process式介面表示：driver把register／interrupt轉成message。這是統一操作模型，並非硬體真的成為能執行任意usercode的普通process。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2個buffers的ownership守恆

本例固定pool={b0,b1}，一個request/reply完成前保留其buffer；不是逐指令模擬原機。

步	free	server queue／處理中	awaiting client reply
初始	b0,b1	空	空
C1 send req	b1	b0 queued	空
C2 send req	空	b0,b1 queued	空
S receive b0後answer	空	b1 queued	b0
C1 receive answer並歸還	b0	b1 queued	空

每個buffer恰屬一狀態；free+queued+processing+reply=2。第三request遇free=0要等／依API錯誤處理，不得重複配出b0。若S等C1的額外訊息、C1又等S回答，仍可能deadlock；小kernel與atomic send不會單獨解循環等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CTSS將短互動與長計算區分對待

CTSS 在 MIT 示範互動式 time sharing，並保留既有 batch 程式的執行環境。user image 在有限 memory 與 backing store 間移動，因此過短 quantum 可能使搬移成本占比過大。「compatible」不代表所有後來的 API 均二進位相容；7090/7094 的記憶體單位是 36-bit word，不可直接換成 KiB。

指數 feedback 將用完整個時間片仍未完成的工作降級，並給較低層較長時間片；I/O 完成後是否提升、aging 與初始層級需另訂。

自行定義的比較模型：單 CPU、切換成本暫不計，L 在 t=0 到達且需 11 單位，短工作 S 在 t=1 到達且需 1 單位。所有新工作進 level 0，$q_0=1$，較高層優先；L 在 [0,1) 用完時間片後降至 level 1。S 在 [1,2) 執行，response time 為 0；若採非搶先 FCFS，L 持續到 t=11，S 的 response time 為 10。這只是該負載與規則的比較，不是對所有互動工作的保證。

固定指數feedback模型的完整burst軌跡

固定 $q_i=2^i$、i 從 0 開始：只有一個 CPU burst，長度 11，從 level 0 開始；用完時間片且尚未完成才降一層，無競爭工作及 promotion。

dispatch	level	時間片	實際用量	累計 CPU	剩餘
1	0	1	1	1	10
2	1	2	2	3	8
3	2	4	4	7	4
4	3	8	4	11	0

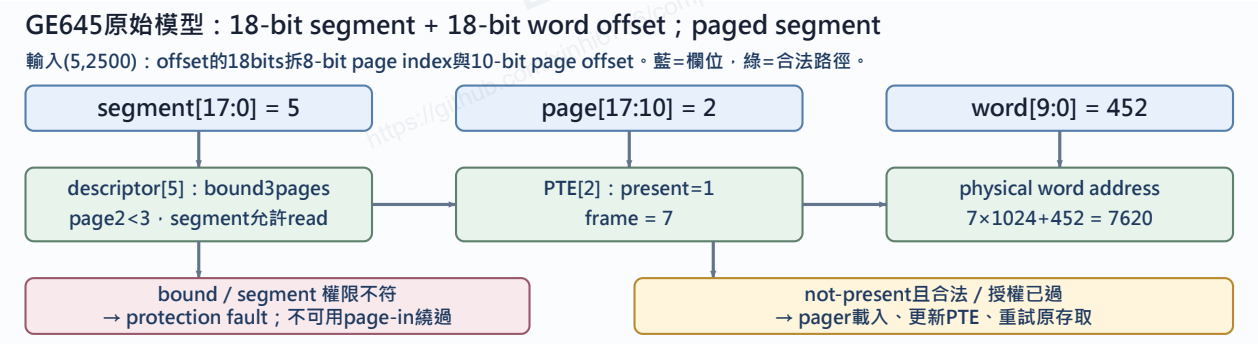
完整用完 level 0 至 k 的時間是 2^(k+1)−1。**成本模型**假設每次 dispatch 均支付 0.5，包括首次載入，也包括同一工作連續執行的 dispatch；它不主張真實 CTSS 每次都必然換出。四次成本合計 2，總時間 13；固定 RR 的 q=1 需 11 次 dispatch，成本 5.5，總時間 16.5。比較僅適用此長 burst 與成本假設。

Multics：以持續服務與選擇性共享組織系統

Multics 的 computer utility 結合階層命名、segment、paging 與受控共享。GE-645 的 virtual address 包含 18-bit segment number 與 18-bit word offset；page 大小為 1024 words，故 offset 可拆成 8-bit page index 與 10-bit page offset。GE-645 原始設計

追蹤 segment 5、word offset 2500：2500=2×1024+452，因此 page index=2、page offset=452。**教學狀態**假設 descriptor 已有效，段界限容許 page 0–2，且允許 read；PTE 2 已駐留且指向 frame 7。逐步檢查 segment→界限／讀取權限→PTE 2→frame 7，得到實體 word 位址 7×1024+452=7620。

若 segment descriptor 無效，先處理 segment fault；segment 合法但超出界限或未獲授權時，存取被拒絕，不能靠載入頁面繞過；合法且授權的 page 未駐留時，才可能交 pager 載入並重試。處理方式依系統狀態而定，並非每個 fault 都可恢復。



rings與分層不是同一種保證

Protection rings 約束跨權限層的存取與入口；THE layers 描述軟體抽象及依賴次序。軟體模組不必各有自己的 ring，畫出層次也不足以證明存取控制正確。

硬體世代要分清楚：GE-645 提供 segment access mode 等保護機制，但 ring 跨層呼叫與返回需 trap 至 supervisor，由軟體介入；後續 Honeywell 6180 的設計才將大部分 ring 檢查、跨層呼叫與返回移入硬體。詳見 Schroeder 與 Saltzer 原始論文。

能解析 segment 名稱不等於有權寫入；仍須通過存取權限與入口規則。共享 code、私人可寫狀態也必須由映射和保護規則維持。Multics 開發者回顧記錄實作整合後依實測成本反覆調整設計。

OS/360 family：相容目標與memory配置版本分開

IBM System/360家族希望不同容量機器可使用較一致的程式／工具；OS/360仍存在不同memory配置方案。MFT使用固定regions，MVT使用variable regions，不能把一個版本的配置行為當所有家族唯一規格。

固定region例可用RAM256KiB分4×64KiB，jobs50/50/70：兩個50各占64，共內部碎片28KiB；70即使總剩128仍不能放進單一64region。這是本例固定region限制，不是說OS/360不能有其他region配置。

variable allocation例holes [50,30,40]KiB，總120卻無連續70KiB，出現external fragmentation。移動／compaction是否可行還需relocation支援與成本；load-time binding的地址不能想改就改。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

virtual memory版本與CP/CMS的不同抽象

System/370之後的OS/VS及MVS利用新增virtual-memory硬體，與原始360沒有相同paging契約。CP/67提供virtual machine環境、CMS是其上可運作的single-user系統；「多個CMS」的隔離由CP的虛擬機資源管理支撐，不是CMS自行變成多使用者kernel。

給兩guest都見virtual address1000，經guest的memory視圖再由VMM映射到不同host資源，便能各執行自己的systemcode。guest的特權操作須由VMM中介，不能因guest認為自己supervisor就可改host所有memory。

family compatibility、硬體relocation、VM隔離與使用者互動是四個不同維度。JCL提供batch資源／步驟描述，不從選項數量直接推論correctness或某個固定效能比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TENEX／TOPS-20把互動命令及memory支援一起設計

依原書脈絡，BBN在PDP-10相關平台加入paging支援並開發TENEX；DEC後續取得權利，形成DECSYSTEM-20／TOPS-20。command interpreter的互動輔助與virtual memory都是其案例重點，兩者作用不同。

命令completion／help降低使用者記憶完整語法的負擔；paging處理程式位址與駐留。shell能提示參數不代表它有權直接變更page table，pager也不能從memory保護替使用者猜出想下哪個命令。

將「像獨占自己的machine」的執行視圖與「多人同時使用」並列看，仍需CPU排程、terminal I/O、file權限與回收。互動性不能只由processor更快推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

機制與可用性是互補的兩層

本例命令有`COPY source destination`，解析器辨識合法語法後只產生request；執行還需查來源read與目標parent／file的write契約。語法正確但無權限，應回授權錯誤，而不是以autocomplete成功當已准許。

若shell先檢查destination再隔一段時間以pathname寫入，名稱仍可能變，會遇第11章的TOCTOU。受管理的file reference、明確錯誤與可理解的互動回饋必須配合。

本例只是把介面／授權分工具體化，不宣稱TOPS-20逐字使用此toy命令語法；歷史UI特色與一般安全推論分開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CP/M與MS-DOS：小型資源限制不等於完整隔離

CP/M在早期微電腦提供command與程式／裝置服務；MS-DOS在IBM PC生態延續單使用者／相容性路徑。原書強調其保護能力與後來多使用者受保護系統不同，不能把它們直接當縮小的現代NT或Linux。

64KiB、640KiB等描述需要區分CPU addressability、conventional memory layout、OS保留與擴充方案；不把一個常見program區間當成所有版本／硬體絕對physical RAM上限。

缺少受強制的memory隔離時，程式可誤寫system資料或其他內容。即使API規定應透過服務呼叫，也不能假設惡意程式一定遵守；需硬體／特權及受信任軟體共同中介。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單一address-space模型的越界反例

固定教學RAM [0,1024)，monitor占[0,256)，job占[256,1024)，沒有硬體write範圍檢查。job store address100可覆寫monitor，即使正確job的library從不這樣做。這是界限模型，不是MS-DOS真實位址配置。

加入受保護的base=256、limit=768且每次user位址a先驗0≤a<768，再轉physical256+a，便限制user存取自己的區間。a=−1及768都拒，a=0與767對應256與1023；一般user也不能改base/limit。

此保護不保證job不死loop，還需timer；也不解共享file政策。補上一個hardware feature只建立一類邊界，不會一次得到完整現代OS。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Macintosh與Windows：GUI不能決定核心保證

早期Macintosh與Windows使桌面互動成為重要介面；後續桌面平台逐步具有更完整memory、multitasking、networking等支援。比較時必須指定世代；本課Windows7為NT家族案例，不把早期依附DOS的Windows與其kernel契約混用。

GUI的windows、mouse、menu是使用者互動模型；preemption、memory protection、API相容與driver模型是另外的實作／隔離問題。畫出兩個視窗不能證明兩個process可真正平行、也不能證明彼此有受保護address space。

同樣，GUI卡住可能因UI thread阻塞，而非全部kernel不再排程。反應時間需追到事件、queue、UI handler及所等服務。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

cooperative與preemptive的可觀察差別

固定單CPU、工作A執行無限loop且從不yield，B已ready。cooperative模型只在A主動yield／阻塞時選B，因此B可永遠不跑；preemptive模型若timer可被服務且scheduler輪流分配，A最遲在slice邊界被搶占，B才有CPU。

若A其實在kernel持鎖不可搶占且無界loop，光有user slice也不能保證B進度；界限仍需包括kernel與interrupt延遲。這個toy反例比較兩種協定，不將每一代桌面OS都粗分成同一種。

對互動式系統，UI設計與scheduler的界限都重要；若某task持有全域資源，得到CPU的B仍可能等待它。因此「preemptive」不等於沒有deadlock或priority inversion。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach版本與task／thread／port角色

Mach源於Accent相關研究，將task/address-space、thread、IPC與virtual memory作為核心抽象。task持address space與port rights等資源，thread是可排程執行context；多thread可在同task共享memory。

版本／模型	BSD功能放置	能推得的邊界
Mach2.x／2.5脈絡	大量BSD功能仍在kernel	不能說全部file服務都跨user server
Mach3 microkernel構想	UNIX功能移至user server	透過受控IPC跨address space

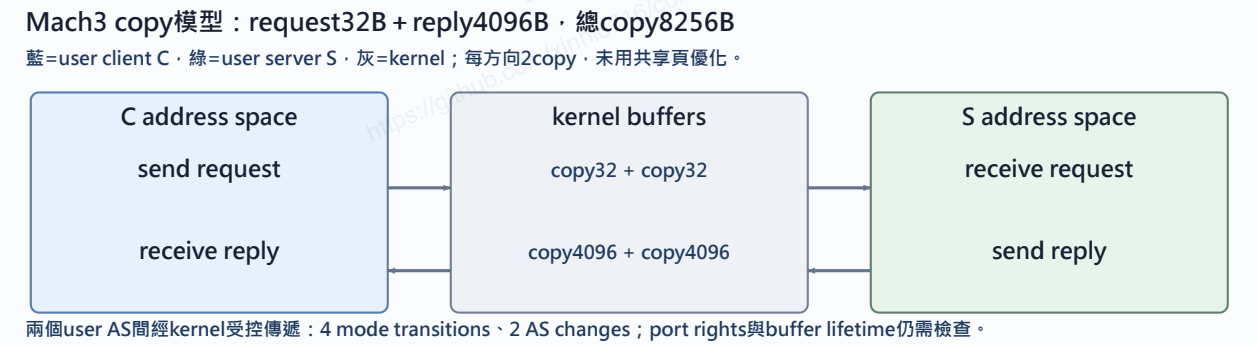
port right是可被kernel檢查的訊息authority；同名整數不是跨task自動有效權利。Mach亦支援shared memory，不能把「訊息是重要介面」擴寫成所有資料交換都必逐byte copy message。

同一request的IPC代價與隔離邊界

固定copy-based模型：client C送32B request給user server S，S回4096B；每方向都user→kernel buffer→另一user兩次copy，不使用shared-page／out-of-line優化。

路徑	mode transitions	address-space changes	copy bytes
C user→K→S user	2	C→S一次	32×2=64
S user→K→C user	2	S→C一次	4096×2=8192
合計	4	2	8256

server crash可被address-space邊界限制直接記憶體損害，但client仍需timeout／重建／錯誤處理；更小kernel不自動證明全部服務可用。訊息VM整合可減少某些copy，代價則轉到mapping、rights、page lifetime與COW，不等於零成本。



其他系統：不要把不同角色的處理器都當對等SMP

原書補充Burroughs MCP的系統程式語言、segmentation及多處理支援，以及CDC6600相關系統的多個處理活動／協調。這些例子提醒：processor角色、記憶體權限與I/O責任要先分清，不能只看「有多顆」就套相同scheduler模型。

高階語言可改善表達、型別／模組及維護，但仍需正確的同步、ABI與硬體控制。反過來，以assembly寫並不邏輯上推出錯誤；語言選擇改變可表達與可檢查的條件，不代替系統契約。

歷史系統是否延續還受相容軟體、硬體成本與部署組織影響。本章沒有從單一設計特色推論商業成敗，也不把書寫當時的市場敘述當成現況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨案例機制對照

問題	案例	不可省略的條件
CPU與慢I/O重疊	resident monitor／spool	queue、buffer、裝置所有權及真正可重疊工作
資訊不全駐core	Atlas	mapping、fault重啟與完整victim選擇
非連續放置／共享code	XDS-940	relocation、唯讀code與private可變狀態
分解正確性推理	THE	無循環的依賴與逐層契約
將政策移出nucleus	RC4000	message ownership、pool上限及等待關係
互動／長burst取捨	CTSS	feedback／promotion／swap成本明列
選擇性共享與命名	Multics	segment、page與protection區分
跨服務隔離	Mach3模型	IPC authority、address space與failure處理

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從歷史演示回到一般證明

- Atlas三規則對非空frames必可選victim：有①用①；否則若有t≠0可在有限集合取最大score；其餘全t=0以③選T最大。這保證定義完整，不保證fault最少。
- buffer守恆由初始狀態與每次原子轉移維持；若buffer同時出現在free及reply，表示ownership轉移破壞不變量。
- layer歸納依Ci只依賴較低層且保留其契約；有cycle或不受控共享修改時，不能沿用。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 20. Atlas one-level store; THE system; Multics GE-645; Multics protection.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study